

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

이 책의 PDF 사본, 코드 번들 등을 받으세요

QR 코드를 스캔하세요 (또는 packtpub.com/unlock으로 이동하세요). 이 책의 이름으로 검색하고, 판을 확인한 후 페이지의 단계를 따르세요.



UNLOCK NOW

참고: 송장을 준비하세요. Packt 웹사이트에서 직접 구매한 경우 송장이 필요하지 않습니다.

7

Advanced Form Validation and Model Forms

Continuing your journey with the `form_project` application you started in the previous chapter, you'll begin this chapter by adding a new form to your app with custom multi-field validation and form cleaning. You'll learn how to set the initial values on your form and customize the widgets (the HTML input elements that are being generated). Then, you'll be introduced to the `ModelForm` class, which allows a form to be automatically created from a model. You'll use it in a view to automatically save the new or changed `Model` instance.

In this chapter, we will take our knowledge of Django form validation further by introducing concepts that form the fundamentals of most production websites.

For instance, a certain field might only be required if another field is set. Let's say we want to add a checkbox to allow users to sign up for our monthly newsletter. It has a text box below it that lets them enter their email address. With some basic validation, we can check the following:

- Whether the user has checked the checkbox
- Whether the user has entered their email address

When the user clicks the **Submit** button, we'll be able to validate whether both fields are actioned. But what if the user doesn't want to sign up for our newsletter? If they click the **Submit** button, ideally, both fields should be blank. That's where validating each field might not work.

Another example could be a case where we have two fields where each has a maximum value of, say, 50. But the total values added to each one must be less than 75. We'll start this chapter by looking at how to write custom validation rules to solve such problems.

7

고급 양식 유효성 검사 및 모델 양식

이전 장에서 시작한 `form_project` 애플리케이션과 함께 여정을 계속하면서, 이 장에서는 사용자 정의 다중 필드 유효성 검사 및 양식 정리를 통해 앱에 새 양식을 추가하는 것으로 시작합니다. 양식의 초기 값을 설정하고 생성되는 HTML 입력 요소인 위젯을 사용자 정의하는 방법을 배우게 됩니다. 그런 다음, 모델에서 자동으로 양식을 생성할 수 있는 `ModelForm` 클래스를 소개합니다. 이를 사용하여 보기에서 새 모델 인스턴스를 자동으로 저장합니다.

이 장에서는 대부분의 프로덕션 웹사이트의 기본을 형성하는 개념을 도입하여 Django 양식 유효성 검사에 대한 지식을 더욱 발전시킬 것입니다.

예를 들어, 특정 필드는 다른 필드가 설정된 경우에만 필수일 수 있습니다. 사용자가 월간 뉴스레터에 가입할 수 있도록 체크박스를 추가하고 싶다고 가정해 보겠습니다. 그 아래에는 이메일 주소를 입력할 수 있는 텍스트 상자가 있습니다. 기본적인 유효성 검사를 통해 다음을 확인할 수 있습니다:

- 사용자가 체크박스를 선택했는지 여부
- 자가 이메일 주소를 입력했는지 여부

사용자가 제출 버튼을 클릭하면 두 필드가 모두 작동했는지 유효성을 검사할 수 있습니다. 하지만 사용자가 뉴스레터에 가입하고 싶지 않다면 어떻게 될까요? 제출 버튼을 클릭하면 이상적으로 두 필드는 비어 있어야 합니다. 이 경우 각 필드를 유효성 검사하는 것이 작동하지 않을 수 있습니다.

또 다른 예는 각 필드의 최대 값이 50인 두 개의 필드가 있는 경우일 수 있습니다. 그러나 각 필드에 추가된 총 값은 75 미만이어야 합니다. 이 장에서는 이러한 문제를 해결하기 위해 사용자 정의 유효성 검사 규칙을 작성하는 방법을 살펴보겠습니다.

Later, as we progress in this chapter, we will look at how to set initial values on a form. This can be useful when automatically filling out information that is already known to the user. For example, we can automatically put a user's contact information into a form if that user is logged in.

We'll finish this chapter by looking at model forms, which will let us automatically create a form from a Django `Model` class. This cuts down the amount of code that needs to be written to create a new `Model` instance.

In this chapter, we will cover the following topics:

- Custom field validation and cleaning
- Adding placeholders and initial values
- Creating or editing Django models

By the end of this chapter, you will know how to add extra multi-field validation to Django forms, how to customize and set form widgets for fields, how to use `ModelForms` to automatically create a form from a Django model, and how to automatically create `Model` instances from `ModelForms`.

Technical requirements

Find the solution in the `Chapter07` folder in this book's GitHub repo. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

Custom field validation and cleaning

In the previous chapter, we saw how a Django form converts values from an HTTP request, which are strings, into Python objects. In a non-custom Django form, the target type is dependent on the field class. For example, the Python type derived from `IntegerField` is `int`, and string values are given to us verbatim, as the user enters them. But we can also implement methods on our `Form` class to alter the output values from our fields in any way we choose. This allows us to clean or filter the user's input data to fit what we expect better. We could round an integer to the nearest multiple of 10 so that it fits into a batch size for ordering specific items. Or we could transform an email address into lowercase so that the data is consistent for searching.

We can also implement some custom validators. We'll look at a couple of different ways of validating fields: by writing a custom validator and by writing a custom `clean` method for the field. Each method has its pros and cons: a custom validator can be applied to different fields and forms, so you don't have to write the validation logic for each field; a custom `clean`

이 장이 진행됨에 따라, 우리는 양식에 초기 값을 설정하는 방법을 살펴볼 것입니다. 이는 사용자가 이미 알고 있는 정보를 자동으로 입력할 때 유용할 수 있습니다. 예를 들어, 사용자가 로그인한 경우 그 사용자의 연락처 정보를 양식에 자동으로 입력할 수 있습니다.

이 장을 마치면서, 우리는 모델 양식을 살펴볼 것입니다. 이는 Django 모델 클래스에서 자동으로 양식을 생성할 수 있게 해줍니다. 이는 새로운 모델 인스턴스를 생성하기 위해 작성해야 하는 코드의 양을 줄여줍니다.

이 장에서는 다음 주제를 다룰 것입니다

- 다: 사용자 정의 필드 검증 및 정
- 리, 플레이스홀더 및 초기 값 추가,
- Django 모델 생성 또는 편집
-

이 장이 끝날 때쯤이면, Django 양식에 추가 다중 필드 검증을 추가하는 방법, 필드에 대한 양식 위젯을 사용자 정의하고 설정하는 방법, Django 모델에서 자동으로 양식을 생성하기 위해 ModelForms를 사용하는 방법, 그리고 ModelForms에서 Model 인스턴스를 자동으로 생성하는 방법을 알게 될 것입니다.

기술 요구 사항

이 책의 GitHub 저장소의 Chapter07 폴더에서 솔루션을 찾으십시오. 저장소 링크에 접근하려면, 서문에 있는 예제 코드 파일 다운로드 섹션의 단계를 따르십시오.

사용자 정의 필드 검증 및 정리

이전 idx_8154900b장에서는 Django 폼이 HTTP 요청에서 값을 문자열로 변환하여 Python 객체로 변환하는 방법을 보았습니다. 비커스터마이즈 Django 폼에서는 대상 유형이 필드 클래스에 따라 달라집니다. 예를 들어, IntegerField에서 파생된 Python 유형은 int이며, 문자열 값은 사용자가 입력한 대로 그대로 제공됩니다. 그러나 우리는 또한 Form 클래스에서 메서드를 구현하여 필드의 출력 값을 원하는 방식으로 변경할 수 있습니다. 이를 통해 사용자의 입력 데이터를 우리가 기대하는 것에 더 잘 맞도록 정리하거나 필터링할 수 있습니다. 우리는 정수를 가장 가까운 10의 배수로 반올림하여 특정 항목을 주문하기 위한 배치 크기에 맞출 수 있습니다. 또는 이메일 주소를 소문자로 변환하여 데이터가 검색을 위해 일관되도록 할 수 있습니다.

우리는 또한 몇 가지 사용자 정의 유효성 검사기를 구현할 수 있습니다. 필드를 검증하는 몇 가지 다른 방법을 살펴보겠습니다: 사용자 정의 유효성 검사기를 작성하고 필드에 대한 사용자 정의 클린 메서드를 작성하는 것입니다. 각 방법에는 장단점이 있습니다: 사용자 정의 유효성 검사기는 서로 다른 필드와 폼에 적용할 수 있으므로 각 필드에 대한 유효성 검사 논리를 작성할 필요가 없습니다; 사용자 정의 클린

method must be implemented on each form you want to clean, but is more powerful and allows validation since you can use other fields in the form or change the cleaned value that the field returns.

Custom validators

A validator is simply a function that accepts a value and raises `django.core.exceptions.ValidationError` if the value is invalid – the validity is determined by the code you write. The value is a Python object (that is, `cleaned_data`, which has already been converted from the POST request string).

Here's a simple example that validates whether a value is in lowercase:

```
from django.core.exceptions import ValidationError

def validate_lowercase(value):
    if value.lower() != value:
        raise ValidationError(f"{value} is not lowercase.")
```

Notice the function does not return anything, for either success or failure. It will just raise `ValidationError` if the value is not valid.

Note

Note that the behavior and handling of `ValidationError` are different from how we've seen other exceptions behave in Django. Normally, if you raise an exception in your view, you'll end up with an HTTP response with a 500 status code from Django (if you don't handle the exception in your code).

When raising `ValidationError` in your validation/cleaning code, the Django form class will catch the error for you; then, the `is_valid` method of form will return `False`. You don't have to write try/except handlers around the code that might raise `ValidationError`.

The validator can be passed to the `validators` argument of a field constructor on a form, inside a list – for example, to our `text_input` field from `ExampleForm`:

```
class ExampleForm(forms.Form):
    text_input = forms.CharField(
        validators=[validate_lowercase])
```

각 폼에서 정리하려는 메서드는 구현되어야 하지만, 더 강력하며 유효성 검사를 허용합니다. 왜냐하면 폼의 다른 필드를 사용하거나 필드가 반환하는 정리된 값을 변경할 수 있기 때문입니다.

사용자 정의 검증기

검증기 `idx_28721089`는 단순히 값을 받아들이고 값이 유효하지 않은 경우 `django.core.exceptions.ValidationError`를 발생시키는 함수입니다. 유효성은 당신이 작성한 코드에 의해 결정됩니다. 값은 Python 객체입니다(즉, POST 요청 문자열에서 이미 변환된 `cleaned_data`).

값이 소문자인지 검증하는 간단한 예는 다음과 같습니다:

```
from django.core.exceptions import ValidationError

def validate_lowercase(value):
    if value.lower() != value:
        raise ValidationError(f"{value}는 소문자가 아닙니다.")
```

함수가 성공이나 실패에 관계없이 아무것도 반환하지 않는다는 점에 유의하십시오. 값이 유효하지 않으면 `ValidationError`를 발생시킬 뿐입니다.

참고

`ValidationError`의 동작 및 처리 방식이 Django에서 다른 예외가 동작하는 방식과 다르다는 점에 유의하십시오. 일반적으로, 뷰에서 예외를 발생시키면 Django에서 500 상태 코드가 있는 HTTP 응답을 받게 됩니다(코드에서 예외를 처리하지 않는 경우).

유효성 검사/정리 코드에서 `ValidationError`를 발생시킬 때, Django 폼 클래스가 오류를 잡아줍니다. 그런 다음, 폼의 `is_valid` 메서드는 `False`를 반환합니다.

`ValidationError`를 발생시킬 수 있는 코드 주위에 `try/except` 핸들러를 작성할 필요가 없습니다.

검증기는 폼의 필드 생성자에 있는 `validators` 인수에 리스트로 전달될 수 있습니다. 예를 들어, `ExampleForm`의 `text_input` 필드에 대해 다음과 같이 할 수 있습니다:

```
class ExampleForm(forms.Form):
    text_input = forms.CharField(
        validators=[validate_lowercase])
```

Now, if we submit the form and the fields contain uppercase values, we'll get an error, as shown in the following figure:

- Text is not lowercase.

Text input:

Figure 7.1 – Lowercase text validator in action

The validator function can be used on any number of fields. In our example, if we wanted lots of fields to have lowercase enforced, `validate_lowercase` could be passed to all of them. Now, let's look at how we could implement this another way, with a custom `clean` method.

Cleaning methods

A `clean` method is created on the `Form` class and is named in the `clean_field_name` format. For example, the `clean` method for `text_input` would be called `clean_text_input`, the `clean` method for `books_you_own` would be called `clean_books_you_own`, and so on.

Cleaning methods take no arguments; instead, they should use the `cleaned_data` attribute on `self` to access the field data. This dictionary will contain the data after being cleaned in the standard Django way, as we saw in the previous example. The `clean` method must return the cleaned value, which will replace the original value in the `cleaned_data` dictionary. Even if the method does not change the value, a value must be returned. You can also use the `clean` method to raise `ValidationError`, and the error will be attached to the field (the same as with a validator).

Let's re-implement the lowercase validator as a `clean` method, like this:

```
class ExampleForm(forms.Form):
    text_input = forms.CharField()
    ...

    def clean_text_input(self):
        value = self.cleaned_data["text_input"]
        if value.lower() != value:
            raise ValidationError(
                f"{value} is not lowercase.")
        return value
```

이제, 우리가 양식을 제출하고 필드에 대문자 값이 포함되어 있으면, 다음 그림과 같이 오류가 발생합니다:

- Text is not lowercase.

Text input:

그림 7.1 - 소문자 텍스트 검증기 작동 중

검증기 `idx_a3218c3b` 함수는 필드 수에 관계없이 사용할 수 있습니다. 예를 들어, 많은 필드에 소문자를 강제 적용하고 싶다면, `validate_lowercase`를 모든 필드에 전달할 수 있습니다. 이제, 사용자 정의 클린 메서드를 사용하여 이를 구현하는 방법을 살펴보겠습니다.

클린 메서드

클린 메서드 `idx_7cb91354`는 `Form` 클래스에서 생성되며 `clean_field_name` 형식으로 명명됩니다. 예를 들어, `text_input`의 클린 메서드는 `clean_text_input`이라고 불리며, `books_you_own`의 클린 메서드는 `clean_books_you_own`이라고 불립니다.

클린 메서드는 인수를 받지 않습니다. 대신, `cleaned_data` 속성을 사용하여 필드 데이터를 접근해야 합니다. 이 사전은 이전 예제에서 보았듯이 표준 Django 방식으로 정리된 후의 데이터를 포함합니다. 클린 메서드는 `cleaned_data` 사전에서 원래 값을 대체할 정리된 값을 반환해야 합니다. 메서드가 값을 변경하지 않더라도, 값을 반환해야 합니다. 또한 클린 메서드를 사용하여 `ValidationError`를 발생시킬 수 있으며, 오류는 필드에 첨부됩니다(검증기와 동일).

소문자 검증기를 클린 메서드로 다시 구현해 보겠습니다, 다음과 같이:

```
class ExampleForm(forms.Form):
    text_input = forms.CharField()
    ...

    def clean_text_input(self):
        value = self.cleaned_data["text_input"]
        if value.lower() != value:
            raise ValidationError(
                f"{value}는 소문자가 아닙니다.")
        return value
```

You can see that the logic is essentially the same, except we must return the validated value at the end. If we submit the form, we will get the same result as the previous time we tried (Figure 7.1).

Let's look at one more cleaning example. Instead of raising an exception when the value is invalid, we could just convert the value into lowercase. We would implement that with this code:

```
class ExampleForm(forms.Form):
    text_input = forms.CharField()
    ...

    def clean_text_input(self):
        value = self.cleaned_data['text_input']
        return value.lower()
```

Now, let's say we enter text into the input as uppercase:

Text input:

Figure 7.2 – ALL UPPERCASE text entered

If we were to examine the cleaned data using our debug output from the view, we would see that it is lowercase:

text_input: (<class 'str'>) all uppercase

Figure 7.3 – The cleaned data has been transformed into lowercase

These were just a couple of simple examples of how to validate fields using both validators and clean methods. You can, of course, make each type of validation much more complex if you wish, and transform the data in more complex ways using a clean method.

So far, you've only learned simple methods for form validation, where you've treated each field independently. A field is valid (or not) based only on the information it contains and nothing else. What if the validity of one field depends on what the user entered into another field? An example of this might be that you have an email field to collect someone's email address if they want to be signed up for a mailing list. The field is only required if they check a checkbox that indicates they want to be signed up. Neither of these fields is required on its own – we do not want the checkbox to be required to be checked, but if it is checked, then the email field should be required too.

idx_7300ced4에서 볼 수 있듯이, 논리는 본질적으로 동일하지만, 마지막에 검증된 값을 반환해야 합니다. 폼을 제출하면, 이전에 시도했을 때와 동일한 결과를 얻을 수 있습니다 (그림 7.1).

또 다른 정리 예제를 살펴보겠습니다. 값이 유효하지 않을 때 예외를 발생시키는 대신, 값을 소문자로 변환할 수 있습니다. 우리는 이 코드를 사용하여 이를 구현할 것입니다:

```
class ExampleForm(forms.Form):
    text_input = forms.CharField()
    ...

    def clean_text_input(self):
        value = self.cleaned_data['text_input']
        return value.lower()
```

이제 입력란에 대문자로 텍스트를 입력한다고 가정해 보겠습니다:

Text input:

그림 7.2 - 대문자만 입력된 텍스트

우리가 뷰에서 디버그 출력을 사용하여 정리된 데이터를 검사한다면, 그것이 소문자임을 알 수 있습니다:

text_input: (<class 'str'>) all uppercase

그림 7.3 - 정리된 데이터가 소문자로 변환되었습니다

이것은 검증기와 정리 방법을 사용하여 필드를 검증하는 방법에 대한 간단한 예시 몇 가지였습니다. 물론 원하신다면 각 유형의 검증을 훨씬 더 복잡하게 만들 수 있으며, 정리 방법을 사용하여 데이터를 더 복잡한 방식으로 변환할 수 있습니다.

지금까지 여러분은 각 필드를 독립적으로 다루는 간단한 양식 검증 방법만 배웠습니다. 필드는 그 자체로 포함된 정보에만 기반하여 유효(또는 유효하지 않음)합니다. 한 필드의 유효성이 사용자가 다른 필드에 입력한 내용에 따라 달라진다면 어떻게 될까요? 예를 들어, 사용자가 메일링 리스트에 가입하기를 원할 경우 이메일 주소를 수집하기 위한 이메일 필드가 있을 수 있습니다. 이 필드는 사용자가 가입하기를 원한다는 것을 나타내는 체크박스를 체크할 경우에만 필수입니다. 이 두 필드는 각각 독립적으로 필수가 아닙니다 - 체크박스가 체크되어야 하는 것은 원하지 않지만, 체크되어 있다면 이메일 필드도 필수여야 합니다.

In the next section, we will show how you can validate a form whose fields depend on each other by overriding the `clean` method in your form.

Multi-field validation

We've just looked at the `clean_field_name` methods that can be added to a Django form to clean a specific field. Django also allows us to override the `clean` method, in which we can access all cleaned data from all fields, and we know that all custom field methods have been called. This allows the fields to be validated based on another field's data.

Referring to our previous example with a form that has an email address that is only required if a checkbox is checked, we'll see how we can implement this using the `clean` method.

First, create a Form class and add two fields – make them both optional with the `required=False` argument:

```
class NewsletterSignupForm(forms.Form):
    signup = forms.BooleanField(
        label="Sign up to newsletter?", required=False)
    email = forms.EmailField(
        help_text="Enter your email address to subscribe",
        required=False)
```

We've also introduced two new arguments that can be used for any field:

- `label`: This allows you to set the label text for a field. As we've seen, Django will automatically generate label text from the field name. If you set the `label` argument, you can override this default. Use this argument if you want to have a more descriptive label.
- `help_text`: If you need more information to be displayed regarding what input a field requires, you can use this argument. By default, it's displayed after the field.

When rendered, the form looks like this:

Sign up to newsletter? ☐

Email: Enter your email address to subscribe

Figure 7.4 – Email signup form with a custom label and help text

다음 섹션에서는 필드가 서로 의존하는 양식을 검증하는 방법을 양식의 clean 메서드를 재정의하여 보여드리겠습니다.

다중 필드 검증

우리는 특정 필드를 정리하기 위해 Django 양식에 추가할 수 있는 clean_field_name 메서드를 살펴보았습니다. Django는 또한 clean 메서드를 재정의할 수 있도록 허용하며, 이 메서드에서는 모든 필드의 정리된 데이터에 접근할 수 있고, 모든 사용자 정의 필드 메서드가 호출되었음을 알 수 있습니다. 이를 통해 필드가 다른 필드의 데이터에 기반하여 검증될 수 있습니다.

체크박스가 선택된 경우에만 이메일 주소가 필요한 양식의 이전 예를 참조하여, clean 메서드를 사용하여 이를 구현하는 방법을 살펴보겠습니다.

먼저, Form 클래스를 만들고 두 개의 필드를 추가합니다. 두 필드 모두 required=False 인수로 선택 사항으로 만듭니다:

```
class NewsletterSignupForm(forms.Form):
    signup = forms.BooleanField(
        label="뉴스레터에 가입하시겠습니까?", required=False)
    email = forms.EmailField(
        help_text="구독하려면 이메일 주소를 입력하세요",
        required=False)
```

우리는 또한 모든 필드에 사용할 수 있는 두 가지 새로운 인수를 도입했습니다:

- **label:** 이 옵션은 필드의 라벨 텍스트를 설정할 수 있게 해줍니다. 앞서 본 것처럼, Django는 필드 이름에서 자동으로 라벨 텍스트를 생성합니다. label 인자를 설정하면 이 기본값을 덮어쓸 수 있습니다. 더 설명적인 라벨을 원할 경우 이 인자를 사용하세요.
- **help_text:** 필드에 어떤 입력이 필요한지에 대해 더 많은 정보를 표시해야 하는 경우 필요한 경우 이 인수를 사용할 수 있습니다. 기본적으로 필드 뒤에 표시됩니다. 렌더링되면 폼은 다음과 같이 보입니다:

시됩니다. 렌더링되면 폼은 다음과 같이 보입니다:

Sign up to newsletter? ☐

Email: Enter your email address to subscribe

그림 7.4 - 사용자 지정 레이블과 도움말 텍스트가 포함된 이메일 가입 양식

If we were to submit the form now, without entering any data, nothing would happen. Neither field is required, so the form validates fine.

Now, we can add the multi-field validation to the `clean` method. We will check whether the **Sign up to newsletter?** checkbox is checked, and then check that the **Email** field has a value. The built-in Django methods have already validated that the email address is valid at this point, so we just need to check that a value exists for it. We will then use the `add_error` method to set an error for the email field. This is a method you haven't seen before, but it's very simple; it takes two arguments – the name of the field to set the error on, and the text of the error.

Here's the code for the `clean` method:

```
class NewsletterSignupForm(forms.Form):
    ...

    def clean(self):
        cleaned_data = super().clean()

        if (cleaned_data["signup"] and not
            cleaned_data.get("email")):
            self.add_error("email", "Your email address is required "
                           "if signing up for the newsletter.")
```

Your `clean` method must always call the `super().clean()` method to retrieve the cleaned data. When `add_error` is called to add errors to the form, the form will no longer validate (the `is_valid` method returns `False`).

Now, if we submit the form without the checkbox checked, still, no error is generated, but if you check the checkbox without an email address, you will receive the error we just wrote the code for:

지금 데이터를 입력하지 않고 양식을 idx_a642c499 제출하면 아무 일도 일어나지 않습니다. 필드가 필수는 아니므로 양식은 정상적으로 유효성을 검사합니다.

이제 우리는 clean 메서드에 다중 필드 유효성 검사를 추가할 수 있습니다. 뉴스레터에 가입하시겠습니까? 체크박스가 선택되었는지 확인한 다음, 이메일 필드에 값이 있는지 확인합니다. 내장된 Django 메서드는 이미 이 시점에서 이메일 주소가 유효하다고 확인했으므로, 우리는 단지 값이 존재하는지 확인하면 됩니다. 그런 다음 add_error 메서드를 사용하여 이메일 필드에 대한 오류를 설정합니다. 이것은 당신이 이전에 보지 못한 메서드이지만 매우 간단합니다. 오류를 설정할 필드의 이름과 오류 텍스트라는 두 개의 인수를 받습니다.

다음은 clean 메서드의 코드입니다:

```
class NewsletterSignupForm(forms.Form):
    ...

    def clean(self):
        cleaned_data = super().clean()

        if (cleaned_data["signup"] and not
            cleaned_data.get("email")):
            self.add_error("email", "이메일 주소는 필수입니다 ",
                           "뉴스레터에 가입하는 경우.")
```

clean 메서드는 항상 super().clean() 메서드를 호출하여 cleaned_data를 검색해야 합니다. add_error가 호출되어 폼에 오류를 추가하면 폼은 더 이상 유효성을 검사하지 않습니다(이 메서드는 False를 반환합니다).

이제 체크박스를 체크하지 않고 폼을 제출하면 여전히 오류가 생성되지 않지만, 이메일 주소 없이 체크박스를 체크하면 방금 작성한 오류가 발생합니다:

Sign up to newsletter? ☒

- Enter your email address to subscribe

Email: Enter your email address to subscribe

Submit Input

Figure 7.5 – Error displayed when attempting to sign up with no email address

You might notice that we are retrieving the email from the `cleaned_data` dictionary using the `get` method. The reason for doing this is that if the `email` value in the form is invalid, then the `email` key will not exist in the dictionary. The browser should prevent the user from submitting the form if an invalid email has been entered, but a user might be using an older browser that does not support this client-side validation, so, for safety, we use the `get` method. Since the `signup` field is a `BooleanField` type and not required, it will only be invalid if a custom validation function is used. We're not using one here, so it's safe to access its value using square bracket notation.

There's one more validation scenario to consider before moving on to our first exercise, and that's adding errors that are not specific to any field. Django calls these **non-field errors**. There are many scenarios where you might want to use these when multiple fields are dependent on each other.

Take, for example, a shopping website. Your order form could have two numeric fields whose totals can't exceed a certain value. If the total were exceeded, the value of either field could be decreased to bring the total below the maximum value, so the error is not specific to either of the fields. To add a non-field error, call the `add_error` method with `None` as the first argument.

Let's look at how to implement this. In this example, we'll have a form where the user can specify a certain number of items to order, for item A or item B. The user cannot order more than 100 items in total. The fields will have a maximum value of 100, and a minimum value of 0, but custom validation in the `clean` method will need to be written to handle the validation of the total amount:

```
class OrderForm(forms.Form):
    item_a = forms.IntegerField(min_value=0, max_value=100)
    item_b = forms.IntegerField(min_value=0, max_value=100)
```

Sign up to newsletter? ☒

- Enter your email address to subscribe

Email: Enter your email address to subscribe

Submit Input

그림 7.5 - 이메일 주소 없이 가입하려고 할 때 표시되는 오류

우리는 cleaned_data 사전에서 get 메서드를 사용하여 이메일을 가져오고 있다는 점을 주목할 수 있습니다. 이렇게 하는 이유는 양식의 이메일 값이 유효하지 않으면 사전에서 email 키가 존재하지 않기 때문입니다. 브라우저는 유효하지 않은 이메일이 입력된 경우 사용자가 양식을 제출하지 못하도록 해야 하지만, 사용자가 이 클라이언트 측 유효성 검사를 지원하지 않는 오래된 브라우저를 사용하고 있을 수 있으므로 안전을 위해 get 메서드를 사용합니다. signup 필드는 BooleanField 유형이며 필수는 아니므로 사용자 정의 유효성 검사 함수가 사용되지 않는 한 유효하지 않게 됩니다. 여기서는 사용하지 않으므로 대괄호 표기법을 사용하여 안전하게 그 값을 접근할 수 있습니다.

첫 번째 연습으로 넘어가기 전에 고려해야 할 또 다른 유효성 검사 시나리오가 있으며, 그것은 특정 필드에 국한되지 않은 오류를 추가하는 것입니다. Django는 이러한 비필드 오류를 비필드 오류라고 부릅니다. 여러 필드가 서로 의존하는 경우 이러한 오류를 사용하고 싶을 수 있는 많은 시나리오가 있습니다.

예를 들어 쇼핑 웹사이트를 생각해 보십시오. 주문 양식에는 총합이 특정 값을 초과할 수 없는 두 개의 숫자 필드가 있을 수 있습니다. 총합이 초과되면 두 필드 중 하나의 값을 줄여서 총합을 최대값 이하로 가져올 수 있으므로 오류는 두 필드 중 어느 것에도 특정되지 않습니다. 비필드 오류를 추가하려면 add_error 메서드를 호출하고 첫 번째 인수로 None을 전달합니다.

이것을 구현하는 방법을 살펴보겠습니다. 이 예제에서는 사용자가 A 항목 또는 B 항목에 대해 주문할 특정 수량을 지정할 수 있는 양식이 있습니다. 사용자는 총 100개 이상의 항목을 주문할 수 없습니다. 필드는 최대값이 100이고 최소값이 0이지만, 총 금액의 유효성을 검사하기 위해 clean 메서드에서 사용자 정의 유효성 검사를 작성해야 합니다:

```
class OrderForm(forms.Form):
    item_a = forms.IntegerField(min_value=0, max_value=100)
    item_b = forms.IntegerField(min_value=0, max_value=100)
```

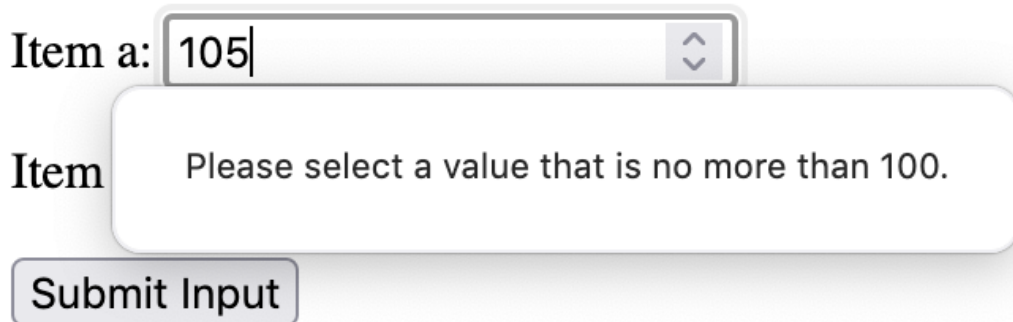
```
def clean(self):
    cleaned_data = super().clean()
    if (cleaned_data.get("item_a", 0) +
        cleaned_data.get("item_b", 0) > 100):
        self.add_error(None, "The total number of items "
                             "must be 100 or less.")
```

The fields (`item_a` and `item_b`) are added in the normal way, with standard validation rules. You can see that we have used the `clean` method the same way we used it before. Moreover, we have implemented the maximum item logic inside this method. The following line is what registers the non-field error if the maximum number of items is exceeded:

```
self.add_error(None, "The total number of items must be 100 or less.")
```

Once again, we access the values of `item_a` and `item_b` using the `get` method, with a default value of `0`. This is in case the user has an older browser (from 2011 or earlier) and is able to submit the form with invalid values.

In a browser, the field-level validation ensures values between `0` and `100` have been entered in each field, and prevents the form from being submitted otherwise:



Item a: 105

Item Please select a value that is no more than 100.

Submit Input

Figure 7.6 – The form cannot be submitted if one field exceeds the maximum value

However, if we put in two values that sum to more than 100, we can see how Django displays the non-field error:

```
def clean(self):
    cleaned_data = super().clean()
    if (cleaned_data.get("item_a", 0) +
        cleaned_data.get("item_b", 0) > 100):
        self.add_error(None, "항목의 총 수는 "
            "100 이하이어야 합니다.")
```

항목(item_a 및 item_b)은 표준 검증 규칙에 따라 일반적인 방식으로 추가됩니다. 우리는 clean 메서드를 이전과 동일한 방식으로 사용했음을 알 수 있습니다. 또한, 이 메서드 내에서 최대 항목 논리를 구현했습니다. 다음 줄은 최대 항목 수를 초과할 경우 비필드 오류를 등록하는 것입니다:

```
self.add_error(None, "항목의 총 수는 100 이하이어야 합니다.")
```

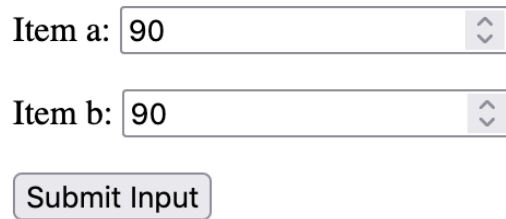
다시 한 번, 우리는 get 메서드를 사용하여 item_a와 item_b의 값을 접근하며, 기본값은 0입니다. 이는 사용자가 오래된 브라우저(2011년 또는 그 이전)를 사용하고 잘못된 값으로 양식을 제출할 수 있는 경우를 대비한 것입니다.

브라우저에서 필드 수준 유효성 검사는 각 필드에 0과 100 사이의 값이 입력되었는지 확인하고, 그렇지 않으면 양식이 제출되는 것을 방지합니다:

그림 7.6 - 한 필드가 최대 값을 초과하면 양식을 제출할 수 없습니다

그러나 우리가 100을 초과하는 두 값을 입력하면 Django가 비필드 오류를 어떻게 표시하는지 볼 수 있습니다:

- The total number of items must be 100 or less.



Item a:

Item b:

Figure 7.7 – Django non-field error displayed at the start of the form

Django non-field errors are always displayed at the start of a form, before other fields or errors.

In the following exercise, we will build a form that implements a validation function, a field clean method, and a form clean method.

Exercise 7.01 – Custom clean and validation methods

In this exercise, you will build a new form that allows the user to create an order for books or magazines. It must have the following validation criteria:

- The user may order up to 80 magazines and/or 50 books, but the total number of items must not be more than 100
- The user can choose to receive an order confirmation, and if they do, they must enter an email address
- The user should not enter an email address if they have not chosen to receive an order confirmation
- To ensure they are part of our company, the email address must be part of our company domain (in our case, we'll just use `example.com`)
- For consistency with other email addresses in our fictional company, the address should be converted to lowercase

This sounds like a lot of rules, but with Django, it's simple if we tackle them one by one. We'll carry on with the `form_project` app you started in *Chapter 6, Forms*. If you haven't completed *Chapter 6, Forms*, you can download the code from the `Chapter07` folder in this book's GitHub repo.

Follow these steps:

1. In PyCharm, open the `form_example` app's `forms.py` file.

- The total number of items must be 100 or less.

Item a:

Item b:

그림 7.7 - Django 비필드 오류가 양식의 시작 부분에 표시됨

Django 비필드 오류는 항상 양식의 시작 부분에 표시되며, 다른 필드나 오류보다 먼저 나타납니다.

다음 연습에서는 유효성 검사 기능, 필드 클린 메서드 및 양식 클린 메서드를 구현하는 양식을 만들 것입니다.

연습 7.01 – 사용자 정의 클린 및 유효성 검사 메서드

이 연습에서는 사용자가 책이나 잡지 주문을 생성할 수 있는 새로운 양식을 만들 것입니다. 다음 유효성 검사 기준을 충족해야 합니다:

- 사용자는 최대 80개의 잡지와/또는 50권의 책을 주문할 수 있지만, 총 항목 수는 100개를 초과할 수 없습니다.
- 사용자는 주문 확인을 받을지 선택할 수 있으며, 선택할 경우 이메일 주소를 입력해야 합니다.
- 주문 확인을 받지 않기로 선택한 경우 이메일 주소를 입력하지 않아야 합니다.
- 회사의 일원임을 보장하기 위해 이메일 주소는 우리 회사 도메인의 일부여야 합니다(우리 경우에는 example.com을 사용하겠습니다).
- 우리 허구의 회사의 다른 이메일 주소와 일관성을 위해 주소는 소문자로 변환되어야 합니다.

이것은 많은 규칙처럼 들리지만, Django를 사용하면 하나씩 해결하면 간단합니다. 6장, 양식에서 시작한 form_project 앱을 계속 진행하겠습니다. 6장, 양식을 완료하지 않았다면 이 책의 GitHub 저장소의 Chapter07 폴더에서 코드를 다운로드할 수 있습니다.

다음 단계를 따르세요:

1. PyCharm에서 form_example 앱의 forms.py 파일을 엽니다.

The function splits the email address on the @ symbol, and then checks whether the part after it is equal to `example.com`. This function alone would validate non-email addresses. For example, the `not-valid@some other domain@example.com` string would not cause `ValidationError` to be raised in this function. This is acceptable in our case because, since we're using `EmailField`, the other standard field validators will check the email address's validity.

5. Add the `validate_email_domain` function as a validator to the email field on `OrderForm`. Update the `EmailField` constructor call to add a `validators` argument, passing in a list containing the validation function:

```
class OrderForm(forms.Form):
    ...
    email = forms.EmailField(required=False,
                             validators=[validate_email_domain])
```

6. Add a `clean_email` method to the form to make sure the email address is lowercase:

```
class OrderForm(forms.Form):
    # truncated for brevity

    def clean_email(self):
        return self.cleaned_data['email'].lower()
```

7. Now, add the `clean` method to perform all the cross-field validation. First, we will just add the logic for making sure that an email address is only entered if an order confirmation is requested:

```
class OrderForm(forms.Form):
    # truncated for brevity

    def clean(self):
        cleaned_data = super().clean()
        if (cleaned_data["send_confirmation"] and
            not cleaned_data.get("email")):
            self.add_error("email",
                           "Please enter an email address to "
                           "receive the confirmation message.")

        elif (cleaned_data.get("email") and
              not cleaned_data["send_confirmation"]):
```

함수는 이메일 주소를 @ 기호로 나누고, 그 뒤의 부분이 example.com과 같은지 확인합니다. 이 함수만으로는 유효하지 않은 이메일 주소를 검증할 수 있습니다. 예를 들어, not-valid@some other domain@example.com 문자열은 이 함수에서 ValidationError를 발생시키지 않습니다. 이는 우리가 EmailField를 사용하고 있기 때문에 다른 표준 필드 검증기가 이메일 주소의 유효성을 확인할 것이므로 우리의 경우에는 허용됩니다.

5. OrderForm의 이메일 필드에 validate_email_domain 함수를 검증기로 추가합니다. 검증 함수가 포함된 목록을 전달하여 EmailField 생성자 호출을 업데이트합니다:

```
class OrderForm(forms.Form):
    ...
    email = forms.EmailField(required=False,
                             validators=[validate_email_domain])
```

6. 이메일 주소가 소문자인지 확인하기 위해 폼에 clean_email 메서드를 추가합니다:

```
class OrderForm(forms.Form):
    # 간결성을 위해 생략됨

    def clean_email(self):
        return self.cleaned_data['email'].lower()
```

7. 이제 모든 교차 필드 유효성 검사를 수행하기 위해 clean 메서드를 추가합니다. 먼저, 우리는 단지 주문이 요청된 경우에만 이메일 주소가 입력되도록 하는 논리를 추가할 것입니다.

idx_4dfb9b27confirmation이 요청되었습니다:

```
class OrderForm(forms.Form):
    # 간결성을 위해 생략됨

    def clean(self):
        cleaned_data = super().clean()
        if (cleaned_data["send_confirmation"] and
            not cleaned_data.get("email")):
            self.add_error("email",
                           "확인 메시지를 받으려면 이메일 주소를 입력하세요."
                           "receive the confirmation message.")

        elif (cleaned_data.get("email") and
              not cleaned_data["send_confirmation"]):
```

```
self.add_error("send_confirmation",
               "Please check this if you want to "
               "receive a confirmation email.")
```

This will add an error to the **Email** field if **Send confirmation** is checked but no email address is added:

Method: POST

Magazine count: 10

Book count: 20

Send confirmation: ☒

- Please enter an email address to receive the confirmation message.

Email:

Submit Input

Button With **Styled** Text

Figure 7.8 – Error if Send confirmation is checked but no email address is added

Similarly, an error will be added to **Email** if an email address is entered but **Send confirmation** is not checked:

Method: POST

Magazine count: 10

Book count: 20

- Please check this if you want to receive a confirmation email.

Send confirmation: ☐

Email:

Submit Input

Button With **Styled** Text

Figure 7.9 – Error because an email has been entered but the user has not chosen to receive confirmation

8. Add the final check, also inside the `clean` method. The total number of items should not be more than 100. We will add a non-field error if the sum of `magazine_count` and `book_count` is greater than 100:

```
class OrderForm(forms.Form):
    ...
```

```
self.add_error("send_confirmation",
               "확인 이메일을 받으려면 이 항목을 체크하세요 "
               "확인 이메일을 받으려면 체크하세요.")
```

Send confirmation이 체크되었지만 이메일 주소가 추가되지 않은 경우 이메일 필드에 오류가 추가됩니다:

The screenshot shows a web form with the following elements:

- Method: POST**
- Magazine count:** 10 (dropdown menu)
- Book count:** 20 (dropdown menu)
- Send confirmation:** ☒
- Please enter an email address to receive the confirmation message.
- Email:**
- Submit Input** button
- Button With **Styled** Text** button

그림 7.8 - Send confirmation이 체크되었지만 이메일 주소가 추가되지 않은 경우 오류

유사하게, 이메일 주소가 입력되었지만 Send confirmation이 체크되지 않은 경우 Email에 오류 idx_56ea2c2a가 추가됩니다:

The screenshot shows a web form with the following elements:

- Method: POST**
- Magazine count:** 10 (dropdown menu)
- Book count:** 20 (dropdown menu)
- Please check this if you want to receive a confirmation email.
- Send confirmation:** ☐
- Email:**
- Submit Input** button
- Button With **Styled** Text** button

그림 7.9 - 이메일이 입력되었지만 사용자가 확인을 받기로 선택하지 않은 경우 오류

- 최종 검사를 추가하세요. 클린 메서드 안에서도 마찬가지입니다. 항목의 총 수는 100을 초과해서는 안 됩니다. magazine_count와 book_count의 합이 100을 초과하면 비필드 오류를 추가할 것입니다:

```
class OrderForm(forms.Form):
```

```
...
```

```
def clean(self):
    ...
    item_total = cleaned_data.get(
        "magazine_count", 0) + cleaned_data.get(
        "book_count", 0)

    if item_total > 100:
        self.add_error(None,
            "The total number of items must be "
            "100 or less.")
```

This will add a non-field error by passing `None` as the first argument to the `add_error` call.

Note

Refer to `/form_project/form_example/forms.py` folder in the Chapter07 folder in this book's GitHub repo for the complete code.

Save `forms.py`.

9. Open the reviews app's `views.py` file. We will change the form's import so that `OrderForm` is being imported instead of `ExampleForm`. Consider the following import line:

```
from .forms import ExampleForm
```

Change it as follows:

```
from .forms import OrderForm
```

10. In the `form_example` view, change the two lines that use `ExampleForm` to use `OrderForm` instead. Consider the following line of code:

```
form = ExampleForm(request.POST)
```

Change this as follows:

```
form = OrderForm(request.POST)
```

```
def clean(self):
    ...
    item_total = cleaned_data.get(
        "magazine_count", 0) + cleaned_data.get(
        "book_count", 0)

    if item_total > 100:
        self.add_error(None,
            "항목의 총 수는 "
            "100 이하입니다.")
```

이것은 `add_error`에 첫 번째 인수로 `None`을 전달하여 비필드 오류를 추가합니다.

참고

Chapter07의 `/form_project/form_example/forms.py` 폴더를 참조하십시오.
이 책의 GitHub 리포지토리에서 전체 코드를 확인하십시오.

`forms.py`를 저장하십시오.

- 리뷰 앱의 `views.py` 파일을 엽니다. `OrderForm`이 `ExampleForm` 대신 가져오도록 폼의 가져오기를 변경할 것입니다. 다음 가져오기 줄을 고려하십시오:

```
from .forms import ExampleForm
```

다음과 같이 변경하십시오:

```
from .forms import OrderForm
```

- `form_example` 뷰에서 `ExampleForm`을 사용하는 두 줄을 `OrderForm`을 사용하도록 변경하세요. 대신, 다음 코드 줄을 고려하세요:

```
form = ExampleForm(request.POST)
```

다음과 같이 변경하세요:

```
form = OrderForm(request.POST)
```

Similarly, consider the following line of code:

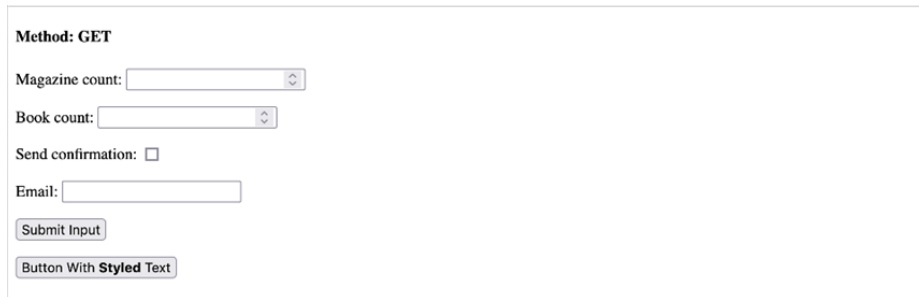
```
form = ExampleForm()
```

Change this as follows:

```
form = OrderForm()
```

The rest of the function can stay as-is.

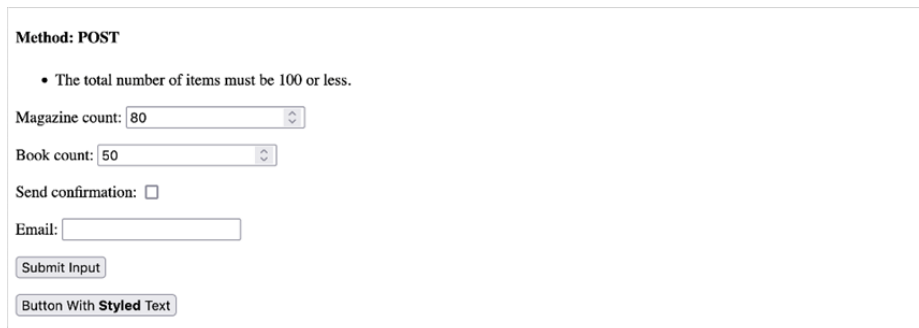
We don't have to make changes to the template. Start the Django dev server and navigate to `http://127.0.0.1:8000/form-example/` in your browser. You should see the form rendered similarly to what's shown in *Figure 7.10*:



The screenshot shows a web browser window displaying a form titled "Method: GET". The form contains the following elements: a "Magazine count:" label followed by a text input field with a value of 80; a "Book count:" label followed by a text input field with a value of 50; a "Send confirmation:" label followed by an unchecked checkbox; an "Email:" label followed by an empty text input field; a "Submit Input" button; and a "Button With **Styled** Text" button.

Figure 7.10 – OrderForm in the browser

11. Try submitting the form with a **Magazine count** value of 80 and a **Book count** value of 50. The browser will allow this, but since they sum to more than 100, an error will be triggered by the `clean` method in the form and displayed on the page:



The screenshot shows a web browser window displaying a form titled "Method: POST". At the top, there is a non-field error message: "• The total number of items must be 100 or less." Below this, the form contains the same elements as in Figure 7.10, but the "Magazine count:" input field now has a value of 80 and the "Book count:" input field has a value of 50. The "Send confirmation:" checkbox is still unchecked, and the "Email:" field is empty. The "Submit Input" and "Button With **Styled** Text" buttons are also present.

Figure 7.11 – A non-field error displayed on the form when the maximum number of allowed items is exceeded

유사하게, 다음 코드 줄을 고려하십시오:

```
form = ExampleForm()
```

다음과 같이 변경하십시오:

```
form = OrderForm()
```

함수의 나머지는 그대로 두어도 됩니다.

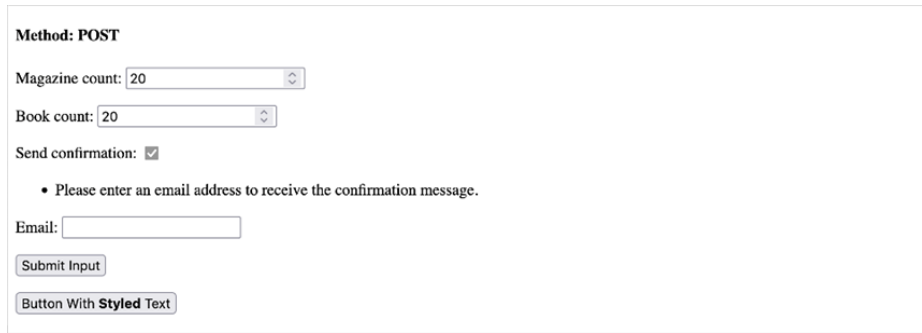
템플릿을 변경할 필요가 없습니다. Django 개발 서버를 시작하고 브라우저에서 <http://127.0.0.1:8000/form-example/>로 이동하십시오. 그러면 그림 7.10에 표시된 것과 유사하게 렌더링된 폼을 볼 수 있어야 합니다:

그림 7.10 - 브라우저의 OrderForm

11. idx_cd40cf9b 양식에 잡지 수치 80과 책 수치 50을 입력하여 제출해 보세요. 브라우저는 이를 허용하지만, 두 수치의 합이 100을 초과하므로 양식의 clean 메서드에 의해 오류가 발생하고 페이지에 표시됩니다:

그림 7.11 - 허용된 항목의 최대 수를 초과했을 때 양식에 표시된 비필드 오류

12. Try submitting the form with **Send confirmation** checked but the **Email** field blank. Then, fill in the **Email** text box, but uncheck **Send confirmation**. Either combination will give an error stating that both must be present. The error will differ based on which field is missing:



The screenshot shows a web form with the following elements:

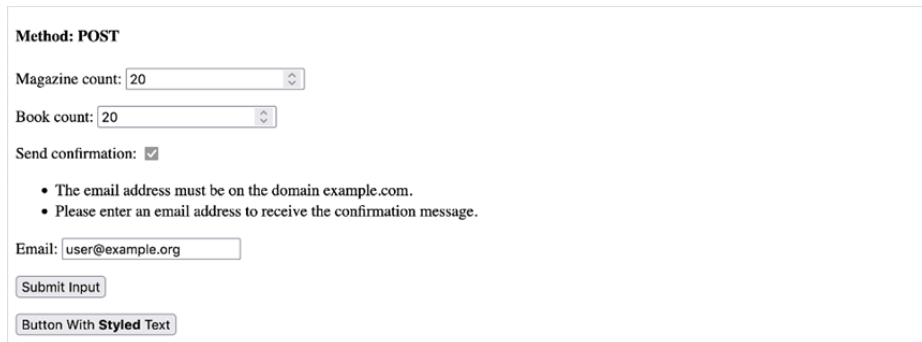
- Method: POST**
- Magazine count:** A dropdown menu with the value 20.
- Book count:** A dropdown menu with the value 20.
- Send confirmation:** A checkbox that is checked.
- Email:** An empty text input field.
- Submit Input** button.
- Button With **Styled** Text** button.

An error message is displayed below the **Send confirmation** checkbox:

- Please enter an email address to receive the confirmation message.

Figure 7.12 – Error message if no email address is present

13. Now, try submitting the form with **Send confirmation** checked, and an email address that is not on the `example.com` domain. You should receive a message that your email address must have the `example.com` domain. You should also receive a message that **Email** must be set – this is because the email does not end up in the `cleaned_data` dictionary. After all, it is not valid:



The screenshot shows the same web form as Figure 7.12, but with the **Email** field filled with `user@example.org`. The **Send confirmation** checkbox remains checked.

Two error messages are displayed below the **Send confirmation** checkbox:

- The email address must be on the domain `example.com`.
- Please enter an email address to receive the confirmation message.

Figure 7.13 – The error message that's shown when the email domain is not `example.com`

14. Finally, enter valid values for **Magazine count** and **Book count** (such as 20 and 20). Check **Send confirmation** and enter `UserName@Example.Com` as the email (make sure you match the letter case, including the mixed uppercase and lowercase characters):

12. 확인 전송이 체크된 상태에서 양식을 제출해 보되 이메일 필드는 비워 두십시오. 그런 다음 이메일 텍스트 상자를 채우되 확인 전송의 체크를 해제하십시오. 두 조합 idx_57742481 모두 두 필드가 모두 존재해야 한다는 오류가 발생합니다. 누락된 필드에 따라 오류가 다르게 나타납니다:

The screenshot shows a web form with the following elements:

- Method: POST**
- Magazine count:** 20 (dropdown menu)
- Book count:** 20 (dropdown menu)
- Send confirmation:** ☒
- Error message:**
 - Please enter an email address to receive the confirmation message.
- Email:** (empty text input field)
- Submit Input** (button)
- Button With Styled Text** (button)

그림 7.12 - 이메일 주소가 없을 경우의 오류 메시지

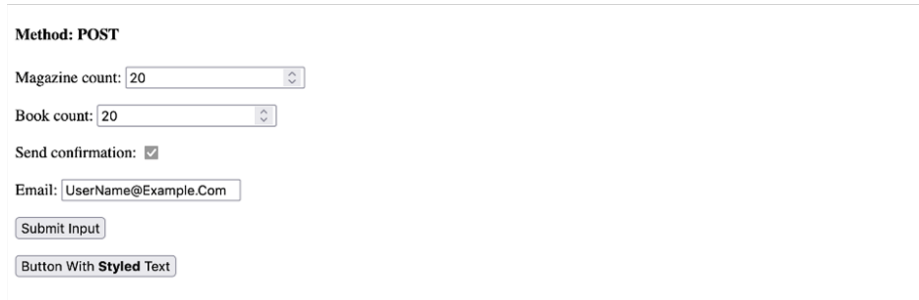
13. 이제 확인 전송이 체크된 상태에서 example.com 도메인에 없는 이메일 주소로 양식을 제출해 보십시오. 이메일 주소는 example.com 도메인을 가져야 한다는 메시지를 받아야 합니다. 또한 이메일이 설정되어야 한다는 메시지를 받아야 합니다. 이는 이메일이 cleaned_data 사전으로 들어가지 않기 때문입니다. 결국 유효하지 않기 때문입니다:

The screenshot shows the same web form as in Figure 7.12, but with the email address 'user@example.org' entered in the Email field. The error messages are:

- The email address must be on the domain example.com.
- Please enter an email address to receive the confirmation message.

그림 7.13 - 이메일 도메인이 example.com 이 아닐 때 표시되는 오류 메시지

14. 마지막으로 잡지 수와 책 수에 대한 유효한 값을 입력하십시오(예: 20 및 20). 확인 전송을 체크하고 UserName@Example.Com을 idx_769242e7 이메일로 입력하십시오(대소문자를 포함하여 혼합된 대문자와 소문자를 일치시키는 것을 잊지 마십시오):



Method: POST

Magazine count: 20

Book count: 20

Send confirmation: ☒

Email: UserName@Example.Com

Submit Input

Button With **Styled Text**

Figure 7.14 – The form after being submitted with valid values

15. Switch to PyCharm and look in the debug console. You'll see that the email is converted into lowercase when it is printed by our debug code:

```
magazine_count: (<class 'int'>) 20
book_count: (<class 'int'>) 20
send_confirmation: (<class 'bool'>) True
email: (<class 'str'>) username@example.com
```

Figure 7.15 – Email in lowercase, as well as other fields

This is our `clean_email` method in action – even though we entered data in both uppercase and lowercase, it has been converted into all lowercase.

In this exercise, we created a new `OrderForm` that implemented form and field `clean` methods. We used a custom validator to ensure that the **Email** field met our specific validation rules – only a specific domain was allowed. We used a custom field cleaning method (`clean_email`) to convert the email address into lowercase. We then implemented a `clean` method to validate the forms that were dependent on each other. In this method, we added both field and non-field errors.

So far, we have learned about multi-field validations and how to implement them to make sure that we can validate the form in one go rather than highlighting errors one by one.

In the next section, we will cover how to add placeholders and initial values to the form.

그림 7.14 - 유효한 값으로 제출된 후의 폼

15. PyCharm으로 전환하고 디버그 콘솔을 확인하세요. 디버그 코드에 의해 출력될 때 이메일이 소문자로 변환되는 것을 볼 수 있습니다:

```
magazine_count: (<class 'int'>) 20
book_count: (<class 'int'>) 20
send_confirmation: (<class 'bool'>) True
email: (<class 'str'>) username@example.com
```

그림 7.15 - 소문자의 이메일과 다른 필드들

이것이 우리의 clean_email 메서드가 작동하는 모습입니다 – 대문자와 소문자로 데이터를 입력했음에도 불구하고 모두 소문자로 변환되었습니다.

이번 연습에서는 폼 및 필드 클린 메서드를 구현한 새로운 OrderForm을 만들었습니다. 이메일 필드가 특정 유효성 검사 규칙을 충족하도록 보장하기 위해 사용자 정의 검증기를 사용했습니다 – 특정 도메인 idx_6420b2c1만 허용되었습니다. 이메일 주소를 소문자로 변환하기 위해 사용자 정의 필드 클리닝 메서드(clean_email)를 사용했습니다. 그런 다음 서로 의존하는 폼을 검증하기 위해 클린 메서드를 구현했습니다. 이 메서드에서는 필드 오류와 비필드 오류를 모두 추가했습니다.

지금까지 우리는 다중 필드 유효성 검사에 대해 배우고 이를 구현하여 오류를 하나씩 강조하는 대신 한 번에 폼을 검증할 수 있도록 하는 방법을 배웠습니다.

다음 섹션에서는 폼에 플레이스홀더와 초기 값을 추가하는 방법을 다룰 것입니다.

Adding placeholders and initial values

There are two things our first manually-built form had that our current Django form still does not have – placeholders and initial values. Adding placeholders is simple; they are just added as attributes to the widget constructor for the form field. This is similar to what we've already seen for setting the type of `DateField` in our previous examples.

Here's an example:

```
class ExampleForm(forms.Form):
    text_field = forms.CharField(
        widget=forms.TextInput(
            attrs={"placeholder": "Text Placeholder"})
    )
    password_field = forms.CharField(
        widget=forms.PasswordInput(
            attrs={"placeholder": "Password Placeholder"})
    )
    email_field = forms.EmailField(
        widget=forms.EmailInput(
            attrs={"placeholder": "Email Placeholder"})
    )
    text_area = forms.CharField(
        widget=forms.Textarea(
            attrs={"placeholder": "Text Area Placeholder"})
    )
```

Here is what the preceding form looks like when rendered in the browser:

자리 표시자 및 초기 값 추가

idx_c7052b8four 첫 번째 수동으로 작성된 양식에는 idx_c2eafcb2our 현재 Django 양식이 여전히 가지고 있지 않은 두 가지가 있습니다 idx_92702e4c – 자리 표시자와 초기 값. idx_dd5e6f66 자리 표시자를 추가하는 것은 간단합니다; 그것들은 양식 필드의 위젯 생성자에 속성으로 추가됩니다. 이것은 우리가 이전 예제에서 idx_f36ac50a DateField의 idx_73050c60type idx_e3293f6b을 설정하는 것과 유사합니다.

예시입니다:

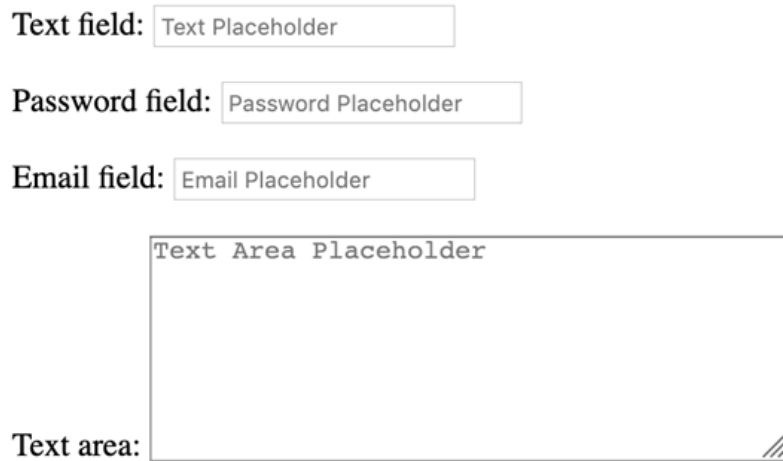
```
class ExampleForm(forms.Form):
    text_field = forms.CharField(
        widget=forms.TextInput(
            attrs={"placeholder": "텍스트 자리 표시자"})
    )

    password_field = forms.CharField(
        widget=forms.PasswordInput(
            attrs={"placeholder": "비밀번호 자리 표시자"})
    ),

    email_field = forms.EmailField(
        widget=forms.EmailInput(
            attrs={"placeholder": "이메일 자리 표시자"})
    ),

    text_area = forms.CharField(
        widget=forms.Textarea(
            attrs={"placeholder": "텍스트 영역 자리 표시자"})
    ),
```

브라우저에 렌더링될 때 이전 양식이 어떻게 보이는지입니다:



The image shows a Django form with four fields, each with a placeholder text:

- Text field:** A single-line text input with the placeholder "Text Placeholder".
- Password field:** A single-line password input with the placeholder "Password Placeholder".
- Email field:** A single-line email input with the placeholder "Email Placeholder".
- Text area:** A multi-line text area with the placeholder "Text Area Placeholder".

Figure 7.16 – Django form with placeholders

Of course, if we're manually setting `Widget` for each field, we need to know which `Widget` class to use. The ones that support placeholders are `TextInput`, `NumberInput`, `EmailInput`, `URLInput`, `PasswordInput`, and `Textarea`.

While we're examining the `Form` class itself, we'll look at the first of two ways of setting an initial value for a field. We can do this by using the `initial` argument on a `Field` constructor, like this:

```
text_field = forms.CharField(initial="Initial Value", ...)
```

The other method is to pass in a dictionary of data when instantiating the form in our view. The keys are the field names. The dictionary should have zero or more items (that is, an empty dictionary is valid). Any extra keys are ignored. This dictionary should be supplied as the `initial` argument in our view, as follows:

```
initial = {"text_field": "Text Value", "email_field": "user@example.com"}
form = ExampleForm(initial=initial)
```

Or, for a `POST` request, pass in `request.POST` as the first argument, as usual:

```
initial = {"text_field": "Text Value", "email_field": "user@example.com"}
form = ExampleForm(request.POST, initial=initial)
```

Text field:

Password field:

Email field:

Text area:

Text Area Placeholder

그림 7.16 - 플레이스홀더가 있는 Django 폼

물론, 각 필드에 대해 위젯을 수동으로 설정하는 경우, 어떤 위젯 클래스를 사용해야 하는지 알아야 합니다. 플레이스홀더를 지원하는 것들은 `TextInput`, `NumberInput`, `EmailInput`, `URLInput`, `PasswordInput` 및 `Textarea`입니다.

폼 클래스 자체를 살펴보는 동안, 필드의 초기 값을 설정하는 두 가지 방법 중 첫 번째 방법을 살펴보겠습니다. 필드 생성자에서 초기 인수를 사용하여 이렇게 할 수 있습니다:

```
text_field = forms.CharField(initial="초기 값", ...)
```

다른 방법은 뷰에서 폼을 인스턴스화할 때 데이터의 사전을 전달하는 것입니다. 키는 필드 이름입니다. 사전은 0개 이상의 항목을 가질 수 있으며(즉, 빈 사전도 유효합니다). 추가 키는 무시됩니다. 이 사전은 뷰에서 초기 인수로 제공되어야 하며, 다음과 같이 작성합니다:

```
initial = {"text_field": "텍스트 값", "email_field": "user@example.com"}
form = ExampleForm(initial=initial)
```

또는 POST 요청의 경우, 요청을 첫 번째 인수로 전달합니다, 평소와 같이:

```
initial = {"text_field": "텍스트 값", "email_field": "user@example.com"}
form = ExampleForm(request.POST, initial=initial)
```

Values in `request.POST` will override values in `initial`. This means that even if we have an initial value for a required field, if it's left blank when submitted, then it will not validate. The field will not fall back to the value in `initial`.

Whether you decide to set initial values in the `Form` class itself or in the view is up to you and depends on your use case. If you had a form that was used in multiple views but usually had the same value, it would be better to set the initial value in the form. Otherwise, it can be more flexible to use setting in the view.

In the next exercise, we will add placeholders and initial values to the `OrderForm` class from the previous exercise.

Exercise 7.02 – Placeholders and initial values

In this exercise, you will enhance the `OrderForm` class by adding placeholder text. You will simulate passing an initial email address to the form. It will be a hardcoded address, but once the user can log in, it could be an email address associated with their account – you will learn about sessions and authentication in *Chapter 9, Sessions and Authentication*.

Follow these steps:

1. In PyCharm, open the reviews app's `forms.py` file. You will add placeholders to the `magazine_count`, `book_count`, and `email` fields on `OrderForm`, which means also setting a widget.

To the `magazine_count` field, add a `NumberInput` widget with placeholder in the `attrs` dictionary. This placeholder value should be set to `Number of Magazines`. Write the following code:

```
magazine_count = forms.IntegerField(
    min_value=0,
    max_value=80,
    widget=forms.NumberInput(
        attrs={"placeholder": "Number of Magazines"}),
)
```

2. Add a placeholder to the `book_count` field in the same manner. The placeholder text should be `Number of Books`:

```
book_count = forms.IntegerField(
    min_value=0,
    max_value=50,
    widget=forms.NumberInput(
```

request.POST의 값은 초기 값들을 덮어씁니다. 이는 필수 필드에 대한 초기 idx_b8c8fc22 값이 있더라도, idx_94258d11 제출 시 비워두면 유효성 검사를 통과하지 못한다는 것을 의미합니다. 필드 idx_8725701b는 초기의 값 idx_da2b0021으로 되돌아가지 않습니다.

폼 클래스 자체에서 초기 값을 설정할지 또는 뷰에서 설정할지는 귀하에게 달려 있으며 사용 사례에 따라 다릅니다. 여러 뷰에서 사용되지만 보통 동일한 값을 가지는 폼이 있다면, 폼에서 초기 값을 설정하는 것이 더 좋습니다. 그렇지 않으면 뷰에서 설정을 사용하는 것이 더 유연할 수 있습니다.

다음 연습에서는 이전 연습의 OrderForm 클래스에 플레이스홀더와 초기 값을 추가할 것입니다.

연습 7.02 – 플레이스홀더와 초기 값

이번 연습에서는 idx_34540f36 OrderForm 클래스를 idx_5b8df06c 플레이스홀더 텍스트를 추가하여 향상시킬 것입니다. 초기 이메일 주소 idx_9bc29188를 폼에 전달하는 것을 시뮬레이션할 것입니다. 이는 하드코딩된 주소가 될 것이지만, 사용자가 로그인할 수 있게 되면 그들의 계정과 연결된 이메일 주소가 될 수 있습니다 – 9장, 세션 및 인증에서 세션과 인증에 대해 배울 것입니다.

다음 단계를 따르세요:

1. PyCharm에서 reviews 앱의 forms.py 파일을 엽니다. OrderForm의 magazine_count, book_count 및 email 필드에 플레이스홀더를 추가할 것이며, 이는 위젯을 설정하는 것을 의미합니다.

magazine_count 필드에 attrsdictionary의 플레이스홀더가 있는 NumberInput 위젯을 추가합니다. 이 플레이스홀더 값은 '잡지 수'로 설정해야 합니다. 다음 코드를 작성하세요:

```
magazine_count = forms.IntegerField(
    min_value=0,
    max_value=80,
    widget=forms.NumberInput(
        attrs={"placeholder": "잡지 수"}),
    )
```

2. book_count 필드에 idx_8c398559placeholder를 같은 방식으로 추가하세요. idx_ae5ac2bcplaceholder 텍스트는 idx_ca3efa01be 책 수:

```
book_count = forms.IntegerField(
    min_value=0,
    max_value=50,
    widget=forms.NumberInput(
```

```
        attrs={"placeholder": "Number of Books"}),  
    )
```

3. The final change to `OrderForm` is to add a placeholder to the email field. This time, the widget is `EmailInput`. The placeholder text should be `Your company email address`:

```
email = forms.EmailField(  
    required=False,  
    validators=[validate_email_domain],  
    widget=forms.EmailInput(attrs={  
        "placeholder": "Your company email address"}),  
)
```

Note that the `clean_email` and `clean` methods should remain as they were in *Exercise 7.01 – Custom clean and validation methods*. Save the file.

4. Open the `form_project` app's `views.py` file. In the `form_example` view function, create a new dictionary variable called `initial` with one key, `email`, like this:

```
initial = {"email": "user@example.com"}
```

5. In the two places that you are instantiating `OrderForm`, also pass in the `initial` variable using the `initial` keyword argument. The first instance is as follows:

```
form = OrderForm(request.POST, initial=initial)
```

The second instance is as follows:

```
form = OrderForm(initial=initial)
```

The complete code for `views.py` can be found in the `Chapter07` folder in this book's GitHub repo.

Save the `views.py` file.

6. Start the Django dev server if it is not already running. Browse to `http://127.0.0.1:8000/form-example/` in your browser. You should see that your form now has placeholders and an initial value set:

```
        attrs={"placeholder": "책 수"}),
    )
```

3. OrderForm에 대한 마지막 변경 사항은 이메일 필드에 플레이스홀더를 추가하는 것입니다. 이번에는, 위젯은 EmailInput입니다. 플레이스홀더 텍스트는 귀사의 이메일 주소여야 합니다:

```
email = forms.EmailField(
    required=False,
    validators=[validate_email_domain],
    widget=forms.EmailInput(attrs={
        "placeholder": "귀사의 이메일 주소"}),
)
```

clean_email 및 clean 메서드는 Exercise7.01 – 사용자 정의 클린 및 검증 메서드에서와 같이 유지되어야 합니다. 파일을 저장하세요.

4. idx_3991370dform_project 앱의 views.py 파일을 엽니다. idx_95f86f3dform_example 뷰 idx_cd4a515efunction에서, 다음과 같이 email이라는 하나의 키를 가진 새로운 idx_d9469629dictionary 변수를 initial로 생성합니다:

```
initial = {"email": "user@example.com"}
```

5. OrderForm을 인스턴스화하는 두 곳에서 initial 키워드 인수를 사용하여 initial 변수를 전달합니다. 첫 번째 인스턴스는 다음과 같습니다:

```
form = OrderForm(request.POST, initial=initial)
```

두 번째 예는 다음과 같습니다:

```
form = OrderForm(initial=initial)
```

views.py의 전체 코드는 이 책의 GitHub 저장소의 Chapter07 폴더에서 찾을 수 있습니다.

views.py 파일을 저장하세요.

6. Django 개발 서버가 이미 실행 중이지 않다면 시작하세요. 브라우저에서 <http://127.0.0.1:8000/form-example/>로 이동하세요. 이제 양식에 자리 표시자와 초기 값이 설정된 것을 볼 수 있어야 합니다:

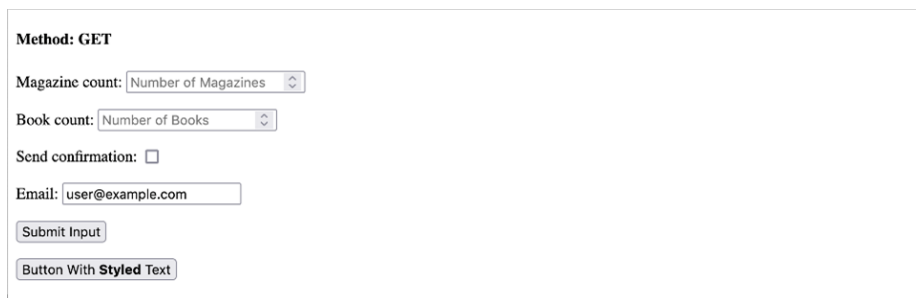


Figure 7.17 – Order form with initial values and placeholders

In this exercise, we added placeholders to form fields. This was done by setting a form widget when defining the form field on the form class and setting a placeholder value in the `attrs` dictionary. We also set an initial value for the form using a dictionary and passed it to the form instance using the `initial` keyword argument.

In the next section, we will talk about how to work with Django models using data from forms, and how `ModelForm` makes this easier.

Creating or editing Django models

We've seen how to define a form, and in *Chapter 2, Models and Migrations*, you learned how to create Django model instances. By using these things together, you can build a view that displays a form and also saves a model instance to the database. This gives you an easy way to save data without having to write a lot of boilerplate code or create custom forms. In *Bookr*, we will use this method to allow users to add reviews without requiring access to the Django admin site. Without using `ModelForm`, you can do something like this:

- You can create a form based on an existing model, for example, `Publisher`. The form would be called `PublisherForm`.
- You can manually define the fields on `PublisherForm`, using the same rules defined on the `Publisher` model, as shown here:

```
class PublisherForm(forms.Form):
    name = forms.CharField(max_length=50)
    website = forms.URLField()
    ...
```

그림 7.17 - 초기 값과 자리 표시자가 있는 주문 양식

이 연습에서는 양식 필드에 자리 표시자를 추가했습니다. 이는 양식 필드를 정의할 때 form widget을 설정하고 attrsdictionary에서 자리 표시자 값을 설정하여 이루어졌습니다. 또한 사전을 사용하여 양식의 초기 값을 설정하고 이를 initial 키워드 인수를 사용하여 forminstance에 전달했습니다.

다음 섹션에서는 양식의 데이터를 사용하여 Django 모델로 작업하는 방법과 ModelForm이 이를 어떻게 쉽게 만드는데 대해 이야기할 것입니다.

Django 모델 생성 또는 편집

우리는 양식을 정의하는 방법을 보았고, 2장 모델 및 마이그레이션에서 Django 모델 인스턴스를 생성하는 방법을 배웠습니다. 이러한 것들을 함께 사용하여 양식을 표시하고 데이터베이스에 모델 인스턴스를 저장하는 뷰를 구축할 수 있습니다. 이렇게 하면 많은 보일러플레이트 코드를 작성하거나 사용자 정의 양식을 만들 필요 없이 데이터를 쉽게 저장할 수 있습니다. Bookr에서는 이 방법을 사용하여 사용자가 Django 관리 사이트에 접근하지 않고도 리뷰를 추가할 수 있도록 할 것입니다. ModelForm을 사용하지 않고도 다음과 같은 작업을 수행할 수 있습니다:

- 기존 모델을 기반으로 양식을 생성할 수 있습니다. 예를 들어, Publisher. 양식은 PublisherForm이라고 불릴 것입니다.
- PublisherForm에서 Publisher 모델에 정의된 동일한 규칙을 사용하여 필드를 수동으로 정의할 수 있습니다. 다음과 같이 표시됩니다:

```
class PublisherForm(forms.Form):
    name = forms.CharField(max_length=50)
    website = forms.URLField()
    ...
```

- In the view, the initial values would be retrieved from the model queried from the database, and then passed to the form using the initial argument. If you were creating a new instance, the initial value would be blank – something like this:

```
if create:
    initial = {}
else:
    publisher = Publisher.objects.get(pk=pk)
    initial = {"name": publisher.name,
              "website": publisher.website, ...}

form = PublisherForm(initial=initial)
```

- Then, in the POST flow of the view, you can either create or update the model based on `cleaned_data`:

```
form = PublisherForm(request.POST, initial=initial)
if create:
    publisher = Publisher()
else:
    publisher = Publisher.objects.get(pk=pk)

publisher.name = form.cleaned_data['name']
publisher.website = forms.cleaned_data['website']
...
publisher.save()
```

This is a lot of work, and we have to consider how much duplicated logic we have. For example, we're defining the length of the name in the name form field. If we made a mistake here, we could allow a longer name in the field than the model allows. We also have to remember to set all the fields in the initial dictionary, as well as set the values on the new or updated model with `cleaned_data` from the form. There are many opportunities to make mistakes here, as well as to remember to add or remove field setting data for each of these steps if the model changes. All this code would have to be duplicated for each Django model you work with as well, exacerbating the duplication problem.

- 뷰에서 초기 값은 데이터베이스에서 쿼리된 모델에서 검색된 후 초기 인수를 사용하여 폼에 전달됩니다. 새 인스턴스를 생성하는 경우 초기 값은 비어 있습니다 – 다음과 같은 형태입니다:

```
if create:
    initial = {}
else:
    publisher = Publisher.objects.get(pk=pk)
    initial = {"name": publisher.name,
              "website": publisher.website, ...}

form = PublisherForm(initial=initial)
```

- 그런 다음, 뷰의 POST 흐름에서 모델을 생성하거나 업데이트할 수 있습니다. cleaned_data:

```
form = PublisherForm(request.POST, initial=initial)
if create:
    publisher = Publisher()
else:
    publisher = Publisher.objects.get(pk=pk)

publisher.name = form.cleaned_data['name']
publisher.website = forms.cleaned_data['website']
...
publisher.save()
```

이것은 많은 idx_f16e8cd4작업이며, 우리는 idx_96ee27e에서 얼마나 많은 중복 로직이 있는지 고려해야 합니다. 예를 들어, 우리는 이름 양식 필드에서 이름의 길이를 정의하고 있습니다. 여기서 실수를 하면 모델이 허용하는 것보다 필드에 더 긴 이름을 허용할 수 있습니다. 또한 초기 사전에서 모든 필드를 설정하고, 양식의 cleaned_data로 새 모델이나 업데이트된 모델의 값을 설정하는 것을 기억해야 합니다. 여기에는 실수를 할 수 있는 많은 기회가 있으며, 모델이 변경될 경우 이러한 각 단계에 대한 필드 설정 데이터를 추가하거나 제거하는 것을 기억해야 합니다. 이 모든 코드는 작업하는 각 Django 모델에 대해 중복되어야 하며, 중복 문제를 악화시킵니다.

The ModelForm class

Luckily, Django provides a method of building `Model` instances from forms much more simply, with the `ModelForm` class. `ModelForm` is a form that is built automatically from a particular model. It will inherit the validation rules from the model (such as whether fields are required, the maximum length of `CharField` instances, and so on). It provides an extra `__init__` argument (called `instance`) for automatically populating the initial values from an existing model. It also adds a `save` method to automatically persist the form data to the database. All you need to do to set up `ModelForm` is specify its model and what fields should be used: this is done on the class `Meta` attribute of the form class. Let's see how to build a form from `Publisher`.

Inside the file that contains the form (for example, the `forms.py` file we have been working with), the only change is that the model must be imported:

```
from .models import Publisher
```

Then, the `Form` class can be defined. The class requires a class `Meta` attribute, which, in turn, must define a `model` attribute and either the `fields` or `excludes` attributes:

```
class PublisherForm(forms.ModelForm):  
    class Meta:  
        model = Publisher  
        fields = ("name", "website", "email")
```

`fields` is a list or tuple of the fields to include in the form. When manually setting the list of fields, if you add extra fields to the model, you must also add their name here to have them displayed on the form.

You can also use the special `__all__` value instead of a list or tuple to automatically include all the fields, like this:

```
class PublisherForm(forms.ModelForm):  
    class Meta:  
        model = Publisher  
        fields = "__all__"
```

If the `model` field has its `editable` attribute set to `False`, then it will not be automatically included.

ModelForm 클래스

다행히도, Django `idx_c0b7df87`는 `ModelForm` 클래스를 사용하여 `idx_a49a0955` 폼에서 `Model` 인스턴스를 훨씬 더 간단하게 구축하는 방법을 제공합니다. `ModelForm`은 특정 모델에서 자동으로 생성되는 폼입니다. 이 폼은 모델에서 유효성 검사 규칙(예: 필드가 필수인지, `CharField` 인스턴스의 최대 길이 등)을 상속받습니다. 또한 기존 모델에서 초기 값을 자동으로 채우기 위한 추가 `__init__` 인수(인스턴스라고 함)를 제공합니다. 또한 폼 데이터를 데이터베이스에 자동으로 저장하기 위한 `save` 메서드를 추가합니다. `ModelForm`을 설정하기 위해 필요한 것은 모델과 사용할 필드를 지정하는 것입니다: 이는 폼 클래스의 클래스 `Meta` 속성에서 수행됩니다. 출판사로부터 폼을 구축하는 방법을 살펴보겠습니다.

폼이 포함된 파일(예: 우리가 작업하고 있는 `forms.py` 파일) 내에서 유일한 변경 사항은 모델을 가져와야 한다는 것입니다:

```
from .models import Publisher
```

그런 다음, `Form` 클래스를 정의할 수 있습니다. 이 클래스는 클래스 `Meta` 속성이 필요하며, 이 속성은 차례로 모델 속성과 필드 또는 제외 속성을 정의해야 합니다:

```
class PublisherForm(forms.ModelForm):
    class Meta:
        model = Publisher
        fields = ("name", "website", "email")
```

`fields`는 양식에 포함할 필드의 목록 또는 튜플입니다. 필드 목록을 수동으로 설정할 때, 모델에 추가 필드를 `idx_e3c238deadd` 추가하면, 양식에 표시되도록 여기에도 그 `idx_b90336c9name`을 추가해야 합니다.

모든 필드를 자동으로 포함하려면 목록이나 튜플 대신 특별한 `__all__` 값을 사용할 수 있습니다. 다음과 같이:

```
class PublisherForm(forms.ModelForm):
    class Meta:
        model = Publisher
        fields = "__all__"
```

모델 필드의 `editable` 속성이 `False`로 설정되어 있으면, 자동으로 포함되지 않습니다.

On the contrary, the `exclude` attribute sets the fields to not display in the form. Any fields added to the model will automatically be added to the form. We could define the preceding form using `exclude` with any empty tuple, as we want all the fields. The code is like this:

```
class PublisherForm(forms.ModelForm):  
    class Meta:  
        model = Publisher  
        exclude = ()
```

This saves you some work because you don't need to add a field to both the model and the fields list; however, it is not as safe since you might automatically expose fields to the end user that you don't want to. For example, if you had a `User` model with `UserForm`, you might add an `is_admin` field to the `User` model to give admin users extra privileges. If this field were not in `exclude`, it would be displayed to the user. A user would then be able to make themselves an administrator, which is something you probably wouldn't want.

Whichever of these three approaches to choosing the forms to display we decide to use, in our case, they will display the same in the browser. This is because we are choosing to display *all* the fields. They all look like this when rendered in the browser:

Name: The name of the Publisher.

Website: The Publisher's website.

Email: The Publisher's email address.

Figure 7.18 – *PublisherForm*

Note that `help_text` from the `Publisher` model is automatically rendered as well.

Usage in a view is similar to the other forms we have seen. Also, as mentioned, there is an extra argument that can be provided, called `instance`. This can be set to `None`, which will render an empty form.

Assuming, in your view function, you have some method of determining whether you are creating or editing a model instance (we will discuss how to do this later), this will determine a

반대로, `exclude` 속성은 양식에 표시되지 않을 필드를 설정합니다. 모델에 추가된 모든 필드는 자동으로 양식에 추가됩니다. 모든 필드를 원하므로 빈 튜플과 함께 `exclude`를 사용하여 이전 양식을 정의할 수 있습니다. 코드는 다음과 같습니다:

```
class PublisherForm(forms.ModelForm):
    class Meta:
        model = Publisher
        exclude = ()
```

이것은 모델과 필드 목록 모두에 필드를 추가할 필요가 없기 때문에 작업을 줄여줍니다. 그러나 사용자가 원하지 않는 필드를 자동으로 노출할 수 있기 때문에 안전하지 않습니다. 예를 들어, `UserForm`이 있는 `User` 모델이 있다면, 관리 사용자에게 추가 권한을 부여하기 위해 `User` 모델에 `is_admin` 필드를 추가할 수 있습니다. 이 필드가 `exclude`에 없으면 사용자에게 표시됩니다. 그러면 사용자가 자신을 관리자로 만들 수 있게 되며, 이는 아마 원하지 않을 것입니다.

이 세 가지 접근 방식 중 어떤 것을 사용하든, 우리의 경우에는 브라우저에서 동일하게 표시됩니다. 이는 모든 필드를 표시하기로 선택했기 때문입니다. 브라우저에서 렌더링 될 때 모두 다음과 같이 보입니다:

Name: The name of the Publisher.

Website: The Publisher's website.

Email: The Publisher's email address.

그림 7.18 - *PublisherForm*

퍼블리셔 모델의 `help_text`가 자동으로 렌더링된다는 점에 유의하세요.

뷰에서의 사용은 우리가 본 다른 폼들과 유사합니다. 또한 언급했듯이, `instance`라는 추가 인수를 제공할 수 있습니다. 이는 `None`으로 설정할 수 있으며, 그러면 빈 폼이 렌더링됩니다.

당신의 뷰 함수에서 모델 인스턴스를 생성 중인지 편집 중인지 판단하는 방법이 있다고 가정하면(이 방법에 대해서는 나중에 논의할 것입니다), 이는 다음을 결정할 것입니다.

variable called `is_create` (True if creating an instance, or False if editing an existing one). Your view function to create the form could then be written like this:

```
if is_create:
    instance = None
else:
    instance = get_object_or_404(Publisher, pk=pk)

if request.method == "POST":
    form = PublisherForm(request.POST, instance=instance)
    if form.is_valid():
        # we'll cover this branch soon
    else:
        form = PublisherForm(instance=instance)
```

As you can see, in either branch, the instance is passed to the `PublisherForm` constructor, although it is `None` if we're in create mode.

If the form is valid, we can save the model instance. This can be done simply by calling the `save` method on the form. This will automatically create the instance, or simply save changes to the old one:

```
if form.is_valid():
    form.save()
    return redirect(success_url)
```

The `save` method returns the model instance that was saved. It takes one optional argument, `commit`, which determines whether the changes should be written to the database. You can pass `False` instead, which allows you to make additional changes to the instance before manually saving the changes. This would be required to set attributes that have not been included in the form. As we mentioned, maybe you would set the `is_admin` flag to `False` on a `User` instance:

```
if form.is_valid():
    new_user = form.save(False)
    new_user.is_admin = False
    new_user.save()
    return redirect(success_url)
```

인스턴스를 생성하는 경우 True, 기존 인스턴스를 편집하는 경우 False인 `is_create` 라는 변수. 양식을 생성하기 위한 뷰 함수는 다음과 같이 작성할 수 있습니다:

```
if is_create:
    instance = None
else:
    instance = get_object_or_404(Publisher, pk=pk)

if request.method == "POST":
    form = PublisherForm(request.POST, instance=instance)
    if form.is_valid():
        # 이 분기를 곧 다룰 것입니다
    else:
        form = PublisherForm(instance=instance)
```

보시다시피, `idx_df89e3d0`의 어느 분기에서든 인스턴스가 `idx_b1e8c1e1`의 `PublisherForm` 생성자에 전달됩니다. 생성 모드일 경우 None입니다.

폼이 유효하면 모델 인스턴스를 저장할 수 있습니다. 이는 폼에서 `save_method`를 호출하여 간단히 수행할 수 있습니다. 이렇게 하면 인스턴스가 자동으로 생성되거나 기존 인스턴스에 대한 변경 사항이 저장됩니다:

```
if form.is_valid():
    form.save()
    return redirect(success_url)
```

`save` 메서드는 저장된 모델 인스턴스를 반환합니다. 이는 선택적 인수 `commit`을 받아들이며, 이는 변경 사항이 데이터베이스에 기록될지 여부를 결정합니다. 대신 `False`를 전달할 수 있으며, 이는 변경 사항을 수동으로 저장하기 전에 인스턴스에 추가 변경을 할 수 있게 해줍니다. 이는 폼에 포함되지 않은 속성을 설정하는 데 필요할 수 있습니다. 언급했듯이, 아마도 `User` 인스턴스의 `is_admin` 플래그를 `False`로 설정할 것입니다:

```
if form.is_valid():
    new_user = form.save(False)
    new_user.is_admin = False
    new_user.save()
    return redirect(success_url)
```

In *Activity 7.02 – Review creation UI*, at the end of this chapter, we will be using this feature as well.

If your model uses `ManyToMany` fields, and you call `form.save(False)`, you should also call `form.save_m2m()` to save any many-to-many relationships that have been set. It's not necessary to call this method if you call the `form.save` method with `commit` set to `True` (that is, the default).

Model forms can be customized by making changes to their Meta attributes. The `widgets` attribute can be set. It can contain a dictionary keyed on the field names, with widget classes or instances as the values. For example, this is how to set up `PublisherForm` to have placeholders:

```
class PublisherForm(forms.ModelForm):
    class Meta:
        model = Publisher
        fields = "__all__"
        widgets = {
            "name": forms.TextInput(attrs={
                "placeholder": "The publisher's name."
            })
        }
```

The values behave the same as setting the `kwargs` widget in the field definition; they can be a class or an instance. For example, to display `CharField` as a password input, the `PasswordInput` class can be used; it does not need to be instantiated:

```
widgets = {
    "password": forms.PasswordInput
}
```

Model forms can also be augmented with extra fields added in the same way as they are added to a normal form. For example, suppose we wanted to provide the option of sending a notification email after saving a `Publisher`. We can add an `email_on_save` field to `PublisherForm` like this:

```
class PublisherForm(forms.ModelForm):
    email_on_save = forms.BooleanField(required=False,
        help_text="Send notification email on save")

    class Meta:
```

활동 7.02 – 생성 UI 검토에서, 이 장의 끝에서 우리는 이 기능을 사용할 것입니다.

모델이 ManyToMany 필드를 사용하는 경우, `form.save(False)`를 호출하면 설정된 모든 다대다 관계를 저장하기 위해 `form.save_m2m()`도 호출해야 합니다. `commit`이 `True`로 설정된 `form.save` 메서드를 호출하는 경우 이 메서드를 호출할 필요는 없습니다(즉, 기본값입니다).

모델 양식은 메타 속성에 대한 변경을 통해 사용자 정의할 수 있습니다. 위젯 속성을 설정할 수 있습니다. 필드 이름을 키로 하는 사전에 포함할 수 있으며, 위젯 클래스나 인스턴스가 값으로 사용됩니다. 예를 들어, `PublisherForm`에 플레이스홀더를 설정하는 방법은 다음과 같습니다:

```
class PublisherForm(forms.ModelForm):
    class Meta:
        model = Publisher
        fields = "__all__"
        widgets = {
            "name": forms.TextInput(attrs={
                "placeholder": "출판사의 이름."
            })
        }
```

값은 필드 정의에서 `kwargs` 위젯을 설정하는 것과 동일하게 동작합니다. 클래스나 인스턴스가 될 수 있습니다. 예를 들어, `CharField`를 비밀번호 입력으로 표시하려면 `PasswordInput` 클래스를 사용할 수 있습니다. 인스턴스화할 필요는 없습니다:

```
widgets = {
    "password": forms.PasswordInput
}
```

모델 폼은 일반 `idx_7eff1090form`에 추가하는 것과 같은 방식으로 추가 필드를 추가하여 확장할 수 있습니다. 예를 들어, `Publisher`를 저장한 후 알림 이메일을 보내는 옵션을 제공하고 싶다고 가정해 보겠습니다. `PublisherForm`에 `email_on_save` 필드를 다음과 같이 추가할 수 있습니다:

```
class PublisherForm(forms.ModelForm):
    email_on_save = forms.BooleanField(required=False,
        help_text="저장 시 알림 이메일 전송")

    class Meta:
```

```
model = Publisher
fields = "__all__"
```

When rendered, the form looks like this:

Name: The name of the Publisher.

Website: The Publisher's website.

Email: The Publisher's email address.

Email on save: ☒ Send notification email on save

Figure 7.19 – PublisherForm with an additional field

Additional fields are placed after the Model fields. The extra fields are not handled automatically – they do not exist on the model, so Django won't attempt to save them on the model instance. Instead, you should handle how their values are saved by examining the `cleaned_data` values of the form, as you would with a standard form, for example (inside your view function):

```
if form.is_valid():
    if form.cleaned_data.get("email_on_save"):
        send_email()
        # assume this function is defined elsewhere
    form.save()
    # save the instance regardless of sending
    # the email or not
    return redirect(success_url)
```

In the following exercise, you will write a new view function in Bookr that will allow you to create and edit a Publisher.

```
model = Publisher
fields = "__all__"
```

렌더링되면, 폼은 다음과 같이 보입니다:

Name: The name of the Publisher.

Website: The Publisher's website.

Email: The Publisher's email address.

Email on save: ☒ Send notification email on save

그림 7.19 - 추가 필드가 있는 *PublisherForm*

추가 필드는 모델 필드 뒤에 배치됩니다. 추가 필드는 자동으로 처리되지 않으며 – 모델에 존재하지 않기 때문에 Django는 모델 인스턴스에 저장하려고 시도하지 않습니다. 대신, 표준 폼과 마찬가지로 폼의 `cleaned_data` 값을 검사하여 값이 어떻게 저장되는지 처리해야 합니다(예: 뷰 함수 내에서):

```
if form.is_valid():
    if form.cleaned_data.get("email_on_save"):
        send_email()
        # 이 함수는 다른 곳에 정의되어 있다고 가정합니다
    form.save()
    # 이메일을 보내든 보내지 않은 인스턴스를 저장합니다
    return redirect(success_url)
```

다음 `idx_8a2abb4fexercise`에서, Bookr에 새로운 뷰 함수를 작성하여 `idx_3d688694` 출판사를 생성하고 편집할 수 있도록 할 것입니다.

Exercise 7.03 – Creating and editing a publisher

In this exercise, we will return to Bookr. We want to add the ability to create and edit a Publisher without using the Django admin. To do this, we'll add a `ModelForm` for the Publisher model. It will be used in a new view function. This view function will take an optional argument, `pk`, which will either be the ID of the Publisher object being edited or `None` to create a new Publisher. We will add two new URL maps to facilitate this. When this is complete, we will be able to see and update any publisher using their ID. For example, information for *Publisher 1* will be viewable/editable at the `/publishers/1` URL path:

1. In PyCharm, open the reviews app's `forms.py` file. After the forms import, import the Publisher model:

```
from .models import Publisher
```

2. Create a `PublisherForm` class, inheriting from `forms.ModelForm`:

```
class PublisherForm(forms.ModelForm):
```

3. Define the class `Meta` attribute on `PublisherForm`. The attributes that `Meta` requires are the model (`Publisher`) and fields (`"__all__"`):

```
class PublisherForm(forms.ModelForm):
    class Meta:
        model = Publisher
        fields = "__all__"
```

Save `forms.py`.

Note

The full file can be found in the `Chapter07` folder in this book's GitHub repo.

4. Open the reviews app's `views.py` file. At the top of the file, import `PublisherForm`:

```
from .forms import PublisherForm, SearchForm
```

연습 7.03 – 출판사 생성 및 편집

이 연습에서는 Bookr로 돌아갑니다. Django 관리자를 사용하지 않고 출판사를 생성하고 편집할 수 있는 기능을 추가하고자 합니다. 이를 위해 Publisher 모델에 대한 ModelForm을 추가할 것입니다. 이 ModelForm은 새로운 뷰 함수에서 사용될 것입니다. 이 뷰 함수는 선택적 인수인 pk를 받아들이며, 이는 편집 중인 Publisher 객체의 ID이거나 새로운 Publisher를 생성하기 위한 None일 것입니다. 이를 용이하게 하기 위해 두 개의 새로운 URL 맵을 추가할 것입니다. 이 작업이 완료되면 ID를 사용하여 모든 출판사를 보고 업데이트할 수 있게 됩니다. 예를 들어, 출판사 1에 대한 정보는 /publishers/1 URL 경로에서 볼 수 있고 편집할 수 있습니다:

1. PyCharm에서 reviews 앱의 forms.py 파일을 엽니다. forms 가져오기 후, Publisher 모델을 가져옵니다:

```
from .models import Publisher
```

2. PublisherForm 클래스를 생성하고 forms.ModelForm에서 상속받습니다:

```
class PublisherForm(forms.ModelForm):
```

3. PublisherForm에서 클래스 Meta 속성을 정의합니다. Meta가 요구하는 속성은 모델(Publisher)과 필드("__all__")입니다:

```
class PublisherForm(forms.ModelForm):
    class Meta:
        model = Publisher
        fields = "__all__"
```

forms.py를 저장합니다.

참고

전체 파일은 이 책의 GitHub 저장소의 Chapter07 폴더에서 찾을 수 있습니다.

4. 리뷰 앱의 views.py 파일을 엽니다. 파일의 맨 위에 PublisherForm을 가져옵니다:

```
from .forms import PublisherForm, SearchForm
```

5. Make sure you import the `get_object_or_404` and `redirect` functions from `django.shortcuts`, if you aren't already:

```
from django.shortcuts import (
    render, get_object_or_404, redirect)
```

6. Also, make sure you're importing the `Publisher` model if you aren't already. You may already be importing this and other models:

```
from .models import Book, Contributor, Publisher
```

7. The final import you will need is the `messages` module. This will allow us to register a message letting the user know that a `Publisher` object was edited or created:

```
from django.contrib import messages
```

Once again, add this import if you do not already have it.

8. Create a new view function called `publisher_edit`. It takes two arguments: `request` and `pk` (the ID of the `Publisher` object to edit). This is optional, and if it is `None`, then a `Publisher` object will be created instead:

```
def publisher_edit(request, pk=None):
```

9. Inside the view function, we need to try to load the existing `Publisher` instance if `pk` is not `None`. Otherwise, `publisher` should be `None`:

```
def publisher_edit(request, pk=None):
    if pk is not None:
        publisher = get_object_or_404(Publisher, pk=pk)
    else:
        publisher = None
```

10. After getting a `Publisher` instance or `None`, complete the branch for a POST request. Instantiate the form in the same way as you saw earlier in this chapter, but now, make sure that it takes `instance` as a kwarg. Then, if the form is valid, save it using the `form.save()` method. This method will return the updated `Publisher` instance, which is stored in the `updated_publisher` variable. Then, register a different success message depending on whether the `Publisher` was created or updated. Finally, redirect back to

5. `django.shortcuts`에서 `get_object_or_404` 및 `redirect` 함수를 이미 가져오지 않았다면 가져오도록 하세요:

```
from django.shortcuts import (
    render, get_object_or_404, redirect)
```

6. 또한, 이미 가져오지 않았다면 `Publisher` 모델을 가져오고 있는지 확인하세요. 당신은 이미 이와 다른 모델을 가져오고 있을 수 있습니다:

```
from .models import Book, Contributor, Publisher
```

7. 필요한 마지막 `idx_b614076fimport`는 `messages` 모듈입니다. 이것을 통해 우리는 등록할 수 있습니다. 퍼블리셔 객체가 수정되었거나 생성되었음을 사용자에게 알리는 메시지:

```
from django.contrib import messages
```

다시 한 번, 이미 추가하지 않았다면 이 `import`를 추가하세요.

8. `publisher_edit`라는 새로운 뷰 함수를 만드세요. 이 함수는 두 개의 인수, 즉 `request`와 `pk`(수정할 `Publisher` 객체의 ID)를 받습니다. 이 인수는 선택 사항이며, 만약 `None`이라면 대신 `Publisher` 객체가 생성됩니다.

```
def publisher_edit(request, pk=None):
```

9. 뷰 함수 내부에서 `pk`가 있으면 기존 `Publisher` 인스턴스를 로드하려고 시도해야 합니다. `None`이 아닙니다. 그렇지 않으면, `publisher`는 `None`이어야 합니다:

```
def publisher_edit(request, pk=None):
    pk가 None이 아닐 경우:
        publisher = get_object_or_404(Publisher, pk=pk)
    그렇지 않으면:
        publisher = None
```

10. `Publisher` 인스턴스 또는 `None`을 가져온 후, `POST` 요청을 위한 분기를 완료합니다. 이 장에서 `idx_0988c20ee`와 같이 양식을 인스턴스화하지만, 이제는 인스턴스를 `kwargs`로 받도록 해야 합니다. 그런 다음, 양식이 유효하면 `theform.save()` 메서드를 사용하여 저장합니다. 이 `idx_093646af` 메서드는 업데이트된 `Publisher` 인스턴스를 반환하며, 이는 `updated_publisher` 변수에 저장됩니다. 그런 다음, `Publisher`가 생성되었는지 업데이트되었는지에 따라 다른 성공 메시지를 등록합니다. 마지막으로, 다시 리디렉션합니다.

this `publisher_edit` view, since `updated_publisher` will always have an ID at this point:

```
def publisher_edit(request, pk=None):
    ...
    if request.method == "POST":
        form = PublisherForm(request.POST, instance=publisher)
        if form.is_valid():
            updated_publisher = form.save()
            if publisher is None:
                messages.success(request,
                                f'Publisher "{updated_publisher}" 'f'was created.')
            else:
                messages.success(request, "Publisher
                                f'Publisher "{updated_publisher}" 'f'was updated.')
        return redirect("publisher_edit",
                        updated_publisher.pk)
```

If the form is not valid, the execution will fall through and just return the render function call with the invalid form (this will be implemented in *Step 12*). The redirect uses a named URL map, which will be added later in this exercise.

11. Next, fill in the non-POST branch of the code. In this case, just instantiate the form with your instance:

```
def publisher_edit(request, pk=None):
    ...
    if request.method == "POST":
        ...
    else:
        form = PublisherForm(instance=publisher)
```

12. In *Step 14*, you'll create a `form-example.html` file to render, but we can add the call to the render function now. Render it in the view with the render function by passing in the HTTP method and form as the context:

```
def publisher_edit(request, pk=None):
    ...
    return render(request, "form-example.html",
                  {"method": request.method, "form": form})
```

이 `publisher_edit` 뷰에서는 `updated_publisher`가 항상 이 시점에서 ID를 가집니다:

```
def publisher_edit(request, pk=None):
    ...
    if request.method == "POST":
        form = PublisherForm(request.POST, instance=publisher)
        if form.is_valid():
            updated_publisher = form.save()
            if publisher is None:
                messages.success(request,
                                f'발행자 "{updated_publisher}" 'f'가 생성되었습니다.')
            else:
                messages.success(request, "발행자 "
                                f'발행자 "{updated_publisher}" 'f'가 업데이트되었습니다.')
            return redirect("publisher_edit",
                           updated_publisher.pk)
```

양식이 유효하지 않으면 실행이 넘어가고 유효하지 않은 양식으로 `renderfunction idx_5f71280ccall`을 반환합니다(이는 Step 12에서 `idx_f665f60c` 구현됩니다). 리디렉션은 나중에 이 연습에서 추가될 이름이 지정된 `idx_82b9e469` URL 맵을 사용합니다.

11. 다음으로, 코드의 비-POST 분기를 채워십시오. 이 경우, 인스턴스로 양식을 인스턴스화하기만 하면 됩니다:

```
def publisher_edit(request, pk=None):
    ...
    if request.method == "POST":
        ...
    다른:
        form = PublisherForm(instance=publisher)
```

12. 14단계에서는 `form-example.html` 파일을 생성하여 렌더링할 것이지만, 지금은 `render` 함수 호출을 추가할 수 있습니다. HTTP 메서드와 `form`을 컨텍스트로 전달하여 `render` 함수로 뷰에서 렌더링하세요:

```
def publisher_edit(request, pk=None):
    ...
    return render(request, "form-example.html",
                  {"method": request.method, "form": form})
```

Save this file. You can refer to this in the Chapter07 folder in this book's GitHub repo.

13. Open `urls.py` in the reviews directory. Add two new URL maps; they will both go to the `publisher_edit` view. One will capture the ID of `Publisher`, which we will want to edit and pass into the view as the `pk` argument. The other will use `new` instead, and will not pass the `pk` argument, which will indicate that we want to create a new `Publisher`. To your `urlpatterns` variable, add the `"publishers/<int:pk>/"` path mapping to the `reviews.views.publisher_edit` view with the name `"publisher_edit"`.

Also, add the `"publishers/new/"` path mapping to the `reviews.views.publisher_edit` view with the name `"publisher_create"`:

```
urlpatterns = [
    ...
    path("publishers/<int:pk>",
         views.publisher_edit, name="publisher_edit"),
    path("publishers/new/",
         views.publisher_edit, name="publisher_create")
]
```

Since the second mapping does not capture anything, whichever `pk` is passed to the `publisher_detail` view function is `None`.

Save the `urls.py` file. The completed version for reference is at the Chapter07 folder in this book's GitHub repo.

14. Create a `form-example.html` file inside the reviews app's templates directory by using the **New HTML File** function in PyCharm. Since this is a standalone template (it does not extend any other templates), we need to render the messages inside it. Add this code just after the opening `<body>` tag to iterate through all the messages and display them:

```
{% for message in messages %}
<p><em>{{ message.level_tag|title }}:</em>
{{ message }}</p>
{% endfor %}
```

This will loop over the messages we have added and display the tag (in our case, `Success`) and then the message.

이 파일을 저장하세요. 이 파일은 이 책의 GitHub 저장소의 Chapter07 폴더에서 참조할 수 있습니다.

13. reviews 디렉토리에서 urls.py를 엽니다. 두 개의 새로운 URL 맵을 추가하세요. 두 개 모두 publisher_edit 뷰로 이동합니다. 하나는 편집할 Publisher의 ID를 캡처하고, 이를 idx_ca5a212c 뷰에 pk 인수로 전달할 것입니다. 다른 하나는 대신 new를 사용하고, idx_aca9c180은 pk 인수를 전달하지 않을 것입니다. 이는 새로운 Publisher를 생성하고 싶다는 것을 나타냅니다. urlpatterns 변수에 "publishers/<int:pk>/" 경로 매핑을 reviews.views.publisher_edit 뷰에 "publisher_edit"라는 이름으로 추가하세요.

또한, "publishers/new/" 경로 매핑을 reviews.views.publisher_edit 뷰에 "publisher_create"라는 이름으로 추가하세요:

```
urlpatterns = [
    ...
    path("publishers/<int:pk>",
         views.publisher_edit, name="publisher_edit"),
    path("publishers/new/",
         views.publisher_edit, name="publisher_create")
]
```

두 번째 매핑이 아무것도 캡처하지 않기 때문에, publisher_detail 뷰 함수에 전달되는 pk는 None입니다.

urls.py 파일을 저장하세요. 참조용으로 완성된 버전은 이 책의 GitHub 리포지토리의 Chapter07 폴더에 있습니다.

14. PyCharm의 New HTML File 기능을 사용하여 reviews 앱의 templates 디렉토리 안에 form-example.html 파일을 만드세요. 이것은 독립형 idx_03fd3cectemplate 이므로(다른 템플릿을 확장하지 않음), 우리는 그 안에서 메시지를 렌더링해야 합니다. 모든 메시지를 반복하고 표시하기 위해 <body> 태그 바로 뒤에 이 idx_9ceaececcode를 추가하세요:

```
{% for message in messages %}<p><em>
{{ message.level_tag|title }}:</em>
{{ message }}</p>
{% endfor %}
```

이것은 우리가 추가한 메시지를 반복하고 태그(우리 경우에는 Success)와 메시지를 표시합니다.

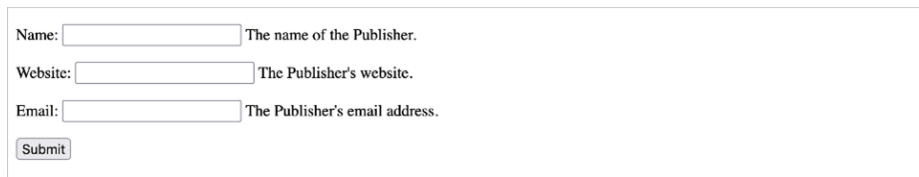
15. Then, add the normal form rendering and submission code:

```
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <p>
        <input type="submit" value="Submit">
    </p>
</form>
```

Save and close this file.

You can refer to the full version of this file at the Chapter07 folder in this book's GitHub repo.

16. Start the Django dev server, then navigate to `http://127.0.0.1:8000/publishers/new/`. You should see a blank PublisherForm being displayed:



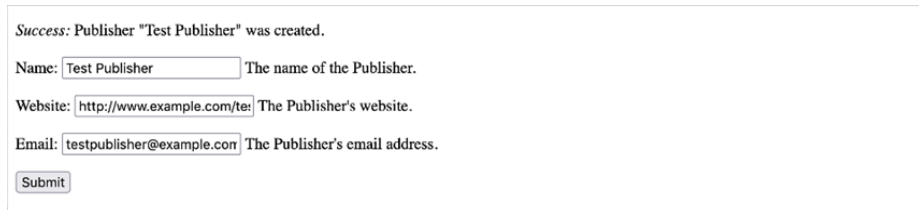
Name: The name of the Publisher.

Website: The Publisher's website.

Email: The Publisher's email address.

Figure 7.20 – Blank publisher form

17. The form has inherited the model's validation rules, so you cannot submit the form with too many characters for **Name** or with an invalid **Website** or **Email** value. Put in some valid information, then submit the form. After submission, you should see the **Success** message; the form will be populated with information that was saved to the database:



Success: Publisher "Test Publisher" was created.

Name: The name of the Publisher.

Website: The Publisher's website.

Email: The Publisher's email address.

Figure 7.21 – Form after submission

Notice that the URL has also been updated and now includes the ID of the publisher that was created. In this case, it is `http://127.0.0.1:8000/publishers/10/`; the ID in

15. 그런 다음, 일반 양식 렌더링 및 제출 코드를 추가하십시오:

```
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <p>
        <input type="submit" value="제출">
    </p>
</form>
```

idx_8e1184c7을 저장하고 이 파일을 닫습니다.

이 파일의 전체 버전은 이 책의 GitHub repo의 Chapter07 폴더에서 idx_6ccb6292를 참조할 수 있습니다.

16. Django 개발 서버를 시작한 후, <http://127.0.0.1:8000/publishers/new/>로 이동합니다. 빈 PublisherForm이 표시되는 것을 볼 수 있어야 합니다:

Name: The name of the Publisher.

Website: The Publisher's website.

Email: The Publisher's email address.

그림 7.20 - 빈 출판사 양식

17. 양식은 모델의 유효성 검사 규칙을 상속받았으므로, 이름에 너무 많은 문자를 입력하거나 잘못된 웹사이트 또는 이메일 값을 입력하여 양식을 제출할 수 없습니다. 유효한 정보를 idx_5b3864e5 입력한 후 양식을 제출하십시오. 제출 후, 성공 메시지를 idx_b6736233 볼 수 있어야 하며, 양식은 데이터베이스에 저장된 정보로 idx_5e32ba4abe idx_8af5c749 채워질 것입니다:

Success: Publisher "Test Publisher" was created.

Name: The name of the Publisher.

Website: The Publisher's website.

Email: The Publisher's email address.

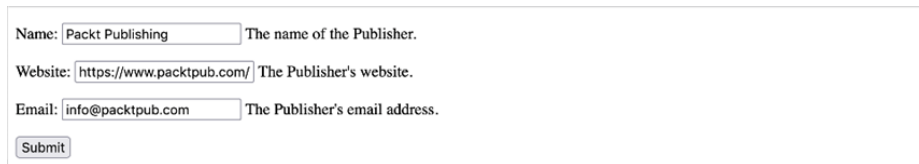
그림 7.21 - 제출 후 양식

URL도 업데이트되었으며 생성된 출판사의 ID가 포함되어 있음을 주목하십시오. 이 경우, <http://127.0.0.1:8000/publishers/10/>입니다; ID는

your setup will depend on how many Publisher instances were already in your database.

Notice that if you refresh the page, you will not receive a message confirming whether you want to resend the form data. This is because we redirected after saving, so it is safe to refresh this page as many times as you want, and no new Publisher instances will be created. If you had not redirected it, then every time the page was refreshed, a new Publisher instance would be created.

If you have other Publisher instances in your database, you can change the ID in the URL to edit other ones. Since the ID in this instance is 3, we can assume that *Publisher 1* and *Publisher 2* already exist and can substitute their IDs to see the existing data. Here is the view of the existing *Publisher 1* information (at `http://127.0.0.1:8000/publishers/1/`) – your information may be different:



A screenshot of a web form for editing a publisher. The form has three text input fields: 'Name' with the value 'Packt Publishing', 'Website' with the value 'https://www.packtpub.com/', and 'Email' with the value 'info@packtpub.com'. Each field is followed by a label: 'The name of the Publisher.', 'The Publisher's website.', and 'The Publisher's email address.' respectively. At the bottom left of the form is a 'Submit' button.

Figure 7.22 – Existing Publisher 1 information

Try making changes to the existing Publisher instance. Notice that after you save, the message is different – it is telling the user that the Publisher instance was *updated* rather than *created*:



A screenshot of the same web form as in Figure 7.22, but with a success message at the top: 'Success: Publisher "Packt Publishing" was updated.' The form fields and labels are identical to the previous figure, and the 'Submit' button is still present at the bottom left.

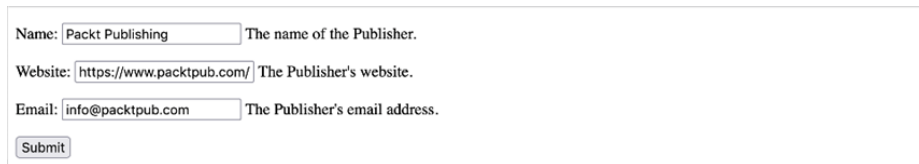
Figure 7.23 – Publisher after updating instead of creating

In this exercise, we implemented a `ModelForm` from a model (`PublisherForm` was created from `Publisher`) and saw how Django automatically generated the form fields with the correct validation rules. We then used the form's built-in save method to save changes to the Publisher instance (or automatically create it) inside the `publisher_edit` view. We mapped two URLs to the view. The first was for editing an existing Publisher and passing a `pk` to the view. The other did not pass this `pk` to the view, indicating that the Publisher instance should

당신의 설정은 데이터베이스에 이미 존재하는 Publisher 인스턴스의 수에 따라 달라질 것입니다.

페이지를 새로 고치면 양식 데이터를 다시 전송할 것인지 확인하는 메시지를 받지 않는다는 점에 유의하세요. 이는 저장 후 리디렉션을 했기 때문에 이 페이지를 원하는 만큼 새로 고쳐도 안전하며, 새로운 Publisher 인스턴스가 생성되지 않습니다. 만약 리디렉션을 하지 않았다면, 페이지가 새로 고쳐질 때마다 새로운 Publisher 인스턴스가 생성되었을 것입니다.

데이터베이스에 다른 Publisher 인스턴스가 있는 경우, URL에서 ID를 변경하여 다른 인스턴스를 편집할 수 있습니다. 이 인스턴스의 ID가 3이므로, Publisher 1과 Publisher 2가 이미 존재한다고 가정할 수 있으며, 그들의 ID를 대체하여 기존 데이터를 볼 수 있습니다. 다음은 기존 Publisher 1 정보의 보기입니다 (<http://127.0.0.1:8000/publishers/1/>) – 당신의 정보는 다를 수 있습니다:



The screenshot shows a web form for editing a publisher. It has three input fields: 'Name' with the value 'Packt Publishing', 'Website' with the value 'https://www.packtpub.com/', and 'Email' with the value 'info@packtpub.com'. Each field has a label to its right: 'The name of the Publisher.', 'The Publisher's website.', and 'The Publisher's email address.' respectively. Below the fields is a 'Submit' button.

그림 7.22 – 기존 Publisher 1 정보

idx_cdb83481을 사용하여 기존 Publisher 인스턴스에 변경을 시도해 보세요. 저장한 후 메시지가 다르다는 점에 유의하세요 – 이는 Publisher 인스턴스가 생성된 것이 아니라 업데이트되었다고 사용자에게 알리는 idx_1d12804e입니다:



The screenshot shows the same web form as before, but with a success message at the top: 'Success: Publisher "Packt Publishing" was updated.' The input fields and the 'Submit' button are still present and unchanged.

그림 7.23 – 생성하는 대신 업데이트한 Publisher

이 연습에서는 모델(Publisher로부터 생성된 PublisherForm)에서 ModelForm을 구현하고 Django가 자동으로 올바른 검증 규칙으로 양식 필드를 생성하는 방법을 보았습니다. 그런 다음 양식의 내장 저장 메서드를 사용하여 publisher_edit 뷰 내에서 Publisher 인스턴스에 대한 변경 사항을 저장(또는 자동으로 생성)했습니다. 우리는 두 개의 URL을 뷰에 매핑했습니다. 첫 번째는 기존 Publisher를 편집하고 pk를 뷰에 전달하는 것이었습니다. 다른 하나는 이 pk를 뷰에 전달하지 않아 Publisher 인스턴스가 있어야 함을 나타냅니다.

be created. Finally, we used the browser to experiment with creating a new Publisher instance and then editing an existing one.

Activity 7.01 – Styling and integrating the publisher form

In *Exercise 7.03 – Creating and editing a publisher*, you added `PublisherForm` to create and edit Publisher instances. You built this with a standalone template that did not extend any other templates, so it lacked the global styles. In this activity, you will build a generic form detail page that will display a Django form, similar to `form-example.html`, but extending from a base template. The template will accept a variable to display the type of model being edited. You will also update the main `base.html` template to render the Django messages, using Bootstrap styling.

These steps will help you complete this activity:

1. Start by editing the project's `base.html` file. Wrap the content block in a `div` container for a nicer layout with some spacing. Surround the existing content block with a `<div>` element with `class="container-fluid"`.
2. Render each message in `messages` (similar to *Step 14 of Exercise 7.03 – Creating and editing a publisher*). Add the `{% for %}` block after the `<div>` container you just created, but before the content block. You should use the Bootstrap framework classes – this snippet will help you:

```
<div role="alert"
      class="alert alert-{{ {'error': 'danger'}.get(level_tag, level_tag)}}"
>
    {{ message }}
</div>
```

The Bootstrap class and Django message tags have corresponding names for the most part (for example, `success` and `alert-success`). The exception is Django's `error` tag. The corresponding Bootstrap class is `alert-danger`. See more information about Bootstrap alerts at <https://getbootstrap.com/docs/5.3/components/alerts/>. This is why you need to use the `if` template tag in this snippet.

3. Create a new template called `instance-form.html`, inside the `reviews` app's namespaced templates directory.
4. Now, `instance-form.html` should extend from the `reviews` app's `base.html` file.

생성되었습니다. 마지막으로, 우리는 브라우저를 사용하여 새로운 게시자 인스턴스를 생성하고 기존 인스턴스를 편집하는 실험을 했습니다.

활동 7.01 – 게시자 양식 스타일링 및 통합

연습 7.03 – 게시자 생성 및 편집에서, 당신은 게시자 인스턴스를 생성하고 편집하기 위해 PublisherForm을 추가했습니다. 당신은 다른 템플릿을 확장하지 않는 독립형 템플릿으로 이것을 구축했기 때문에 전역 스타일이 부족했습니다. 이 활동에서는 Django 양식을 표시할 일반 양식 세부 페이지를 구축할 것입니다. 이는 form-example.html과 유사하지만 기본 템플릿에서 확장됩니다. 템플릿은 편집 중인 모델의 유형을 표시하기 위해 변수를 수용할 것입니다. 또한 Bootstrap 스타일을 사용하여 Django 메시지를 렌더링하기 위해 기본 base.html 템플릿을 업데이트할 것입니다.

이 단계들은 당신이 이 활동을 완료하는 데 도움이 될 것입니다:

1. 프로젝트의 base.html 파일을 편집하는 것으로 시작하세요. 더 나은 레이아웃을 위해 콘텐츠 블록을 div 컨테이너로 감싸고 약간의 간격을 추가하세요. 기존 콘텐츠 블록을 class="container-fluid"인 <div> 요소로 감싸세요.
2. 메시지에서 각 메시지를 렌더링하세요 (연습 7.03 – 게시자 생성 및 편집의 14단계와 유사). 당신이 방금 생성한 <div> 컨테이너 뒤에 {% for %} 블록을 추가하되, 콘텐츠 블록 앞에 추가하세요. Bootstrap 프레임워크 클래스를 사용해야 합니다 – 이 스니펫이 도움이 될 것입니다:

```
<div role="alert"
    class="alert alert-{{ {'error': 'danger'}.get(level_tag, level_tag)}}">
    >
    {{ message }}
</div>
```

부트스트랩 클래스와 장고 메시지 태그는 대부분 대응하는 이름을 가지고 있습니다 (예: success와 alert-success). 예외는 장고의 error 태그입니다. 해당하는 부트스트랩 클래스는 alert-danger입니다. 부트스트랩 알림에 대한 자세한 정보는 <https://getbootstrap.com/docs/5.3/components/alerts/>에서 확인하세요. 이것이 이 코드 조각에서 if 템플릿 태그를 사용해야 하는 이유입니다.

3. reviews 앱의 네임스페이스가 있는 템플릿 디렉토리 안에 instance-form.html이라는 새 템플릿을 만드세요.
4. 이제 instance-form.html은 reviews 앱의 base.html 파일을 확장해야 합니다.

5. The context that's being passed to this template will contain a variable called `instance`. This will be the `Publisher` instance being edited, or `None` if we are creating a new `Publisher` instance. The context will also contain a `model_type` variable, which is a string indicating the model type (in this case, `Publisher`). Use these two variables to populate the `title` block template tag:
 - If the instance is `None`, the title should be `New Publisher`
 - Otherwise, the title should be `Editing Publisher <Publisher Name>`
6. `instance-form.html` should contain a content block template tag to override the `base.html` content block.
7. Add an `<h2>` element inside the content block and populate it using the same logic as the title. For better styling, wrap the publisher name in an `<em>` element.
8. Add a `<form>` element to the template with a `method` value of `post`. Since we are posting back to the same URL, an `action` value does not need to be specified.
9. Include the CSRF token template tag in the `<form>` body.
10. Render the Django form (its context variable will be `form`) inside `<form>`, using the `as_p` method.
11. Add a submit `<button>` element to the form. Its text should depend on whether you're editing or creating. Use the `Save` text for editing or `Create` for creating. You can use the Bootstrap classes for the button styling here. It should have the `class="btn btn-primary"` attribute.
12. In `reviews/views.py`, the `publisher_edit` view does not need many changes. Update the render call to render `instance-form.html` instead of `form-example.html`.
13. Update the context dictionary being passed to the render call. It should include the `Publisher` instance (the `publisher` variable that was already defined) and the `model_type` string. The context dictionary already includes `form` (a `PublisherForm` instance). You can remove the `method` key.
14. Since we're finished with the `form-example.html` template, it can be deleted.

When you've finished, the `Publisher` creation page (at `http://127.0.0.1:8000/publishers/new/`) should look like what's shown in *Figure 7.24*:

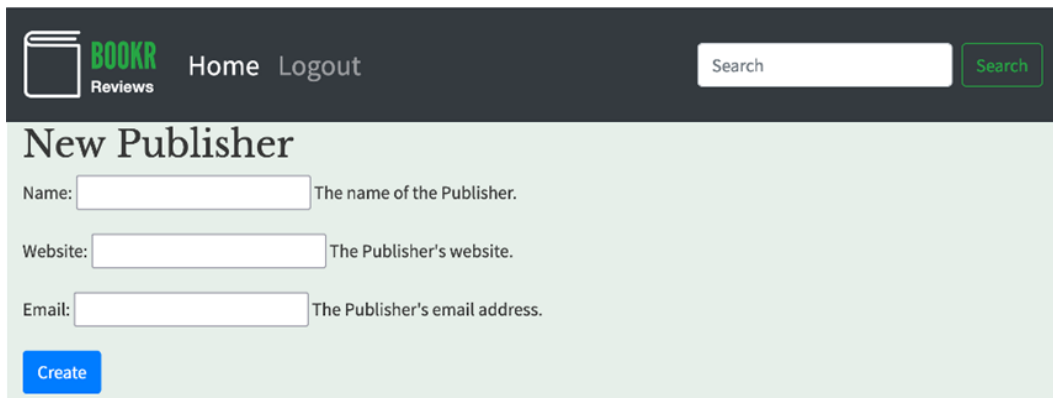
5. 이 템플릿에 전달되는 컨텍스트는 instance라는 변수를 포함할 것입니다. 이는 편집 중인 Publisher 인스턴스이거나, 새 Publisher 인스턴스를 생성하는 경우 None이 될 것입니다. 컨텍스트에는 모델 유형을 나타내는 문자열인 model_type 변수도 포함됩니다(이 경우 Publisher). 이 두 변수를 사용하여 제목 블록 템플릿 태그를 채워주세요:

- 인스턴스가 None인 경우 제목은 '새 Publisher'여야 합니다.

- 그렇지 않으면 제목은 '편집 중인 Publisher <Publisher Name>'이어야 하며, instance-form.html에는 base.html 콘텐츠를 블록을 재정의하기 위한 콘텐츠 블록 템플릿 태그가 포함되어야 합니다.
7. 콘텐츠 블록 안에 <h2> 요소를 추가하고 제목과 동일한 논리를 사용하여 채웁니다. 더 나은 스타일링을 위해 출판사 이름을 요소로 감쌉니다.
 8. 템플릿에 메서드 값이 post인 <form> 요소를 추가합니다. 같은 URL로 다시 게시하므로 액션 값은 지정할 필요가 없습니다.
 9. <form> 본문에 CSRF 토큰 템플릿 태그를 포함합니다.
 10. <form> 안에 Django 폼(그 컨텍스트 변수는 form입니다.)을 as_p 메서드를 사용하여 렌더링합니다.
 11. 폼에 제출 <button> 요소를 추가합니다. 그 텍스트는 편집 중인지 생성 중인지에 따라 달라져야 합니다. 편집 중에는 Save 텍스트를 사용하고, 생성 중에는 Create를 사용합니다. 여기서 버튼 스타일링을 위해 Bootstrap 클래스를 사용할 수 있습니다. 클래스="btn btnprimary" 속성이 있어야 합니다.
 12. reviews/views.py에서 publisher_edit 뷰는 많은 변경이 필요하지 않습니다. 렌더 호출을 idx_ff212a0bform-instance.html로 업데이트하고 idx_08ec58aform-example.html 대신에 업데이트합니다. 렌더 호출에 전달되는 컨텍스트
 13. idx_6425fc9e 사전에 업데이트합니다. 여기에는 이미 정의된 publisher 변수인 Publisher 인스턴스와 model_type 문자열이 포함되어야 합니다. 컨텍스트 사전에는 이미 form(PublisherForm 인스턴스)이 포함되어 있습니다. 메서드 키를 제거할 수 있습니다.

14. form-example.html 템플릿 작업이 끝났으므로 삭제할 수 있습니다.

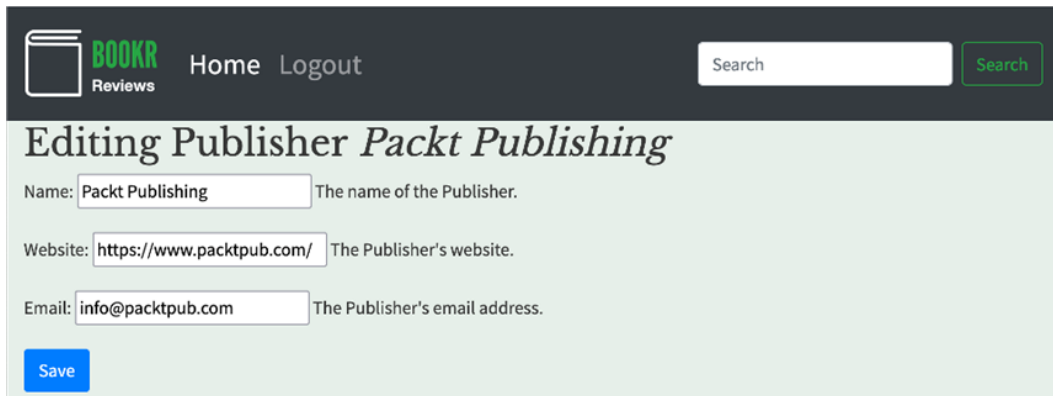
작업이 끝나면, 퍼블리셔 생성 페이지(<http://127.0.0.1:8000/publishers/new/>)는 그림 7.24에 표시된 것처럼 보여야 합니다:



The screenshot shows the 'New Publisher' form. At the top is a dark navigation bar with a 'BOOKR Reviews' logo, 'Home' and 'Logout' links, and a search bar with a 'Search' button. The main content area has a light green background. The title 'New Publisher' is in a large, dark serif font. Below it are three form fields: 'Name:' with a text input and the label 'The name of the Publisher.', 'Website:' with a text input and the label 'The Publisher's website.', and 'Email:' with a text input and the label 'The Publisher's email address.'. A blue 'Create' button is at the bottom left.

Figure 7.24 – The Publisher creation page

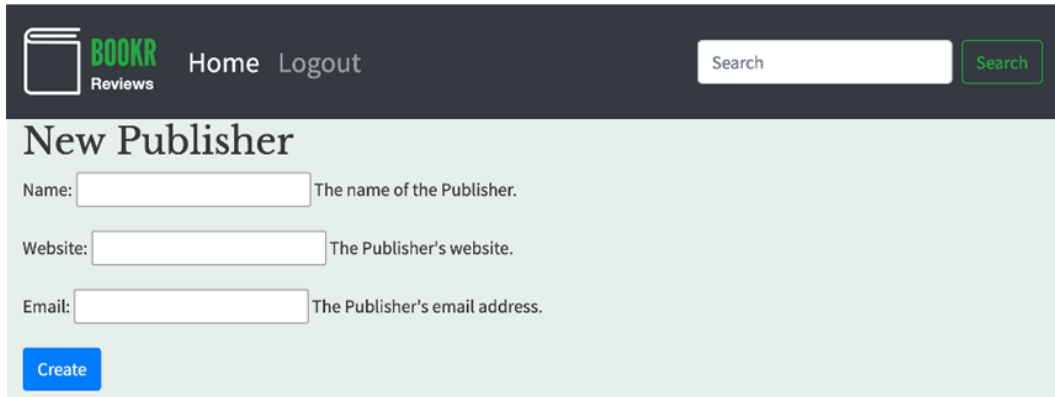
When editing a Publisher (for example, at <http://127.0.0.1:8000/publishers/1/>), your page should look like what's shown in Figure 7.25:



The screenshot shows the 'Editing Publisher' form. The navigation bar is identical to Figure 7.24. The title 'Editing Publisher' is in a large, dark serif font, followed by the publisher's name 'Packt Publishing' in a smaller, italicized serif font. Below it are three form fields: 'Name:' with a text input containing 'Packt Publishing' and the label 'The name of the Publisher.', 'Website:' with a text input containing 'https://www.packtpub.com/' and the label 'The Publisher's website.', and 'Email:' with a text input containing 'info@packtpub.com' and the label 'The Publisher's email address.'. A blue 'Save' button is at the bottom left.

Figure 7.25 – The Editing Publisher page

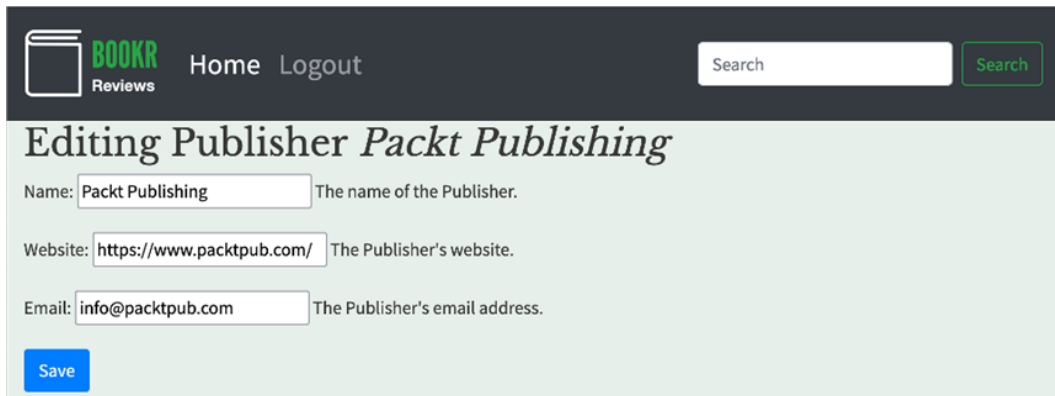
After saving a Publisher instance, whether creating or editing, you should see a success message at the top of the page (Figure 7.26):



The screenshot shows the 'New Publisher' form in the BOOKR Reviews application. The header bar is dark grey with the BOOKR Reviews logo on the left, 'Home' and 'Logout' links in the center, and a search bar on the right. The main content area has a light green background. The form title 'New Publisher' is at the top. Below it are three input fields: 'Name' with a placeholder 'The name of the Publisher.', 'Website' with a placeholder 'The Publisher's website.', and 'Email' with a placeholder 'The Publisher's email address.'. A blue 'Create' button is at the bottom left.

그림 7.24 - 출판사 생성 페이지

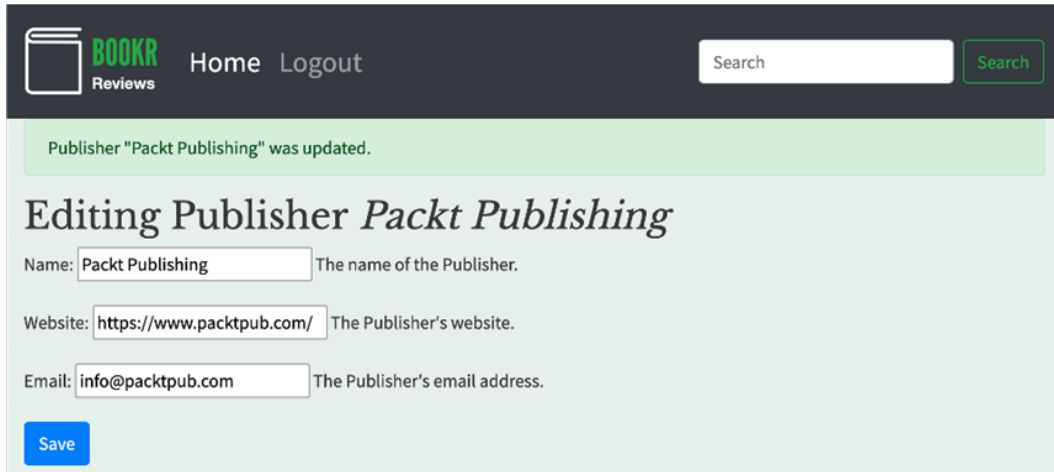
출판사를 편집할 때 (예: <http://127.0.0.1:8000/publishers/1/>), 당신의 페이지는 그림 7.25에 표시된 것처럼 보여야 합니다:



The screenshot shows the 'Editing Publisher Packt Publishing' form in the BOOKR Reviews application. The header bar is dark grey with the BOOKR Reviews logo on the left, 'Home' and 'Logout' links in the center, and a search bar on the right. The main content area has a light green background. The form title 'Editing Publisher Packt Publishing' is at the top. Below it are three input fields: 'Name' with the value 'Packt Publishing' and a placeholder 'The name of the Publisher.', 'Website' with the value 'https://www.packtpub.com/' and a placeholder 'The Publisher's website.', and 'Email' with the value 'info@packtpub.com' and a placeholder 'The Publisher's email address.'. A blue 'Save' button is at the bottom left.

그림 7.25 - 출판사 편집 페이지

출판사 인스턴스를 저장한 후, 생성하든 편집하든, 페이지 상단에 성공 메시지가 표시되어야 합니다 (그림 7.26):



The screenshot shows a web application interface. At the top, there is a dark navigation bar with a logo on the left that says "BOOKR Reviews" (with "BOOKR" in green and "Reviews" in white). To the right of the logo are links for "Home" and "Logout". Further right is a search bar with a "Search" button. Below the navigation bar, a light green alert box displays the message: "Publisher 'Packt Publishing' was updated." Below the alert, the main content area has the heading "Editing Publisher *Packt Publishing*". There are three form fields: "Name:" with the value "Packt Publishing" and a description "The name of the Publisher.", "Website:" with the value "https://www.packtpub.com/" and a description "The Publisher's website.", and "Email:" with the value "info@packtpub.com" and a description "The Publisher's email address.". At the bottom left of the form is a blue "Save" button.

Figure 7.26 – Success message rendered as a Bootstrap alert

Activity 7.02 – Review creation UI

Activity 7.01 – Styling and integrating the publisher form, was quite extensive; however, by completing it, you have created a foundation that makes it easier to add other edit and create views. You will experience this first-hand in this activity when you build forms for creating and editing reviews. Because the `instance-form.html` template was made generically, you can reuse it in other views.

In this activity, you will create a `review ModelForm` and then add a `review_edit` view to create or edit a `Review` instance. You can reuse `instance-form.html` from *Activity 7.01 – Styling and integrating the Publisher form*, and pass in different context variables to make it work with the `Review` model. When working with reviews, you'll operate within the context of a book – that is, the `review_edit` view must accept a `Book pk` as an argument. You'll fetch this `Book` separately and assign it to the `Review` instance that you create.

These steps will help you complete this activity:

1. In `forms.py`, add a `ReviewForm` subclass of `ModelForm`; its model should be `Review` (make sure you import the `Review` model).
`ReviewForm` should exclude the `date_edited` and `book` fields since the user should not be setting these in the form. The database allows any rating, but we can override the rating field with an `IntegerField` that requires a minimum value of 0 and a maximum value of 5.

BOOKR Reviews Home Logout Search Search

Publisher "Packt Publishing" was updated.

Editing Publisher *Packt Publishing*

Name: The name of the Publisher.

Website: The Publisher's website.

Email: The Publisher's email address.

Save

그림 7.26 - 부트스트랩 경고로 렌더링된 성공 메시지

활동 7.02 – 생성 UI 검토

활동 7.01 – 게시자 양식의 스타일링 및 통합은 상당히 광범위했습니다. 그러나 이를 완료함으로써, 당신은 다른 편집 및 생성 뷰를 추가하기 쉽게 만드는 기초를 마련했습니다. 이 활동에서 리뷰를 생성하고 편집하기 위한 양식을 구축할 때 이를 직접 경험하게 될 것입니다. `instance-form.html` 템플릿이 일반적으로 만들어졌기 때문에 다른 뷰에서도 재사용할 수 있습니다.

이 활동에서는 리뷰 `ModelForm`을 생성한 다음 리뷰 인스턴스를 생성하거나 편집하기 위해 `review_edit` 뷰를 추가합니다. 활동 7.01 – 게시자 양식의 스타일링 및 통합에서 `instance-form.html`을 재사용하고, 리뷰 모델과 함께 작동하도록 다른 컨텍스트 변수를 전달할 수 있습니다. 리뷰 작업을 할 때는 책의 맥락 내에서 작업하게 됩니다. 즉, `review_edit` 뷰는 `Book pk`를 인수로 받아야 합니다. 이 책은 별도로 가져와서 생성하는 리뷰 인스턴스에 할당합니다.

이 단계는 이 활동을 완료하는 데 도움이 될 것입니다:

1. `forms.py`에서 `ModelForm`의 `ReviewForm` 하위 클래스를 추가합니다. 그 모델은 `Review`여야 합니다(`Review` 모델을 가져오는 것을 잊지 마세요). `ReviewForm`은 사용자가 양식에서 설정하지 않아야 하므로 `date_edited` 및 `book` 필드를 제외해야 합니다. 데이터베이스는 어떤 평점도 허용하지만, 우리는 최소값 0과 최대값 5가 필요한 `IntegerField`로 평점 필드를 재정의할 수 있습니다.

2. Create a new view called `review_edit`. It should accept two arguments after request: `book_pk`, which is required, and `review_pk`, which is optional (defaults to `None`). Fetch the `Book` instance and `Review` instance using the `get_object_or_404` shortcut (call it once for each type). When fetching the review, make sure the review belongs to the book. If `review_pk` is `None`, then the `Review` instance should be `None` too.
3. If the request method is `POST`, then instantiate a `ReviewForm` using `request.POST` and the `Review` instance. Make sure you import this `ReviewForm`.
If the form is valid, save the form but change the `commit` argument from `save` to `False`. Then, set the `book` attribute on the returned `Review` instance to the book that was fetched in *Step 2*.

If the `Review` instance is being updated instead of created, then you should also set the `date_edited` attribute to the current date and time. Use the `from django.utils.timezone.now()` function. Then, save the `Review` instance.
4. Finish the valid form branch by registering a success message and redirecting back to the `book_detail` view. Since the `Review` model doesn't contain a meaningful text description, use the `Book` title in the message – for example, `Review for "<book title>" created`.
5. If the request method is not `POST`, instantiate a `ReviewForm` and just pass in the `Review` instance.
6. Render the `instance-form.html` template. In the context dictionary, include the same items as were used in `publisher_view`: `form`, `instance`, and `model_type` (`Review`). Include two extra items – `related_model_type`, which should be `Book`, and `related_instance`, which will be the `Book` instance.
7. Edit `instance-form.html` to add a place to display the related instance information added in *Step 6*. Under the `<h2>` element, add a `<p>` element that is only displayed if both `related_model_type` and `related_instance` are set. It should show the text `For <related_model_type> <related_instance>` – for example, `For Book Advanced Deep Learning with Keras`. Put the `related_instance` output in an `<em>` element for better readability.
8. In the `reviews` app's `urls.py`, add URL maps to the `review_edit` view. The `/books/` and `/books/<pk>/` URLs have already been configured. Add `/books/<book_pk>/reviews/new/` to create a review and `/books/<book_pk>/reviews/<review_pk>/` to

2. review_edit라는 새 뷰를 만듭니다. 이 뷰는 요청 후 두 개의 인수를 받아야 합니다: book_pk(필수)와 review_pk(선택적, 기본값은 None). get_object_or_404 단축키를 사용하여 Book 인스턴스와 Review 인스턴스를 가져옵니다(각 유형에 대해 한 번 호출). 리뷰를 가져올 때, 리뷰가 책에 속하는지 확인하십시오. review_pk가 None이면 Review 인스턴스도 None이어야 합니다.
3. 요청 방법이 POST인 경우, request.POST와 Review 인스턴스를 사용하여 ReviewForm을 인스턴스화합니다. 이 ReviewForm을 가져오는 것을 잊지 마십시오.

폼이 유효하면 폼을 저장하되, commit 인수를 save에서 False로 변경합니다. 그런 다음, 반환된 Review 인스턴스의 book 속성을 2단계에서 가져온 책으로 설정합니다.

Review 인스턴스가 생성되는 대신 업데이트되고 있는 경우, date_edited 속성을 현재 날짜와 시간으로 설정해야 합니다. django.utils.timezone.now() 함수를 사용하십시오. 그런 다음, Review 인스턴스를 저장합니다.
4. 유효한 양식 분기를 완료하여 성공 메시지를 등록하고 book_detail 뷰로 다시 리디렉션합니다. Review 모델에는 의미 있는 텍스트 설명이 없으므로 메시지에 책 제목을 사용하세요 – 예를 들어, "<booktitle>"에 대한 리뷰가 생성되었습니다.
5. 요청 방법이 POST가 아닌 경우 ReviewForm을 인스턴스화하고 Review 인스턴스를 전달합니다.
6. instance-form.html 템플릿을 렌더링합니다. 컨텍스트 사전에는 publisher_view에서 사용된 것과 동일한 항목인 form, instance 및 model_type (Review)을 포함합니다. idx_e65d161d 두 개의 추가 항목인 related_model_type (Book이어야 함)과 related_instance (Book 인스턴스가 될 것)를 포함합니다.
7. instance-form.html을 편집하여 6단계에서 추가된 관련 인스턴스 정보를 표시할 수 있는 공간을 추가합니다. <h2> 요소 아래에 관련_model_type과 related_instance가 모두 설정된 경우에만 표시되는 <p> 요소를 추가합니다. 예를 들어, For Book Advanced Deep Learning with Keras와 같이 For <related_model_type> <related_instance> 텍스트를 표시해야 합니다. 관련_instance 출력을 더 나은 가독성을 위해 요소에 넣습니다.
8. 리뷰 앱의 urls.py에서 review_edit 뷰에 대한 URL 맵을 추가합니다. /books/ 및 /books/<pk>/ URL은 이미 구성되어 있습니다. 리뷰를 생성하기 위해 /books/<book_pk>/reviews/new/를 추가하고 /books/<book_pk>/reviews/<review_pk>/로 추가합니다.

edit a review. Make sure you give these names, such as `review_create` and `review_edit`.

9. Inside the `book_detail.html` template, add links that a user can click on to create or edit a review. Add a link inside the content block, just before the `endblock` closing template tag. It should use the `url` template tag to link to the `review_edit` view when in creation mode. Also, use the `class="btn btn-primary"` attribute to make the link display like a Bootstrap button. The link text should be `Add Review`.
10. Finally, add a link to edit a review, inside the `for` loop that iterates over `Reviews` for `Book`. After all the instances of `text-info <span>`, add a link to the `review_edit` view using the `url` template tag. You will need to provide `book.pk` and `review.pk` as arguments. The text of the link should be `Edit Review`.

When you're finished, the **Review Comments** page should look like what's shown in *Figure 7.27*:

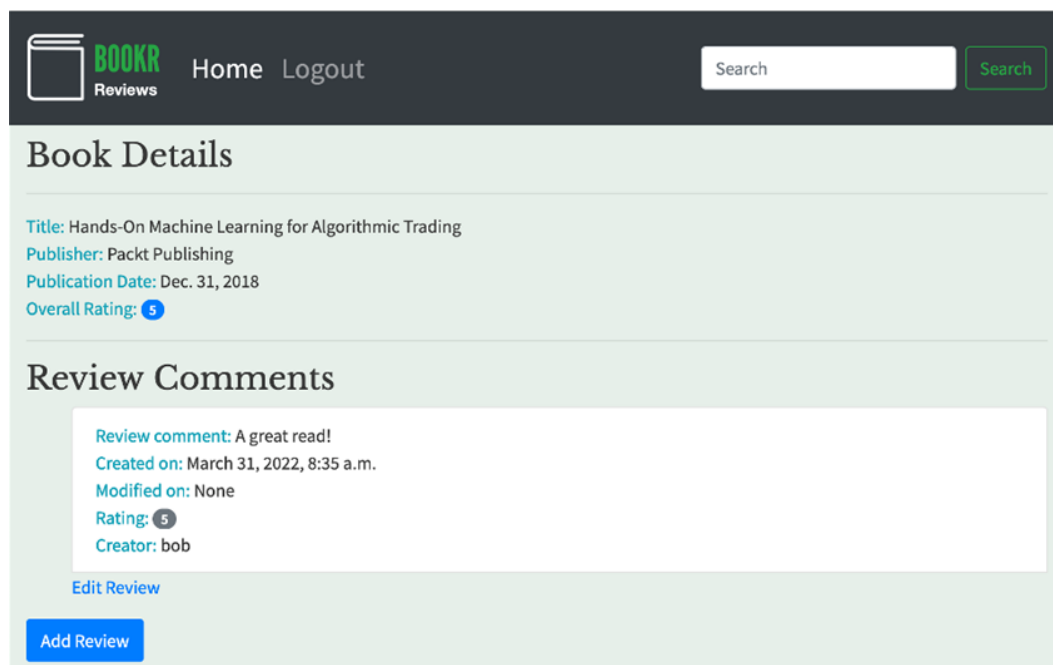


Figure 7.27 – The Book Details page with the added Add Review button

리뷰를 편집하세요. `review_create`와 `review_edit`와 같은 이름을 반드시 사용하세요.

9. `book_detail.html` 템플릿 안에 사용자가 클릭하여 리뷰를 생성하거나 편집할 수 있는 링크를 추가하세요. `content` 블록 안에, `endblock` 닫는 템플릿 태그 바로 전에 링크를 추가해야 합니다. 생성 모드일 때 `review_edit` 뷰에 연결하기 위해 `url` 템플릿 태그를 사용해야 합니다. 또한, 링크가 Bootstrap 버튼처럼 보이도록 `class="btn btn-primary"` 속성을 사용하세요. 링크 텍스트는 '리뷰 추가'여야 합니다.

10. 마지막으로, Book에 대한 Reviews를 반복하는 `for` 루프 안에 리뷰를 편집하는 링크를 추가하세요. `text-info <span>`의 모든 인스턴스 뒤에 `url` 템플릿 태그를 사용하여 `review_edit` 뷰에 대한 링크를 추가해야 합니다. `book.pk`와 `review.pk`를 인수로 제공해야 합니다. 링크의 텍스트는 '리뷰 편집'이어야 합니다.

`idx_2deac6a9` 작업이 완료되면, 리뷰 댓글 페이지는 그림 7.27에 표시된 것처럼 보여야 합니다:

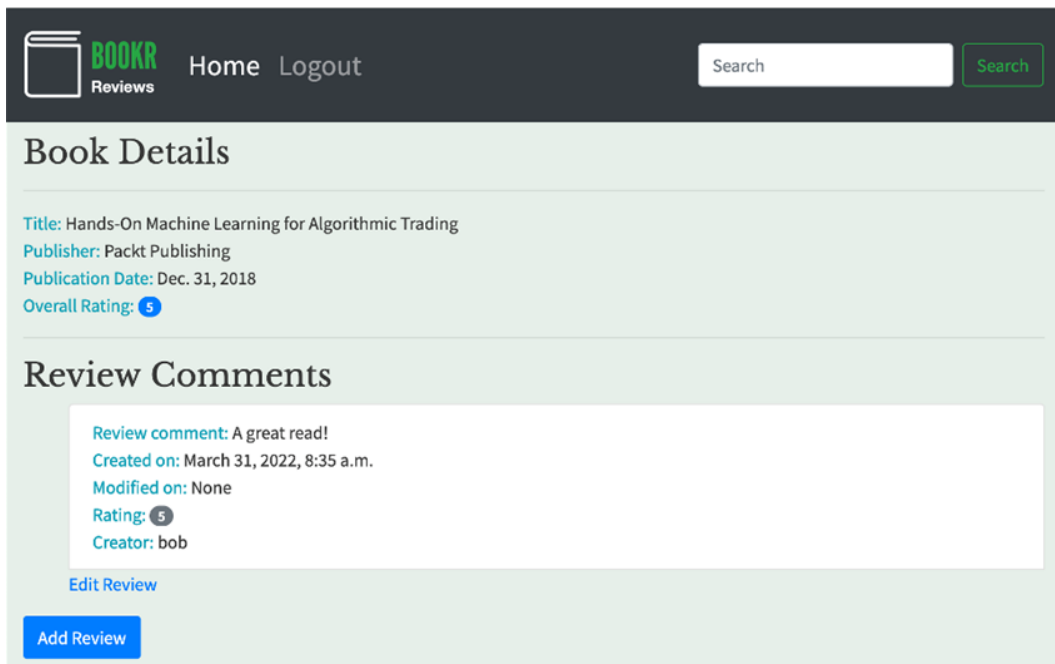
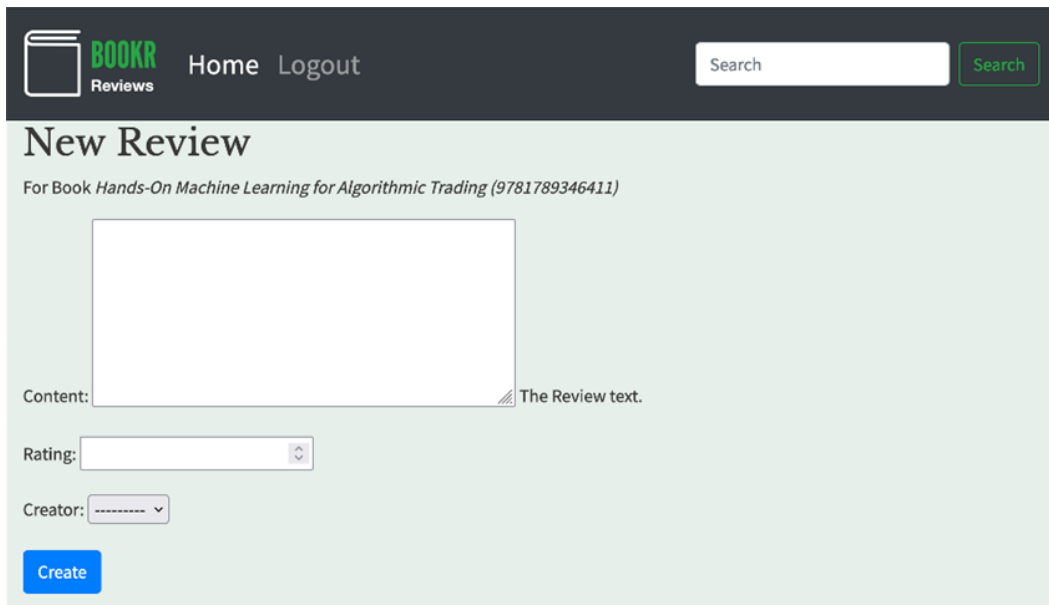


그림 7.27 - 추가된 리뷰 추가 버튼이 있는 도서 세부정보 페이지

You will see the **Add Review** button. Clicking it will take you to the **New Review** page, which should look like what's shown in *Figure 7.28*:



The screenshot shows a web application interface for adding a new review. At the top is a dark navigation bar with a logo on the left that says "BOOKR Reviews" (with "BOOKR" in green and "Reviews" in white). To the right of the logo are links for "Home" and "Logout". On the far right of the navigation bar is a search bar with the placeholder text "Search" and a green "Search" button. Below the navigation bar, the main content area has a light green background. The heading "New Review" is displayed in a large, dark font. Below the heading, the text "For Book Hands-On Machine Learning for Algorithmic Trading (9781789346411)" is shown in a smaller, italicized font. The main form consists of three fields: a large text area labeled "Content:" with a placeholder "The Review text.", a "Rating:" field which is a dropdown menu, and a "Creator:" field which is a dropdown menu showing "-----". At the bottom left of the form is a blue "Create" button.

Figure 7.28 – The New Review page

리뷰 추가 버튼이 보일 것입니다. 클릭하면 그림 7.28에 표시된 것처럼 보이는 새 리뷰 페이지로 이동합니다:

BOOKR Reviews Home Logout Search Search

New Review

For Book *Hands-On Machine Learning for Algorithmic Trading* (9781789346411)

Content: The Review text.

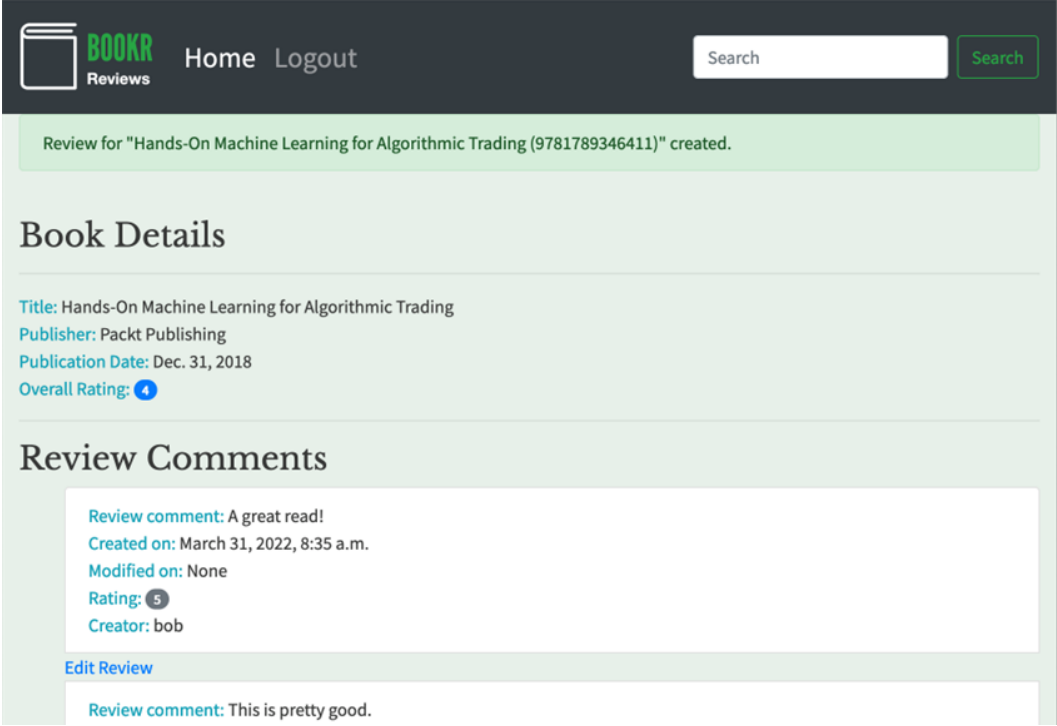
Rating:

Creator:

Create

그림 7.28 - 새 리뷰 페이지

Enter some details in the form and click **Create**. You'll be redirected to the **Book Details** page. You should see the success message and your review, as shown in *Figure 7.29*:



BOOKR Reviews Home Logout Search Search

Review for "Hands-On Machine Learning for Algorithmic Trading (9781789346411)" created.

Book Details

Title: Hands-On Machine Learning for Algorithmic Trading
Publisher: Packt Publishing
Publication Date: Dec. 31, 2018
Overall Rating: 4

Review Comments

Review comment: A great read!
Created on: March 31, 2022, 8:35 a.m.
Modified on: None
Rating: 5
Creator: bob

[Edit Review](#)

Review comment: This is pretty good.

Figure 7.29 – The Book Details page with a review added

양식에 몇 가지 세부정보를 입력하고 생성 클릭하세요. 책 세부정보 페이지로 리디렉션됩니다. 성공 메시지와 귀하의 리뷰가 표시되어야 합니다. 그림 7.29와 같이:

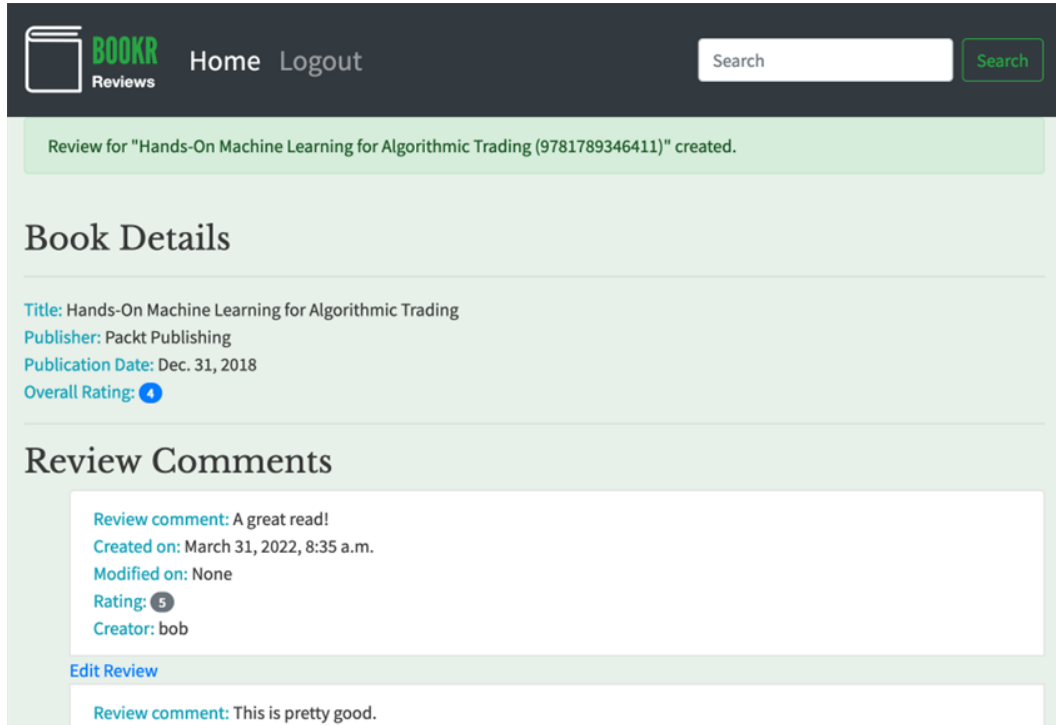


그림 7.29 - 리뷰가 추가된 책 세부정보 페이지

You will also see the **Edit Review** link. If you click it, you will be taken to a form that has been pre-populated with your review data (see *Figure 7.30*):

The screenshot shows a web application interface for editing a review. At the top, there is a dark navigation bar with a logo on the left that says 'BOOKR Reviews' next to a book icon. To the right of the logo are links for 'Home' and 'Logout'. Further right is a search bar with the placeholder text 'Search' and a green 'Search' button. Below the navigation bar, the main heading reads 'Editing Review *bob - Hands-On Machine Learning for Algorithmic Trading*'. Underneath the heading, a line of text identifies the book: 'For Book *Hands-On Machine Learning for Algorithmic Trading* (9781789346411)'. The main form area has a light green background. It contains a large text input field with the text 'A great read!'. To the left of this field is the label 'Content:'. To the right of the field is a small icon of a document with a diagonal line and the text 'The Review text.'. Below the text field is a 'Rating:' label followed by a dropdown menu showing the value '5'. Below the rating is a 'Creator:' label followed by a dropdown menu showing the value 'bob'. At the bottom left of the form is a blue 'Save' button.

Figure 7.30 – Review form when editing a review

편집 리뷰 링크도 idx_00d6fe6esee를 볼 수 있습니다. 클릭하면 리뷰 데이터로 미리 채워진 양식으로 이동합니다(그림 7.30 참조):

The screenshot shows the BOOKR Reviews website interface. At the top, there is a dark navigation bar with the BOOKR logo, 'Home' and 'Logout' links, and a search bar. The main content area has a light green background. The title 'Editing Review bob - Hands-On Machine Learning for Algorithmic Trading' is displayed in a large, bold font. Below the title, the book title 'Hands-On Machine Learning for Algorithmic Trading (9781789346411)' is shown. The review content area contains a text box with the text 'A great read!' and a rating of 5. The creator is listed as 'bob'. A 'Save' button is at the bottom left.

BOOKR Reviews Home Logout Search

Editing Review bob - *Hands-On Machine Learning for Algorithmic Trading*

For Book *Hands-On Machine Learning for Algorithmic Trading* (9781789346411)

A great read!

Content:  The Review text.

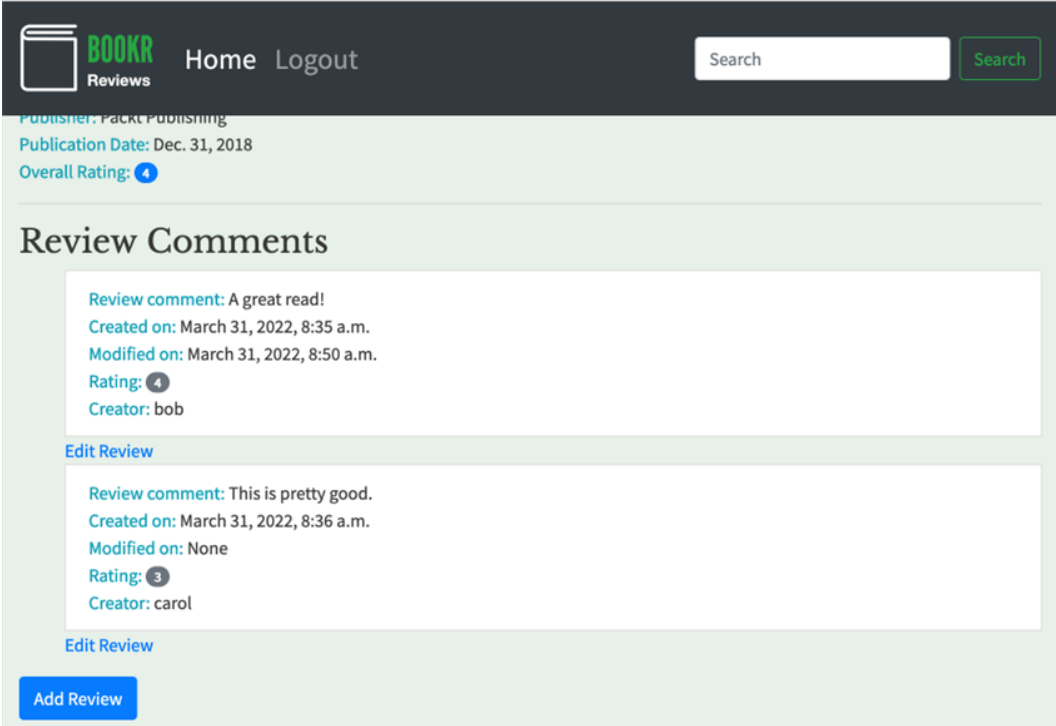
Rating: 5

Creator: bob

Save

그림 7.30 - 리뷰 편집 시 리뷰 양식

After saving an existing review, you should see that the **Modified on** date has been updated on the **Book Details** page (Figure 7.31):



The screenshot shows the 'Bookr Reviews' page. At the top, there's a dark navigation bar with the 'BOOKR Reviews' logo, 'Home' and 'Logout' links, and a search bar. Below the navigation bar, the page displays book details: 'Publisher: Packt Publishing', 'Publication Date: Dec. 31, 2018', and 'Overall Rating: 4'. The main section is titled 'Review Comments'. It contains two review entries. The first entry, by 'bob', has a rating of 4 and a 'Modified on' date of 'March 31, 2022, 8:50 a.m.'. The second entry, by 'carol', has a rating of 3 and a 'Modified on' date of 'None'. Each entry has an 'Edit Review' link below it. At the bottom left, there is a blue 'Add Review' button.

Figure 7.31 – The *Modified on* date field has now been populated

Summary

This chapter provided a deep dive into forms. We saw how to enhance Django forms with custom validation, advanced rules for cleaning data, and validating fields. We also saw how custom cleaning methods can transform the data that we get out of forms. A nice feature we saw that can be added to forms is the ability to set initial and placeholder values on fields so that the user does not have to fill them out.

We then looked at how to use the `ModelForm` class to automatically create a form from a Django model. We saw how to only show some fields to the user and how to apply custom form validation rules to `ModelForm`. We also saw how Django can automatically save the new or updated model instance to the database inside the view. In the activities for this chapter, we enhanced Bookr further by adding forms for creating and editing publishers and submitting reviews.

기존 리뷰를 저장한 후, 도서 세부정보 페이지에서 수정된 날짜가 업데이트된 것을 확인할 수 있습니다 (그림 7.31):

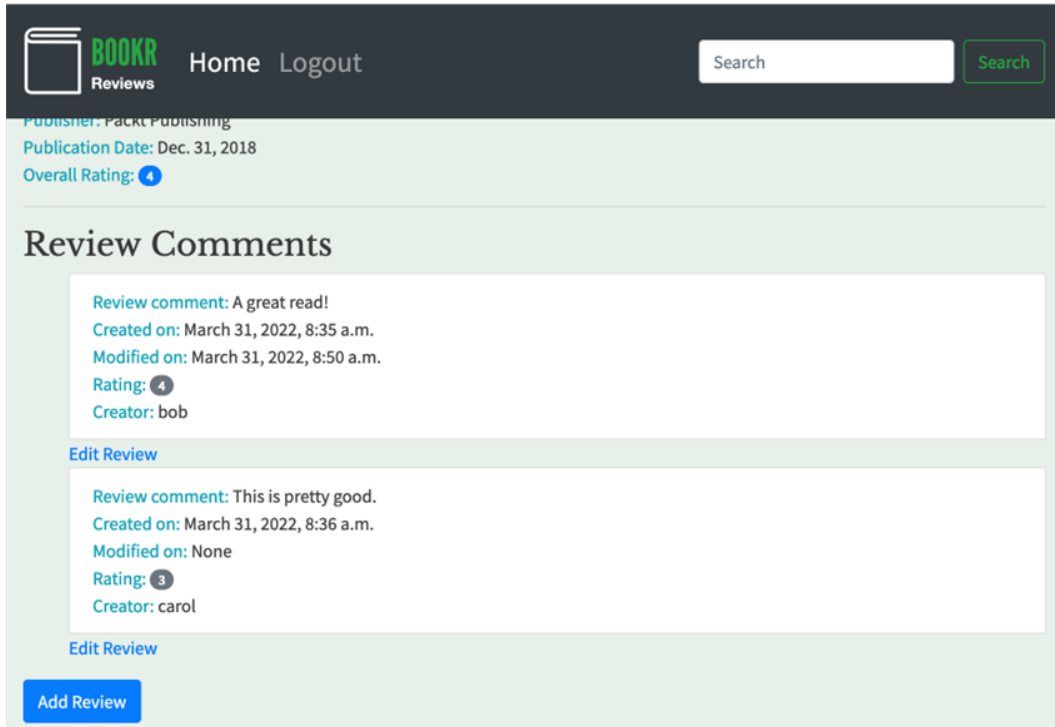


그림 7.31 - 수정된 날짜 필드가 이제 채워졌습니다

요약

이 장에서는 폼에 대해 깊이 있게 다루었습니다. 우리는 Django 폼을 사용자 정의 유효성 검사, 데이터 정리를 위한 고급 규칙, 필드 유효성 검사로 향상시키는 방법을 보았습니다. 또한 사용자 정의 정리 방법이 폼에서 얻은 데이터를 어떻게 변환할 수 있는지도 보았습니다. 폼에 추가할 수 있는 멋진 기능은 사용자가 필드를 작성할 필요가 없도록 초기 및 플레이스홀더 값을 설정할 수 있는 기능입니다.

그런 다음 ModelForm 클래스를 사용하여 Django 모델에서 자동으로 폼을 생성하는 방법을 살펴보았습니다. 우리는 사용자에게 일부 필드만 표시하는 방법과 ModelForm에 사용자 정의 폼 유효성 검사 규칙을 적용하는 방법을 보았습니다. 또한 Django가 뷰 내에서 새 모델 인스턴스나 업데이트된 모델 인스턴스를 데이터베이스에 자동으로 저장할 수 있는 방법도 보았습니다. 이 장의 활동에서는 출판사를 생성하고 편집하는 폼과 리뷰를 제출하는 폼을 추가하여 Bookr를 더욱 향상시켰습니다.

In the next chapter, we will continue with the theme of submitting user input, and we'll discuss how Django handles file uploads and downloads.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

다음 장에서는 사용자 입력 제출이라는 주제를 계속 진행하며, Django가 파일 업로드 및 다운로드를 처리하는 방법에 대해 논의할 것입니다.

이 책의 PDF 사본, 코드 번들 등을 받으세요

QR 코드를 스캔하거나 (packtpub.com/unlock으로 이동) 이 책을 이름으로 검색하고, 판을 확인한 후 페이지의 단계를 따르세요.



UNLOCK NOW

참고: 송장을 준비하세요. Packt 웹사이트에서 직접 구매한 경우 송장이 필요하지 않습니다.

8

Media Serving and File Uploads

Media files refer to extra files that can be added after deployment to enrich your Django application. Usually, they are extra images that you would use on your site, but any type of file (including video, audio, PDF, text, documents, or even HTML) can be served as media.

You can think of them as somewhere between dynamic data and static assets. They aren't dynamic data that Django generates on the fly, such as when rendering a template. They also aren't the static files that are included by the site developer when the site is deployed. Instead, they are extra files that can be uploaded by users or generated by your application for later retrieval.

Some common examples of media files (which you'll see in *Activity 8.01 – image and PDF upload for books*, later in this chapter) are book covers and preview PDFs that can be attached to a Book object. You can also use media files to allow users to upload images for a blog post or avatars for a social media site. If you wanted to use Django to build your own video-sharing platform, you would store the uploaded videos as media. Your website will not function well if all these files are static files, as users won't be able to upload their own book covers, videos, and so on, and will be stuck with the ones you deployed.

In this chapter, we will be covering the following topics:

- Settings for media uploads and serving
- File uploads using HTML forms
- File uploads with Django forms
- Image uploads with Django forms
- Serving uploaded (and other) files using Django
- `ModelForm` and file uploads

8

미디어 제공 및 파일 업로드

미디어 파일은 Django 애플리케이션을 풍부하게 하기 위해 배포 후 추가할 수 있는 추가 파일을 의미합니다. 일반적으로 사이트에서 사용할 추가 이미지이지만, 비디오, 오디오, PDF, 텍스트, 문서 또는 HTML을 포함한 모든 유형의 파일이 미디어로 제공될 수 있습니다.

이들을 동적 데이터와 정적 자산 사이의 어딘가로 생각할 수 있습니다. 이들은 템플릿을 렌더링할 때와 같이 Django가 즉석에서 생성하는 동적 데이터가 아닙니다. 또한 사이트가 배포될 때 사이트 개발자가 포함한 정적 파일도 아닙니다. 대신, 이들은 사용자가 업로드하거나 나중에 검색하기 위해 애플리케이션에서 생성할 수 있는 추가 파일입니다.

미디어 파일의 몇 가지 일반적인 예(이 장의 8.01 활동에서 볼 수 있는 책의 이미지 및 PDF 업로드)는 책 객체에 첨부할 수 있는 책 표지 및 미리보기 PDF입니다. 또한 미디어 파일을 사용하여 사용자가 블로그 게시물이나 소셜 미디어 사이트의 아바타를 위한 이미지를 업로드할 수 있도록 할 수 있습니다. Django를 사용하여 자체 비디오 공유 플랫폼을 구축하려는 경우 업로드된 비디오를 미디어로 저장합니다. 모든 파일이 정적 파일인 경우 웹사이트가 제대로 작동하지 않으며, 사용자는 자신의 책 표지, 비디오 등을 업로드할 수 없고 배포한 파일에 갇히게 됩니다.

이 장에서는 다음 주제를 다룰 것입니다: 미디어 업

- 로드 및 제공 설정
- HTML 양식을 사용한 파일 업로드
- Django 양식을 사용한 파일 업로드
- Django 양식을 사용한 이미지 업로드
- Django를 사용하여 업로드된 (및 기타) 파일 제공
- ModelForm 및 파일 업로드

- Activity 8.01 – image and PDF upload for books
- Activity 8.02 – displaying the cover and sample link

Technical requirements

Find the solution in the Chapter08 folder in this book's GitHub repo. To access the repository link, follow the steps in the *Download the example code file* section in the *Preface*.

Settings for media uploads and serving

In *Chapter 5, Serving Static Files*, we looked at how Django can be used to serve static files. Serving media files is quite similar. Two settings must be configured in `settings.py`: `MEDIA_ROOT` and `MEDIA_URL`. These are analogous to `STATIC_ROOT` and `STATIC_URL` for serving static files. They are as follows:

- `MEDIA_ROOT`: This is the path on the disk where the media (such as uploaded files) will be stored. As with static files, your web server should be configured to serve directly from this directory to take the load off Django.
- `MEDIA_URL`: This is similar to `STATIC_URL`, but as you might guess, it's the URL that should be used to serve media. It must end in `/`. Generally, you will use something such as `/media/`.

Note

For security reasons, the path for `MEDIA_ROOT` must not be the same as the path for `STATIC_ROOT`, and `MEDIA_URL` must not be the same as `STATIC_URL`. If they were the same, a user might replace your static files (such as JavaScript or CSS files) with malicious code and exploit your users.

`MEDIA_URL` is designed to be used in templates so that you are not hardcoding the URL, and it can be changed easily. For example, you might want to set it to a specific host or **content delivery network (CDN)** when you deploy to production. We will discuss the use of `MEDIA_URL` in templates in the next section.

Serving media files in development

As with static files, when serving media in production, your web server should be configured to serve directly from the `MEDIA_ROOT` directory to prevent Django from being tied up in servicing the request. The Django dev server can serve media files in development. However, unlike static files, the URL mapping and view are not set up automatically for media files.

- 활동 8.01 – 책을 위한 이미지 및 PDF 업로드할
- 동 8.02 – 표지 및 샘플 링크 표시

기술 요구 사항

이 책의 GitHub 저장소의 Chapter08 폴더에서 솔루션을 찾으세요. 저장소 링크에 접근하려면 서문에 있는 예제 코드 파일 다운로드 섹션의 단계를 따르세요.

미디어 업로드 및 제공 설정

5장, 정적 파일 제공에서 우리는 Django가 정적 파일을 제공하는 데 어떻게 사용될 수 있는지 살펴보았습니다. 미디어 파일 제공은 매우 유사합니다. settings.py에서 두 가지 설정을 구성해야 합니다: MEDIA_ROOT 및 MEDIA_URL. 이들은 정적 파일 제공을 위한 STATIC_ROOT 및 STATIC_URL과 유사합니다. 설정은 다음과 같습니다:

- MEDIA_ROOT: 이것은 미디어(업로드된 파일 등)가 저장될 디스크의 idx_95eb6656 경로입니다. 정적 파일과 마찬가지로, 웹 서버는 Django의 부하를 줄이기 위해 이 디렉토리에서 직접 제공하도록 구성되어야 합니다.
- MEDIA_URL: 이것은 STATIC_URL과 유사하지만, 추측할 수 있듯이 미디어를 제공하는 데 사용해야 하는 URL입니다. 반드시 /로 끝나야 합니다. 일반적으로 /media/와 같은 것을 사용합니다.

참고

보안상의 이유로, MEDIA_ROOT의 경로는 STATIC_ROOT의 경로와 같아서는 안 되며, MEDIA_URL은 STATIC_URL과 같아서는 안 됩니다. 만약 같다면, 사용자가 정적 파일(예: JavaScript 또는 CSS 파일)을 악성 코드로 교체하여 사용자를 악용할 수 있습니다.

MEDIA_URL은 URL을 하드코딩하지 않도록 템플릿에서 사용하도록 설계되었으며, 쉽게 변경할 수 있습니다. 예를 들어, 프로덕션에 배포할 때 특정 호스트나 콘텐츠 전송 네트워크(CDN)로 설정하고 싶을 수 있습니다. MEDIA_URL을 템플릿에서 사용하는 방법에 대해서는 다음 섹션에서 논의하겠습니다.

개발 중 미디어 파일 제공

정적 파일과 마찬가지로, 프로덕션에서 미디어를 제공할 때 웹 서버는 Django가 요청 처리에 묶이지 않도록 MEDIA_ROOT 디렉토리에서 직접 제공하도록 구성되어야 합니다. Django 개발 서버는 개발 중에 미디어 파일을 제공할 수 있습니다. 그러나 정적 파일과 달리, 미디어 파일에 대한 URL 매핑 및 뷰는 자동으로 설정되지 않습니다.

Django provides the static URL mapping, which can be added to your existing URL maps to serve media files. It is added to your `urls.py` file like this:

```
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    # your existing URL maps
]
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL,
        document_root=settings.MEDIA_ROOT)
```

This will serve the `MEDIA_ROOT` setting defined in `settings.py` to the `MEDIA_URL` setting that is also defined there. The reason we check for `settings.DEBUG` before appending the map is so that we don't add this map in production.

For example, if your `MEDIA_ROOT` was set to `/var/www/bookr/media`, and your `MEDIA_URL` was set to `/media/`, then the `/var/www/bookr/media/image.jpg` file would be available at `http://127.0.0.1:8000/media/image.jpg`.

The static URL map does not work when the Django `DEBUG` setting is `False`, so it can't be used in production. However, as has been mentioned, in production, your web server should be serving these requests, so Django will not need to handle them.

In the first exercise, you will add `MEDIA_ROOT` and `MEDIA_URL` to your `settings.py`. You will then add the static media-serving URL map and add a test file to ensure media serving is configured correctly.

Exercise 8.01 – configuring media storage and serving

In this exercise, you will set up a new Django project as an example project to use throughout this chapter. Then, you'll configure it to be able to serve media files. You'll do this by creating a media directory and adding the `MEDIA_ROOT` and `MEDIA_URL` settings. Then, you'll set up the URL mapping for `MEDIA_URL`.

장고는 미디어 파일을 제공하기 위해 기존 URL 맵에 추가할 수 있는 정적 URL 매핑을 제공합니다. 이는 `urls.py` 파일에 다음과 같이 추가됩니다:

```
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    # 기존 URL 맵
]

if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL,
                          document_root=settings.MEDIA_ROOT)
```

이것은 `settings.py`에 정의된 `MEDIA_ROOT` 설정을 거기에서 또한 정의된 `MEDIA_URL` 설정에 제공할 것입니다. 우리가 맵을 추가하기 전에 `settings.DEBUG`를 확인하는 이유는 프로덕션에서 이 맵을 추가하지 않기 위함입니다.

예를 들어, `MEDIA_ROOT`가 `/var/www/bookr/media`로 설정되어 있고, `MEDIA_URL`이 `/media/`로 설정되어 있다면, `/var/www/bookr/media/image.jpg` 파일은 `http://127.0.0.1:8000/media/image.jpg`에서 사용할 수 있습니다.

Django DEBUG 설정이 False일 때 static URL 맵 `idx_a2bb22d1`은 작동하지 않으므로 프로덕션에서 사용할 수 없습니다. 그러나 언급했듯이, 프로덕션에서는 웹 서버가 이러한 요청을 처리해야 하므로 Django가 이를 처리할 필요는 없습니다.

첫 번째 연습에서는 `MEDIA_ROOT`와 `MEDIA_URL`을 `settings.py`에 추가할 것입니다. 그런 다음 정적 미디어 제공 URL 맵을 추가하고 미디어 제공이 올바르게 구성되었는지 확인하기 위해 테스트 파일을 추가할 것입니다.

연습 8.01 – 미디어 저장소 및 제공 구성

이 연습에서는 이 장 전반에 걸쳐 사용할 예제 프로젝트로 `idx_e84030a6` 새로운 Django 프로젝트를 설정할 것입니다. 그런 다음 미디어 파일을 제공할 수 있도록 구성할 것입니다. 이를 위해 `media` 디렉토리를 만들고 `MEDIA_ROOT` 및 `MEDIA_URL` 설정을 추가할 것입니다. 그런 다음 `MEDIA_URL`에 대한 URL 매핑을 설정할 것입니다.

To check that everything is configured and being served correctly, you will put a test file inside the media directory:

1. As with the previous example Django projects you've set up, you can reuse the existing djangoenv virtual environment. In a terminal, activate the djangoenv virtual environment, using `pyenv local djangoenv`. Then, start a new project named `media_project` using `django-admin.py`:

```
django-admin startproject media_project
```

Note

To learn how to create and activate a virtual environment using pyenv, refer to *Chapter 1* of this book.

Change (or `cd`) into the `media_project` directory that was created, then use the `startapp` management command to start an app called `media_example`:

```
python manage.py startapp media_example
```

2. Open the `media_project` directory in PyCharm. Set up a run configuration for the `runserver` command in the same manner as for the other Django projects you've opened:

모든 것이 올바르게 구성되고 제공되고 있는지 확인하기 위해, 미디어 디렉토리 안에 테스트 파일을 넣을 것입니다:

1. 이전에 설정한 Django 프로젝트와 마찬가지로, 기존의 djangoenv 가상 환경을 재사용할 수 있습니다. 터미널에서 `pyenv local djangoenv`를 사용하여 `djangoenv` 가상 환경을 활성화합니다. 그런 다음, `django-admin.py`를 사용하여 `media_project`라는 새 프로젝트를 시작합니다:

```
django-admin startproject media_project
```

참고

`pyenv`를 사용하여 가상 환경을 생성하고 활성화하는 방법을 배우려면 이 책의 1장을 참조하세요.

생성된 `media_project` 디렉토리로 변경(`cd`)한 후, `startapp` 관리 명령을 사용하여 `media_example`이라는 앱을 시작합니다:

```
python manage.py startapp media_example
```

2. PyCharm에서 `media_project` 디렉토리를 엽니다. 실행 구성을 설정합니다. `runserver` 명령은 다른 Django 프로젝트에서와 같은 방식으로 실행됩니다. 열림:

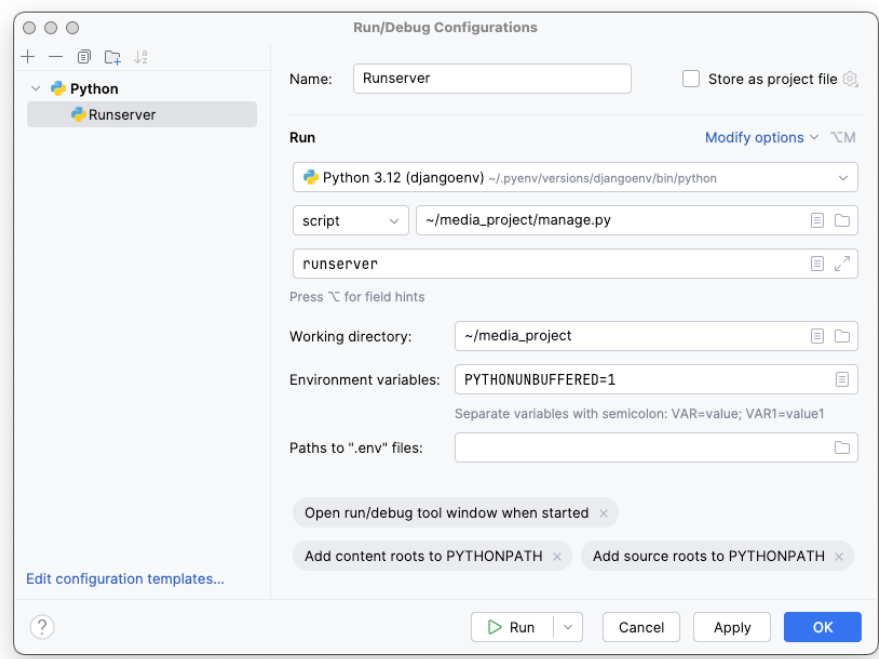


Figure 8.1: Runserver configuration

Figure 8.1 shows the **Runserver** configuration of the project in PyCharm.

- 3. Create a new directory named `media` inside the `media_project` project directory. Then, create a new file in this directory named `test.txt`. The directory structure of this will look like Figure 8.2:

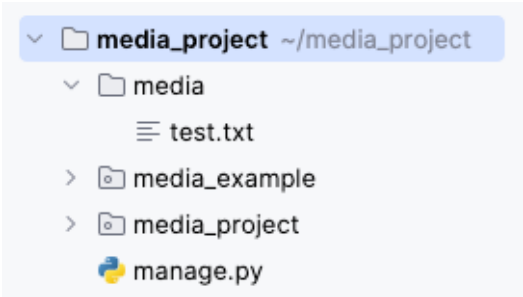


Figure 8.2: The media directory and the test.txt layout

- 4. The `test.txt` file will also open automatically. Enter `Hello world!` into it, then you can save and close the file.

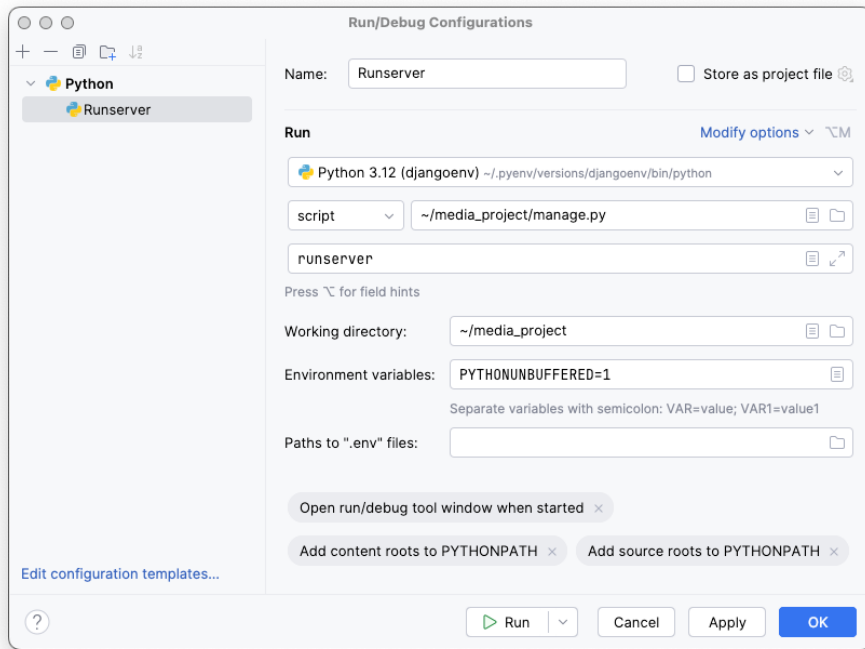


그림 8.1: Runserver 구성

그림 8.1은 PyCharm에서 프로젝트의 Runserver 구성을 보여줍니다.

3. media_project 프로젝트 디렉토리 안에 media라는 새 디렉토리를 만듭니다. 그런 다음 이 디렉토리 안에 test.txt라는 새 파일을 만듭니다. 이 디렉토리 구조는 그림 8.2와 같을 것입니다:

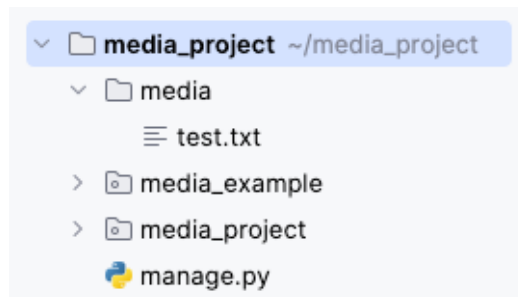


그림 8.2: media 디렉토리와 test.txt 레이아웃

4. test.txt 파일은 idx_5683f024도 자동으로 열립니다. 여기에 Hello world!를 입력한 후 파일을 저장하고 닫을 수 있습니다.

5. Open `settings.py` inside the `media_project` package directory. At the end of the file, add a setting for `MEDIA_ROOT` using the path to the `media` directory you just created. Join the name of the directory (`media`) onto `BASE_DIR`:

```
MEDIA_ROOT = BASE_DIR / "media"
```

6. Directly below the line added in step 5, add another setting for `MEDIA_URL`; this should just be `"/media/"`:

```
MEDIA_URL = "/media/"
```

With these changes made, `settings.py` should look like the file placed in the `Chapter08` folder in this book's GitHub repo.

7. Open the `media_project` package's `urls.py` file. After the `urlpatterns` definition, add the following code to add the media-serving URL if running in the `DEBUG` mode. First, you will need to import the Django settings and static serving view by adding the highlighted import lines above the `urlpatterns` definition:

```
from django.contrib import admin
from django.urls import path
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path("admin/", admin.site.urls),
]
```

8. Then, add the following code right after your `urlpatterns` definition (refer to the code block in the previous step) to conditionally add a mapping from the `MEDIA_URL` setting to the static view, which will serve from `MEDIA_ROOT`:

```
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL,
        document_root=settings.MEDIA_ROOT)
```

5. `media_project` 패키지 디렉토리 내의 `settings.py`를 엽니다. 파일의 끝에 방금 생성한 미디어 디렉토리의 경로를 사용하여 `MEDIA_ROOT`에 대한 설정을 추가합니다. 디렉토리 이름(미디어)을 `BASE_DIR`에 추가합니다:

```
MEDIA_ROOT = BASE_DIR / "media"
```

6. 5단계에서 추가한 줄 바로 아래에 MEDIA_URL에 대한 또 다른 설정을 추가합니다. 이것은 단순히 `"/media/"`이어야 합니다:

```
MEDIA_URL = "/media/"
```

이러한 변경이 이루어지면 settings.py는 이 책의 GitHub 리포지토리의 Chapter08 폴더에 있는 파일과 같아야 합니다.

7. media_project 패키지의 urls.py 파일을 엽니다. urlpatterns 정의 후, DEBUG 모드에서 실행 중일 경우 미디어 제공 URL을 추가하기 위해 다음 코드를 추가합니다. 먼저, urlpatterns 정의 위에 강조된 import 라인을 추가하여 Django 설정 및 정적 제공 뷰를 가져와야 합니다:

```
from django.contrib import admin
from django.urls import path
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path("admin/", admin.site.urls),
],
```

8. 그런 다음, `urlpatterns` 정의 바로 뒤에 다음 `idx_2ea7f765` 코드를 추가합니다 (코드 참조).
이전 단계에서 차단) `MEDIA_URL` 설정에서 `aidx_d8d35e48` 매핑을 조건부로 추가합니다.
정적 뷰에, 이는 `MEDIA_ROOT`에서 제공됩니다:

```
settings.DEBUG인 경우:
    urlpatterns += static(settings.MEDIA_URL,
        document_root=settings.MEDIA_ROOT)
```

You can now save this file.

9. Start the Django dev server, if it is not already running, then visit `http://127.0.0.1:8000/media/test.txt`. If you did everything correctly, you should see the **Hello world!** text in your browser:

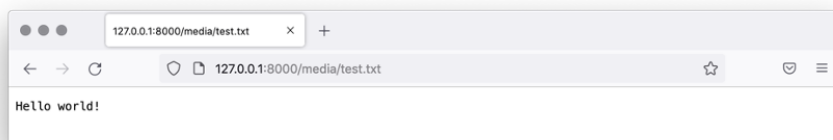


Figure 8.3: Serving a media file

If your browser looks like *Figure 8.3*, it means that the media files are being served from the `MEDIA_ROOT` directory.

The `test.txt` file we created was just for testing, but we will use it in *Exercise 8.02 – template settings and using `MEDIA_URL` in templates*, so don't delete it yet.

In this exercise, we configured Django to serve media files. We served a test file just to make sure everything worked as expected, and it did. We'll now look at how we can automatically generate media URLs in templates.

Context processors and using `MEDIA_URL` in templates

To use `MEDIA_URL` in a template, we could pass it in through the rendering context dictionary in our view. An example is shown in the following code block:

```
from django.conf import settings

def my_view(request):
    return render(request, "template.html",
                  {"MEDIA_URL": settings.MEDIA_URL,
                   "username": "jbloggs"})
```

This will work, but the problem is that `MEDIA_URL` is a common variable that we might want to use in many places, so we'd have to pass it through in practically every view.

Instead, we can use a **context processor**, which is a way of adding one or more variables automatically to the context dictionary on every render call.

이제 이 파일을 저장할 수 있습니다.

9. Django 개발 서버를 시작하세요. 이미 실행 중이지 않다면

`http://127.0.0.1:8000/media/test.txt`를 방문하세요. 모든 것을 올바르게 수행했다면 브라우저에서 Hello world! 텍스트를 볼 수 있어야 합니다:

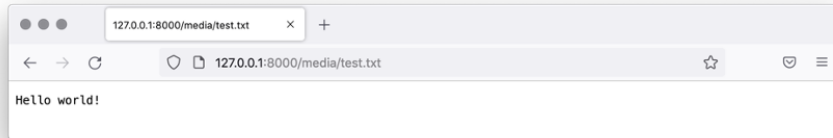


그림 8.3: 미디어 파일 제공

브라우저 idx_3046ee01가 그림 8.3처럼 보인다면, 이는 미디어 파일이 MEDIA_ROOT 디렉토리에서 제공되고 있다는 의미입니다.

우리가 만든 test.txt 파일은 테스트용이었지만, 우리는 이를 연습 8.02 – 템플릿 설정 및 템플릿에서 MEDIA_URL 사용에 사용할 것입니다. 그러니 아직 삭제하지 마세요.

이번 연습에서는 Django가 미디어 파일을 제공하도록 구성했습니다. 모든 것이 예상대로 작동하는지 확인하기 위해 테스트 파일을 제공했으며, 잘 작동했습니다. 이제 템플릿에서 미디어 URL을 자동으로 생성하는 방법을 살펴보겠습니다.

컨텍스트 프로세서 및 템플릿에서 MEDIA_URL 사용

템플릿에서 MEDIA_URL을 사용하려면, weidx_ad93dade 뷰의 렌더링 컨텍스트 사전에 통해 전달할 수 있습니다. 다음 코드 블록에 예가 나와 있습니다:

```
from django.conf import settings

def my_view(request):
    return render(request, "template.html",
                  {"MEDIA_URL": settings.MEDIA_URL,
                   "username": "jbloggs"})
```

이것은 작동하지만, 문제는 MEDIA_URL이 우리가 여러 곳에서 사용하고 싶어하는 일반 변수라는 것입니다. 그래서 사실상 모든 뷰에서 이를 전달해야 합니다.

대신, 우리는 컨텍스트 프로세서를 사용할 수 있습니다. 이는 모든 렌더 호출에서 컨텍스트 사전에 하나 이상의 변수를 자동으로 추가하는 방법입니다.

A context processor is a function that accepts one argument, the current request. It returns a dictionary of context information that will be merged with the dictionary that was passed to the render call.

We can look at the source code of the `media` context processor, which illustrates how it works:

```
def media(request):  
    """  
    Add media-related context variables to the context.  
    """  
    return {"MEDIA_URL": settings.MEDIA_URL}
```

With the `media` context processor activated, `MEDIA_URL` will be added to your context dictionaries. We could change our render call, seen previously, to this:

```
return render(request, "template.html",  
              {"username": "jbloggs"})
```

The same data would be sent to the template, as the context processor would add `MEDIA_URL`.

The full module path to the `media` context processor is

`django.template.context_processors.media`. Some examples of other context processors that Django provides are the following:

- `django.template.context_processors.debug`: This returns the `{"DEBUG": settings.DEBUG}` dictionary
- `django.template.context_processors.request`: This returns the `{"request": request}` dictionary; that is, it just adds the current HTTP request to the context

To enable a context processor, its module path must be added to the `context_processors` option of your `TEMPLATES` setting. For example, to enable the `media` context processor, add `django.template.context_processors.media`. We will cover how to do this in detail in *Exercise 8.02 – template settings and using `MEDIA_URL` in templates*.

Once the `media` context processor is enabled, the `MEDIA_URL` variable can be accessed inside a template just like a normal variable:

```
{{ MEDIA_URL }}
```

컨텍스트 프로세서는 현재 요청이라는 하나의 인수를 받는 함수입니다. 이는 렌더 호출에 전달된 사전과 병합될 컨텍스트 정보의 사전을 반환합니다.

media 컨텍스트 프로세서의 소스 코드를 살펴볼 수 있으며, 이는 작동 방식을 보여줍니다:

```
def media(request):
    """
    미디어 관련 컨텍스트 변수를 컨텍스트에 추가합니다."""

    return {"MEDIA_URL": settings.MEDIA_URL}
```

미디어 컨텍스트 프로세서가 활성화되면 MEDIA_URL이 컨텍스트 사전에 추가됩니다. 이전에 보았던 렌더 호출을 다음과 같이 변경할 수 있습니다:

```
return render(request, "template.html",
              {"username": "jbloggs"})
```

같은 데이터가 템플릿으로 전송되며, 컨텍스트 프로세서가 MEDIA_URL을 추가합니다.

미디어 컨텍스트 프로세서의 전체 모듈 경로는

django.template.context_processors.media입니다. Django가 제공하는 다른 컨텍스트 프로세서의 몇 가지 예는 다음과 같습니다:

- django.template.context_processors.debug: 이것은 {"DEBUG": settings.DEBUG} 사전을 반환합니다.
django.template.context_processors.request: 이것은 {"request": request} 사전을 반환합니다. 즉, 현재 HTTP 요청을 컨텍스트에 추가합니다. 컨텍스트 프로세서를 활성화하려면 해당 모듈 경로를 TEMPLATES 설정의 context_processors 옵션에 추가해야 합니다. 예를 들어, 미디어 컨텍스트 프로세서를 활성화하려면 django.template.context_processors.media를 추가합니다. 우리는 Exercise 8.02 - 템플릿 설정 및 템플릿에서 MEDIA_URL 사용에 대해 자세히 다룰 것입니다.

미디어 컨텍스트 프로세서가 활성화되면, MEDIA_URL 변수를 템플릿 내에서 일반 변수처럼 접근할 수 있습니다:

```
{{ MEDIA_URL }}
```

You could use it, for example, to source an image:

```

```

Note that, unlike with static files, there is no template tag for loading media files (i.e., there is no equivalent to the `{% static %}` template tag).

Custom context processors can also be written. For example, referring back to the Bookr application that we built, we might want to show a list of the five latest reviews in a sidebar that's on every page. A context processor like this would perform the following code:

```
from reviews.models import Review

def latest_reviews(request):
    return {"latest_reviews": Review.objects.order_by(
        "-date_created")[:5]}.
```

This would be saved in a file named `context_processors.py` in the Bookr project directory, then referred to in the `context_processors` setting by its module path, `context_processors.latest_reviews`. Or we could save it inside the reviews app and refer to it as `reviews.context_processors.latest_reviews`. It is up to you to decide whether a context processor should be considered project-wide or app-specific. However, bear in mind that regardless of where it is stored, once activated, it applies to all the render calls for all apps.

A context processor can return a dictionary with multiple items or even zero items. It would do this if it had conditions to only add items if certain criteria were met. For example, show the latest reviews only if the user is logged in.

Let us explore this in detail in the next exercise.

예를 들어 이미지를 소스하는 데 사용할 수 있습니다:

```

```

idx_391a8ac3정적 파일과 달리 미디어 파일을 로드하는 템플릿 태그가 없다는 점에 유의하십시오 (즉, {% static %} 템플릿 태그에 해당하는 것이 없습니다).

사용자 정의 컨텍스트 프로세서 idx_0a88f88b도 작성할 수 있습니다. 예를 들어, 우리가 만든 Bookr 애플리케이션을 다시 참조하면, 모든 페이지에 있는 사이드바에 최신 리뷰 다섯 개의 목록을 표시하고 싶을 수 있습니다. 이러한 컨텍스트 프로세서는 다음 코드를 수행합니다:

```
from reviews.models import Review

def latest_reviews(request):
    return {"latest_reviews": Review.objects.order_by(
        "-date_created")[:5]}.
```

이것은 Bookr 프로젝트 디렉토리의 context_processors.py라는 파일에 저장되며, context_processors 설정에서 모듈 경로인 context_processors.latest_reviews로 참조됩니다. 또는 reviews 앱 안에 저장하고 reviews.context_processors.latest_reviews로 참조할 수 있습니다. 컨텍스트 프로세서가 프로젝트 전체에 적용될지 앱 특정으로 적용될지는 여러분이 결정할 수 있습니다. 그러나 저장 위치와 관계없이 활성화되면 모든 앱의 모든 렌더 호출에 적용된다는 점을 명심하세요.

컨텍스트 프로세서는 여러 항목이 포함된 사전이나 심지어 항목이 없는 사전을 반환할 수 있습니다. 특정 기준이 충족될 경우에만 항목을 추가하도록 조건이 있다면 그렇게 할 것입니다. 예를 들어, 사용자가 로그인한 경우에만 최신 리뷰를 표시합니다.

다음 연습에서 이것을 자세히 살펴보겠습니다.

Exercise 8.02 — template settings and using MEDIA_URL in templates

In this exercise, you will continue with `media_project` and configure Django to automatically add the `MEDIA_URL` setting to every template. You do this by adding `django.template.context_processors.media` to the `TEMPLATES` `context_processors` setting. You'll then add a template that uses this new variable and an example view to render it. You will make changes to the view and template throughout the exercises in this chapter:

1. In PyCharm, open `settings.py`. First, you'll need to add `media_example` to the `INSTALLED_APPS` setting since it wasn't done when the project was set up:

```
INSTALLED_APPS = [  
    # other apps truncated for brevity  
    "media_example",  
]
```

2. About halfway down the file, you'll find the `TEMPLATES` setting, which is a dictionary. Inside it is the `OPTIONS` item (another dictionary). Inside `OPTIONS` is the `context_processors` setting. Add the following to the end of this list:

```
"django.template.context_processors.media"
```

The full list should look like this:

```
TEMPLATES = [{  
    "BACKEND":  
        "django.template.backends.django.DjangoTemplates",  
    "DIRS": [],  
    "APP_DIRS": True,  
    "OPTIONS": {  
        "context_processors": [  
            "django.template.context_processors.debug",  
            "django.template.context_processors.request",  
            "django.contrib.auth.context_processors.auth",  
            "django.contrib.messages.context_processors.messages",  
            "django.template.context_processors.media",  
        ],  
    },  
}]
```

연습 8.02 – 템플릿 설정 및 MEDIA_URL 사용하기

이 연습에서는 media_project를 계속 진행하고 Django를 구성하여 MEDIA_URL 설정을 모든 템플릿에 자동으로 추가합니다. 이를 위해 django.template.context_processors.media를 TEMPLATES context_processors 설정에 추가합니다. 그런 다음 이 새로운 변수를 사용하는 템플릿과 이를 렌더링할 예제 뷰를 추가합니다. 이 장의 연습을 진행하면서 뷰와 템플릿에 변경을 가할 것입니다:

1. PyCharm에서 settings.py를 엽니다. 먼저, 프로젝트 설정 시 media_example이 추가되지 않았으므로 INSTALLED_APPS 설정에 media_example을 추가해야 합니다:

```
INSTALLED_APPS = [
    # 다른 앱은 간결성을 위해 생략됨
    "media_example",
]
```

2. 파일의 중간쯤에 TEMPLATES 설정이 있으며, 이는 사전입니다. 그 안에는 OPTIONS 항목(또 다른 사전)이 있습니다. OPTIONS 안에는 context_processors 설정이 있습니다. 다음 내용을 이 목록의 끝에 추가하세요:

```
"django.template.context_processors.media"
```

전체 목록은 다음과 같아야 합니다:

```
TEMPLATES = [{
    "BACKEND":
        "django.template.backends.django.DjangoTemplates",
    "DIRS": [],
    "APP_DIRS": True,
    "OPTIONS": {
        "context_processors": [
            "django.template.context_processors.debug",
            "django.template.context_processors.request",
            "django.contrib.auth.context_processors.auth",
            "django.contrib.messages.context_processors.messages",
            "django.template.context_processors.media",
        ],
    },
}]
```

```
    },  
    },]
```

3. Open the `media_example` app's `views.py` file and create a new view called `media_example`. For now, it can just render a template named `media-example.html` (you will create this in step 5). The entire code of the view function is like this:

```
def media_example(request):  
    return render(request, "media-example.html")
```

4. Save `views.py`. You need a URL mapping to the `media_example` view. Open the `media_project` package's `urls.py` file.
First, import `media_example.views` with the other imports in the file:

```
import media_example.views
```

Then add `path` into `urlpatterns` to map `media-example/` to the `media_example` view:

```
path('media-example/', media_example.views.media_example)
```

Your full `urlpatterns` should look like this code block:

```
from django.conf.urls.static import static  
  
import media_example.views  
  
urlpatterns = [  
    path("admin/", admin.site.urls),  
    path("media-example/",  
        media_example.views.media_example),  
]  
  
if settings.DEBUG:  
    urlpatterns += static(settings.MEDIA_URL,  
                           document_root=settings.MEDIA_ROOT)
```

You can save and close the file.

```
    },
    },
    ...
}
```

3. `media_example` 앱의 `views.py` 파일을 열고 새로운 뷰를 만드세요.
`media_example`. 지금은 `media-example.html`이라는 템플릿을 렌더링할 수 있습니다.
 (이것은 5단계에서 생성할 것입니다). 뷰 함수의 전체 코드는 다음과 같습니다:

```
def media_example(request):
    ...
    return render(request, "media-example.html")
```

4. `views.py`를 저장하세요. `media_example` 뷰에 대한 URL `idx_bb48b84f` mapping이 필요합니다. 다음을 열어보세요:
`media_project` 패키지의 `urls.py` 파일입니다.
 먼저, 파일의 다른 import와 함께 `media_example.views`를 import하세요:

```
import media_example.views
```

그런 다음 `urlpatterns`에 `path`를 추가하여 `media-example/`을 `media_example` 뷰에 매핑하세요:

```
path('media-example/', media_example.views.media_example)
```

전체 `urlpatterns`는 다음 코드 블록과 같아야 합니다:

```
from django.conf.urls.static import static

import media_example.views

urlpatterns = [
    path("admin/", admin.site.urls),
    path("media-example/",
        media_example.views.media_example),
]

if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL,
                          document_root=settings.MEDIA_ROOT)
```

파일을 저장하고 닫을 수 있습니다.

5. Create a templates directory inside the media_example app directory. Then, create a new HTML file inside the media_project project's templates directory. Select **HTML File** and name the file media-example.html.
6. The media-example.html file should open automatically. You are just going to add a link inside the file to the test.txt file you created in *Exercise 8.01 – configuring media storage and serving*. Inside the <body> element, add the highlighted code:

```
<body>  
    <a href="{ { MEDIA_URL }}test.txt">Test Text  
    File</a>  
</body>
```

7. Note that there is no / between MEDIA_URL and the filename – this is because we already added a trailing slash when we defined it in settings.py. You can save the file. Start the Django dev server if it's not already running, then visit `http://127.0.0.1:8000/media-example/`. You should see a simple page like in *Figure 8.4*:

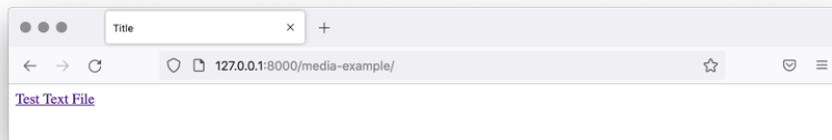


Figure 8.4: Basic media link page

If you click the link, you will be taken to the test.txt display and see the *Hello world!* text you created in *Exercise 8.01 – configuring media storage and serving* (Figure 8.3). This means you have configured the Django context_processors settings correctly.

We have finished with test.txt, so you can delete the file now. We will use the media_example view and template in the other exercises, so retain them. In the next section, we will talk about how to upload files using a web browser and how Django accesses them in a view.

File uploads using HTML forms

In *Chapter 6, Forms*, we learned about HTML forms. We discussed how to use the method attribute of <form> for the GET or POST requests. So far, we have only submitted text data using a form, but it is also possible to submit one or more files using a form.

5. media_example 앱 디렉토리 안에 templates 디렉토리를 생성합니다. 그런 다음 media_project 프로젝트의 templates 디렉토리 안에 새로운 HTMLid_x_2658f056 파일을 생성합니다. HTMLFile을 선택하고 파일 이름을 media-example.html로 지정합니다.
6. media-example.html 파일은 자동으로 열려야 합니다. 파일 안에 Exercise 8.01 – 미디어 저장소 구성 및 제공에서 생성한 test.txt 파일에 대한 링크를 추가할 것입니다. <body> 요소 안에 강조된 코드를 추가하세요:

```
<body>
    <a href="{{ MEDIA_URL }}"test.txt">테스트 텍스트
    파일</a>
</body>
```

7. MEDIA_URL과 파일 이름 사이에 /가 없다는 점에 유의하세요. 이는 settings.py에서 정의할 때 후행 슬래시를 이미 추가했기 때문입니다. 파일을 저장할 수 있습니다. Django devidx_6edc902e 서버가 이미 실행 중이지 않다면 시작한 후 http://127.0.0.1:8000/media-example/를 방문하세요. 그림 8.4와 같은 간단한 페이지를 볼 수 있어야 합니다:

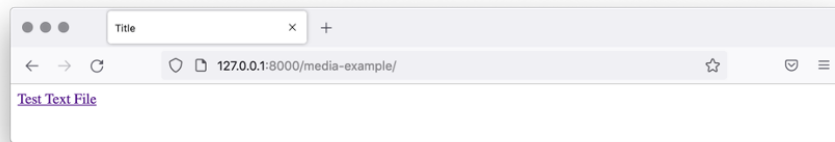


그림 8.4: 기본 미디어 링크 페이지

링크를 클릭하면 test.txt 디스플레이로 이동하게 되며, Exercise 8.01에서 생성한 Hello world! 텍스트를 볼 수 있습니다 – 미디어 저장소 구성 및 제공 (그림 8.3). 이는 Django context_processors 설정을 올바르게 구성했음을 의미합니다.

test.txt 작업을 마쳤으므로 이제 파일을 삭제할 수 있습니다. 다른 연습에서 media_example 뷰와 템플릿을 사용할 것이므로 유지하세요. 다음 섹션에서는 웹 브라우저를 사용하여 파일을 업로드하는 방법과 Django가 뷰에서 이를 액세스하는 방법에 대해 이야기하겠습니다.

HTML 양식을 사용한 파일 업로드

6장, 양식에서 우리는 idx_12042557 HTML 양식에 대해 배웠습니다. 우리는 <form>의 method 속성을 GET 또는 POST 요청에 사용하는 방법에 대해 논의했습니다. 지금까지 우리는 양식을 사용하여 텍스트 데이터만 제출했지만, 양식을 사용하여 하나 이상의 파일을 제출하는 것도 가능합니다.

When submitting files, we must ensure that there are at least two attributes on the form: method and enctype. You may still also need other attributes, such as action. A form that supports file uploads might look like this:

```
<form method="post" enctype="multipart/form-data">
```

File uploads are only available for POST requests. They are not possible with GET requests, as it would be impossible to send all the data for a file through a URL. The enctype attribute must be set to let the browser know it should send the form data as multiple parts, one part for the text data of the form and separate parts for each of the files that have been attached to the form. This encoding is seamless to the user; they don't know how the browser is encoding the form, and they do not need to do anything different.

To attach files to a form, you need to create an input of the file type. You can manually write the HTML code like this:

```
<input type="file" name="file-upload-name">
```

When the input is rendered in the browser, it looks like this when empty:

No file selected.

Figure 8.5: Empty file input

The title of the button might be different depending on your browser.

Clicking the **Browse...** button will display a *file open* dialog box:

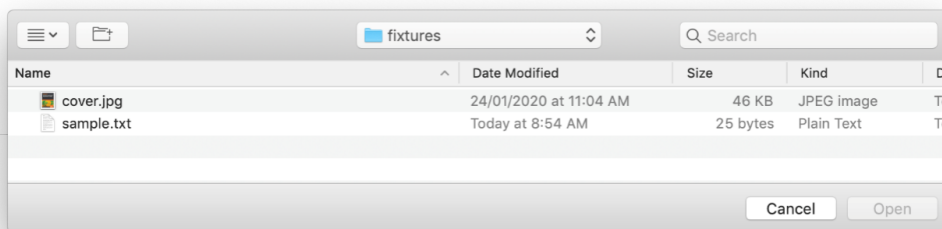


Figure 8.6: File browser on macOS

After selecting a file, the name of the file is shown in the field:

파일을 제출할 때, 양식에 최소 두 개의 속성: method와 enctype이 있어야 합니다. action과 같은 다른 속성이 필요할 수도 있습니다. 파일 업로드를 지원하는 양식은 다음과 같을 수 있습니다:

```
<form method="post" enctype="multipart/form-data">
```

파일 업로드는 POST 요청에서만 가능합니다. GET 요청에서는 URL을 통해 파일의 모든 데이터를 전송하는 것이 불가능하기 때문에 불가능합니다. enctype 속성은 브라우저에 양식 데이터를 여러 부분으로 전송해야 함을 알리기 위해 설정되어야 하며, 하나의 부분은 양식의 텍스트 데이터이고, 각 파일에 대해 별도의 부분이 있어야 합니다. 이 인코딩은 사용자에게는 매끄럽게 진행되며, 그들은 브라우저가 양식을 어떻게 인코딩하는지 알지 못하고, 다른 작업을 할 필요도 없습니다.

양식에 파일을 첨부하려면 파일 유형의 입력을 생성해야 합니다. HTML 코드를 수동으로 다음과 같이 작성할 수 있습니다:

```
<input type="file" name="file-upload-name">
```

입력이 브라우저에서 렌더링될 때, 비어 있을 때는 다음과 같이 보입니다:

Browse... No file selected.

그림 8.5: 비어 있는 파일 입력

버튼의 제목은 브라우저에 따라 다를 수 있습니다. 찾아보기...

버튼을 클릭하면 파일 열기 대화 상자가 표시됩니다:

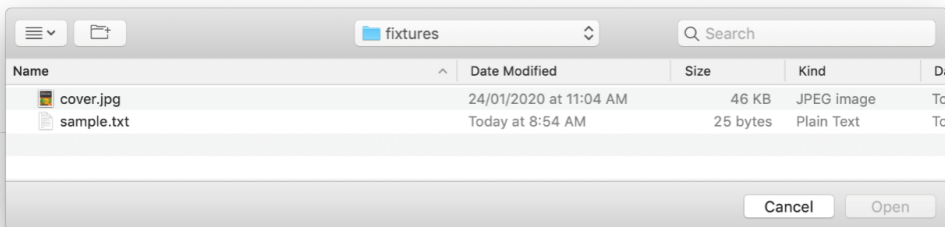


그림 8.6: macOS의 파일 브라우저

파일을 선택한 후, 파일의 이름이 필드에 표시됩니다:



Figure 8.7: File input with *cover.jpg* selected

Figure 8.7 shows a file input with a file named `cover.jpg` having been selected.

We briefly looked at how file upload fields look in the browser. Let's start looking at the Django side of the file upload process.

Working with uploaded files in a view

In addition to text data, if a form also contains file uploads, Django will populate the `request.FILES` attribute with uploaded files. The `request.FILES` attribute is a dictionary-like object that is keyed on the name attribute given to the file input.

In the form example in the previous section, the file input had the name `file-upload-name`. So, the file would be accessible in the view using `request.FILES["file-upload-name"]`.

The objects that `request.FILES` contains are file-like objects (specifically a `django.core.files.uploadedfile.UploadFile` instance), so to use them, you must read their data. For example, to get the content of an uploaded file in your view, you can write the following:

```
content = request.FILES["file-upload-name"].read()
```

A more common action is to write the file contents to disk. When files are uploaded, they are stored in a temporary location (in memory if they are under 2.5 MB, otherwise in a temporary file on disk). To store the file data in a known location, the contents must be read and then written to disk at that location. `UploadFile` has a `chunks` method that will read the file data one chunk at a time to prevent too much memory from being used by reading the entirety of the file at once.

So, instead of simply using the `read` and `write` functions, use the `chunks` method to only read small chunks of the file into memory at a time:

```
with open("/path/to/output.jpg", "wb+") as output_file:
    uploaded_file = request.FILES["file-upload-name"]
    for chunk in uploaded_file.chunks():
        output_file.write(chunk)
```



그림 8.7: cover.jpg가 선택된 파일 입력

그림 8.7은 cover.jpg라는 파일이 선택된 파일 입력을 보여줍니다.

우리는 브라우저에서 idx_277b4912 파일 업로드 필드가 어떻게 보이는지 간단히 살펴 보았습니다. 이제 파일 업로드 프로세스의 Django 측면을 살펴보겠습니다.

업로드된 파일을 뷰에서 작업하기

텍스트 데이터 외에도 양식에 파일 업로드가 포함된 경우 Django는 업로드된 파일로 request.FILES 속성을 채웁니다. request.FILES 속성은 파일 입력에 주어진 이름 속성을 키로 하는 사전과 유사한 객체입니다.

이전 섹션의 양식 예제에서 파일 입력의 이름은 file-upload-name이었습니다. 따라서 뷰에서 request.FILES["file-upload-name"]을 사용하여 파일에 접근할 수 있습니다.

request.FILES에 포함된 객체는 파일과 유사한 객체(특히 django.core.files.uploadedfile.UploadedFile 인스턴스)이므로 이를 사용하려면 데이터를 읽어야 합니다. 예를 들어, 뷰에서 업로드된 파일의 내용을 가져오려면 다음과 같이 작성할 수 있습니다:

```
content = request.FILES["file-upload-name"].read()
```

파일 내용을 디스크에 쓰는 것이 더 일반적인 작업입니다. 파일이 업로드되면 임시 위치에 저장됩니다(2.5MB 이하일 경우 메모리에, 그렇지 않으면 디스크의 임시 파일에 저장됨). 파일 데이터를 알려진 위치에 저장하려면 내용을 읽고 해당 위치에 디스크에 써야 합니다. UploadedFile에는 파일 데이터를 한 번에 한 청크씩 읽어 메모리 사용을 방지하는 chunks 메서드가 있습니다.

따라서 단순히 read 및 write 함수를 사용하는 대신, chunks 메서드를 사용하여 파일의 작은 청크만 메모리에 한 번에 읽도록 하십시오:

```
with open("/path/to/output.jpg", "wb+") as output_file:
    uploaded_file = request.FILES["file-upload-name"]
    for chunk in uploaded_file.chunks():
        output_file.write(chunk)
```

Note that in some of the upcoming examples, we will refer to this code as the `save_file_upload` function. Assume that the function is defined like this:

```
def save_file_upload(upload, save_path):  
    with open(save_path, "wb+") as output_file:  
        for chunk in upload.chunks():  
            output_file.write(chunk)
```

The previous example code could then be refactored to call the function:

```
uploaded_file = request.FILES["file-upload-name"]  
save_file_upload(uploaded_file, "/path/to/output.jpg")
```

Each `UploadedFile` object (the `uploaded_file` variable in the previous example code snippets) also contains extra metadata about the uploaded file, such as the file's name, size, and content type. The attributes you'll find most useful are the following:

- **size:** As the name suggests, this is the size of the uploaded file in bytes.
- **name:** This refers to the name of the uploaded file, for example, `image.jpg`, `file.txt`, `document.pdf`, and so on. This value is sent by the browser.
- **content_type:** The content type (MIME type) of the uploaded file, for example, `image/jpeg`, `text/plain`, `application/pdf`, and so on. Like `name`, this value is sent by the browser.
- **charset:** This refers to the charset or text encoding of the uploaded file for text files. This will be something such as UTF-8 or ASCII. Once again, this value is also determined and sent by the browser.

Here is a quick example of accessing these attributes (such as inside a view):

```
upload = request.FILES["file-upload-name"]  
size = upload.size  
name = upload.name  
content_type = upload.content_type  
charset = upload.charset
```

다음 예제들 중 일부에서 이 코드를 `thesave_file_upload` 함수라고 언급할 것입니다. 함수가 다음과 같이 정의되어 있다고 가정하십시오:

```
def save_file_upload(upload, save_path):
    with open(save_path, "wb+") as output_file:
        for chunk in upload.chunks():
            output_file.write(chunk)
```

이전 예제 코드는 함수를 호출하도록 리팩토링될 수 있습니다:

```
uploaded_file = request.FILES["file-upload-name"]
save_file_upload(uploaded_file, "/path/to/output.jpg")
```

각 `UploadedFile` 객체(이전 예제 코드 스니펫의 `uploaded_file` 변수)는 업로드된 파일에 대한 추가 메타데이터도 포함하고 있습니다. 예를 들어 파일의 이름, 크기 및 콘텐츠 유형 등이 있습니다. 가장 유용한 속성은 다음과 같습니다:

- `size`: 이름에서 알 수 있듯이, 이는 업로드된 파일의 크기를 바이트 단위로 나타냅니다.
- `name`: 이는 업로드된 파일의 이름을 나타내며, 예를 들어 `image.jpg`, `file.txt`, `document.pdf` 등이 있습니다. 이 값은 브라우저에 의해 전송됩니다.
- `content_type`: 업로드된 파일의 콘텐츠 유형(MIME 유형)으로, 예를 들어 `image/jpeg`, `text/plain`, `application/pdf` 등이 있습니다. `name`과 마찬가지로 이 값은 브라우저에 의해 전송됩니다.
- `charset`: 이는 텍스트 파일의 업로드된 파일의 문자 집합 또는 텍스트 인코딩을 나타냅니다. 이는 UTF-8 또는 ASCII와 같은 값이 될 것입니다. 다시 말해, 이 값도 브라우저에 의해 결정되고 전송됩니다.

다음은 이러한 속성에 접근하는 간단한 예입니다(예: 뷰 내부에서):

```
upload = request.FILES["file-upload-name"]
size = upload.size
name = upload.name
content_type = upload.content_type
charset = upload.charset
```

Security and trust of browsers' sent values

As we just described, `UploadedFile` values for `name`, `content_type`, and `charset` are determined by the browser. This is important to consider because a malicious user could send fake values in place of real ones to disguise the actual files being uploaded. Django does not automatically try to determine the content type or charset of the uploaded file, so it relies on the client to be accurate when it sends this information.

If we manually handle the saving of file uploads without suitable checks, then a scenario such as this could happen:

1. A user of the site uploads a malicious `malware.exe` executable but sends the `image/jpeg` content type.
2. Our code checks the content type and considers it to be safe, so it saves `malware.exe` to the `MEDIA_ROOT` file.
3. Another user of the site downloads what they think is a book cover image but is actually the `malware.exe` executable. They open the file, and their computer is infected with malware.

This scenario has been simplified; the malicious file would probably have a name that wasn't so obvious (maybe something such as `cover.jpg.exe`), but the general process has been illustrated.

How you choose to handle the security of your uploads will depend on the specific use case, but for most cases, these tips will help:

- When you save the file to disk, generate a name instead of using the one provided by the uploader. You should replace the file extension with what you expect. For example, if a file is named `cover.exe` but the content type is `image/jpeg`, save the file as `cover.jpg`. You could also generate a completely random filename for extra security.
- Check that the filename extension matches the content type. This method is not foolproof, as there are so many MIME types that if you are handling uncommon files, you might not get a match. The built-in Python `mimetypes` module can help you here. Its `guess_type` function takes a filename and returns a `mimetype` tuple (content type) and encoding. Here's a short snippet showing its use in a Python console:

```
>>> import mimetypes
>>> mimetypes.guess_type('file.jpg')
('image/jpeg', None)
>>> mimetypes.guess_type('text.html')
```

브라우저가 전송한 값의 보안 및 신뢰성

우리가 방금 설명한 바와 같이, UploadedFile의 name, content_type 및 charset 값은 브라우저에 의해 결정됩니다. 이는 악의적인 사용자가 실제 파일을 위장하기 위해 실제 값 대신 가짜 값을 보낼 수 있기 때문에 중요합니다. Django는 업로드된 파일의 콘텐츠 유형이나 문자 집합을 자동으로 결정하려고 하지 않으므로, 클라이언트가 이 정보를 보낼 때 정확해야 합니다.

적절한 검증 없이 파일 업로드 저장을 수동으로 처리하면 다음과 같은 시나리오가 발생할 수 있습니다:

1. 사이트의 사용자가 악성 malware.exe 실행 파일을 업로드하지만 image/jpeg 콘텐츠를 유형을 보냅니다.
2. 우리의 코드는 콘텐츠 유형을 확인하고 이를 안전하다고 간주하여 malware.exe를 MEDIA_ROOT 파일에 저장합니다.
3. 사이트의 또 다른 사용자가 책 표지 이미지라고 생각하는 파일을 다운로드하지만 실제로는 malware.exe 실행 파일입니다. 그들은 파일을 열고, 그들의 컴퓨터는 맬웨어에 감염됩니다.

이 시나리오는 단순화되었습니다; 악성 파일은 아마도 그렇게 명백하지 않은 이름을 가질 것입니다 (예: cover.jpg.exe와 같은 것일 수 있습니다), 하지만 일반적인 과정은 설명되었습니다.

업로드의 보안을 처리하는 방법은 특정 사용 사례에 따라 달라지겠지만, 대부분의 경우 이러한 팁이 도움이 될 것입니다:

- idx_e91f8751 파일을 디스크에 저장할 때, 업로더가 제공한 이름 대신 이름을 생성하세요. 파일 확장자는 예상하는 것으로 교체해야 합니다. 예를 들어, 파일 이름이 cover.exe이지만 콘텐츠 유형이 image/jpeg인 경우, 파일을 cover.jpg로 저장하세요. 추가 보안을 위해 완전히 무작위 파일 이름을 생성할 수도 있습니다.
- 파일 이름 확장자가 콘텐츠 유형과 일치하는지 확인하세요. 이 방법은 완벽하지 않으며, 드문 파일을 처리하는 경우 일치하지 않을 수 있는 많은 MIME 유형이 있습니다. 내장된 Python mimetypes 모듈이 여기에서 도움이 될 수 있습니다. 그 guess_type 함수는 파일 이름을 받아서 mimetype 튜플(콘텐츠 유형)과 인코딩을 반환합니다. 다음은 Python 콘솔에서의 사용을 보여주는 짧은 코드 조각입니다:

```
>>> import mimetypes>>>
mimetypes.guess_type('file.jpg')
('image/jpeg', None)>>>
mimetypes.guess_type('text.html')
```

```
('text/html', None)
>>> mimetypes.guess_type('unknownfile.abc')
(None, None)
>>> mimetypes.guess_type('archive.tar.gz')
('application/x-tar', 'gzip')
```

Either element of the tuple might be `None` if the type or encoding can't be guessed. Once imported into your file by doing `import mimetypes`, you can use it like this in your view function:

```
upload = request.FILES["file-upload-name"]
mimetype, encoding = mimetypes.guess_type(upload.name)
if mimetype != upload.content_type:
    raise TypeError("Mimetype doesn't match file extension.")
```

This method will work for common file types such as images, but as mentioned, many uncommon types may return `None` for `mimetype`.

- If you are expecting image uploads, use the Pillow library to try to open the uploaded file as an image. If it is not a valid image, then Pillow will be unable to open it. This is what Django does when using `ImageField` to upload images. We will show how to use this technique to open and manipulate an image in *Exercise 8.05 – image uploads using Django forms*.
- You can also consider the `python-magic` Python package, which examines the actual content of files to try to determine their type. It is installed using `pip`, and its GitHub project is found here: <https://github.com/ahupp/python-magic>. Once installed and imported into your file with `import magic`, you can use it like this in your view function:

```
upload = request.FILES["field_name"]
mimetype = magic.from_buffer(upload.read(2048), mime=True)
```

You can then verify that `mimetype` is in a list of allowed types.

This is not a definitive list of all the ways of protecting against malicious file uploads. The best approach will depend on what type of application you are building. You might build a site for hosting arbitrary files, in which case you wouldn't need any kind of content checking at all.

```
( 'text/html', None)>>> mimetypes.guess_type
( 'unknownfile.abc')(None, None)>>>
mimetypes.guess_type( 'archive.tar.gz')
( 'application/x-tar', 'gzip')
```

튜플의 어느 요소도 유형이나 인코딩을 추측할 수 없는 경우 None일 수 있습니다. `mimetypes`를 import한 후, 뷰 함수에서 다음과 같이 사용할 수 있습니다:

```
upload = request.FILES["file-upload-name"]
mimetype, encoding = mimetypes.guess_type(upload.name)
if mimetype != upload.content_type:
    raise TypeError("Mimetype doesn't match file extension.")
```

이 방법은 이미지와 같은 일반 파일 유형에 대해 작동하지만, 언급했듯이 많은 비정상적인 유형은 `mimetype`에 대해 None을 반환할 수 있습니다.

- `idx_303d493d` 이미지 업로드를 예상하는 경우, `Pillow` 라이브러리를 사용하여 업로드된 파일을 이미지로 열어보십시오. 유효한 이미지가 아닌 경우 `Pillow`는 이를 열 수 없습니다. 이것이 Django가 `ImageField`를 사용하여 이미지를 업로드할 때 수행하는 작업입니다. 우리는 이 기술을 사용하여 이미지를 열고 조작하는 방법을 연습 8.05 - Django 양식을 사용한 이미지 업로드에서 보여줄 것입니다.
- `python-magic` Python 패키지를 고려할 수도 있습니다. 이 패키지는 파일의 실제 내용을 검사하여 유형을 결정하려고 시도합니다. `pip`를 사용하여 설치되며, GitHub 프로젝트는 여기에서 찾을 수 있습니다:
<https://github.com/ahupp/python-magic>. 설치한 후 `import magic`으로 파일에 가져오면, 뷰 함수에서 다음과 같이 사용할 수 있습니다:

```
upload = request.FILES["field_name"]
mimetype = magic.from_buffer(upload.read(2048), mime=True)
```

그런 다음 `mimetype`이 허용된 유형 목록에 있는지 확인할 수 있습니다.

이것은 악의적인 파일 업로드로부터 보호하는 모든 방법의 확정적인 목록이 아닙니다. 최선의 접근 방식은 구축 중인 애플리케이션의 유형에 따라 달라집니다. 임의의 파일을 호스팅하는 사이트를 구축하는 경우, 어떤 종류의 콘텐츠 확인도 필요하지 않을 수 있습니다.

Let us now see how we can build an HTML form and view that allows files to be uploaded. We will then store them inside the media directory and retrieve the downloaded files in our browser.

Exercise 8.03 – file upload and download

In this exercise, you will add a form with a file field to the `media-example.html` template. This will allow you to upload a file to the `media_example` view using your browser. You'll also update the `media_example` view to save the file to the `MEDIA_ROOT` directory so that it's available for download. You will then test that this all works by downloading the file again:

1. In PyCharm, open the `media-example.html` template located inside the `templates` folder. Inside the `<body>` element, remove the `<a>` link that was added in step 6 of *Exercise 8.02 – template settings and using MEDIA_URL in templates*. Replace it with a `<form>` element (highlighted in the following code snippet). Make sure the opening tag has `method="post"` and `enctype="multipart/form-data"`:

```
</head>
<body>
    <form method="post" enctype="multipart/form-data">
    </form>
</body>
```

2. Insert the `{% csrf_token %}` template tag inside the `<form>` body.
3. After `{% csrf_token %}`, add an `<input>` element, with `type="file"` and `name="file_upload"`:

```
<input type="file" name="file_upload">
```

4. Finally, before the closing `</form>` tag, add a `<button>` element with `type="submit"` and the Submit text content:

```
<button type="submit">Submit</button>
```

Your HTML body should now look like this:

```
<body>
    <form method="post" enctype="multipart/form-data">
        {% csrf_token %}
```

이제 HTML 양식과 파일 업로드를 허용하는 뷰를 어떻게 구축할 수 있는지 살펴보겠습니다. 그런 다음 미디어 디렉토리에 파일을 저장하고 브라우저에서 다운로드한 파일을 검색할 것입니다.

연습 8.03 – 파일 업로드 및 다운로드

이 연습에서는 media-example.html 템플릿에 파일 필드가 있는 양식을 추가합니다. 이를 통해 브라우저를 사용하여 media_example 뷰에 파일을 업로드할 수 있습니다. 또한 MEDIA_ROOT 디렉토리에 파일을 저장하도록 media_example 뷰를 업데이트하여 다운로드할 수 있도록 합니다. 그런 다음 파일을 다시 다운로드하여 모든 것이 작동하는지 테스트합니다:

1. PyCharm에서 templates 폴더에 있는 media-example.html 템플릿을 엽니다.
`<body>` 요소 안에서 연습 8.02 – 템플릿 설정 및 템플릿에서 MEDIA_URL 사용의 6단계에서 추가된 `<a>` 링크를 제거합니다. 다음 코드 스니펫에서 강조 표시된 `<form>` 요소로 교체합니다. 여는 태그에 `method="post"` 및 `enctype="multipart/form-data"`가 포함되어 있는지 확인합니다:

```
</head>
<body>
    <form method="post" enctype="multipart/form-data">
    </form>
</body>
```

2. `<form>` 본문 안에 `{% csrf_token %}` 템플릿 태그를 삽입합니다. `{% csrf_token %}` 다음에 `type="file"`과 `name="file_upload"`인 `<input>` 요소를 추가합니다:

```
<input type="file" name="file_upload">
```

4. 마지막으로, 닫는 `</form>` 태그 전에 `type="submit"`과 제출 텍스트 내용을 가진 `<button>` 요소를 추가합니다:

```
<button type="submit">제출</button>
```

이제 당신의 HTML 본문은 다음과 같아야 합니다:

```
<body>
    <form method="post" enctype="multipart/form-data">
        {% csrf_token %}
```

```
<input type="file" name="file_upload">
<button type="submit">Submit</button>
</form>
</body>
```

Now, save and close the file.

5. Open the `media_example` app's `views.py` file. Inside the `media_example` view, add code to save the uploaded file to the `MEDIA_ROOT` directory. For this, you need access to `MEDIA_ROOT` from the settings, so import the Django settings at the top of the file:

```
from django.conf import settings
```

6. The uploaded file should only be saved if the request method is POST. Inside the `media_example` view, add an if statement to validate that `request.method` is POST:

```
def media_example(request):
    if request.method == "POST":
        ...
```

7. Inside the if statement added in the previous step, generate the output path by joining the uploaded filename to the `MEDIA_ROOT` directory. Then, open this path in the `wb` mode and iterate over the uploaded file using the `chunks` method. Finally, write each chunk to the saved file:

```
def media_example(request):
    if request.method == 'POST':
        save_path = (settings.MEDIA_ROOT /
                     request.FILES["file_upload"].name)

        with open(save_path, "wb") as output_file:
            for chunk in request.FILES[
                "file_upload"].chunks():
                output_file.write(chunk)

        return render(request, "media-example.html")
```

Note that the uploaded file and its metadata are being accessed from the `request.FILES` dictionary using the key that matches the name given to the file input (in our case, this is `file_upload`). You can save and close `views.py`.

```

<input type="file" name="file_upload">
<button type="submit">제출</button>
</form>
</body>

```

이제, save andidx_bd2289cf 파일을 닫으세요.

5. media_example 앱의 views.py 파일을 엽니다. media_example 뷰 안에 idx_244b6e02codeto를 추가하여 업로드된 파일을 MEDIA_ROOT 디렉토리에 저장합니다. 이를 위해서는 settings에서 MEDIA_ROOT에 접근해야 하므로, 파일 상단에 Django 설정을 가져옵니다:

```
from django.conf import settings
```

6. 업로드된 파일은 요청 방식이 POST인 경우에만 저장되어야 합니다. 내부에 있는 경우 media_example 뷰에서 request.method가 POST인지 확인하는 if 문을 추가하세요:

```

def media_example(request):
    ...
    if request.method == "POST":
        ...

```

7. 이전 단계에서 추가한 if 문 안에서 출력 경로를 결합하여 생성합니다. 업로드된 파일 이름을 MEDIA_ROOT 디렉토리에 저장합니다. 그런 다음, 이 경로를 wb 모드로 열고 chunks 메서드를 사용하여 업로드된 파일을 반복 처리합니다. 마지막으로, 각 청크를 저장된 파일에 씁니다:

```

def media_example(request):
    ...
    if request.method == 'POST':
        save_path = (settings.MEDIA_ROOT
                     /request.FILES["
                     file_upload"].name)

        with open(save_path, "wb") as output_file:
            for chunk in request.FILES[
                "file_upload"].chunks
                ():output_file.write(chunk)

        return render(request, "media-example.html")

```

idx_4ca321db업로드된 파일과 해당 메타데이터는 파일 입력에 주어진 이름과 일치하는 키를 사용하여 request.FILES 사전에서 액세스되고 있습니다(우리 경우, 이것은 file_upload입니다). views.py를 저장하고 닫을 수 있습니다.

8. Start the Django dev server if it's not already running, then navigate to `http://127.0.0.1:8000/media-example/`. You should see the file upload field and the **Submit** button, as can be seen here:

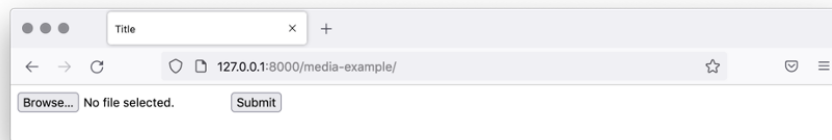


Figure 8.8: The file upload form

Click **Browse...** (or the equivalent on your browser) and select a file to upload. The name of the file will appear in the file input. Then, click **Submit**.

The page will reload, and the form will be empty again. This is normal – in the background, the file should have been saved.

9. Try to download the file you uploaded using `MEDIA_URL`. In this example, a file named `cover.jpg` was uploaded. It will be downloadable at `http://127.0.0.1:8000/media/cover.jpg`. Your URL will depend on the name of the file you uploaded:

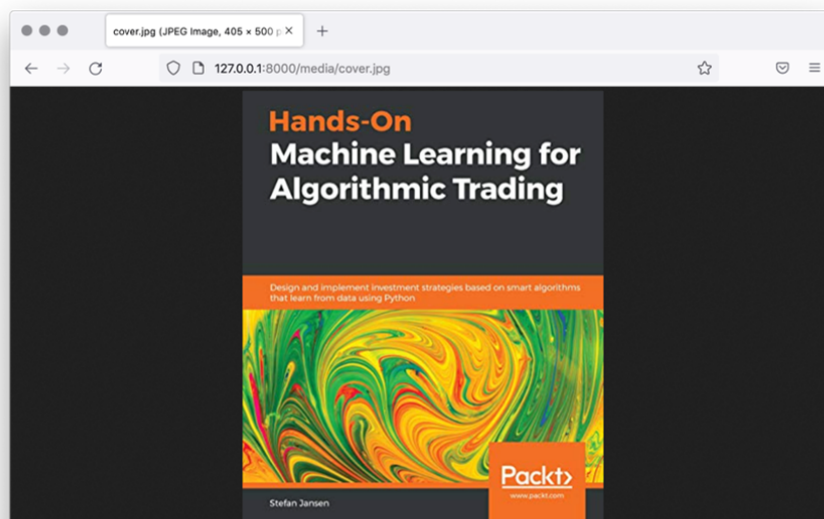


Figure 8.9: Uploaded file visible inside `MEDIA_URL`

8. Django 개발 서버가 이미 실행 중이지 않다면 시작하고,
<http://127.0.0.1:8000/media-example/>로 이동하세요. 여기에서 파일 업로드 필드와 제출 버튼을 볼 수 있어야 합니다:

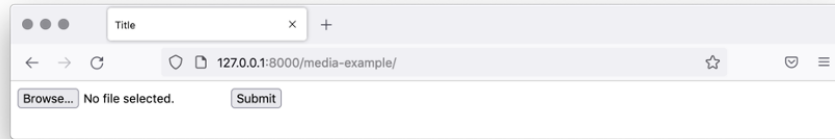


그림 8.8: 파일 업로드 양식

찾아보기... (또는 브라우저에서 해당하는 항목)을 클릭하고 업로드할 파일을 선택하세요. 파일의 이름이 파일 입력란에 나타납니다. 그런 다음, 제출을 클릭하세요.

페이지가 새로 고쳐지고 양식이 다시 비어 있을 것입니다. 이는 정상입니다 – 백그라운드에서 파일이 저장되었어야 합니다.

9. MEDIA_URL을 사용하여 업로드한 파일을 다운로드해 보세요. 이 예제에서는 cover.jpg 라는 이름의 파일이 업로드되었습니다. 이는 <http://127.0.0.1:8000/media/cover.jpg>에서 다운로드할 수 있습니다. 귀하의 URL은 업로드한 파일의 이름에 따라 달라집니다:

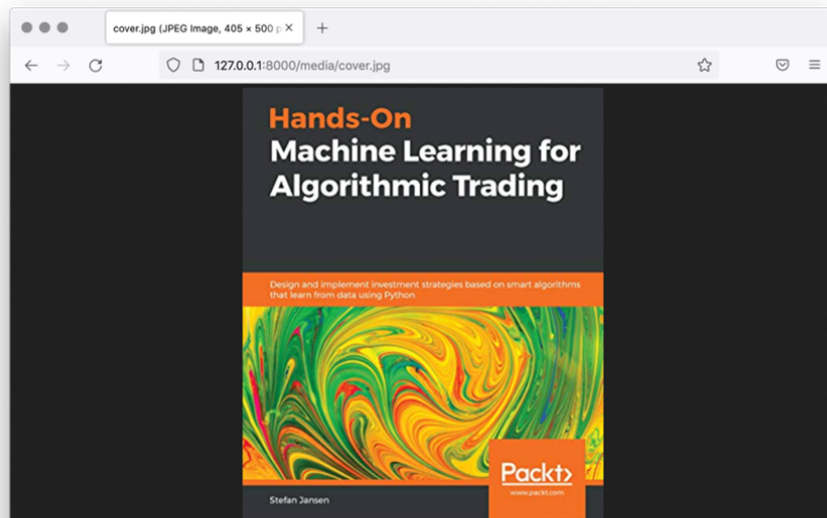


그림 8.9: MEDIA_URL 내에서 볼 수 있는 업로드된 파일

If you upload an image file, HTML file, or another type of file that your browser can display, you will be able to view it inside the browser. Otherwise, your browser will just download it to disk again. In both cases, it means the upload was successful.

You can also confirm that the upload was successful by looking inside the media directory in the `media_project` project directory:

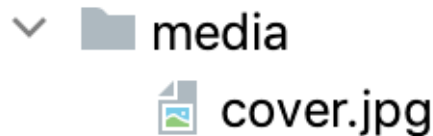


Figure 8.10: `cover.jpg` inside the media directory

Figure 8.10 shows the `cover.jpg` file inside the `media` directory in PyCharm.

In this exercise, you added an HTML form with `enctype` set to `multipart/form-data` so that it allowed file uploads. It contained a file input to select a file to upload. You then added a saving functionality to the `media_example` view to save the uploaded file to disk.

In the next section, we will look at how to simplify form generation and add validation using Django forms for file upload handling.

File uploads with Django forms

In *Chapter 6, Forms*, we saw how Django makes it easy to define forms and automatically render them to HTML. In the previous example, we defined our form manually and wrote the HTML. We can replace this with a Django form and implement the file input with a `FileField` constructor.

Here's how `FileField` is defined on a form:

```
from django import forms

class ExampleForm(forms.Form):
    file_upload = forms.FileField()
```

The `FileField` constructor can take the following keyword arguments:

- **Required:** This should be `True` for required fields and `False` if the field is optional

이미지 파일, HTML 파일 또는 브라우저가 표시할 수 있는 다른 유형의 파일을 업로드하면 idx_517bbc87브라우저 내에서 볼 수 있습니다. 그렇지 않으면 브라우저가 다시 idx_1cf42a2ddisk에 다운로드합니다. 두 경우 모두 업로드가 성공적이었다는 것을 의미합니다.

미디어_project 프로젝트 디렉토리의 미디어 디렉토리 안을 살펴보면 업로드가 성공적이었는지 확인할 수 있습니다:



그림 8.10: 미디어 디렉토리 안의 cover.jpg 그림
8.10은 PyCharm의 미디어 디렉토리 안에 있는 cover.jpg 파일을 보여줍니다.

이 연습에서는 파일 업로드를 허용하기 위해 enctype이 multipart/form-data로 설정된 HTML 양식을 추가했습니다. 업로드할 파일을 선택하기 위한 파일 입력이 포함되어 있었습니다. 그런 다음 업로드된 파일을 디스크에 저장하기 위해 media_example 뷰에 저장 기능 idx_3a7ba342를 추가했습니다.

다음 섹션에서는 파일 업로드 처리를 위해 Django 양식을 사용하여 양식 생성을 단순화하고 유효성 검사를 추가하는 방법을 살펴보겠습니다.

Django 양식으로 파일 업로드

6장, 양식에서 Django가 양식을 정의하고 자동으로 HTML로 렌더링하는 방법을 쉽게 만드는 것을 보았습니다. 이전 예제에서는 양식을 수동으로 정의하고 HTML을 작성했습니다. 이를 Django 양식으로 교체하고 FileField 생성자를 사용하여 파일 입력을 구현할 수 있습니다.

양식에서 FileField가 정의되는 방법은 다음과 같습니다:

```
from django import forms

class ExampleForm(forms.Form):
    file_upload = forms.FileField()
```

FileField 생성자는 다음과 같은 키워드 인수를 받을 수 있습니다:

- 필수: 필수 필드의 경우 True여야 하며, 필드가 선택 사항인 경우 False여야 합니다.

- `max_length`: This refers to the maximum length of the filename of the file being uploaded
- `allow_empty_file`: A field with this argument is considered to be valid even if the uploaded file is empty (has a size of 0)

Apart from these three keyword arguments, the constructor can also accept the standard `Field` arguments, such as `widget`. The default widget class for `FileField` is `ClearableFileInput`; this is a file input that can display a checkbox that can be checked to send a null value and clear the saved file on a `Model` field.

Using a form with a `FileField` constructor in a view is similar to other forms, but when the form has been submitted (i.e., `request.METHOD` is `POST`), then `request.FILES` should be passed into the form constructor as well. This is because Django needs to access `request.FILES` to find information about uploaded files when validating the form.

The basic flow in a view function is, therefore, similar to this:

```
def view(request):
    if request.method == "POST":
        # instantiate the form with POST data and files
        form = ExampleForm(request.POST, request.FILES)
        if form.is_valid():
            # process the form and save files
            form.save()
            return redirect("success-url")
    else:
        # instantiate an empty form as we've seen before
        form = ExampleForm()

    # render a template, the same as for other forms
    return render(request, "template.html", {"form": form})
```

When working with uploaded files and forms, you can interact with the uploaded files by accessing them through `request.FILES`, or through `form.cleaned_data`; the values will return to the same object. In the preceding example, we could process the uploaded file like this:

```
if form.is_valid():
    save_file_upload("/path/to/save.jpg", request.FILES["file_upload"])
    return redirect("/success-url/")
```

- `max_length`: 이는 업로드되는 파일의 파일 이름의 최대 길이를 나타냅니다
- `allow_empty_file`: 이 인수가 있는 필드는 업로드된 파일이 비어 있어도(크기가 0인 경우) 유효한 것으로 간주됩니다

이 세 가지 키워드 `idx_499ac678` 인수 외에도 생성자는 위젯과 같은 표준 필드 인수를 사용할 수 있습니다. `FileField`의 기본 위젯 클래스는 `ClearableFileInput`입니다. 이는 체크박스를 표시할 수 있는 파일 입력으로, 체크하여 `null` 값을 전송하고 모델 필드에 저장된 파일을 지울 수 있습니다.

`FileField` 생성자가 있는 양식을 `idx_4df341f7` 뷰에서 사용하는 것은 다른 양식과 유사하지만, 양식이 제출되면(즉, `request.METHOD`가 `POST`인 경우) `request.FILES`도 양식 생성자에 전달되어야 합니다. 이는 Django가 양식을 검증할 때 업로드된 파일에 대한 정보를 찾기 위해 `request.FILES`에 접근해야 하기 때문입니다.

따라서 뷰 함수의 기본 흐름은 다음과 유사합니다:

```
def view(request):
    if request.method == "POST":
        # POST 데이터와 파일로 양식 인스턴스화
        form = ExampleForm(request.POST, request.FILES)
        if form.is_valid():
            # 양식을 처리하고 파일을 저장
            form.save()
            return redirect("success-url")
    else:
        # 이전에 본 것처럼 빈 양식 인스턴스화
        form = ExampleForm()

    # 템플릿을 렌더링합니다. 다른 양식과 동일합니다.
    return render(request, "template.html", {"form": form})
```

업로드된 파일과 양식 작업 시, `request.FILES`를 통해 업로드된 파일에 접근하거나 `form.cleaned_data`를 통해 상호작용할 수 있습니다; 값은 동일한 객체로 반환됩니다. 앞의 예에서 우리는 `idx_48546442` 업로드된 파일을 다음과 같이 처리할 수 있습니다:

```
if form.is_valid():
    save_file_upload(
        "/path/to/save.jpg", request.FILES["file_upload"])
    return redirect("/success-url/")
```

Or, since they contain the same object, you can use `form.cleaned_data`:

```
if form.is_valid():
    save_file_upload("/path/to/save.jpg", form.cleaned_data["file_upload"])
    return redirect("/success-url/")
```

The data that is saved will be the same.

Note

In *Chapter 6, Forms*, you experimented with forms and submitted them with invalid values. When the page refreshed to show the form errors, the data that you had previously entered was populated when the page reloaded. This does not occur with file fields; instead, the user will have to navigate and select the file again if the form is invalid.

In the next exercise, we will put what we have seen with `FileField`s into practice by building an example form and then modifying our view to save the file only if the form is valid.

Exercise 8.04 – file uploads with a Django form

In the previous exercise, you created a form in HTML and used it to upload a file to a Django view. If you tried submitting the form without selecting a file, you would get a Django exception screen. You did not do any validation on the form, so this method is quite fragile.

In this exercise, you will create a Django form with `FileField`, which will allow you to use form validation functions to make the view more robust, as well as reducing the amount of code:

1. In PyCharm, inside the `media_example` app, create a new file named `forms.py`. It will open automatically. At the start of the file, import the Django forms library:

```
from django import forms
```

Then, create a `forms.Form` subclass, and name it `UploadForm`. Add to it a `FileField` field named `file_upload`. Your class should have this code:

```
class UploadForm(forms.Form):
    file_upload = forms.FileField()
```

You can save and close this file.

또는 동일한 객체를 포함하고 있으므로 `form.cleaned_data`를 사용할 수 있습니다:

```
if form.is_valid(): save_file_upload("
    /path/to/save.jpg", form.cleaned_data["
    file_upload"]) return redirect("/success-url/")
```

저장된 데이터는 동일할 것입니다.

참고

6장, 폼에서, 당신은 폼을 실험하고 유효하지 않은 값으로 제출했습니다. 페이지가 새로 고쳐져 폼 오류를 보여줄 때, 이전에 입력한 데이터는 페이지가 다시 로드될 때 채워졌습니다. 그러나 파일 필드에서는 이러한 일이 발생하지 않습니다. 대신, 사용자는 폼이 유효하지 않은 경우 파일을 다시 탐색하고 선택해야 합니다.

다음 연습에서는 `FileFields`에서 본 내용을 실제로 적용하여 예제 폼을 만들고, 폼이 유효할 때만 파일을 저장하도록 뷰를 수정할 것입니다.

연습 8.04 – Django 폼을 사용한 파일 업로드

이전 연습에서, 당신은 HTML로 폼을 만들고 이를 사용하여 Django 뷰에 파일을 업로드했습니다. 파일을 선택하지 않고 폼을 제출하려고 하면 Django 예외 화면이 표시됩니다. 폼에 대한 유효성 검사를 수행하지 않았기 때문에 이 방법은 상당히 취약합니다.

이 연습에서는 `FormField`가 있는 Django 양식을 생성하여, 뷰를 더 견고하게 만들고 코드 양을 줄이기 위해 양식 유효성 검사 함수를 사용할 수 있습니다:

1. PyCharm에서 `media_example` 앱 안에 `forms.py`라는 새 파일을 생성합니다. 파일이 자동으로 열립니다. 파일의 시작 부분에 Django 양식 라이브러리를 가져옵니다:

```
from django import forms
```

그런 다음, `forms.Form` 서브클래스를 생성하고 이름을 `UploadForm`으로 지정합니다. `file_upload`라는 이름의 `FormField` 필드를 추가합니다. 클래스는 다음과 같은 코드를 가져야 합니다:

```
class UploadForm(forms.Form):
    file_upload = forms.FileField()
```

이 파일을 저장하고 닫을 수 있습니다.

2. Open the `form_example` app's `views.py` file. At the start of the file, right below the existing import statements, you will need to import your new class like this:

```
from .forms import UploadForm
```

3. If you are in the POST branch of the view, `UploadForm` needs to be instantiated with both `request.POST` and `request.FILES`. If you don't pass in `request.FILES`, then the form instance will not be able to access the uploaded files. Under the `if request.method == "POST"` check, instantiate `UploadForm` with these two arguments:

```
form = UploadForm(request.POST, request.FILES)
```

4. The existing lines that define `save_path` and store the file contents can be retained, but they should be indented by one block and put inside a form validity check, so they are only executed if the form is valid. Add the `if form.is_valid():` line, and then indent the other lines, so the code looks like this:

```
if form.is_valid():
    save_path = os.path.join(settings.MEDIA_ROOT,
                             request.FILES["file_upload"].name)

    with open(save_path, "wb") as output_file:
        for chunk in request.FILES["file_upload"].chunks():
            output_file.write(chunk)
```

5. Since you are using a form now, you can access the file upload through the form. Replace usages of `request.FILES["file_upload"]` with `form.cleaned_data["file_upload"]`:

```
if form.is_valid():
    save_path = settings.MEDIA_ROOT /
    form.cleaned_data["file_upload"].name

    with open(save_path, "wb") as output_file:
        for chunk in form.cleaned_data["file_upload"].chunks():
            output_file.write(chunk)
```

2. `form_example` 앱의 `views.py` 파일을 엽니다. 파일의 시작 부분, 기존 `import` 문 바로 아래에 새로운 클래스를 다음과 같이 `import`해야 합니다:

```
from .forms import UploadForm
```

3. 뷰의 `POST` 분기에 있는 경우, `UploadForm`은 `request.POST`와 `request.FILES` 모두로 인스턴스화되어야 합니다. `request.FILES`를 전달하지 않으면 폼 인스턴스가 업로드된 파일에 접근할 수 없습니다. `if request.method == "POST"` 확인 아래에서 이 두 인수로 `UploadForm`을 인스턴스화합니다:

```
form = UploadForm(request.POST, request.FILES)
```

4. `save_path`를 정의하고 파일 내용을 저장하는 기존 줄은 유지할 수 있지만 그들은 `idx_f807b057`하나의 블록으로 들여쓰기를 하고 폼 유효성 검사 안에 넣어야 하므로, 그들은 `idx_58358327`폼이 유효할 때만 실행됩니다. `if form.is_valid()`: 줄을 추가하고, 그 다음에 다른 줄들을 들여쓰기 하여 코드가 다음과 같이 보이도록 합니다:

```
if form.is_valid():
    save_path = os.path.join(settings.MEDIA_ROOT,
                             request.FILES["file_upload"].name)

    with open(save_path, "wb") as output_file:
        for chunk in request.FILES["file_upload"].chunks():
            output_file.write(chunk)
```

5. 현재 양식을 사용하고 있으므로, 양식을 통해 파일 업로드에 접근할 수 있습니다. `request.FILES["file_upload"]`의 사용을 `form.cleaned_data["file_upload"]`로 교체하세요:

```
if form.is_valid():
    save_path = settings.MEDIA_ROOT /
    form.cleaned_data["file_upload"].name

    with open(save_path, "wb") as output_file:
        for chunk in form.cleaned_data["file_upload"].chunks():
            output_file.write(chunk)
```

6. Finally, add an else branch to handle non-POST requests, which simply instantiates a form without any arguments:

```
if request.method == "POST":
    ...
else:
    form = UploadForm()
```

7. Add a context dictionary argument to the render call and set the form variable in the form key:

```
return render(request, "media-example.html", {"form": form})
```

You can now save and close this file.

8. Finally, open the `media-example.html` template and remove your manually defined `<input>` file. Replace it with `form`, rendered using the `as_p` method (highlighted):

```
<body>
  <form method="post" enctype="multipart/form-data">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Submit</button>
  </form>
</body>
```

You should not change any other parts of the file. You can save and close this file.

9. Start the Django dev server if it's not already running, then navigate to `http://127.0.0.1:8000/media-example/`. You should see the file upload field and the **Submit** button as follows:

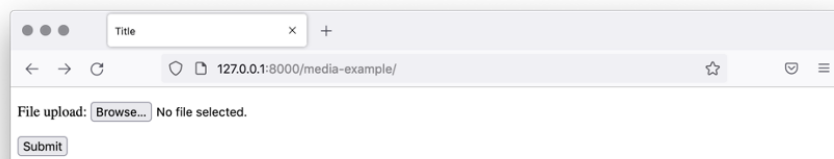


Figure 8.11: The file upload Django form rendered in the browser

6. 마지막으로, 단순히 인스턴스화하는 비-POST 요청을 처리하기 위해 else 분기 idx_8ba49366을 추가합니다.
 ...
 인수 없이 withoutidx_e6112e1c 양식:

```
if request.method == "POST":
    ...
    그렇지 않으면:
        form = UploadForm()
```

7. 렌더 호출에 컨텍스트 사전 인수를 추가하고 폼 변수를 설정합니다.
 폼 키:

```
return render(request, "media-example.html", {"form": form})
```

이제 이 파일을 저장하고 달을 수 있습니다.

8. 마지막으로, media-example.html 템플릿을 열고 수동으로 정의한 <input> 파일을 제거합니다. as_p 메서드를 사용하여 렌더링된 폼으로 교체합니다(강조 표시됨):

```
<body>
  <form method="post" enctype="multipart/form-data">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">제출</button>
  </form>
</body>
```

파일의 다른 부분(idx_184e8fdc)은 변경하지 않아야 합니다. 이 파일을 저장하고 달을 수 있습니다.

9. Django 개발 서버가 아직 실행 중이지 않다면 시작하고,
 http://127.0.0.1:8000/media-example/로 이동하세요. 파일 업로드 필드와 제출
 버튼이 다음과 같이 표시되어야 합니다:

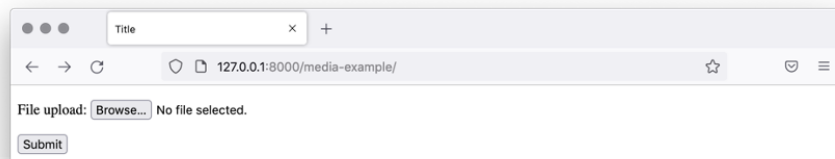


그림 8.11: 브라우저에 렌더링된 파일 업로드 Django 폼

10. Since we're using a Django form, we get its built-in validation automatically. If you try to submit the form without selecting a file, your browser should prevent it and show an error, as can be seen here:

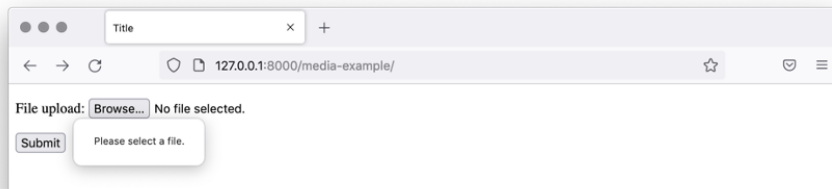


Figure 8.12: Form submission prevented by the browser

11. Finally, repeat the upload test that you performed in *Exercise 8.03 – file upload and download*, by selecting a file and submitting the form. You should then be able to retrieve the file using `MEDIA_URL`. In this case, a file named `cover.jpg` is being uploaded again (see the following screenshot):

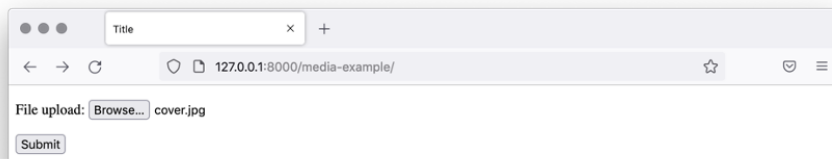


Figure 8.13: Uploading a file named `cover.jpg`

You can then retrieve the file at `http://127.0.0.1:8000/media/cover.jpg`, and you can see it in the browser as follows:

10. Django 폼을 사용하고 있으므로 내장된 유효성 검사를 자동으로 받을 수 있습니다. 파일을 선택하지 않고 폼을 제출하려고 하면 브라우저가 이를 방지하고 오류를 표시해야 합니다. 여기에서 볼 수 있습니다:

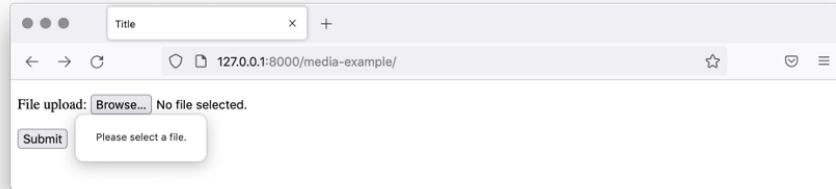


그림 8.12: 브라우저에 의해 제출이 방지된 폼

11. 마지막으로, 연습 8.03에서 수행한 업로드 테스트 idx_0e30bf86을 반복하세요 – 파일 업로드 및 다운로드, 파일을 선택하고 폼을 제출합니다. 그러면 idx_d995ee24를 사용하여 파일을 검색할 수 있어야 합니다. 이 경우 cover.jpg라는 파일이 다시 업로드되고 있습니다 (다음 스크린샷 참조):

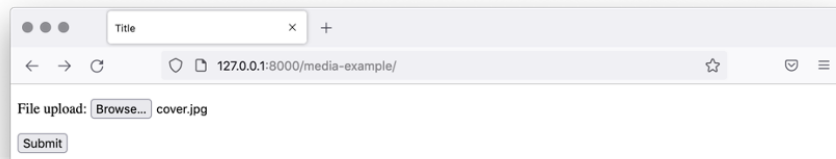


그림 8.13: cover.jpg라는 파일 업로드

그런 다음 <http://127.0.0.1:8000/media/cover.jpg>에서 파일을 검색할 수 있으며, 브라우저에서 다음과 같이 볼 수 있습니다:

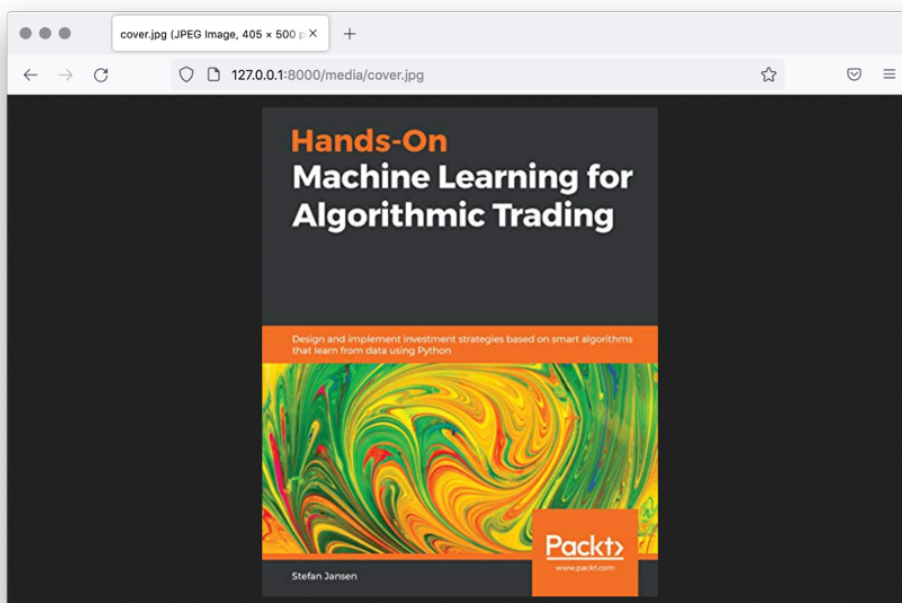


Figure 8.14: The file uploaded using the Django form is also visible in the browser

In this exercise, we replaced a manually built form with a Django form containing `FileField`. We instantiated the form in the view by passing in both `request.POST` and `request.FILES`. We then used the standard `is_valid` method to check the validity of the form and only saved the file upload if the form was valid. We tested the file uploading and saw that we were able to retrieve uploaded files using `MEDIA_URL`.

In the next section, we will look at `ImageField`, which is like `FileField` but specifically for images.

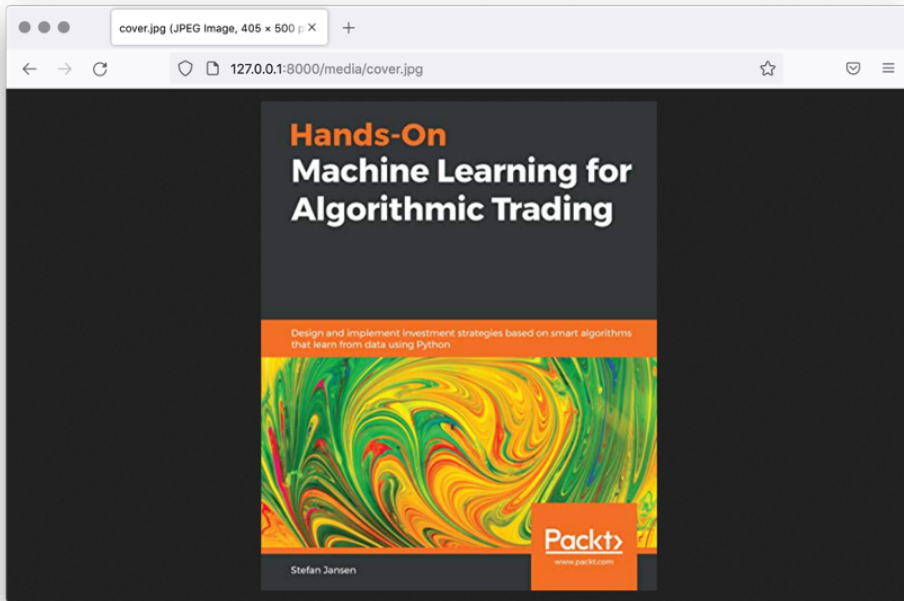


그림 8.14: Django 양식을 사용하여 업로드된 파일이 브라우저에서도 보입니다

이 연습에서는 수동으로 작성된 `aidx_d7fb3891` 양식을 `FileField`가 포함된 Django 양식으로 교체했습니다. 우리는 `request.POST`와 `request.FILES`를 모두 전달하여 뷰에서 양식을 인스턴스화했습니다. 그런 다음 표준 `is_valid` 메서드를 사용하여 양식의 유효성을 확인하고 양식이 유효한 경우에만 파일 업로드를 저장했습니다. 우리는 파일 업로드를 테스트했으며 `MEDIA_URL`을 사용하여 업로드된 파일을 검색할 수 있음을 확인했습니다.

다음 섹션에서는 `FileField`와 유사하지만 이미지 전용인 `ImageField`를 살펴보겠습니다.

Image uploads with Django forms

If you want to work with images in Python, the most common library that you'll use is called Pillow, and this is the library Django uses to validate images. The veteran Python developer Fredrik Lundh first released the **Python Imaging Library (PIL)** in 1995. It was not kept up to date, and eventually, a fork of the library was created and is still maintained, called Pillow. The original project has been discontinued, but development on a fork of the project called Pillow has continued. Pillow strives to maintain a high degree of compatibility with the original PIL package. For this reason, Pillow contains a very similar namespace to PIL, and the package is still called PIL when installed. For example, the Image object is imported from PIL as follows:

```
from PIL import Image
```

The terms *Python Imaging Library*, *PIL*, and *Pillow* are often used interchangeably. You can assume that if someone refers to PIL, they mean the latest Pillow library.

Pillow provides various methods of retrieving data about images or manipulating them. You can find out the width and height of images and scale, crop, and apply transformations to them. There are too many operations available to cover in this chapter, so we will just introduce a simple example (scaling an image), which you will use in the next exercise.

Since images are one of the most common types of files that a user may want to upload, Django also includes `ImageField`. This behaves similarly to `FileField` but also automatically validates that the data is an image file. This helps mitigate against security issues where we expect an image but the user uploads a malicious file.

An `UploadedFile` from an `ImageField` has all the same attributes and methods as `FileField` (size, `content_type`, `name`, `chunks()`, etc.) but adds an extra attribute: `image`. This is an instance of the PIL Image object that is used to verify that the file being uploaded is a valid image.

After checking that the form is valid, the underlying PIL Image object is closed. This is done to free up memory and prevent the Python process from holding too many files open, which could cause performance issues. What this means for the developer is that you can access some of the metadata about the image (such as width, height, and format), but you can't access the actual image data without reopening the image.

To illustrate, we'll have a form with `ImageField`, named `picture`:

```
class ExampleForm(forms.Form):  
    picture = forms.ImageField()
```

Django 폼을 이용한 이미지 업로드

Python에서 idx_fca76e14 이미지를 다루고 싶다면 가장 일반적으로 사용할 라이브러리는 idx_f09cd7baPillow라고 불리며, 이는 Django가 이미지를 검증하는 데 사용하는 라이브러리입니다. 베테랑 Python 개발자 Fredrik Lundh는 1995년에 Python Imaging Library (PIL)를 처음 출시했습니다. 이 라이브러리는 업데이트되지 않았고, 결국 idx_7e5644a9 라이브러리의 포크가 생성되어 현재도 유지 관리되고 있으며, Pillow라고 불립니다. 원래 프로젝트는 중단되었지만, Pillow라는 프로젝트의 포크에 대한 개발은 계속되고 있습니다. Pillow는 원래 PIL 패키지와 높은 호환성을 유지하기 위해 노력합니다. 이러한 이유로 Pillow는 PIL과 매우 유사한 네임스페이스를 포함하고 있으며, 설치 시 여전히 PIL이라고 불립니다. 예를 들어, Image 객체는 다음과 같이 PIL에서 가져옵니다:

```
from PIL import Image
```

Python Imaging Library, PIL 및 Pillow라는 용어는 종종 서로 바꿔 사용됩니다. 누군가 PIL을 언급하면 최신 Pillow 라이브러리를 의미한다고 가정할 수 있습니다.

Pillow는 이미지에 대한 데이터를 검색하거나 조작하는 다양한 방법을 제공합니다. 이미지의 너비와 높이를 확인하고, 크기를 조정하고, 자르고, 변환을 적용할 수 있습니다. 이 장에서 다루기에는 너무 많은 작업이 있으므로, 다음 연습에서 사용할 간단한 예제(이미지 크기 조정)를 소개하겠습니다.

이미지는 사용자가 업로드하고 싶어할 수 있는 가장 일반적인 파일 유형 중 하나이므로, Django는 ImageField도 포함합니다. 이는 FileField와 유사하게 작동하지만 데이터가 이미지 파일인지 자동으로 검증합니다. 이는 사용자가 악성 파일을 업로드할 때 이미지가 필요하다고 예상하는 보안 문제를 완화하는 데 도움이 됩니다.

ImageField에서의 UploadedFile은 FileField와 동일한 모든 속성과 메서드(size, content_type, name, chunks() 등)를 가지고 있지만 추가 속성인 image를 추가합니다. 이는 업로드되는 파일이 유효한 이미지인지 확인하는 데 사용되는 PIL Image 객체의 인스턴스입니다.

폼이 유효한지 확인한 후, 기본 PIL Image 객체는 닫힙니다. 이는 메모리를 확보하고 Python 프로세스가 너무 많은 파일을 열어두지 않도록 방지하기 위해 수행됩니다. 개발자에게는 이미지에 대한 일부 메타데이터(너비, 높이 및 형식 등)에 접근할 수 있지만, 이미지를 다시 열지 않고는 실제 이미지 데이터에 접근할 수 없다는 의미입니다.

예를 들어, 우리는 picture라는 이름의 ImageField가 있는 양식을 가질 것입니다:

```
class ExampleForm(forms.Form):
    picture = forms.ImageField()
```

Inside the view function, the picture field can be accessed in the form's `cleaned_data`:

```
if form.is_valid():  
    picture_field = form.cleaned_data["picture"]
```

Then, the picture field's Image object can be retrieved:

```
image = picture_field.image
```

Now that we have a reference to the image in the view, we can get some metadata:

```
w = image.width # an integer, e.g. 600  
h = image.height # also an integer, e.g. 420  
f = image.format # the format of the image as a string,  
# e.g. "PNG"
```

Django will also automatically update the `content_type` attribute of `UploadedFile` to the correct type for the picture field. This overwrites the value that the browser has sent when uploading the file.

Attempting to use a method that accesses the actual image data (rather than just the metadata) will cause an exception to be raised. This is because Django has already closed the underlying image file.

For example, the following code snippet will raise `AttributeError`:

```
image.getdata()
```

Instead, we need to reopen the image. The image data can be opened with the `ImageField` reference after importing the `Image` class:

```
from PIL import Image  
  
image = Image.open(picture_field)
```

Now that the image has been opened, you can perform operations on it. In the next section, we will look at a simple example – resizing the uploaded image.

뷰 함수 내부에서, 그림 필드는 폼의 `cleaned_data`에서 접근할 수 있습니다:

```
if form.is_valid():
    picture_field = form.cleaned_data["picture"]
```

그런 다음, 그림 필드의 이미지 객체를 가져올 수 있습니다:

```
image = picture_field.image
```

이제 뷰에서 이미지에 대한 참조가 있으므로, 메타데이터를 가져올 수 있습니다:

```
w = image.width # 정수, 예: 600
h = image.height # 또한 정수, 예: 420# 이미지의 형식을 문
f = image.format # 자일로,
# 예: "PNG"
```

Django는 또한 `UploadedFile`의 `content_type` 속성을 자동으로 올바른 유형으로 업데이트합니다. 이는 파일을 업로드할 때 브라우저가 보낸 값을 덮어씁니다.

실제 이미지 데이터에 접근하는 메서드를 사용하려고 하면(메타데이터만이 아니라) 예외가 발생합니다. 이는 Django가 이미 기본 이미지 파일을 닫았기 때문입니다.

예를 들어, 다음 코드 조각은 `AttributeError`를 발생시킵니다:

```
image.getdata()
```

대신 이미지를 다시 열어야 합니다. 이미지 데이터는 `Image` 클래스를 가져온 후 `ImageField` 참조로 열 수 있습니다:

```
from PIL import Image

image = Image.open(picture_field)
```

이제 이미지를 열었으므로, 그 위에서 작업을 수행할 수 있습니다. 다음 섹션에서는 업로드된 이미지를 크기 조정하는 간단한 예제를 살펴보겠습니다.

Resizing an image with Pillow

Pillow supports many operations that you might want to perform on an image before saving it. We can't explain them all in this book, so we will just use a common operation: resizing an image to a specific size before saving it. This will help us save storage space and improve download speed. For example, a user may upload large cover images in Bookr that are bigger than are needed for our purposes. When saving the file (writing it back to disk), we must specify the format to use. We could determine the type of image that was uploaded with a number of methods (such as checking `content_type` of the uploaded file or `format` from the `Image` object), but in our example, we'll always just save it as a JPEG file.

The `PIL Image` class has a `thumbnail` method that will resize an image to a maximum size while retaining the aspect ratio. For example, we could set a maximum size of 50 **pixels (px)** by 50 px. A 200 px by 100 px image would be resized to 50 px by 25 px; the aspect ratio is retained by setting the maximum dimension to 50 px. Each dimension is scaled by a factor of 0.25:

```
from PIL import Image

size = 50, 50 # a tuple of width, height to resize to
image = Image.open(image_field) # open the image as before
image.thumbnail(size) # perform the resize
```

At this point, the resize has been done in memory only. The change is not saved to disk until the `save` method is called, like so:

```
image.save("path/to/file.jpg")
```

The output format is automatically determined from the file extension used, in this case, JPEG. The `save` method can also take a `format` argument to override it, as in this example:

```
image.save("path/to/file.png", "JPEG")
```

Despite having the `.png` extension, the format is specified as JPEG, and so the output will be in JPEG format. As you might imagine, this can be very confusing, so you might decide to stick with specifying the extension only.

In the next exercise, we will change the `UploadForm` we have been working with to use `ImageField` instead of `FileField`, then implement resizing of an uploaded image before saving it to the media directory.

Pillow로 이미지 크기 조정하기

Pillow는 이미지를 저장하기 전에 수행할 수 있는 많은 작업을 지원합니다. 이 책에서 모든 작업을 설명할 수는 없으므로, 일반적인 작업인 특정 크기로 이미지를 크기 조정하는 작업을 사용하겠습니다. 이렇게 하면 저장 공간을 절약하고 다운로드 속도를 개선할 수 있습니다. 예를 들어, 사용자가 Bookr에 필요한 것보다 큰 대형 표지 이미지를 업로드할 수 있습니다. 파일을 저장할 때(디스크에 다시 쓸 때) 사용할 형식을 지정해야 합니다. 업로드된 파일의 content_type을 확인하거나 Image 객체에서 형식을 확인하는 등 여러 방법으로 업로드된 이미지의 유형을 결정할 수 있지만, 우리의 예에서는 항상 JPEG 파일로 저장하겠습니다.

PIL Image 클래스에는 비율을 유지하면서 이미지를 최대 크기로 조정하는 썸네일 메서드가 있습니다. 예를 들어, 최대 크기를 50픽셀(px) x 50픽셀로 설정할 수 있습니다. 200px x 100px 이미지는 50px x 25px로 크기 조정되며, 최대 치수를 50px로 설정하여 비율이 유지됩니다. 각 차원은 0.25의 비율로 조정됩니다:

```
from PIL import Image

size = 50, 50 # 크기 조정할 너비, 높이의 튜플
image = Image.open(image_field) # 이전과 같이 이미지를 엽니다
image.thumbnail(size) # 크기 조정을 수행합니다
```

이 시점에서 크기 조정은 메모리에서만 수행되었습니다. 변경 사항은 save 메서드가 호출될 때까지 디스크에 저장되지 않습니다. 다음과 같이:

```
image.save("path/to/file.jpg")
```

출력 형식은 사용된 파일 확장자에 따라 자동으로 결정됩니다. 이 경우 JPEG입니다. save 메서드는 형식 인수를 받아 이를 재정의할 수도 있습니다. 이 예제와 같이:

```
image.save("path/to/file.png", "JPEG")
```

확장자가 .png임에도 불구하고 형식이 JPEG로 지정되었으므로 출력은 JPEG 형식이 됩니다. 상상할 수 있듯이, 이는 매우 혼란스러울 수 있으므로 확장자만 지정하는 것이 좋습니다.

다음 연습에서는 우리가 작업해온 UploadForm을 FileField 대신 ImageField를 사용하도록 변경한 다음, 업로드된 이미지를 미디어 디렉토리에 저장하기 전에 크기 조정을 구현할 것입니다.

Exercise 8.05 – image uploads using Django forms

In this exercise, you will update the `UploadForm` class you created in *Exercise 8.04 – file uploads with a Django form*, to use `ImageField` instead of `FileField` (this will involve simply changing the field's class). You will then see that the form renders the same in the browser. Next, you will try uploading some non-image files and see how Django validates the form to disallow them. Finally, you will update your view to use PIL to resize the image before saving it, and then test it in action:

1. Open the `media_example` app's `forms.py` file. In the `UploadForm` class, change `file_upload` so that it's an instance of `ImageField` instead of `FileField`. After the update, `UploadForm` should look like this:

```
class UploadForm(forms.Form):  
    file_upload = forms.ImageField()
```

Save and close the file.

2. Start the Django dev server if it's not already running, then navigate to `http://127.0.0.1:8000/media-example/`. You should see the form rendered, and it will look identical to when we used `FileField` (see the following screenshot):

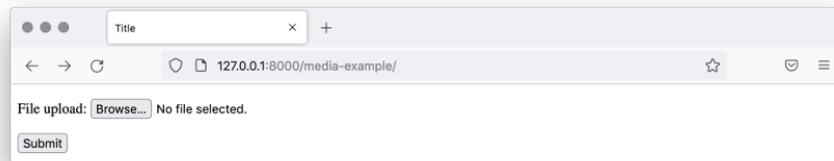


Figure 8.15: `ImageField` looks the same as `FileField`

3. You will notice the difference when you try to upload a non-image file. Click the **Browse...** button and try to select a non-image file. Depending on your browser or operating system, you might not be able to select anything other than an image file, as in *Figure 8.16*:

연습 8.05 – Django 폼을 사용한 이미지 업로드

이 연습에서는 연습 8.04 – Django 폼을 사용한 파일 업로드에서 생성한 UploadForm 클래스를 업데이트하여 FileField 대신 ImageField를 사용합니다(이는 필드의 클래스를 단순히 변경하는 것과 관련이 있습니다). 그런 다음 브라우저에서 폼이 동일하게 렌더링되는 것을 볼 수 있습니다. 다음으로, idx_91c4bfaduploading 일부 비이미지 파일 idx_32c11794를 시도하고 Django가 이를 허용하지 않도록 폼을 검증하는 방법을 확인합니다. 마지막으로, 이미지를 저장하기 전에 크기를 조정하기 위해 PIL을 사용하도록 뷰를 업데이트하고, 이를 실행해 보겠습니다:

1. media_example 앱의 forms.py 파일을 엽니다. UploadForm 클래스에서 file_upload를 FileField 대신 ImageField의 인스턴스가 되도록 변경합니다. 업데이트 후 UploadForm은 다음과 같아야 합니다:

```
class UploadForm(forms.Form):
    file_upload = forms.ImageField()
```

파일을 저장하고 닫습니다.

2. Django 개발 서버가 실행 중이지 않다면 시작한 후 <http://127.0.0.1:8000/media-example/>로 이동합니다. 폼이 렌더링된 것을 볼 수 있으며, FileField를 사용했을 때와 동일하게 보일 것입니다(다음 스크린샷 참조):

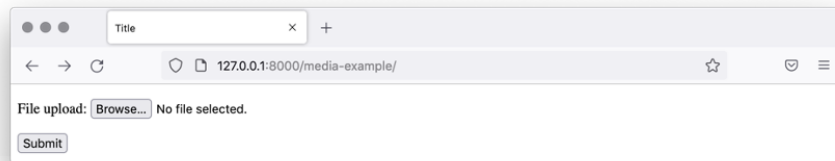


그림 8.15: ImageField는 FileField와 동일하게 보입니다.

3. 비이미지 파일을 업로드하려고 할 때 차이를 알 수 있습니다. Browse... 버튼을 클릭하고 비이미지 파일을 선택해 보세요. 브라우저나 운영 체제에 따라 이미지 파일 이외의 파일을 선택할 수 없을 수도 있습니다, 그림 8.16과 같이:

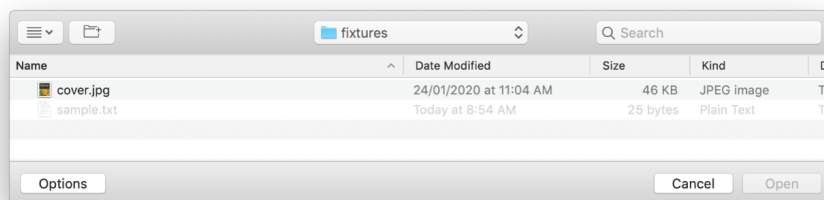


Figure 8.16: Only image files are selectable

Your browser may allow you to select an image but show an error in the form after selection. Or your browser may allow you to select a file and submit the form, and Django will raise `ValidationError`. Regardless, you can be sure that in your view, the form's `is_valid` view will only return `True` if an image has been uploaded.

Note

You don't need to test uploading a file at this point, as the result would be the same as in *Exercise 8.04 – file uploads with a Django form*.

4. The first thing you will need to do is make sure the Pillow library is installed. In a terminal (making sure your virtual environment has been activated), run the following command:

```
pip3 install pillow
```

In Windows, this is `pip install pillow`. You'll get output like *Figure 8.17*:

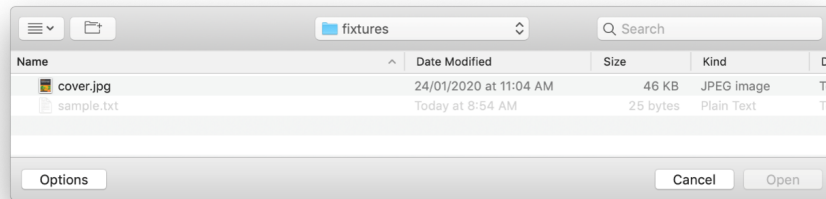


그림 8.16: 이미지 파일만 선택할 수 있습니다

브라우저에서 이미지를 선택할 수 있지만 선택 후 양식에서 오류가 발생할 수 있습니다. 또는 브라우저에서 파일을 선택하고 양식을 제출할 수 있으며, Django는 `ValidationError`를 발생시킵니다. 어쨌든, 뷰에서 `form`의 `is_valid` 뷰는 이미지가 업로드된 경우에만 `True`를 반환한다는 것을 확신할 수 있습니다.

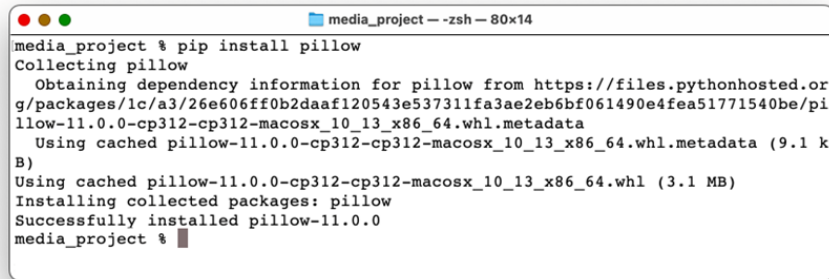
참고

이 시점에서 파일 업로드를 테스트할 필요는 없습니다. 결과는 연습 8.04와 동일할 것이기 때문입니다 - Django 양식으로 파일 업로드.

- 가장 먼저 해야 할 일은 Pillow 라이브러리가 설치되어 있는지 확인하는 것입니다. 터미널에서 (가상 환경이 활성화되어 있는지 확인한 후), 다음 명령을 실행하세요:

```
pip3 install pillow
```

Windows에서는 `pip install pillow`입니다. 그림 8.17과 같은 출력이 표시됩니다:

A terminal window titled 'media_project -- zsh -- 80x14' showing the command 'pip install pillow' being executed. The output shows that Pillow is being collected, dependency information is obtained from a URL, and the package is successfully installed. The terminal text is as follows:

```
media_project % pip install pillow
Collecting pillow
  Obtaining dependency information for pillow from https://files.pythonhosted.org/packages/1c/a3/26e606ff0b2daaf120543e537311fa3ae2eb6bf061490e4fea51771540be/pillow-11.0.0-cp312-cp312-macosx_10_13_x86_64.whl.metadata
    Using cached pillow-11.0.0-cp312-cp312-macosx_10_13_x86_64.whl.metadata (9.1 kB)
Using cached pillow-11.0.0-cp312-cp312-macosx_10_13_x86_64.whl (3.1 MB)
Installing collected packages: pillow
Successfully installed pillow-11.0.0
media_project %
```

Figure 8.17: pip3 installing Pillow

If Pillow is already installed, you'll see the Requirement already satisfied output message.

5. Now, we can update the `media_example` view to resize the image before saving it. Switch back to PyCharm and open the `media_example` app's `views.py` file, then import PIL's `Image` class by adding this import line near the top of the file:

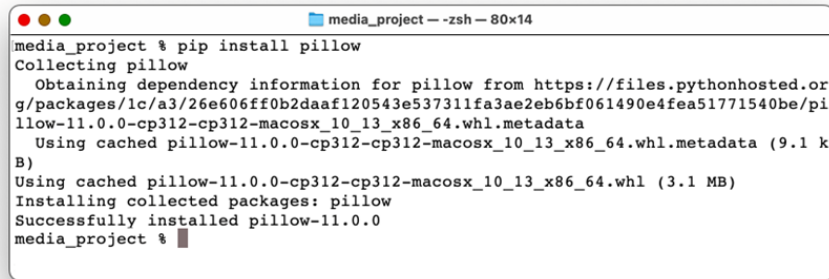
```
from PIL import Image
```

6. Go to the `media_example` view. Under the line that generates `save_path`, take out the three lines that open the output file, iterate over the uploaded file, and write out its chunks. Replace this with the code that opens the uploaded file with PIL, resizes it, and then saves it:

```
image = Image.open(form.cleaned_data["file_upload"])
image.thumbnail((50, 50))
image.save(save_path)
```

The first line creates an `Image` instance by opening the uploaded file, the next performs the thumbnail conversion (to a maximum size of 50 px by 50 px), and the third line saves the file to the same save path that we have been generating in previous exercises. You can save the file.

7. The Django dev server should still be running from step 2, but you should start it if it is not. Then, navigate to `http://127.0.0.1:8000/media-example/`. You'll see the



```
media_project % pip install pillow
Collecting pillow
  Obtaining dependency information for pillow from https://files.pythonhosted.org/packages/1c/a3/26e606ff0b2daaf120543e537311fa3ae2eb6bf061490e4fea51771540be/pillow-11.0.0-cp312-cp312-macosx_10_13_x86_64.whl.metadata
  Using cached pillow-11.0.0-cp312-cp312-macosx_10_13_x86_64.whl.metadata (9.1 kB)
  Using cached pillow-11.0.0-cp312-cp312-macosx_10_13_x86_64.whl (3.1 MB)
Installing collected packages: pillow
Successfully installed pillow-11.0.0
media_project %
```

그림 8.17: pip3로 Pillow 설치하기

Pillow가 이미 설치되어 있다면, Requirement already satisfied 출력 메시지를 볼 수 있습니다.

- 이제 이미지를 저장하기 전에 크기를 조정하기 위해 media_example 뷰를 업데이트할 수 있습니다. PyCharm으로 돌아가 media_example 앱의 views.py 파일을 열고, 파일 상단 근처에 이 import 문을 추가하여 PIL의 Image 클래스를 가져옵니다:

```
from PIL import Image
```

- media_example 뷰로 이동합니다. save_path를 생성하는 줄 아래에서, idx_c7cbde44 출력 파일을 열고, idx_0945a309 업로드된 파일을 반복하며 그 청크를 작성하는 세 줄을 제거합니다. 이를 PIL로 업로드된 파일을 열고 크기를 조정할 다음 저장하는 코드로 교체합니다:

```
image = Image.open(form.cleaned_data["file_upload"])
image.thumbnail((50, 50))
image.save(save_path)
```

첫 번째 줄은 업로드된 파일을 열어 Image 인스턴스를 생성하고, 다음 줄은 썸네일 변환을 수행합니다(최대 크기 50 px x 50 px). 세 번째 줄은 이전 연습에서 생성한 동일한 저장 경로에 파일을 저장합니다. 파일을 저장할 수 있습니다.

- Django 개발 서버는 여전히 2단계에서 실행 중이어야 하지만, 실행 중이 아니라면 시작해야 합니다. 그런 다음 <http://127.0.0.1:8000/media-example/>로 이동하세요. 당신은 다음을 볼 수 있습니다.

familiar `UploadForm`. Select an image and submit the form. If the upload and resize were successful, the form will refresh and be empty again.

8. View the uploaded image using `MEDIA_URL`. For example, a file named `cover.jpg` will be downloadable from `http://127.0.0.1:8000/media/cover.jpg`. You should see that the image has been resized to have a maximum dimension of just 50 px:

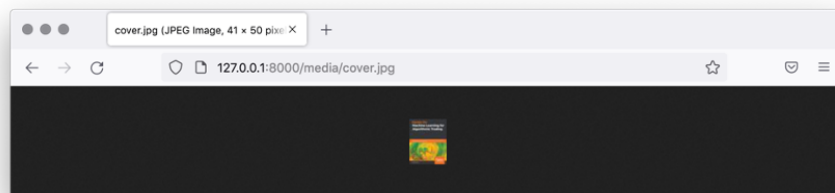


Figure 8.18: The resized cover

While a thumbnail this size might not be that useful, it at least lets us be sure that the image resize has worked correctly.

In this exercise, we changed `FileField` on `UploadForm` to `ImageField`. We saw that the browser wouldn't let us upload anything other than images. We then added code to the `media_example` view to resize the uploaded image using PIL.

We have encouraged the use of a separate web server to serve static and media files for performance reasons. However, in some cases, you might want to use Django to serve files, for example, to provide authentication before allowing access. In the next section, we will discuss how to use Django to serve media files.

Serving uploaded (and other) files using Django

Throughout this chapter and *Chapter 5, Serving Static Files*, we have discouraged serving files using Django. This is because it would needlessly tie up a Python process just to serve a file – something that the web server is capable of handling. Unfortunately, web servers do not usually provide dynamic access control, that is, allowing only authenticated users to download a file. Depending on the web server used in production, you might be able to have it authenticate against Django and then serve the file itself; however, the specific configuration of specific web servers is outside the scope of this book.

One approach you can take is to specify a subdirectory of your `MEDIA_ROOT` directory and have your web server prevent access to just this specific folder. Any protected media should be stored inside it. If you do this, only Django will be able to read the files inside. For example,

익숙한 UploadForm. 이미지를 선택하고 양식을 제출합니다. 업로드 및 크기 조정이 성공하면 양식이 새로 고쳐지고 다시 비어 있게 됩니다.

8. MEDIA_URL을 사용하여 업로드된 이미지를 봅니다. 예를 들어, cover.jpg라는 파일은 `http://127.0.0.1:8000/media/cover.jpg`에서 다운로드할 수 있습니다. 이미지는 최대 크기가 50px로 조정되었음을 확인할 수 있어야 합니다:

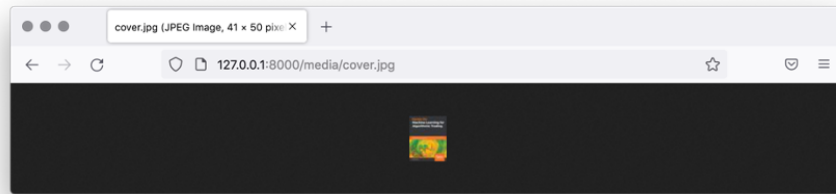


그림 8.18: 조정된 표지

이 크기의 썸네일은 그다지 유용하지 않을 수 있지만, 적어도 이미지 크기 조정이 올바르게 작동했음을 확신할 수 있습니다.

이 연습에서는 UploadForm의 FileField를 ImageField로 변경했습니다. 우리는 브라우저가 이미지를 제외한 다른 파일을 업로드할 수 없도록 막았음을 보았습니다. 그런 다음 media_example 뷰에 코드를 추가하여 업로드된 이미지를 PIL을 사용하여 크기 조정했습니다.

성능상의 이유로 정적 및 미디어 파일을 제공하기 위해 별도의 웹 서버를 사용하는 것을 권장했습니다. 그러나 경우에 따라 Django를 사용하여 파일을 제공하고 싶을 수 있습니다. 예를 들어, 접근을 허용하기 전에 인증을 제공하기 위해서입니다. 다음 섹션에서는 Django를 사용하여 미디어 파일을 제공하는 방법에 대해 논의할 것입니다.

Django를 사용하여 업로드된 (및 기타) 파일 제공하기

이 장과 5장, 정적 파일 제공을 통해 우리는 Django를 사용하여 파일을 제공하는 것을 권장하지 않았습니다. 이는 파일을 제공하기 위해 Python 프로세스를 불필요하게 점유하게 만들기 때문입니다 –

idx_4a1fbe91 웹 서버가 처리할 수 있는 작업입니다. 불행히도, 웹 서버는 일반적으로 동적 접근 제어를 제공하지 않으며, 즉 인증된 사용자만 파일을 다운로드할 수 있도록 허용하지 않습니다. 프로덕션에서 사용되는 웹 서버에 따라 Django에 대해 인증한 후 파일 자체를 제공할 수 있을 수도 있지만, 특정 웹 서버의 구체적인 구성은 이 책의 범위를 벗어납니다.

당신이 취할 수 있는 한 가지 접근 방식은 MEDIA_ROOT 디렉토리의 하위 디렉토리를 지정하고 웹 서버가 이 특정 폴더에 대한 접근을 차단하도록 하는 것입니다. 보호된 미디어는 그 안에 저장하는 것이 가장 좋습니다. 이렇게 하면 Django만 내부의 파일을 읽을 수 있습니다. 예를 들어,

your web server could serve everything in the MEDIA_ROOT directory except for a MEDIA_ROOT/protected directory.

Another approach would be to configure a Django view to serve a specific file from disk. The view will determine the path of the file on disk to send, then send it using the `FileResponse` class. The `FileResponse` class takes an open file handle as an argument and tries to determine the correct content type from the file's content. Django will close the file handle after the request completes.

The view function will accept the request and a relative path to the file to be downloaded as parameters. This relative path is the path inside the MEDIA_ROOT/protected folder.

In our case, we will just check whether the user is anonymous (not logged in). We'll do this by checking the `request.user.is_anonymous` property. If they aren't logged in, then we will raise a `django.core.exceptions.PermissionDenied` exception, which returns an **HTTP 403 Forbidden** response to the browser. This will stop the execution of the view and not return any file:

```
import os.path
from django.conf import settings
from django.http import FileResponse
from django.core.exceptions import PermissionDenied

def download_view(request, relative_path):
    if request.user.is_anonymous:
        raise PermissionDenied
    full_path = os.path.join(settings.MEDIA_ROOT,
                             "protected", relative_path)
    file_handle = open(full_path, "rb")
    # Django sends the file then closes the handle
    return FileResponse(file_handle)
```

The URL mapping to this view could be like this, using the `path` path converter inside your `urls.py` file:

```
urlpatterns = [
    ...
    path("downloads/<path:relative_path>", views.download_view)
]
```

귀하의 웹 서버는 MEDIA_ROOT 디렉토리의 모든 내용을 제공할 수 있지만 MEDIA_ROOT/protected 디렉토리는 제외됩니다.

또 다른 접근 방식은 Django 뷰를 구성하여 디스크에서 특정 파일을 제공하는 것입니다. 뷰는 전송할 파일의 경로를 결정한 다음 FileResponse 클래스를 사용하여 전송합니다. FileResponse 클래스는 열린 파일 핸들을 인수로 받아 파일의 내용에서 올바른 콘텐츠 유형을 결정하려고 시도합니다. Django는 요청이 완료된 후 파일 idx_596d91b8 핸들을 닫습니다.

뷰 함수는 요청과 다운로드할 파일에 대한 상대 경로를 매개변수로 받아들입니다. 이 상대 경로는 MEDIA_ROOT/protected 폴더 내의 경로입니다.

우리의 경우, 우리는 사용자가 익명(로그인하지 않음)인지 확인할 것입니다. 우리는 request.user.is_anonymous 속성을 확인하여 이를 수행할 것입니다. 만약 그들이 로그인하지 않았다면, 우리는 django.core.exceptions.PermissionDenied 예외를 발생시킬 것이며, 이는 idx_bd84975d 브라우저에 HTTP 403 Forbidden 응답을 반환합니다. 이는 뷰의 실행을 중지하고 파일을 반환하지 않습니다:

```
import os.path
from django.conf import settings
from django.http import FileResponse
from django.core.exceptions import PermissionDenied

def download_view(request, relative_path):
    if request.user.is_anonymous:raise PermissionDenied

    full_path = os.path.join(settings.MEDIA_ROOT,
                             "protected", relative_path)
    file_handle = open(full_path, "rb")
    # Django가 파일을 전송한 후 핸들을 닫습니다
    return FileResponse(file_handle)
```

이 뷰에 대한 URL 매핑은 yoururls.py 파일 내에서 경로 변환기를 사용하여 다음과 같을 수 있습니다:

```
urlpatterns = [
    ...
    path("downloads/<path:relative_path>", views.download_view)
]
```

There are many ways that you could choose to implement a view that sends files. The important thing is that you use the `FileResponse` class, which is designed to stream the file to the client in chunks instead of loading it all into memory. This will reduce the load on the server and lessen the impact on resource usage if you must resort to sending files with Django.

Now that we know how to upload and process images, let's look at how to store them on models.

Storing files on model instances

So far, we have manually managed the uploading and saving of files. You can also associate a file with a model instance by assigning the path to which it was saved to `CharField`. However, as with much of Django, this capability (and more) is already provided with the `models.FileField` class. A `FileField` instance does not actually store the file data; instead, it stores the path where the file is stored (like `CharField` would), but it also provides helper methods. These methods assist with loading files (so you don't have to open them manually) and generating disk paths for you based on the ID of the instance (or other attributes).

`FileField` can accept two specific optional arguments in its constructor (as well as the base `Field` arguments, such as `required`, `unique`, `help_text`, etc.):

- `max_length`: Like `max_length` in the form's `ImageField`, this is the maximum length of the filename that is allowed.
- `upload_to`: The `upload_to` argument has three different behaviors depending on what type of variable is passed to it. Its simplest use is with a string or a `pathlib.Path` object. The path is simply appended to `MEDIA_ROOT`.

In this example, `upload_to` is just defined as a string:

```
class ExampleModel(models.Model):
    file_field = models.FileField(upload_to="files/")
```

Files saved to `FileField` will be stored in the `MEDIA_ROOT/files` directory.

You can achieve the same result using a `pathlib.Path` instance, too:

```
import pathlib

class ExampleModel(models.Model):
    file_field = models.FileField(upload_to=pathlib.Path("files/"))
```

파일을 전송하는 뷰를 구현하는 방법은 여러 가지가 있습니다. 중요한 점은 파일을 모두 메모리에 로드하는 대신 청크 단위로 클라이언트에 스트리밍하도록 설계된 `FileResponse` 클래스를 사용하는 것입니다. 이렇게 하면 서버의 부하가 줄어들고 Django로 파일을 전송해야 할 경우 리소스 사용에 미치는 영향을 줄일 수 있습니다.

이제 이미지를 업로드하고 처리하는 방법을 알았으니, 모델에 이미지를 저장하는 방법을 살펴보겠습니다.

모델 인스턴스에 파일 저장하기

지금까지 우리는 파일 업로드와 저장을 수동으로 관리해왔습니다. 또한 파일이 저장된 경로를 `CharField`에 할당하여 모델 인스턴스와 파일을 연결할 수도 있습니다. 그러나 Django의 많은 기능들처럼, 이 기능(및 그 이상)은 이미 `models.FileField` 클래스에 의해 제공됩니다. `FileField` 인스턴스는 실제로 파일 데이터를 저장하지 않고, 대신 파일이 저장된 경로를 저장합니다(`CharField`가 하는 것처럼). 하지만 `FileField`는 파일을 로드하는 데 도움이 되는 메서드도 제공합니다(그래서 파일을 수동으로 열 필요가 없습니다). 인스턴스의 ID(또는 다른 속성)를 기반으로 디스크 경로를 생성합니다. `FileField`는 생성자에서 두 가지 특정 선택적 인수와 기본 필드 인수(예: `required`, `unique`, `help_text` 등)를 받을 수 있습니다:

- `max_length`: 폼의 `ImageField`에 있는 `max_length`와 마찬가지로, 허용되는 파일 이름의 최대 길이입니다.

- `upload_to`: `upload_to` 인자는 `idx_8ca0a50c`에 전달되는 변수의 유형에 따라 세 가지 다른 동작을 합니다. 가장 간단한 사용법은 문자열 또는 `pathlib.Path`와 함께 사용하는 것입니다.

객체. 경로는 단순히 `MEDIA_ROOT`에 추가됩니다. 이 예제에서 `upload_to`는 단순히 문자열로 정의됩니다:

```
class ExampleModel(models.Model):
    file_field = models.FileField(upload_to="files/")
```

`FileField`에 저장된 파일은 `MEDIA_ROOT/files` 디렉토리에 저장됩니다. `pathlib.Path` 인스턴스를 사용하여 동일한 결과를 얻을 수도 있습니다:

```
import pathlib

class ExampleModel(models.Model):
    file_field = models.FileField(upload_to=pathlib.Path("files/"))
```

The next way of using `upload_to` is with a string that contains the `strftime` formatting directives (for example, `%Y` to substitute the current year, `%m` for the current month, and `%d` for the current day of the month). The full list of these directives is extensive and can be found at <https://docs.python.org/3/library/time.html#time.strftime>. Django will automatically interpolate these values when saving the file.

For example, say you defined the model and `FileField` like this:

```
class ExampleModel(models.Model):
    file_field = models.FileField(upload_to="files/%Y/%m/%d/")
```

For the first file uploaded on a specific day, Django would create the directory structure for that day. For example, for the first file uploaded on January 1st, 2025, Django would create the `MEDIA_ROOT/2025/01/01` directory and then store the uploaded file there. The next file (and all subsequent ones) uploaded on the same day would also be stored in that directory. Similarly, on January 2nd, 2025, Django will create the `MEDIA_ROOT/2025/01/02` directory, and files will be stored there.

If you have many thousands of files being uploaded every day, you could even have the files split up further by including the hour and minute in the `upload_to` argument (`upload_to="files/%Y/%m/%d/%H/%M/"`). This may not be necessary if you only have a small volume of uploads, though.

By utilizing this method of the `upload_to` argument, you can have Django automatically segregate uploads and prevent too many files from being stored within a single directory (which can be hard to manage).

The final method of using `upload_to` is by passing a function that will be called to generate the storage path. Note that this is different from the other uses of `upload_to`, as it should generate the full path, including the filename, rather than just the directory. The function takes two arguments: `instance` and `filename`. The `instance` argument is the model instance that `FileField` is attached to, and `filename` is the name of the uploaded file.

Here is an example function that takes the first two characters of a filename to generate the save directory. This means that each uploaded file will be grouped into parent directories, which can help organize files and prevent there from being too many in one directory:

```
def user_grouped_file_path(instance, filename):
    return "/".join([instance.username,
                     filename[0].lower(), filename[1].lower(), filename])
```

upload_to를 사용하는 다음 방법은 strftime 형식 지시어가 포함된 문자열을 사용하는 것입니다 (예: 현재 연도를 대체하기 위한 %Y, 현재 월을 위한 %m, 그리고 현재 월의 날짜를 위한 %d). 이러한 지시어의 전체 목록은 방대하며

<https://docs.python.org/3/library/time.html#time.strftime> 에서 확인할 수 있습니다.

Django는 파일을 저장할 때 이러한 값을 자동으로 보간합니다.

예를 들어, 모델과 FileField를 다음과 같이 정의했다고 가정해 보겠습니다:

```
class ExampleModel(models.Model):
    file_field = models.FileField(upload_to="files/%Y/%m/%d/")
```

특정 날짜에 업로드된 첫 번째 파일의 경우, Django는 해당 날짜에 대한 디렉토리 구조를 생성합니다. 예를 들어, 2025년 1월 1일에 업로드된 첫 번째 파일의 경우, Django는 MEDIA_ROOT/2025/01/01 디렉토리를 생성한 다음 그곳에 업로드된 파일을 저장합니다. 같은 날에 업로드된 다음 파일(및 이후의 모든 파일)도 해당 디렉토리에 저장됩니다. 마찬가지로, 2025년 1월 2일에 Django는 MEDIA_ROOT/2025/01/02 디렉토리를 생성하고 파일이 그곳에 저장됩니다.

매일 수천 개의 파일이 업로드되는 경우, upload_to 인수에 시간과 분을 포함하여 파일을 더 세분화할 수 있습니다 (upload_to="files/%Y/%m/%d/%H/%M/"). 그러나 업로드량이 적은 경우에는 필요하지 않을 수 있습니다.

upload_to 인수의 이 idx_e09f0aaf 방법을 활용하면 Django가 idx_d90171b1 업로드를 자동으로 분리하고 단일 디렉토리에 너무 많은 파일이 저장되는 것을 방지할 수 있습니다 (관리하기 어려울 수 있습니다).

upload_to를 사용하는 마지막 방법은 저장 경로를 생성하기 위해 호출될 함수를 전달하는 것입니다. 이는 upload_to의 다른 사용법과 다르며, 디렉토리뿐만 아니라 파일 이름을 포함한 전체 경로를 생성해야 합니다. 이 함수는 두 개의 인수를 받습니다: instance와 filename. instance 인수는 FileField가 연결된 모델 인스턴스이며, filename은 업로드된 파일의 이름입니다.

여기 파일 이름의 처음 두 문자를 사용하여 저장 디렉토리를 생성하는 예제 함수가 있습니다. 이는 업로드된 각 파일이 상위 디렉토리로 그룹화되어 파일을 정리하고 하나의 디렉토리에 너무 많은 파일이 없도록 도와줍니다:

```
def user_grouped_file_path(instance, filename):
    return "/".join([instance.username,
                     filename[0].lower(), filename[1].lower(), filename])
```

If this function is called with the `Test.jpg` filename, it will return `<username>/t/e/test.jpg`. If called with `example.txt`, it will return `<username>/e/x/example.txt`, and so on. `username` is retrieved from the instance that is being saved. To illustrate, here is a model with `FileField` that uses this function. It also has a `username`, which is `CharField`:

```
class ExampleModel(models.Model):
    file_field = models.FileField(upload_to=user_grouped_file_path)
    username = models.CharField(unique=True)
```

You can use any attribute of the instance in the `upload_to` function, but be aware that if this instance is in the process of being created, then the file save function will be called before it is saved to the database. Therefore, some of the automatically generated attributes on the instance (such as `id/pk`) will not yet be populated and shouldn't be used to generate a path.

Whatever path is returned from the `upload_to` function is appended to `MEDIA_ROOT`, so the uploaded files would be saved at `MEDIA_ROOT/<username>/t/e/test.jpg` and `MEDIA_ROOT/<username>/e/x/example.txt`, respectively.

Note that `user_grouped_file_path` is just an illustrative function that has intentionally been kept short, so it will not work correctly with single-character filenames or if the username has invalid characters. For example, if the username has a `/` character in it, then this would act as a directory separator in the generated path.

Now, we've done a deep dive into setting up `FileField` on a model, but how do we actually save an uploaded file to it? It's as easy as assigning the uploaded file to the attribute of the model, as you would with any type of value. Here's a quick example with a view and the simple `ExampleModel` we were using as an example earlier in this section:

```
class ExampleModel(models.Model):
    file_field = models.FileField(upload_to="files/")

def view(request):
    if request.method == "POST":
        # Create a new ExampleModel Instance
        m = ExampleModel()
        m.file_field = request.FILES["uploaded_file"]
        m.save()
    return render(request, "template.html")
```

이 함수가 Test.jpg 파일 이름으로 호출되면 <username>/t/e/test.jpg를 반환합니다. example.txt로 호출되면 <username>e/x/example.txt를 반환하며, 계속해서 그렇게 됩니다. username은 저장 중인 인스턴스에서 검색됩니다. 이를 설명하기 위해, 이 함수를 사용하는 FileField가 있는 모델을 보여드리겠습니다. 또한 CharField인 username도 있습니다:

```
class ExampleModel(models.Model):
    file_field = models.FileField(upload_to=user_grouped_file_path)
    username = models.CharField(unique=True)
```

업로드_to 함수에서 인스턴스의 모든 속성을 사용할 수 있지만, 이 인스턴스가 생성 중인 경우 파일 저장 함수가 데이터베이스에 저장되기 전에 호출된다는 점에 유의해야 합니다. 따라서 인스턴스의 자동 생성된 속성 idx_e9b2836e (예: id/pk)는 아직 채워지지 않으며 경로를 생성하는 데 사용해서는 안 됩니다.

업로드_to 함수에서 반환된 경로는 MEDIA_ROOT에 추가되므로, 업로드된 파일은 각각 MEDIA_ROOT/<username>/t/e/test.jpg 및 MEDIA_ROOT/<username>/e/x/example.txt에 저장됩니다.

user_grouped_file_path는 의도적으로 짧게 유지된 설명용 함수일 뿐이므로, 단일 문자 파일 이름이나 username에 유효하지 않은 문자가 포함된 경우 올바르게 작동하지 않습니다. 예를 들어, username에 / 문자가 포함되어 있으면 생성된 경로에서 디렉토리 구분자로 작용합니다.

이제 모델에서 FileField를 설정하는 방법에 대해 깊이 살펴보았지만, 실제로 업로드된 파일을 어떻게 저장할까요? 업로드된 파일을 모델의 속성에 할당하는 것만큼 간단합니다. 이전 섹션에서 예로 사용했던 간단한 ExampleModel과 함께하는 빠른 예는 다음과 같습니다:

```
class ExampleModel(models.Model):
    file_field = models.FileField(upload_to="files/")

def view(request):
    if request.method == "POST":
        # Create a new ExampleModel Instance
        m = ExampleModel()
        m.file_field = request.FILES["uploaded_file"]
        m.save()
    return render(request, "template.html")
```

In this example, we create a new `ExampleModel` and assign the uploaded file (called `uploaded_file` on the form) to its `file_field` attribute. When we saved the model instance, Django automatically wrote the file with its name to the `upload_to` directory path. If the uploaded file had the name `image.jpg`, the save path would be `MEDIA_ROOT/upload_to/image.jpg`.

We could just as easily have updated the file field on an existing model or used a form (validating it before saving). Here is another simple example demonstrating this:

```
class ExampleForm(forms.Form):
    uploaded_file = forms.FileField()

def view(request, model_pk):
    form = ExampleForm(request.POST, request.FILES)
    if form.is_valid():
        # Get an existing model instance
        m = ExampleModel.objects.get(pk=model_pk)

        # store the uploaded file on the instance
        m.file_field = form.cleaned_data["uploaded_file"]
        m.save()
    return render(request, "template.html")
```

You can see that updating `FileField` on an existing model instance is the same process as setting it on a new instance, and if you choose to use a Django form or just access `request.FILES` directly, the process is just as simple.

Now we'll look at how to store images on models using `ImageField`.

Storing images on model instances

While `FileField` can store any type of file, including images, there also exists `ImageField`. As you would expect, this is only for storing images. The relationship between the models' `forms.FileField` and `forms.ImageField` is similar to that between `models.FileField` and `models.ImageField`, meaning `ImageField` extends `FileField` and adds extra methods for working with images.

이 예제에서는 새로운 ExampleModel을 생성하고 업로드된 파일(양식에서 calleduploaded_file)을 file_field 속성에 할당합니다. 모델 인스턴스를 저장할 때, Django는 자동으로 파일 이름과 함께 upload_to 디렉토리 경로에 파일을 작성했습니다. 만약 업로드된 파일의 이름이 image.jpg였다면, 저장 경로는 MEDIA_ROOT/upload_to/image.jpg가 됩니다.

기존 모델의 파일 필드를 업데이트하거나 양식을 사용하여(저장하기 전에 유효성을 검사) 쉽게 할 수 있었습니다. 다음은 이를 보여주는 또 다른 간단한 예입니다:

```
class ExampleForm(forms.Form):
    uploaded_file = forms.FileField()

def view(request, model_pk):
    form = ExampleForm(request.POST, request.FILES)
    if form.is_valid():
        # 기존 모델 인스턴스 가져오기
        m = ExampleModel.object.get(pk=model_pk)

        # 업로드된 파일을 인스턴스에 저장합니다.
        m.file_field = form.cleaned_data["uploaded_file"]
        m.save()

    return render(request, "template.html")
```

기존 모델 인스턴스에서 FileField를 업데이트하는 과정은 새로운 인스턴스에서 설정하는 것과 동일하며, Django 폼을 사용하든 request.FILES에 직접 접근하든 과정은 매우 간단합니다.

이제 ImageField를 사용하여 모델에 이미지를 저장하는 방법을 살펴보겠습니다.

모델 인스턴스에 이미지를 저장하기

FileField는 이미지 포함 모든 유형의 파일을 저장할 수 있지만, ImageField도 존재합니다. 예상할 수 있듯이, 이는 이미지만 저장하기 위한 것입니다. 모델의 forms.FileField와 forms.ImageField 간의 관계는 models.FileField와 models.ImageField 간의 관계와 유사하며, 이는 ImageField가 FileField를 확장하고 이미지를 다루기 위한 추가 메서드를 추가함을 의미합니다.

The `ImageField` constructor takes the same arguments as `FileField` and adds two extra optional arguments:

- `height_field`: This is the name of the field on the model that will be updated with the height of the image every time the model instance is saved
- `width_field`: This is the width counterpart to `height_field`: the field that stores the width of the image that is updated every time the model instance is saved

Both of these arguments are optional, but the fields they name must exist if used. That is, it is valid to have `height_field` or `width_field` unset, but if they are set to the name of a field that does not exist, then an error will occur. The purpose of this is to assist with searching the database for files of a particular dimension.

Here is an example model using `ImageField`, which updates the image dimension fields:

```
class ExampleModel(models.Model):
    image = models.ImageField(
        upload_to="images/%Y/%m/%d/",
        height_field="image_height",
        width_field="image_width")
    image_height = models.IntegerField()
    image_width = models.IntegerField()
```

Notice that `ImageField` is using the `upload_to` parameter with date formatting directives that are updated on save. The behavior of `upload_to` is identical to that of `FileField`.

Upon saving an `ExampleModel` instance, its `image_height` field would be updated with the height of the image and `image_width` with the width of the image.

We won't show examples for setting the `ImageField` values in a view, as the process is the same as for a plain `FileField`.

Working with FieldFile

When you access the `FileField` or `ImageField` attribute of a model instance, you will not get a native Python file object. Instead, you will be working with a `FieldFile` object. The `FieldFile` class is a wrapper around file that adds extra methods. Yes, it can be confusing to have classes called `FileField` and `FieldFile`.

The reason that Django uses `FieldFile` instead of just a file object is twofold. First, it adds extra methods to open, read, delete, and generate the URL of the file. Second, it provides an abstraction to allow alternative storage engines to be used.

ImageField 생성자는 FileField와 동일한 인수를 사용하고 두 개의 추가 선택적 인수를 추가합니다:

- `height_field`: 이것은 모델에서 이미지의 높이가 모델 인스턴스가 저장될 때마다 업데이트되는 필드의 이름입니다
- `width_field`: 이것은 `height_field`의 너비에 해당하는 필드로, 모델 인스턴스가 저장될 때마다 업데이트되는 이미지의 너비를 저장하는 필드입니다

이 두 인수는 선택 사항이지만, 사용되는 경우 이름이 지정된 필드는 존재해야 합니다. 즉, `height_field` 또는 `width_field`가 설정되지 않는 것은 유효하지만, 존재하지 않는 필드의 이름으로 설정되면 오류가 발생합니다. 이는 특정 치수의 파일을 데이터베이스에서 검색하는 데 도움을 주기 위한 것입니다.

다음은 ImageField를 사용하는 예제 모델로, 이미지 치수 필드를 업데이트합니다:

```
class ExampleModel(models.Model):
    image = models.ImageField(
        upload_to="images/%Y/%m/%d/",
        height_field="image_height",
        width_field="image_width")
    image_height = models.IntegerField()
    image_width = models.IntegerField()
```

ImageField는 저장 시 업데이트되는 날짜 형식 지시어가 있는 `upload_to` 매개변수를 사용하고 있습니다. `upload_to`의 동작은 FileField와 동일합니다.

ExampleModel 인스턴스를 저장하면 `image_height` 필드는 이미지의 높이로 업데이트되고 `image_width`는 이미지의 너비로 업데이트됩니다.

우리는 view에서 ImageField 값을 설정하는 예제를 `idx_1258422a`로 보여주지 않을 것입니다. `idx_9791737f` 프로세스는 일반 FileField와 동일하기 때문입니다.

FieldFile 작업하기

모델 인스턴스의 FileField 또는 ImageField 속성에 접근할 때, 기본 Python 파일 객체를 얻지 못합니다. 대신, FieldFile 객체와 작업하게 됩니다. FieldFile 클래스는 파일을 감싸는 래퍼로 추가 메서드를 제공합니다. 네, FileField와 FieldFile이라는 클래스가 있어 혼란스러울 수 있습니다.

`idx_68877ff0Django`가 단순한 파일 객체 대신 FieldFile을 사용하는 이유는 두 가지입니다. 첫째, 파일을 열고, 읽고, 삭제하고, URL을 생성하는 추가 메서드를 추가합니다. 둘째, 대체 저장 엔진을 사용할 수 있도록 추상화를 제공합니다.

Custom storage engines

We looked at custom storage engines in *Chapter 5, Serving Static Files*, regarding storing static files. We won't examine custom storage engines in detail for media files since the code outlined in *Chapter 5, Serving Static Files*, for static files also applies to media files. The important thing to note is that which storage engine you're using can be changed without updating your other code. This means that you can have your media files stored on your local drive during development and then save them to a CDN when your application is deployed to production.

The default `storage_engine` class can be set with `DEFAULT_FILE_STORAGE` in `settings.py`. The storage engine can also be specified on a per-field basis (for `FileField` or `ImageField`) with the `storage` argument, as in this example:

```
storage_engine = CustomStorageEngine()

class ExampleModel(models.Model):
    image_field = ImageField(storage=storage_engine)
```

This demonstrates what actually happens when you upload or retrieve a file. Django delegates to the storage engine to write or read it, respectively. This happens even while saving to disk; however, this is automated and is invisible to the user.

Reading a stored FieldFile

Now that we've learned about custom storage engines, let's look at reading from `FieldFile`. In the previous sections, we saw how to set the file on the model instance. Reading the data back again is just as easy – we have a couple of different methods that can help us depending on our use case.

In the following few code snippets, assume we are inside a view and have retrieved our model instance in some manner, and it's stored in an `m` variable:

```
m = ExampleModel.objects.get(pk=model_pk)
```

We can read all the data from the file with the `read` method:

```
data = m.file_field.read()
```

사용자 정의 저장 엔진

우리는 정적 파일 저장과 관련하여 5장, 정적 파일 제공에서 사용자 정의 저장 엔진을 살펴보았습니다. 미디어 파일에 대해서는 5장, 정적 파일 제공에서 설명된 코드가 정적 파일에도 적용되므로 사용자 정의 저장 엔진을 자세히 살펴보지 않을 것입니다. 중요한 점은 사용 중인 저장 엔진을 다른 코드를 업데이트하지 않고도 변경할 수 있다는 것입니다. 이는 개발 중에 미디어 파일을 로컬 드라이브에 저장하고, 애플리케이션이 프로덕션에 배포될 때 CDN에 저장할 수 있음을 의미합니다.

기본 저장 엔진 클래스는 settings.py의 DEFAULT_FILE_STORAGE로 설정할 수 있습니다. 저장 엔진은 필드별로(FileField 또는 ImageField) storage 인수를 사용하여 지정할 수도 있습니다. 예를 들면 다음과 같습니다:

```
storage_engine = CustomStorageEngine()

class ExampleModel(models.Model):
    image_field = ImageField(storage=storage_engine)
```

이것은 파일을 업로드하거나 검색할 때 실제로 발생하는 일을 보여줍니다. Django는 각 파일을 쓰거나 읽기 위해 저장 엔진에 위임합니다. 이는 디스크에 저장하는 동안에도 발생하지만, 이는 자동화되어 사용자에게는 보이지 않습니다.

저장된 FieldFile 읽기

이제 사용자 정의 저장 엔진에 대해 배웠으니, FieldFile에서 읽는 방법을 살펴보겠습니다. 이전 idx_7f2e3627 섹션에서는 모델 인스턴스에 파일을 설정하는 방법을 보았습니다. 데이터를 다시 읽는 것은 그만큼 쉽습니다. 우리의 사용 사례에 따라 도움이 될 수 있는 몇 가지 다른 방법이 있습니다.

다음 몇 개의 코드 스니펫에서는 우리가 뷰 안에 있고 어떤 방식으로든 모델 인스턴스를 검색했으며, 그것이 m 변수에 저장되어 있다고 가정합니다:

```
m = ExampleModel.object.get(pk=model_pk)
```

read 메서드를 사용하여 파일에서 모든 데이터를 읽을 수 있습니다:

```
data = m.file_field.read()
```

Or, we can manually open the file with the `open` method. This might be useful if we want to write our own generated data to the file:

```
with m.file_field.open("wb") as f:
    chunk = f.write(b"test") # write bytes to the file
```

If we want to read the file in chunks, we can use the `chunks` method. This works the same as reading chunks from the uploaded file, as we saw earlier:

```
for chunk in m.file_field.chunks():
    # assume this method is defined somewhere
    write_chunk(open_file, chunk)
```

We can also manually open the file ourselves by using its `path` attribute:

```
open(m.file_field.path)
```

If we want to stream `FileField` for download, the best way is by using the `FileResponse` class, as we saw earlier. Combine this with the `open` method on `FileField`. Note that if we're just trying to serve a media file, we should only implement a view to do this if we're trying to restrict access to the file. Otherwise, we should just serve the file using `MEDIA_URL` and allow the web server to handle the request. Here's how we'd write `download_view` to use `FileField` instead of the manually specified path:

```
def download_view(request, model_pk):
    if request.user.is_anonymous:
        raise PermissionDenied
    m = ExampleModel.objects.get(pk=model_pk)
    # Django sends the file then closes the handle
    return FileResponse(m.file_field.open())
```

Django opens the correct path and closes it after the response. Django will also attempt to determine the correct mime type for the file. We assume that `FileField` has its `upload_to` attribute set to a protected directory that the web server is preventing direct access to.

Storing existing files or content on `FileField`

We've seen how to store an uploaded file on an image field; simply assign it to the field like so:

```
m.file_field = request.FILES["file_upload"]
```

또는 open 메서드를 사용하여 파일을 수동으로 열 수 있습니다. 이는 우리가 생성한 데이터를 파일에 쓰고 싶을 때 유용할 수 있습니다:

```
with m.file_field.open("wb") as f:
    chunk = f.write(b"test") # 파일에 바이트 쓰기
```

파일을 청크 단위로 읽고 싶다면 chunks 메서드를 사용할 수 있습니다. 이는 우리가 이전에 본 업로드된 파일에서 청크를 읽는 것과 동일하게 작동합니다:

```
for chunk in m.file_field.chunks():
    # 이 메서드가 어딘가에 정의되어 있다고 가정합니다
    write_chunk(open_file, chunk)
```

파일의 경로 속성을 사용하여 파일을 수동으로 열 수도 있습니다:

```
open(m.file_field.path)
```

파일 필드를 다운로드를 위해 스트리밍하려면, 가장 좋은 방법은 앞서 본 것처럼 FileResponse 클래스를 사용하는 것입니다. 이를 FileField의 open 메서드와 결합하세요. 미디어 파일을 제공하려는 경우, 파일에 대한 접근을 제한하려고 할 때만 이 idx_9e0b7c1d를 구현해야 합니다. 그렇지 않으면 MEDIA_URL을 사용하여 파일을 제공하고 웹 서버가 요청을 처리하도록 해야 합니다. 다음은 수동으로 지정된 경로 대신 FileField를 사용하도록 download_view를 작성하는 방법입니다:

```
def download_view(request, model_pk):
    if request.user.is_anonymous:
        raise PermissionDenied
    m = ExampleModel.objects.get(pk=model_pk)
    # Django는 파일을 전송한 후 핸들을 닫습니다.
    return FileResponse(m.file_field.open())
```

Django는 올바른 경로를 열고 응답 후에 닫습니다. Django는 또한 파일에 대한 올바른 MIME 유형을 결정하려고 시도합니다. 우리는 FileField의 upload_to 속성이 웹 서버가 직접 접근을 방지하는 보호된 디렉토리에 설정되어 있다고 가정합니다.

FileField에 기존 파일이나 콘텐츠 저장하기

이미지 필드에 업로드된 파일을 저장하는 방법을 보았습니다; 단순히 다음과 같이 필드에 할당하면 됩니다:

```
m.file_field = request.FILES["file_upload"]
```

But how can we set the field value to that of an existing file that we might already have on disk? You might think you can use a standard Python file object, but this won't work:

```
# Don't do this
m.file_field = open("/path/to/file.txt", "rb")
```

You might also try setting the file using some content:

```
# Don't do this
m.file_field = "new file content"
```

This won't work either.

You instead need to use the `FileField` save method, which accepts an instance of a Django `File` object or a `ContentFile` object (these classes' full paths are `django.core.files.File` and `django.core.files.base.ContentFile`, respectively). We'll briefly discuss the save method and its arguments and then return to these classes.

The `FileField` save method takes three arguments:

- **name:** This is the name of the file you are saving and is the name the file will have when saved to the storage engine (in our case, to disk, inside `MEDIA_ROOT`).
- **content:** This is an instance of `File` or `ContentFile` that we just saw; again, we will discuss these soon.
- **save:** This argument is optional and defaults to `True`. This indicates whether or not to save the model instance to the database after saving the file. If set to `False` (i.e., the model is not saved), then the file will still be written to the storage engine (to disk), but the association is not stored on the model. The previous file path (or no file if one was not set) will still be stored in the database until the model instance's save method is called manually. You should only set this argument to `False` if you intend to make other changes to the model instance and then save it manually.

Back to `File` and `ContentFile`: the one to use depends on what you want to store in `FileField`.

`File` is used as a wrapper around a Python file object, and you should use it if you have an existing file or file-like object that you want to save. File-like objects include `io.BytesIO` or

하지만 디스크에 이미 있을 수 있는 기존 파일의 필드 값을 어떻게 설정할 수 있을까요? 표준 Python 파일 객체를 사용할 수 있다고 생각할 수 있지만, 이는 작동하지 않습니다:

```
# 이렇게 하지 마세요
m.file_field = open("/path/to/file.txt", "rb")
```

파일을 일부 콘텐츠로 설정하려고 할 수도 있습니다:

```
# 이렇게 하지 마세요
m.file_field = "new file content"
```

이것도 작동하지 않습니다.

대신 DjangoFile 객체 또는 ContentFile 객체의 인스턴스를 허용하는 FileField save 메서드를 사용해야 합니다(이 클래스의 전체 경로는 각각 `django.core.files.File` 및 `django.core.files.base.ContentFile`입니다). 우리는 save 메서드와 그 인수에 대해 간략하게 논의한 후 이 클래스들로 돌아갈 것입니다.

FileField save 메서드는 세 개의 인수를 받습니다:

- **name:** 이것은 저장하려는 파일의 이름이며, 저장 엔진에 저장될 때 파일이 가질 이름입니다(우리의 경우, 디스크의 MEDIA_ROOT 안에).
- **content:** 이것은 우리가 방금 본 File 또는 ContentFile의 인스턴스입니다; 다시 말해, 우리는 곧 이것들에 대해 논의할 것입니다.
- **save:** 이 인수는 `idx_c82677e8` 선택 사항이며 기본값은 `True`입니다. 이것은 파일을 저장한 후 모델 인스턴스를 데이터베이스에 저장할지 여부를 나타냅니다. `False`로 설정하면(즉, 모델이 저장되지 않음) 파일은 여전히 저장 엔진(디스크)에 기록되지만, 모델에 대한 연관성은 저장되지 않습니다. 이전 파일 경로(또는 설정되지 않은 경우 파일 없음)는 모델 인스턴스의 save 메서드가 수동으로 호출될 때까지 데이터베이스에 여전히 저장됩니다. 이 인수를 `False`로 설정해야 하는 경우는 모델 인스턴스에 다른 변경을 하고 나서 수동으로 저장할 의도가 있을 때 뿐입니다.

File 및 ContentFile로 돌아가기: 사용할 것은 FileField에 저장하려는 내용에 따라 다릅니다.

File은 Python 파일 객체를 감싸는 래퍼로 사용되며, 저장하려는 기존 파일 또는 파일과 유사한 객체가 있는 경우 사용해야 합니다. 파일과 유사한 객체에는 `io.BytesIO` 또는

`io.StringIO` instances. To instantiate a `File` instance, just pass the native file object to the constructor, as in this example:

```
f = open("/path/to/file.txt", "rb")
file_wrapper = File(f)
f.close()
```

Use `ContentFile` when you already have some data loaded, either a `str` object or a `bytes` object. Pass the data to the `ContentFile` constructor:

```
string_content = ContentFile("A string value")
bytes_content = ContentField(b"A bytes value")
```

Now that you have either a `File` instance or a `ContentFile` instance, saving the data to `FileField` is easy, using the `save` method:

```
m = ExampleModel.objects.first()
with open("/path/to/file.txt") as f:
    file_wrapper = File(f)
    m.file_field.save("file.txt", f)
```

Since we did not pass a value for `save` to the `save` method, it will default to `True`, so the model instance is automatically persisted to the database.

Next, we will look at how to store an image that has been manipulated with a PIL in an image field.

Writing PIL images to ImageField

In *Exercise 8.05 – image uploads using Django forms*, you used PIL to resize an image and save it to disk. When working with a model, you might want to perform a similar operation but have Django handle the file storage using `ImageField` so that you don't have to do it manually. As in the exercise, you could save the image to disk and then use the `File` class to wrap the stored path – something like this:

```
image = Image.open(request.FILES["image_field"])
image.thumbnail((150, 150))
# save thumbnail to temp location
image.save("/tmp/thumbnail.jpg")

with open("/tmp/thumbnail.jpg", "rb") as f:
```

io.StringIO 인스턴스. File 인스턴스를 생성하려면, 이 예제와 같이 기본 파일 객체를 생성자에 전달하면 됩니다:

```
f = open("/path/to/file.txt", "rb")
file_wrapper = File(f)
f.close()
```

이미 str 객체나 bytes 객체로 데이터가 로드된 경우 ContentFile을 사용하세요. 데이터를 ContentFile 생성자에 전달하세요:

```
string_content = ContentFile("A string value")
bytes_content = ContentField(b"A bytes value")
```

이제 File 인스턴스나 ContentFile 인스턴스가 있으므로, save 메서드를 사용하여 FileField에 데이터를 저장하는 것은 쉽습니다:

```
m = ExampleModel.objects.first()
with open("/path/to/file.txt") as f:
    file_wrapper = File(f)
    m.file_field.save("file.txt", f)
```

저장 메서드에 대한 값을 전달하지 않았기 때문에 기본값은 True가 되어 모델 인스턴스가 자동으로 데이터베이스에 지속됩니다.

다음으로, PIL로 조작된 이미지를 imagefield에 저장하는 방법을 살펴보겠습니다.

PIL 이미지를 ImageField에 쓰기

연습 8.05 - Django 양식을 사용한 이미지 업로드에서, 이미지를 크기 조정하고 디스크에 저장하기 위해 PIL을 사용했습니다. 모델로 작업할 때, 유사한 작업을 수행하고 Django가 ImageField를 사용하여 파일 저장을 처리하도록 하여 수동으로 할 필요가 없도록 할 수 있습니다. 연습과 마찬가지로 이미지를 디스크에 저장한 다음 File 클래스를 사용하여 저장된 경로를 래핑할 수 있습니다 - 다음과 같은 방식으로:

```
image = Image.open(request.FILES["image_field"])
image.thumbnail((150, 150))
# 썸네일을 임시 위치에 저장
image.save("/tmp/thumbnail.jpg")

with open("/tmp/thumbnail.jpg", "rb") as f:
```

```
image_wrapper = File(f)
m.image_field.save("thumbnail.jpg", image_wrapper, save=True)
# clean up temp file
os.unlink("/tmp/thumbnail.jpg")
```

In this example, we're storing the PIL store in a temporary location with the `Image.save()` method, and then reopening the file.

This method works but is not ideal, as it involves writing the file to disk and then reading it out again, which can sometimes be slow. Instead, we can perform this whole process in memory.

Note

`io.BytesIO` and `io.StringIO` are useful objects. They behave like files but exist in memory only. `BytesIO` is used for storing raw bytes, and `StringIO` accepts Python 3's native Unicode strings. You can use `read`, `write`, and `seek`, just like a normal file. Unlike a normal file, though, they don't get written to disk and, instead, will disappear when your program terminates, or they go out of scope and are garbage collected. They are very useful if a function wants to write to something like a file, but you want to access the data immediately.

First, we will save the image data to an `io.BytesIO` object. Then, we'll wrap the `BytesIO` object in a `django.core.files.images.ImageField` instance (a subclass of `File` that is specifically for images and provides the width and height attributes). Once we have this `ImageFile` instance, we can use it in the `save` method of `ImageField`.

Note

`ImageFile` is a file or file-like wrapper just like `File`. It provides two extra attributes: `width` and `height`. `ImageFile` does not generate any errors if you use it to wrap a non-image. For example, you could use `open()` to open a text file and pass the file handle to the `ImageFile` constructor without issue. You can check whether the image file you passed in was valid by trying to access the `width` or `height` attribute; if these are `None`, then PIL was unable to decode the image data. You can check for the validity of these values yourself and throw an exception if they are `None`.

```

image_wrapper = File(f)
m.image_field.save("thumbnail.jpg", image_wrapper, save=True)
# 임시 파일 정리
os.unlink("/tmp/thumbnail.jpg")

```

이 예제에서는 PIL 저장소를 임시 위치에 저장한 다음 파일을 다시 열고 있습니다.

이 방법은 작동하지만 이상적이지 않으며, 파일을 디스크에 쓰고 다시 읽어야 하므로 때때로 느릴 수 있습니다. 대신, 이 전체 프로세스를 메모리에서 수행할 수 있습니다.

참고

`io.BytesIO` 및 `io.StringIO`는 유용한 객체입니다. 이들은 파일처럼 동작하지만 메모리 내에서만 존재합니다. `BytesIO`는 원시 바이트를 저장하는 데 사용되며, `StringIO`는 Python 3의 기본 유니코드 문자열을 수용합니다. 일반 파일처럼 읽기, 쓰기 및 탐색을 사용할 수 있습니다. 그러나 일반 파일과는 달리 디스크에 쓰이지 않으며, 프로그램이 종료되거나 범위를 벗어나면 사라집니다. 함수가 파일과 같은 것에 쓰기를 원하지만 데이터를 즉시 액세스하고 싶을 때 매우 유용합니다.

먼저, 우리는 이미지 데이터를 `io.BytesIO` 객체에 저장할 것입니다. 그런 다음, 우리는 `BytesIO` 객체를 `django.core.files.images.ImageField` 인스턴스(이미지 전용으로 특별히 설계된 `File`의 하위 클래스)로 감쌀 것입니다. 이 `ImageFile` 인스턴스를 얻으면, 우리는 `ImageField`의 `save` 메서드에서 사용할 수 있습니다.

참고

`ImageFile`은 `File`과 마찬가지로 파일 또는 파일과 유사한 래퍼입니다. 두 개의 추가 속성인 `width`와 `height`를 제공합니다. `ImageFile`은 이미지를 래핑하는 데 사용하더라도 오류를 발생시키지 않습니다. 예를 들어, `open()`을 사용하여 텍스트 파일을 열고 파일 핸들을 `ImageFile` 생성자에 전달할 수 있습니다. 전달한 이미지 파일이 유효한지 확인하려면 `width` 또는 `height` 속성에 접근해 보십시오. 이 값이 `None`이면 PIL이 이미지 데이터를 디코딩할 수 없었던 것입니다. 이러한 값의 유효성을 스스로 확인하고, 값이 `None`인 경우 예외를 발생시킬 수 있습니다.

Let's have a look at this in practice in a view:

```
from io import BytesIO
from PIL import Image
from django.core.files.images import ImageFile

def index(request, pk):
    # trim out logic for checking if method is POST

    # get a model instance, or create a new one
    m = ExampleModel.objects.get(pk=pk)

    # store the uploaded image in a variable for shorter
    # code
    uploaded_image = request.FILES["image_field"]

    # Load a PIL image instance from the uploaded file
    image = Image.open(uploaded)

    # perform the image resize
    image.thumbnail((150, 150))

    # Create a BytesIO file-like object to store
    image_data = BytesIO()

    # Write the Image data back out to the BytesIO object
    # Retain the existing format from the uploaded image
    image.save(fp=image_data, uploaded_image.format)

    # Wrap the BytesIO containing the image data
    image_file = ImageFile(image_data)

    # Save the wrapped image file data with the original
    # name
    m.image_field.save(uploaded_image.name, image_file)
    # this also saves the model instance
    return redirect("/success-url/")
```

You can see that this is a little bit more code, but it saves on writing the data to disk. You can choose to use either method (or another one that you come up with) depending on your needs.

실제로 이것을 살펴보자:

```
from io import BytesIO
from PIL import Image
from django.core.files.images import ImageFile

def index(request, pk):
    # 메시드가 POST인지 확인하는 로직을 잘라내기

    # 모델 인스턴스를 가져오거나 새로 생성하기
    m = ExampleModel.objects.get(pk=pk)

    # 업로드된 이미지를 변수에 저장하여 코드 단축하기

    uploaded_image = request.FILES["image_field"]

    # 업로드된 파일에서 PIL 이미지 인스턴스 로드하기
    image = Image.open(uploaded)

    # 이미지 크기 조정 수행하기
    image.thumbnail((150, 150))

    # BytesIO 파일과 유사한 객체 생성하여 저장하기
    image_data = BytesIO()

    # 이미지 데이터를 BytesIO 객체에 다시 쓰기# 업로드된 이
    # 미지의 기존 형식 유지하기
    image.save(fp=image_data, uploaded_image.format)

    # 이미지 데이터가 포함된 BytesIO 래핑하기
    image_file = ImageFile(image_data)

    # 원래 이름으로 래핑된 이미지 파일 데이터 저장하기

    m.image_field.save(uploaded_image.name, image_file)
    # 이것은 모델 인스턴스도 저장합니다
    return redirect("/success-url/")
```

idx_1e041c1b를 볼 수 있습니다. 이 idx_da27ebdd는 코드가 조금 더 많지만, 데이터를 디스크에 쓰는 것을 절약합니다. 필요에 따라 두 방법 중 하나(또는 당신이 생각해낸 다른 방법)를 선택할 수 있습니다.

Referring to media in templates

Once we have uploaded a file, we want to be able to refer to it in a template. For an uploaded image, such as a book cover, we will want to display the image on the page. We saw in *Exercise 8.02 – template settings and using MEDIA_URL in templates*, how to build a URL using MEDIA_URL in a template. When working with FileField or ImageField on a model instance, it is not necessary to do this, as Django provides this functionality for you.

The url attribute of FileField will automatically generate the full URL to the media file based on the MEDIA_URL in your settings.

Note

Note that references we make to FileField in this section also apply to ImageField, as it is a subclass of FileField.

This can be used anywhere that you have access to the instance and field, such as in a view or a template. The following example is in a view:

```
instance = ExampleModel.objects.first()
url = instance.file_field.url # Get the URL
```

The following example is in a template (assuming instance has been passed to the template context):

```

```

In the next exercise, we will create a new model with FileField and ImageField and then show how Django can automatically save these. We'll also demonstrate how to retrieve the URL for an uploaded file.

Exercise 8.06 – FileField and ImageField on models

In this exercise, we will create a model with FileField and ImageField. After doing this, we will have to generate a migration and apply it. We'll then change the UploadForm we've been using so it has both FileField and ImageField. The media_example view will be updated to store the uploaded files on the model instance. Finally, we'll add into the example template to show the previously uploaded image:

1. In PyCharm, open the media_example app's models.py file. Create a new model called ExampleModel with two fields: an ImageField named image_field, and a FileField

템플릿에서 미디어 참조하기

파일을 업로드한 후, `weidx_1f1b1723` 템플릿에서 이를 참조할 수 있어야 합니다. 책 표지와 같은 업로드된 이미지의 경우, 페이지에 이미지를 표시하고 싶습니다. Exercise 8.02 – 템플릿 설정 및 템플릿에서 `MEDIA_URL` 사용하기에서 `MEDIA_URL`을 사용하여 URL을 만드는 방법을 보았습니다. 모델 인스턴스에서 `FileField` 또는 `ImageField`를 사용할 때는 Django가 이 기능을 제공하므로 이를 수행할 필요가 없습니다.

`FileField`의 `url` 속성은 설정의 `MEDIA_URL`을 기반으로 미디어 파일에 대한 전체 URL을 자동으로 생성합니다.

참고

이 섹션에서 `FileField`에 대한 참조는 `ImageField`에도 적용됩니다. `ImageField`는 `FileField`의 하위 클래스이기 때문입니다.

인스턴스와 필드에 접근할 수 있는 곳이면 어디에서나 사용할 수 있습니다. 예를 들어 뷰나 템플릿에서 사용할 수 있습니다. 다음 예시는 뷰에서의 예입니다:

```
instance = ExampleModel.objects.first() url =
instance.file_field.url # URL 가져오기
```

다음 예시는 템플릿에서의 예입니다 (인스턴스가 템플릿 컨텍스트에 전달되었다고 가정):

```

```

다음 연습에서는 `FileField`와 `ImageField`가 있는 새 모델을 만들고 Django가 이를 자동으로 저장하는 방법을 보여줍니다. 또한 업로드된 파일의 URL을 검색하는 방법도 시연할 것입니다.

연습 8.06 – 모델의 `FileField` 및 `ImageField`

이번 연습에서는 `FileField`와 `ImageField`가 있는 모델을 생성할 것입니다. 이를 수행한 후에는 마이그레이션을 생성하고 적용해야 합니다. 그런 다음 우리가 사용해온 `UploadForm`을 `FileField`와 `ImageField`를 모두 포함하도록 변경할 것입니다. `media_example` 뷰는 모델 인스턴스에 업로드된 파일을 저장하도록 업데이트될 것입니다. 마지막으로, 이전에 업로드된 이미지를 보여주기 위해 예제 템플릿에 `<img>`를 추가할 것입니다:

1. PyCharm에서 `media_example` 앱의 `models.py` 파일을 엽니다. `image_field`라는 이름의 `ImageField`와 `FileField`가 있는 `ExampleModel`이라는 새 모델을 만듭니다.

called `file_field`. `ImageField` should have `upload_to` set to `images/`, and `FileField` should have `upload_to` set to `files/`. The finished model should look like this:

```
class ExampleModel(models.Model):
    image_field = models.ImageField(upload_to="images/")
    file_field = models.FileField(upload_to="files/")
```

2. Open a terminal and navigate to the `media_project` project directory. Make sure your `djangoenv` virtual environment is active. Run the `makemigrations` management command to generate the migrations for this new model:

```
python manage.py makemigrations
```

Note

To learn how to create and activate a virtual environment, refer to the *Preface* section of the book.

The output should be like the following:

```
% python3 manage.py makemigrations
Migrations for 'media_example':
  media_example/migrations/0001_initial.py
  - Create model ExampleModel
```

3. Apply the migration by running the `migrate` management command:

```
python3 manage.py migrate
```

The output will be as follows:

```
% python3 manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes,
  reviews, sessions
Running migrations:
  # output trimmed for brevity
  Applying media_example.0001_initial... OK
```

file_field이라고 불립니다. ImageField는 upload_to를 images/로 설정해야 하고, FileField는 upload_to를 files/로 설정해야 합니다. 완성된 모델은 다음과 같아야 합니다:

```
class ExampleModel(models.Model):
    image_field = models.ImageField(upload_to="images/")
    file_field = models.FileField(upload_to="files/")
```

2. 터미널을 열고 media_project 프로젝트 디렉토리로 이동합니다. djangoenv 가상 환경이 활성화되어 있는지 확인하세요. makemigrations 관리 명령을 실행하여 이 새로운 모델에 대한 마이그레이션을 생성합니다:

```
python manage.py makemigrations
```

참고

가상 환경을 생성하고 활성화하는 방법을 배우려면 책의 서문 섹션을 참조하십시오.

출력은 다음과 같아야 합니다:

```
% python3 manage.py makemigrations
'media_example'에 대한 마이그레이션:
  media_example/migrations/0001_initial.py
  - 모델 ExampleModel 생성
```

3. 마이그레이션 관리 명령을 실행하여 마이그레이션을 적용하십시오:

```
python3 manage.py migrate
```

출력은 다음과 같을 것입니다:

```
% python3 manage.py
migrateOperations 수행할 작업:

  모든 마이그레이션 적용: admin, auth,
  contenttypes, reviews, sessions

마이그레이션 실행 중:
  # 출력은 간결성을 위해 생략됨
  media_example.0001_initial 적용 중... OK
```

Note that all the initial Django migrations will also be applied since we did not apply those after creating the project.

4. Switch back to PyCharm and open the `media_example` app's `forms.py` file. Rename the existing `ImageField` from `file_upload` to `image_upload`. Then, add a new `FileField` named `file_upload`. After making these changes, your `UploadForm` code should look like this:

```
class UploadForm(forms.Form):
    image_upload = forms.ImageField()
    file_upload = forms.FileField()
```

You can save and close the file.

5. Open the `media_example` app's `views.py` file. First, import `ExampleModel` into the file. To do this, add this line at the top of the file after the existing import statements:

```
from .models import ExampleModel
```

Some imports will no longer be required, so you can remove these lines:

```
from PIL import Image
from django.conf import settings
```

6. In the `media_example` view, set a default for the instance you will render in case one is not created. After the function definition, define a variable called `instance` and set it to `None`:

```
def media_example(request):
    instance = None
```

7. You can completely remove the contents of the `form.is_valid()` branch as you no longer need to manually save the file. Instead, it will automatically be saved when the `ExampleModel` instance is saved. You will instantiate an `ExampleModel` instance and set the file and image fields from the uploaded form. Add this code under the `if form.is_valid():` line:

```
instance = ExampleModel()
instance.image_field = form.cleaned_data["image_upload"]
```

프로젝트를 생성한 후에 적용하지 않았기 때문에 모든 초기 Django 마이그레이션도 적용될 것임을 유의하십시오.

4. `idx_99c1e20e`를 PyCharm으로 다시 전환하고 `media_example` 앱의 `forms.py` 파일을 엽니다. 기존의 `ImageField`를 `file_upload`에서 `image_upload`로 이름을 변경합니다. 그런 다음 `file_upload`라는 새로운 `FileField`를 추가합니다. 이러한 `idx_4ba363d0` 변경을 한 후, `UploadForm` 코드는 다음과 같아야 합니다:

```
class UploadForm
    (forms.Form):image_upload =
    forms.ImageField()file_upload =
    forms.FileField()
```

파일을 저장하고 닫을 수 있습니다.

5. `media_example` 앱의 `views.py` 파일을 엽니다. 먼저, `ExampleModel`을 파일에 가져옵니다. 이를 위해 기존 `import` 문 다음에 파일의 맨 위에 이 줄을 추가합니다:

```
from .models import ExampleModel
```

일부 임포트는 더 이상 필요하지 않으므로, 이 줄들을 제거할 수 있습니다:

```
from PIL import Image
from django.conf import settings
```

6. `media_example` 뷰에서 렌더링할 인스턴스가 없는 경우를 대비해 기본값을 설정하세요. 생성되지 않았습니다. 함수 정의 후에 `instance`라는 변수를 정의하고 그것을 다음과 같이 설정하세요 없음:

```
def media_example(request):
    ...
    인스턴스 = 없음
```

7. 폼의 `.is_valid()` 분기의 내용을 완전히 제거할 수 있습니다. 더 이상 수동으로 `idx_377a2b5b` 파일을 저장할 필요가 없습니다. 대신, `ExampleModel` 인스턴스가 저장될 때 자동으로 저장됩니다. 업로드된 폼에서 파일 및 이미지 필드를 설정하고 `ExampleModel` 인스턴스를 인스턴스화합니다. `if form.is_valid():` 줄 아래에 이 코드를 추가하세요:

```
instance = ExampleModel()
instance.image_field = form.cleaned_data["image_upload"]
```

```
instance.file_field = form.cleaned_data["file_upload"]
instance.save()
```

8. Pass the instance through to the template in the context dictionary that is passed to render. Use the instance key:

```
return render(request, "media-example.html",
              {"form": form, "instance": instance})
```

You can now save and close this file.

9. Open the `media-example.html` template. Add an `<img>` element that displays the last uploaded image. Under the closing `</form>` tag, add an `if` template tag that checks whether instance has been provided. If so, display `<img>` with a `src` attribute of `instance.image_field.url`:

```
{% if instance %}
    
{% endif %}
```

10. Start the Django dev server if it's not already running, then navigate to `http://127.0.0.1:8000/media-example/`. You should see the form rendered with two fields:

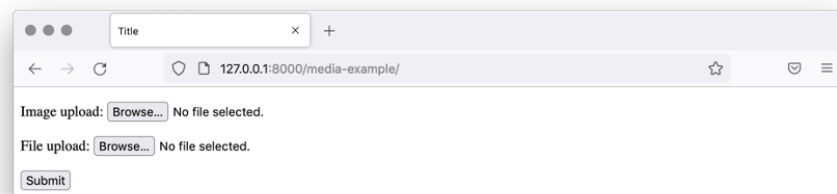


Figure 8.19: UploadForm with two fields

11. Select a file for each field – for `ImageField`, you must select an image, but any type of file is allowed for `FileField`. See Figure 8.20, which shows the fields with files selected:

```
instance.file_field = form.cleaned_data["file_upload"]
instance.save()
```

8. 템플릿에 전달할 때 컨텍스트 사전에서 인스턴스를 전달하세요. 인스턴스 키를 사용하세요:

```
return render(request, "media-example.html",
              {"form": form, "instance": instance})
```

이제 이 파일을 저장하고 달을 수 있습니다.

9. media-example.html 템플릿을 엽니다. 마지막으로 업로드된 이미지를 표시하는 `<img>` 요소를 추가합니다. 닫는 `</form>` 태그 아래에 인스턴스가 제공되었는지 확인하는 `if` 템플릿 태그를 추가합니다. 그렇다면 `src` 속성이 `instance.image_field.url`인 `<img>`를 표시합니다:

```
{% if instance %}
    
{% endif %}
```

10. `idx_6a7ee81dDjango devidx_9f17f5a8` 서버가 이미 실행 중이지 않다면 시작하고, `http://127.0.0.1:8000/media-example/`로 이동합니다. 두 개의 필드가 있는 양식이 렌더링된 것을 볼 수 있어야 합니다:

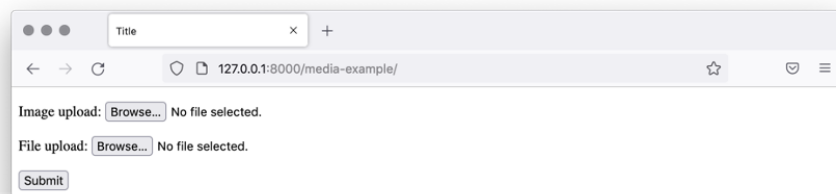


그림 8.19: 두 개의 필드가 있는 UploadForm

11. 각 필드에 대해 파일을 선택합니다 - ImageField의 경우 이미지를 선택해야 하지만 FileField에는 모든 유형의 파일이 허용됩니다. 선택된 파일이 있는 필드를 보여주는 그림 8.20을 참조하세요:

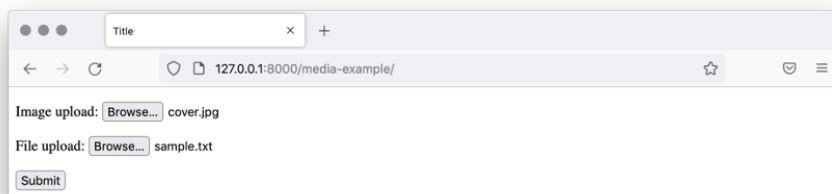


Figure 8.20: ImageField and FileField with files selected

Then, submit the form. If the submission is successful, the page will reload, and the last image we uploaded will be displayed (Figure 8.21):

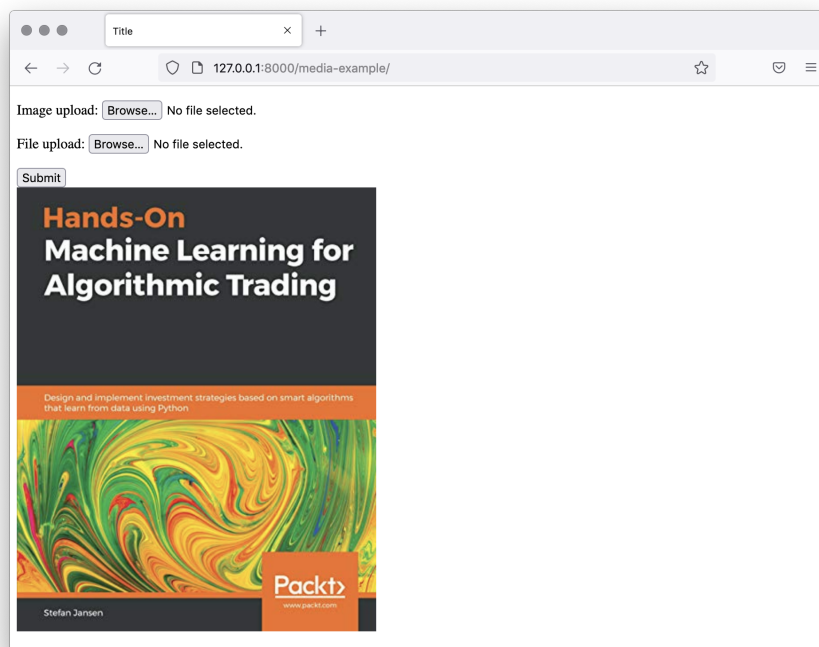


Figure 8.21: The last image that was uploaded is displayed

12. You can see how Django stores the files by looking in the MEDIA_ROOT directory. Figure 8.22 shows the directory layout in PyCharm:

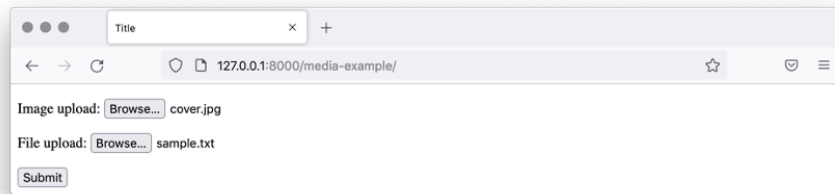


그림 8.20: 선택된 파일이 있는 ImageField 및 FileField

그런 다음 idx_839f63f6 양식을 제출합니다. 제출이 성공하면 페이지가 idx_4d95e951 다시 로드되고, 우리가 업로드한 마지막 이미지가 표시됩니다(그림 8.21):

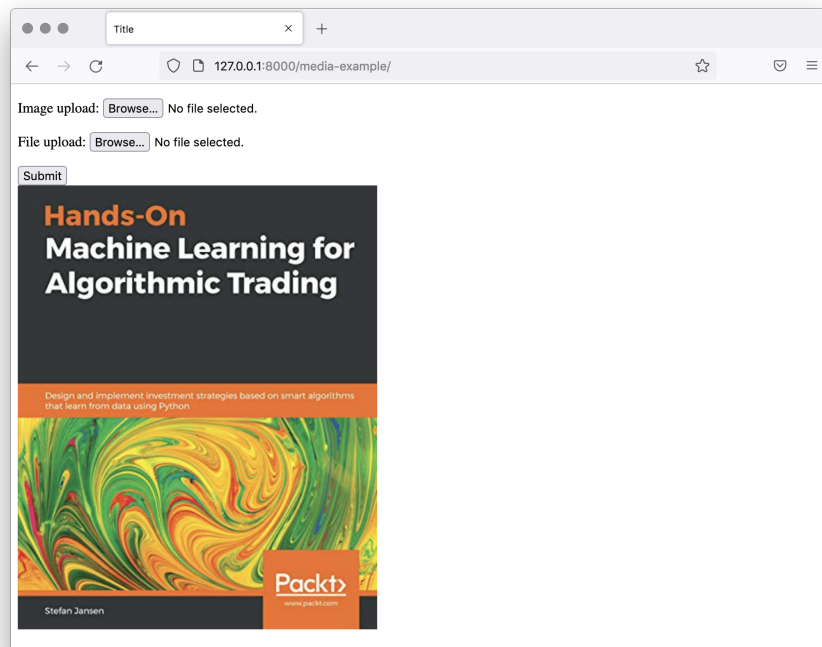


그림 8.21: 업로드된 마지막 이미지가 표시됩니다

12. MEDIA_ROOT 디렉토리를 보면 Django가 파일을 어떻게 저장하는지 idx_cbe5abeb 확인할 수 있습니다. 그림 8.22는 PyCharm의 디렉토리 레이아웃을 보여줍니다:

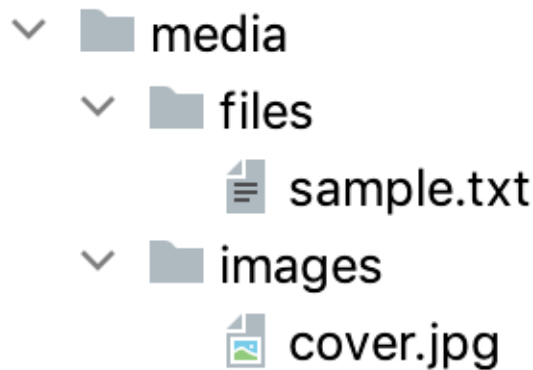


Figure 8.22: Uploaded files that Django has created

You can see that Django has created the `files` and `images` directories. These were what you set in the `upload_to` arguments on `ImageField` and `FileField` of the model.

We could also verify these uploads by attempting to download them, for example, at `http://127.0.0.1:8000/media/files/sample.txt` or `http://127.0.0.1:8000/media/images/cover.jpg`.

In this exercise, we created `ExampleModel` with `FileField` and `ImageField` and saw how to store uploaded files on it. We saw how to generate a URL to an uploaded file for use in a template. We tried uploading some files and saw that Django automatically created the `upload_to` directories (`media/files` and `media/images`) and then stored the files inside.

In the next section, we will look at how we can simplify the process even further by using `ModelForm` to generate the form and save the model without having to manually set the files in the view.

ModelForm and file uploads

We've seen how using `form.ImageField` on a form can prevent non-images from being uploaded. We've also seen how `models.ImageField` makes it easy to store an image for a model. But we need to be aware that Django does not stop us from setting a non-image file to `ImageField`. For example, consider a form that has both `FileField` and `ImageField`:

```
class ExampleForm(forms.Form):
    uploaded_file = forms.FileField()
    uploaded_image = forms.ImageField()
```

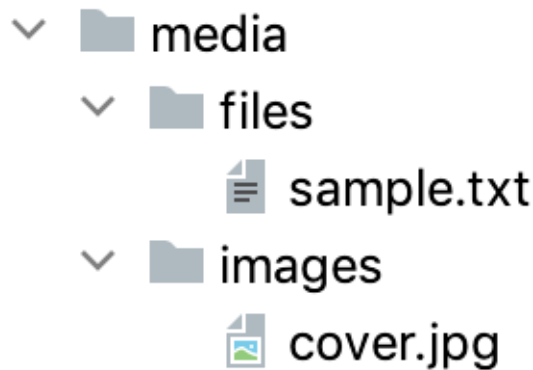


그림 8.22: Django가 생성한 업로드된 파일

Django가 파일 및 이미지 디렉토리를 생성한 것을 볼 수 있습니다. 이것은 모델의 ImageField 및 FileField에서 upload_to 인수에 설정한 것입니다.

예를 들어, `http://127.0.0.1:8000/media/files/sample.txt` 또는 `http://127.0.0.1:8000/media/images/cover.jpg`에서 다운로드를 시도하여 이러한 업로드를 확인할 수 있습니다.

이번 연습에서는 `idx_f4ebe852`가 FileField와 ImageField로 ExampleModel을 생성하고 업로드된 파일을 저장하는 방법을 보았습니다. 우리는 템플릿에서 사용할 업로드된 파일의 URL을 생성하는 방법을 보았습니다. 우리는 몇 개의 파일을 업로드해 보았고 Django가 자동으로 upload_to 디렉토리(media/files 및 media/images)를 생성한 후 파일을 그 안에 저장한 것을 보았습니다.

다음 섹션에서는 ModelForm을 사용하여 양식을 생성하고 뷰에서 파일을 수동으로 설정하지 않고도 모델을 저장하는 과정을 더욱 간소화하는 방법을 살펴보겠습니다.

ModelForm 및 파일 업로드

우리는 양식에서 `form.ImageField`를 사용하여 이미지가 업로드되는 것을 방지하는 방법을 보았습니다. 또한 `models.ImageField`가 모델에 이미지를 저장하는 것을 쉽게 만드는 방법도 보았습니다. 그러나 Django가 ImageField에 이미지 파일을 설정하는 것을 막지 않는다는 점을 인식해야 합니다. 예를 들어, FileField와 ImageField가 모두 있는 양식을 고려해 보십시오:

```
class ExampleForm
    (forms.Form):uploaded_file =
    forms.FileField()uploaded_image =
    forms.ImageField()
```

In the following view, the form would not validate if the `uploaded_image` field on the form was not an image, so some data validity is ensured for uploaded data:

```
def view(request):
    form = ExampleForm(request.POST, request.FILES)
    if form.is_valid():
        m = ExampleModel()
        m.file_field = form.cleaned_data["uploaded_file"]
        m.image_field = forms.cleaned_data["uploaded_image"]
        m.save()
    return render(request, "template.html")
```

Since we're sure that the form is valid, we know that `forms.cleaned_data["uploaded_image"]` must contain an image. Therefore, we would never assign a non-image to the model instance's `image_field`.

However, say we made a mistake in our code and wrote something such as the following:

```
m.image_field = forms.cleaned_data["uploaded_file"]
```

That is, if we accidentally reference `FileField` by mistake, Django does not validate that a (potential) non-image is being assigned to `ImageField`, and so it does not throw an exception or generate any kind of error. We can mitigate the potential for issues such as this by using `ModelForm`.

We introduced `ModelForm` in *Chapter 7, Advanced Form Validation and Model Forms*. This is a type of form whose fields are automatically defined from a model. We saw that `ModelForm` has a `save` method that automatically creates or updates the model data in the database. When used with a model that has `FileField` or `ImageField`, then the `save` method of `ModelForm` will also save uploaded files.

Here's an example of using `ModelForm` to save a new model instance in a view. Here, we are just making sure to pass `request.FILES` to the `ModelForm` constructor:

```
class ExampleModelForm(forms.ModelForm):
    class Meta:
        # The same ExampleModel class we've seen previously
        model = ExampleModel
        fields = "__all__"

    def view(request):
```

다음 보기에서, 양식의 `uploaded_image` 필드가 이미지가 아닌 경우 양식이 유효성을 검사하지 않으므로 `idx_69fd75a6` 일부 데이터 유효성이 업로드된 데이터에 대해 보장됩니다:

```
def view(request):
    form = ExampleForm(request.POST, request.FILES)
    if form.is_valid():
        m = ExampleModel()
        m.file_field = form.cleaned_data["uploaded_file"]
        m.image_field = forms.cleaned_data["uploaded_image"]
        m.save()
    return render(request, "template.html")
```

우리는 `idx_486e6c62` form이 유효하다는 것을 확신하므로, `forms.cleaned_data["uploaded_image"]`에는 이미지가 포함되어야 한다는 것을 알고 있습니다. 따라서 우리는 모델 인스턴스의 `image_field`에 비이미지를 할당하지 않을 것입니다.

하지만, 우리가 코드에서 실수를 하고 다음과 같은 것을 작성했다고 가정해 봅시다:

```
m.image_field = forms.cleaned_data["uploaded_file"]
```

즉, 실수로 `FileField`를 참조하는 경우, Django는 `ImageField`에 비이미지가 할당되는 것을 검증하지 않으므로 예외를 발생시키거나 어떤 종류의 오류를 생성하지 않습니다. 우리는 `ModelForm`을 사용하여 이러한 문제의 가능성을 완화할 수 있습니다.

우리는 7장, 고급 폼 검증 및 모델 폼에서 `ModelForm`을 소개했습니다. 이것은 필드가 모델에서 자동으로 정의되는 일종의 폼입니다. 우리는 `ModelForm`이 데이터베이스에서 모델 데이터를 자동으로 생성하거나 업데이트하는 `save` 메서드를 가지고 있다는 것을 보았습니다. `FileField` 또는 `ImageField`가 있는 모델과 함께 사용될 때, `ModelForm`의 `save` 메서드는 업로드된 파일도 저장합니다.

다음은 뷰에서 새로운 모델 인스턴스를 저장하기 위해 `ModelForm`을 사용하는 예입니다. 여기서는 `request.FILES`를 `ModelForm` 생성자에 전달하는 것을 확실히 하고 있습니다:

```
class ExampleModelForm(forms.ModelForm):
    class Meta:
        # 우리가 이전에 본 것과 동일한 ExampleModel 클래스
        model = ExampleModel
        fields = "__all__"

    def view(request):
```

```
if request.method == "POST":
    form = ExampleModelForm(request.POST,
                             request.FILES)

    form.save()
    return redirect("/success-page")
else:
    form = ExampleModelForm()
    return (request, "template.html", {"form": form})
```

As with any `ModelForm`, the `save` method can be called with the `commit` argument set to `False`. Then, the model instance will not be saved to the database, and the `FileField` and `ImageField` files will not be saved to disk. The `save` method should be called on the model instance itself – this will commit changes to the database and save the files. In this next short example, we set a value on the model instance before saving it:

```
def view(request):
    if request.method == "POST":
        form = ExampleModelForm(request.POST,
                                 request.FILES)
        m = form.save(False)
        # Set arbitrary value on the model instance
        # before save
        m.attribute = "value"
        # save the model instance, also write the
        # files to disk
        m.save()
        return redirect("/success-page/")
    else:
        form = ExampleModelForm()
        return (request, "template.html", {"form": form})
```

Calling the `save` method on the model instance saves both the model data to the database and the uploaded files to disk. In the next exercise, we will build `ModelForm` from the `ExampleModel` we created in *Exercise 8.06 – FileField and ImageField on models*, then test uploading files with it.

```

if request.method == "POST":
    form = ExampleModelForm(request.POST,
                             request.FILES)

    form.save()
    return redirect("/success-page")
else:
    form = ExampleModelForm()
    return (request, "template.html", {"form": form})

```

모든 ModelForm과 마찬가지로, save 메서드는 commit 인자를 False로 설정하여 호출할 수 있습니다. 그러면 모델 인스턴스는 데이터베이스에 저장되지 않고, FileField와 ImageField 파일도 디스크에 저장되지 않습니다. save 메서드는 모델 인스턴스 자체에서 호출되어야 하며, 이는 데이터베이스에 변경 사항을 커밋하고 파일을 저장합니다. 다음 짧은 예제에서는 저장하기 전에 모델 인스턴스에 값을 설정합니다:

```

def view(request):
    ...
    if request.method == "POST":
        form = ExampleModelForm(request.POST,
                                request.FILES)
        m = form.save(False)

        ...
        # 저장 전에 모델 인스턴스에 임의의 값 설정하기 #
        ...

        m.attribute = "value"
        # 모델 인스턴스를 저장하고, 또한 파일을 디스크에 기록합니다

        m.save()
        return redirect("/success-page/")
    else:
        form = ExampleModelForm()
        return (request, "template.html", {"form": form})

```

모델 인스턴스에서 save 메서드 idx_d4e218d0을 호출하면 모델 데이터가 idx_b34e66e2 데이터베이스에 저장되고 업로드된 파일이 디스크에 저장됩니다. 다음 연습에서는 Exercise 8.06 – 모델의 FileField 및 ImageField에서 만든 ExampleModel로부터 ModelForm을 구축한 다음, 이를 사용하여 파일 업로드를 테스트할 것입니다.

Exercise 8.07 – file and image uploads using ModelForm

In this exercise, you will update `UploadForm` to be a subclass of `ModelForm` and have it built automatically from `ExampleModel`. You will then change the `media_example` view to save the instance automatically from the form, so you can see how the amount of code can be reduced:

1. In PyCharm, open the `media_example` app's `forms.py` file. You need to use `ExampleModel` in this chapter, so use `import` to import it at the top of the file after `from django import forms` statement. Insert this line:

```
from .models import ExampleModel
```

2. Change `UploadForm` to be a subclass of `forms.ModelForm`. Remove the class body and replace it with a `class Meta` definition. Its `model` setting should be `ExampleModel`. Set the `fields` attribute to `__all__`. After completing this step, `UploadForm` should look like this:

```
class UploadForm(forms.ModelForm):  
    class Meta:  
        model = ExampleModel  
        fields = "__all__"
```

Save and close the file.

3. Open the `media_example` app's `views.py` file. Since you no longer need to reference `ExampleModel` directly, you can remove its import at the top of the file. Remove the following line:

```
from .models import ExampleModel
```

4. In the `media_example` view, remove the entirety of the `form.is_valid()` branch and replace it with a single line:

```
instance = form.save()
```

The form's `save` method will handle persisting the instance to the database and saving the files. It will return an instance of `ExampleModel`, the same as the other `ModelForm` that we worked with in *Chapter 7, Advanced Form Validation and Model Forms*.

Save and close `views.py`.

연습 8.07 – ModelForm을 사용한 파일 및 이미지 업로드

이 연습에서는 UploadForm을 ModelForm의 하위 클래스로 업데이트하고 ExampleModel에서 자동으로 빌드되도록 합니다. 그런 다음 media_example 뷰를 변경하여 양식에서 인스턴스를 자동으로 저장하여 코드의 양을 어떻게 줄일 수 있는지 확인할 수 있습니다:

1. PyCharm에서 media_example 앱의 forms.py 파일을 엽니다. 이 장에서는 ExampleModel을 사용해야 하므로, django에서 forms를 가져오는 문장 다음에 파일의 맨 위에서 import를 사용하여 가져옵니다. 이 줄을 삽입하세요:

```
from .models import ExampleModel
```

2. UploadForm을 forms.ModelForm의 하위 클래스로 변경합니다. 클래스 본문을 제거하고 클래스 Meta 정의로 교체합니다. 모델 설정은 ExampleModel이어야 합니다. fields 속성을 __all__로 설정합니다. 이 단계를 완료한 후 UploadForm은 다음과 같아야 합니다:

```
class UploadForm(forms.ModelForm):
    class Meta:
        model = ExampleModel
        fields = "__all__"
```

파일을 저장하고 닫습니다.

3. media_example 앱의 views.py 파일을 엽니다. 더 이상 ExampleModel을 직접 참조할 필요가 없으므로, 파일 상단의 import를 제거할 수 있습니다. 다음 줄을 제거하세요:

```
from .models import ExampleModel
```

4. media_example 뷰에서 form.is_valid() 분기를 전체적으로 제거하고 단일 줄로 교체합니다:

```
instance = form.save()
```

품의 저장 메서드는 인스턴스를 데이터베이스에 지속시키고 파일을 저장하는 작업을 처리합니다. 이는 7장, 고급 품 검증 및 모델 품에서 작업한 다른 ModelForm과 동일한 ExampleModel의 인스턴스를 반환합니다.

views.py를 저장하고 닫습니다.

5. Start the Django dev server if it's not already running, then navigate to `http://127.0.0.1:8000/media-example/`. You should see the form rendered with two fields, **Image field** and **File field** (Figure 8.23):

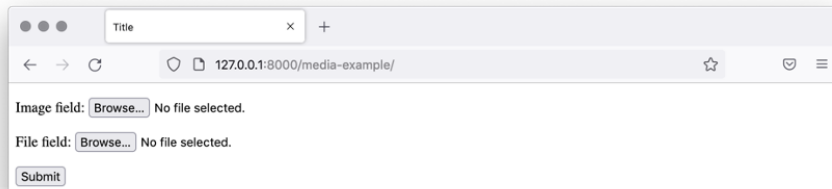
A screenshot of a web browser window. The address bar shows the URL `127.0.0.1:8000/media-example/`. The form contains two fields: 'Image field' and 'File field'. Each field has a 'Browse...' button and the text 'No file selected.' below it. At the bottom of the form is a 'Submit' button.

Figure 8.23: UploadForm as a ModelForm rendered in the browser

Note that the names of these fields now match those of the model rather than the form, since the form just uses the model's fields.

6. Browse and select an image and file (Figure 8.24), then submit the form:

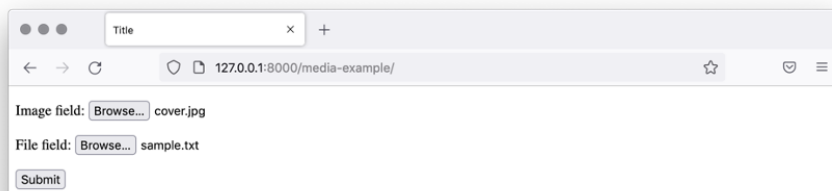
A screenshot of a web browser window, similar to the previous one, but with selected files. The 'Image field' now shows 'cover.jpg' next to the 'Browse...' button. The 'File field' now shows 'sample.txt' next to the 'Browse...' button. The 'Submit' button remains at the bottom.

Figure 8.24: Image and file selected

7. The page will reload, and as in *Exercise 8.06 – FileField and ImageField on models*, you will see the previously uploaded image (Figure 8.25):

5. Django 개발 서버가 이미 실행 중이지 않다면 시작하고, <http://127.0.0.1:8000/media-example/>로 이동합니다. 두 개의 필드인 이미지 필드와 파일 필드가 렌더링된 양식을 볼 수 있어야 합니다(그림 8.23):

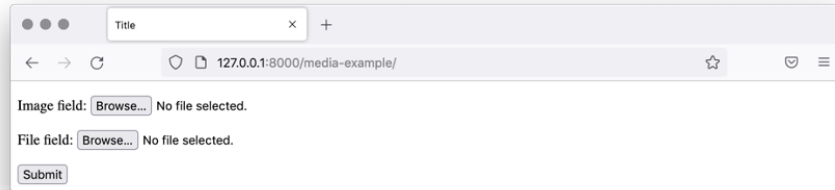


그림 8.23: 브라우저에 렌더링된 ModelForm으로서의 UploadForm

이 필드의 이름이 이제 양식이 아닌 모델의 이름과 일치한다는 점에 유의하세요. 양식 `idx_a2797e08`은 모델의 필드만 사용합니다.

6. 이미지와 파일을 찾아 선택한 후(그림 8.24), 양식을 제출합니다:

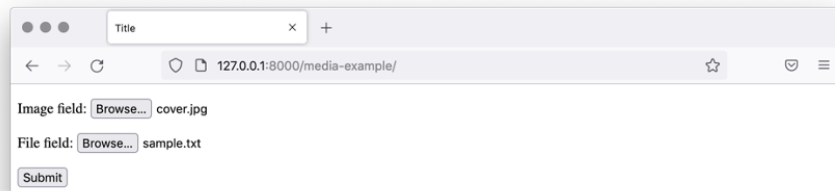


그림 8.24: 선택된 이미지와 파일

7. 페이지가 새로 고쳐지고, 연습 8.06 - 모델의 FileField 및 ImageField에서와 같이, 이전에 업로드한 이미지를 볼 수 있습니다 (그림 8.25):

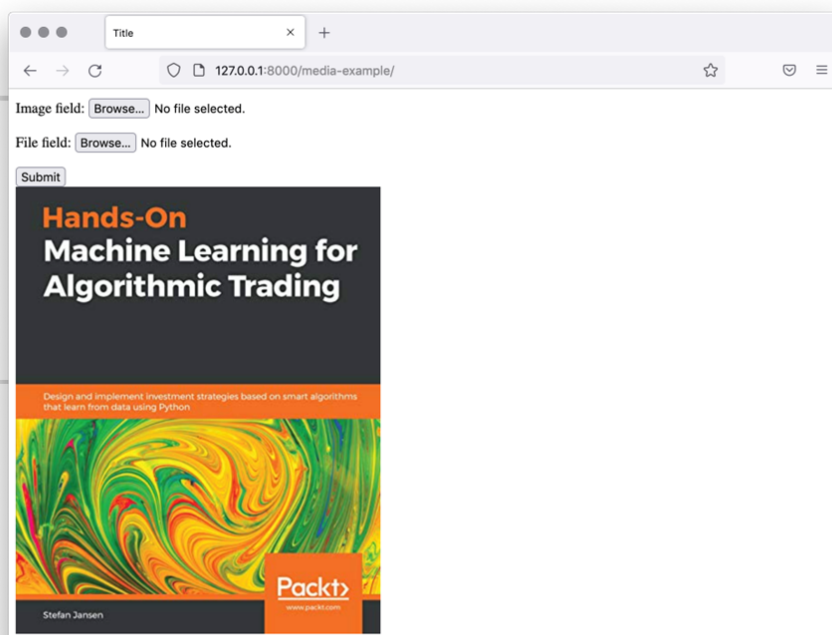


Figure 8.25: Image being displayed after upload

8. Finally, examine the contents of the `media` directory. You should see that the directory layout matches that of *Exercise 8.06 – FileField and ImageField on models*, with images inside the `images` directory and files inside the `files` directory:

```
▼ media
  ▼ files
    sample.txt
    sample_oMGmUVy.txt
  ▼ images
    cover.jpg
    cover_uf1yjUz.jpg
```

Figure 8.26: The uploaded files directory matches Exercise 6

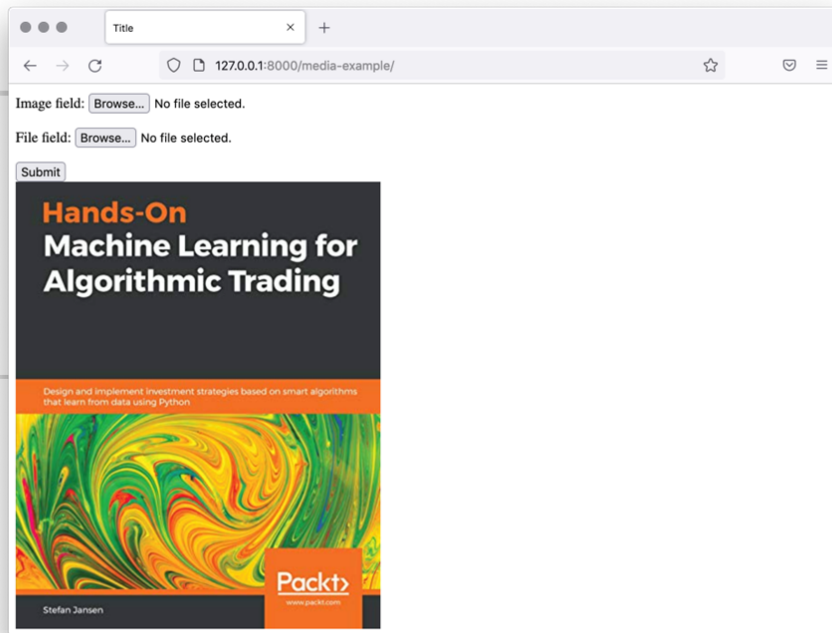


그림 8.25: 업로드 후 표시되는 이미지

8. 마지막으로, 미디어 디렉토리의 `idx_071073b7contents`를 확인하십시오. 디렉토리 레이아웃이 모델의 `FileField` 및 `ImageField`와 일치하는 Exercise 8.06과 일치하는 것을 볼 수 있어야 하며, 이미지 디렉토리 안에는 이미지가 있고, 파일 디렉토리 안에는 파일이 있습니다:

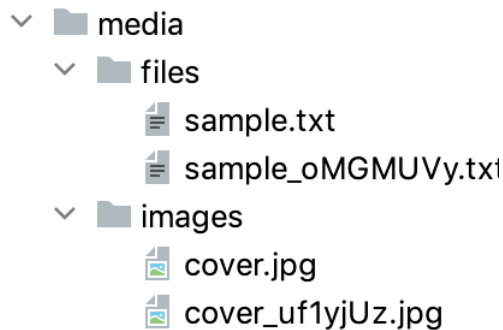


그림 8.26: 업로드된 파일 디렉토리는 Exercise 6과 일치합니다.

In this exercise, we changed `UploadForm` to a `ModelForm` subclass, which allowed us to automatically generate the upload fields. We could replace the code that stored the uploaded files on the models with a call to the form's `save` method.

Before moving on to the activities for this chapter, we need to briefly discuss how to handle file saving when using an instance.

Handling file saving

As you know from working with `ModelForms` in *Chapter 7*, you can pass an `instance` argument to `ModelForm` to cause it to update an existing model instead of creating a new one when saving.

For example, do the following to update an example model in a view:

```
def update_view(request, pk):
    instance = ExampleModel.objects.get(pk=pk)
    form = UploadForm(request.POST, request.FILES,
                      instance=instance)
    if form.valid():
        form.save()
```

When accessing the `cleaned_data` attribute of the form, we will get back the data that was contained in `request.POST`. If there was no value for that key, then we'll get the value from the instance instead.

This normally doesn't cause any problems, but when working with `ImageField` or `FileField`, the type of data being returned will differ from the request or the instance.

If a file was uploaded, then the cleaned data will be `InMemoryUploadedFile` or `TemporaryUploadedFile` (depending on its size; only small files will fit in memory). If no file was uploaded, then the type will be `FieldFile` for `FileField` or `ImageFieldFile` for `ImageField`.

To explain, consider this code:

```
def update_view(request, pk):
    instance = ExampleModel.objects.get(pk=pk)
    form = UploadForm(request.POST, request.FILES,
                      instance=instance)
    if form.is_valid():
```

이 연습에서는 UploadForm을 ModelForm 서브클래스로 변경하여 업로드 필드를 자동으로 생성할 수 있게 되었습니다. 우리는 업로드된 파일을 모델에 저장하는 코드를 폼의 save 메서드를 호출하는 것으로 대체할 수 있었습니다.

이 장의 idx_85a0bf9b 활동으로 넘어가기 전에 인스턴스를 사용할 때 파일 저장을 처리하는 방법에 대해 간단히 논의해야 합니다.

파일 저장 처리

7장에서 ModelForms로 작업하면서 알다시피, ModelForm에 인스턴스 인수를 전달하면 저장할 때 새 모델을 생성하는 대신 기존 모델을 업데이트하도록 할 수 있습니다.

예를 들어, 뷰에서 예제 모델을 업데이트하려면 다음과 같이 하십시오:

```
def update_view(request, pk):
    instance = ExampleModel.objects.get(pk=pk)
    form = UploadForm(request.POST, request.FILES,
                      instance=instance)
    if form.valid():
        form.save()
```

폼의 cleaned_data 속성에 접근할 때, 우리는 request.POST에 포함된 데이터를 다시 얻게 됩니다. 해당 키에 대한 값이 없으면, 대신 인스턴스에서 값을 가져옵니다.

이것은 일반적으로 문제를 일으키지 않지만, ImageField 또는 FileField로 작업할 때 반환되는 데이터의 유형이 요청 또는 인스턴스와 다를 수 있습니다.

파일이 업로드되었다면, 정리된 데이터는 InMemoryUploadedFile 또는 TemporaryUploadedFile이 될 것입니다(크기에 따라; 작은 파일만 메모리에 적합합니다). 파일이 업로드되지 않았다면, 유형은 FileField의 FieldFile 또는 ImageField의 ImageFieldFile이 될 것입니다.

설명을 위해, 이 코드를 고려해 보세요:

```
def update_view(request, pk):
    instance = ExampleModel.objects.get(pk=pk)
    form = UploadForm(request.POST, request.FILES,
                      instance=instance)
    if form.is_valid():
```

```
form_file = form.cleaned_data["file_field"]
form_image = form.cleaned_data["image_field"]
```

In the case where data was uploaded (that is, the user selected an image and file in the form), then `form_file` and `form_image` will be `InMemoryUploadedFile` or `TemporaryUploadedFile`. However, if the user did not upload anything for the fields (assuming the fields are not required in the form), then `form_file` will be a `FieldFile` instance, and `form_image` will be an `ImageFieldFile` instance.

To differentiate between them, there are a few different checks you could do. One of the simplest methods is to check for the existence of a `path` attribute. Since uploaded files haven't been saved to disk yet, they have no `path`:

```
if hasattr(form_file, "path"):
    print("This is a FieldFile")
else:
    print("This is an uploaded file")
```

Sometimes you will need to check what kind of file you are working with before performing some action, and perhaps skip that action if the user hasn't uploaded a new file. For example, we wouldn't want to resize an image that was already assigned to a model. We'd only do the image resize on newly uploaded images:

```
if hasattr(image_file, "path"):
    print("This image was on the instances so no "
          "need to resize")
else:
    perform_resize(image_file)
```

You will need to make a check like this in the first activity for this chapter.

We've now covered everything you need to start enhancing Bookr with file uploads. In the activity for this chapter, we'll add support for uploading a cover image and sample document (PDF, text file, and more) for a book. The book cover will be resized using PIL before it is saved.

Activity 8.01 – image and PDF upload for books

In this activity, you will start by cleaning up (deleting) the example views, templates, forms, models, and URL maps that we were using throughout the exercises in this chapter. You'll then need to generate and apply a migration to delete `ExampleModel` from the database.

```
form_file = form.cleaned_data["file_field"]
form_image = form.cleaned_data["image_field"]
```

데이터가 업로드된 경우(즉, 사용자가 양식에서 이미지와 파일을 선택한 경우), `form_file` 과 `form_image`는 `InMemoryUploadedFile` 또는 `TemporaryUploadedFile`이 됩니다. 그러나 사용자가 필드에 대해 아무것도 업로드하지 않은 경우(필드가 양식에서 필수가 아니라고 가정할 때), `form_file`은 `FieldFile` 인스턴스가 되고, `form_image`는 `ImageFieldFile` 인스턴스가 됩니다.

`idx_8d7fac57`를 구분하기 위해 몇 가지 다른 검사를 수행할 수 있습니다. 가장 간단한 방법 중 하나는 경로 속성의 존재를 확인하는 것입니다. 업로드된 파일은 아직 디스크에 저장되지 않았기 때문에 경로가 없습니다:

```
if hasattr(form_file, "path"):
    print("이것은 FieldFile입니다")
else:
    print("이것은 업로드된 파일입니다")
```

때때로 어떤 작업을 수행하기 전에 어떤 종류의 파일을 다루고 있는지 확인해야 하며, 사용자가 새 파일을 업로드하지 않았다면 그 작업을 건너뛰어야 할 수도 있습니다. 예를 들어, 이미 모델에 할당된 이미지를 크기 조정하고 싶지 않을 것입니다. 우리는 새로 업로드된 이미지에 대해서만 이미지 크기 조정을 수행할 것입니다:

```
if hasattr(image_file, "path"):
    print("이 이미지는 인스턴스에 있었으므로 "
          "크기 조정이 필요하지 않음")
else:
    perform_resize(image_file)
```

이 장의 첫 번째 활동에서 이러한 확인을 수행해야 합니다.

이제 파일 업로드로 Bookr을 향상시키기 위해 필요한 모든 것을 다루었습니다. 이 장의 활동에서는 책의 표지 이미지와 샘플 문서(PDF, 텍스트 파일 등)를 업로드하는 지원을 추가할 것입니다. 책 표지는 저장되기 전에 PIL을 사용하여 크기가 조정됩니다.

활동 8.01 – 책을 위한 이미지 및 PDF 업로드

이 활동에서는 예제 뷰, 템플릿, 양식, 모델 및 URL 맵을 정리(삭제)하는 것으로 시작합니다. 그런 다음 `ExampleModel`을 데이터베이스에서 삭제하기 위해 마이그레이션을 생성하고 적용해야 합니다.

You can then start adding the Bookr enhancements, first by adding `ImageField` and `FileField` to the `Book` model to store the book cover and sample. Then you will create a migration and apply it to add these fields to the database. You can then build a form that will display just these new fields. You'll add a view that uses this form to save the model instance with the uploaded files after first resizing the image to a thumbnail size. You will be able to reuse the `instance-form.html` template from *Chapter 7, Advanced Form Validation and Model Forms*, with a minor change to allow file uploads.

These steps will help you complete the activity:

1. Update the Django settings to add the `MEDIA_ROOT` and `MEDIA_URL` settings.
2. The `/media/` URL mapping should be added to the main `urls.py` file. Use the `static` view and utilize `MEDIA_ROOT` and `MEDIA_URL` from the Django settings. Remember, this mapping should only be added if `DEBUG` is `True`.
3. Add `ImageField` (named `cover`) and `FileField` (named `sample`) to the `Book` model. The fields should upload to `book_covers/` and `book_samples/`, respectively. They should both allow the `null` and `blank` values.
4. Run `makemigrations` and `migrate` again to apply the `Book` model changes to the database.
5. Create `BookMediaForm` as a subclass of `ModelForm`. Its model should be `Book`, and the fields should only be the fields you added in step 3.
6. Add a `book_media` view. This will not allow you to create `Book`; instead, it will only allow you to add media to an existing `Book` (so it must take `pk` as a required argument).
7. The `book_media` view should validate the form and use `save` to save it, but not commit the instance. The uploaded cover should first be resized using the `thumbnail` method, as demonstrated in the *Writing PIL images to ImageField* section. The maximum size should be 300 px by 300 px. It should then be stored on the instance, and the instance should be saved. Remember that the `cover` field is not required, so you should check this before trying to manipulate the image, and you won't need to resize the image if one is already on the instance. On a successful `POST`, register a success message that `Book` was updated, then redirect to the `book_detail` view.
8. Render `instance-form.html`, passing a context dictionary containing `form`, `model_type`, and `instance`, as you did in *Chapter 6, Forms*. Also, pass another item, `is_file_upload`, set to `True`. This variable will be used in the next step.

그런 다음 Book 모델에 ImageField와 FileField를 추가하여 책 표지와 샘플을 저장하는 Bookr 개선 사항을 추가할 수 있습니다. 그런 다음 마이그레이션을 생성하고 이를 데이터베이스에 이러한 필드를 추가하기 위해 적용합니다. 그런 다음 이러한 새로운 필드만 표시하는 양식을 만들 수 있습니다. 먼저 이미지를 썸네일 크기로 조정한 후 업로드된 파일로 모델 인스턴스를 저장하기 위해 이 양식을 사용하는 뷰를 추가할 것입니다. 파일 업로드를 허용하기 위해 Chapter 7, 고급 양식 검증 및 모델 양식의 instance-form.html 템플릿을 약간 수정하여 재사용할 수 있습니다.

이 단계들은 활동을 완료하는 데 도움이 될 것입니다:

1. Django 설정을 업데이트하여 MEDIA_ROOT 및 MEDIA_URL 설정을 추가합니다. /media/ URL 매핑은 main urls.py 파일에 추가되어야 합니다.
2. 다. 정적 뷰를 사용하고 Django 설정에서 MEDIA_ROOT 및 MEDIA_URL을 활용합니다. 이 매핑은 DEBUG가 True일 때만 추가되어야 합니다.
3. Book 모델에 ImageField(이름: cover)와 FileField(이름: sample)를 추가합니다. 필드는 각각 book_covers/ 및 book_samples/에 업로드되어야 합니다. 두 필드 모두 null 및 빈 값을 허용해야 합니다.
4. makemigrations를 실행하고 migrate를 다시 실행하여 Book 모델 변경 사항을 데이터베이스에 적용하세요.
5. BookMediaForm을 ModelForm의 하위 클래스로 만드세요. 그 모델은 Book이어야 하며, 필드는 3단계에서 추가한 필드만 포함해야 합니다.
6. book_media 뷰를 추가하세요. 이 뷰는 Book을 생성할 수 없으며, 대신 기존 Book에 미디어를 추가할 수만 있습니다(따라서 pk를 필수 인수로 받아야 합니다). book_media 뷰는 폼을 검증하고 save를 사용해 저장하되, 인스턴스는 커밋하지 않아야 합니다. 업로드된 커버는 먼저 Writing PIL images to ImageField 섹션에서 보여준 thumbnail 메서드를 사용해 크기를 조정해야 합니다. 최대 크기

300px x 300px여야 합니다. 그런 다음 인스턴스에 저장하고 인스턴스를 저장해야 합니다. cover 필드는 필수가 아니므로 이미지를 조작하기 전에 이를 확인해야 하며, 인스턴스에 이미지가 이미 있는 경우 크기를 조정할 필요가 없습니다. POST가 성공하면 "Book was updated"라는 성공 메시지를 등록한 후 book_detail 뷰로 리디렉션합니다.

8. 6장, 폼에서 했던 것처럼 form, model_type, instance를 포함하는 컨텍스트 딕셔너리를 전달하여 instance-form.html을 렌더링하세요. 또한, is_file_upload라는 항목을 하나 더 전달하는데, 이 값은 True로 설정합니다. 이 변수는 다음 단계에서 사용됩니다.

9. In the `instance-form.html` template, use the `is_file_upload` variable to add the correct enctype attribute to the form. This will allow you to switch the modes for the form to enable file uploads when required.
10. Finally, add a URL map that maps `/books/<pk>/media/` to the `book_media` view.

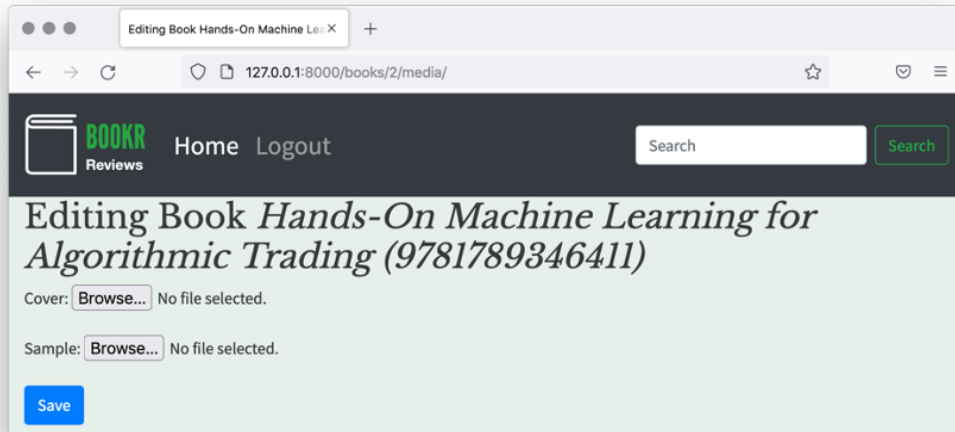


Figure 8.27: BookMediaForm in the browser

Select a cover image and a sample file for the book. You can use the image at `Chapter08/Activity8.01/bookr/media/book_covers/machine-learning-for-algorithmic-trading.png` and the PDF at `Chapter08/Activity8.01/bookr/media/book_samples/machine-learning-for-trading.pdf` (or you can use any other image/PDF of your choosing).

9. instance-form.html 템플릿에서 is_file_upload 변수를 사용하여 양식에 올바른 enctype 속성을 추가합니다. 이렇게 하면 필요할 때 파일 업로드를 활성화하기 위해 양식의 모드를 전환할 수 있습니다.
10. 마지막으로, /books/<pk>/media/를 book_media 뷰에 매핑하는 URL 맵을 추가합니다.

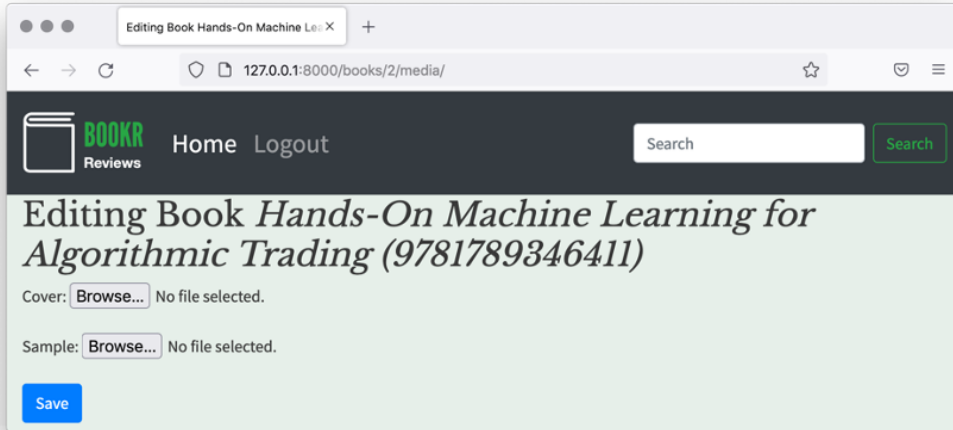


그림 8.27: 브라우저의 BookMediaForm

책을 위한 표지 이미지와 샘플 idx_efe2e851file을 선택합니다. Chapter08/의 이미지를 사용할 수 있습니다.

Activity8.01/bookr/media/book_covers/machine-learning-for-algorithmictrading.png 및 Chapter08/Activity8.01/bookr/media/book_samples/machine-learning-for-trading.pdf 의 PDF를 사용할 수 있습니다(또는 원하는 다른 이미지/PDF를 사용할 수 있습니다).

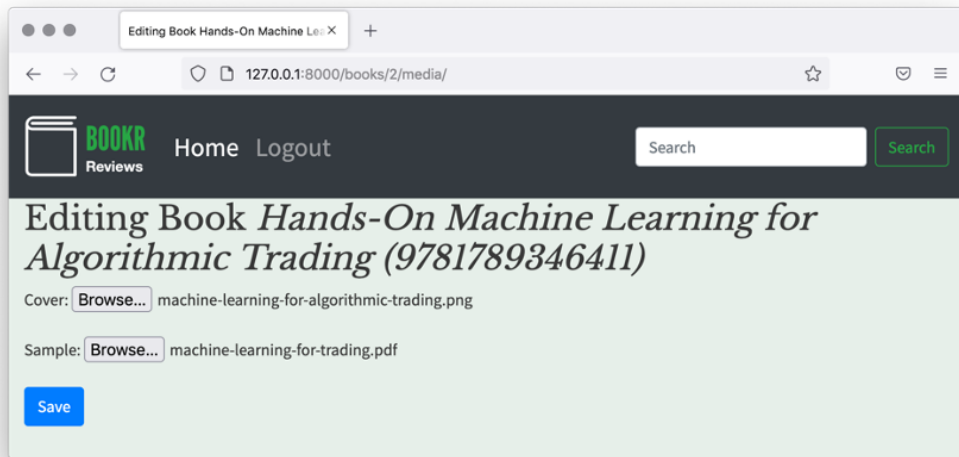


Figure 8.28: Book cover and sample selected

After submitting the form, you will be redirected to the **Book Details** view and see the success message (Figure 8.29):

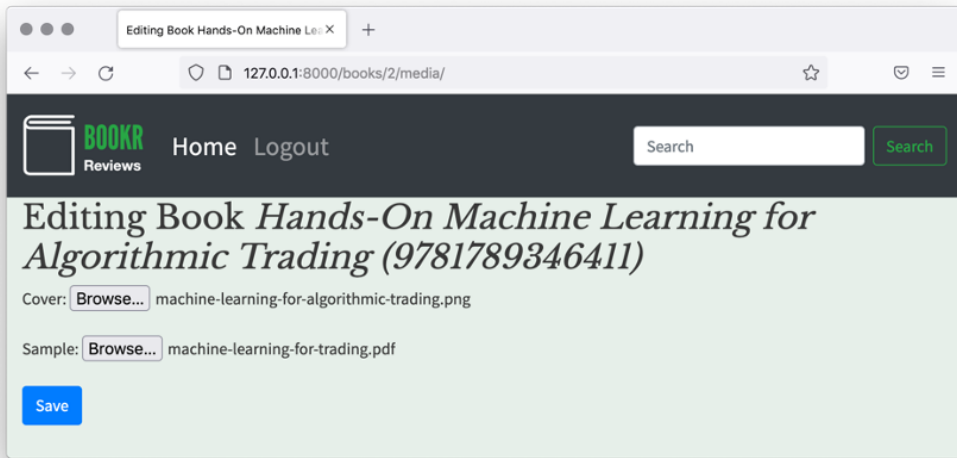


그림 8.28: 책 표지 및 샘플 선택됨

idx_9b570427form을 제출한 후, 책 세부정보 보기로 리디렉션되며 성공 메시지(그림 8.29)를 보게 됩니다:

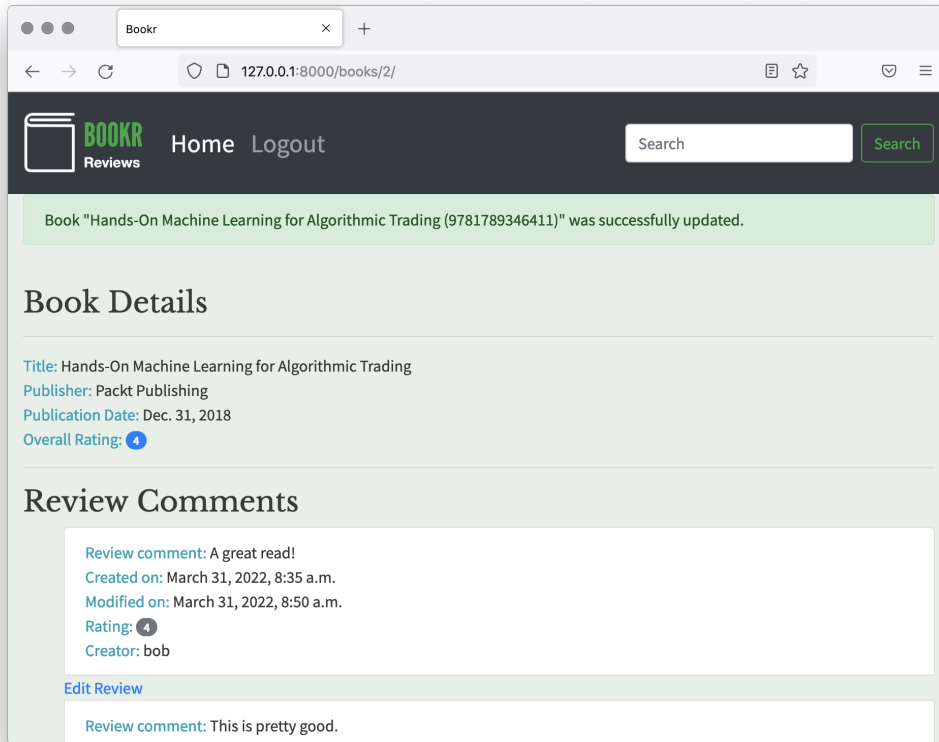


Figure 8.29: Success message on the Book Details page

If you go back to the same book's media page, you should see that the fields are now filled in with an option to clear the data from them:

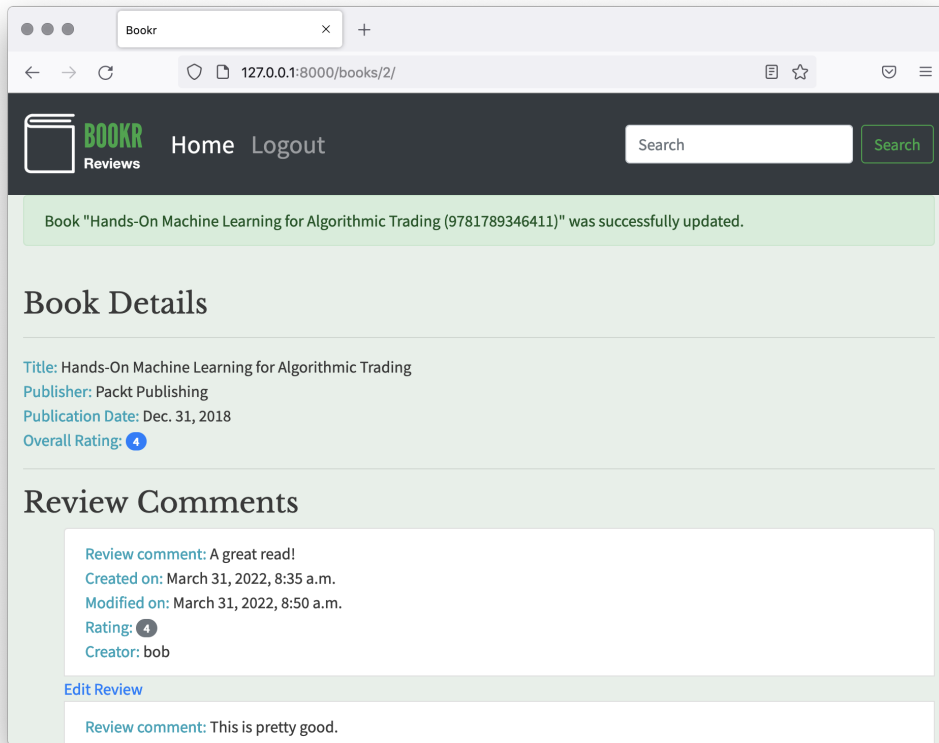


그림 8.29: 도서 세부정보 페이지의 성공 메시지

idx_0afd728asame 도서의 미디어 페이지로 돌아가면 필드가 이제 채워져 있으며 데이터를 지울 수 있는 옵션이 표시됩니다:

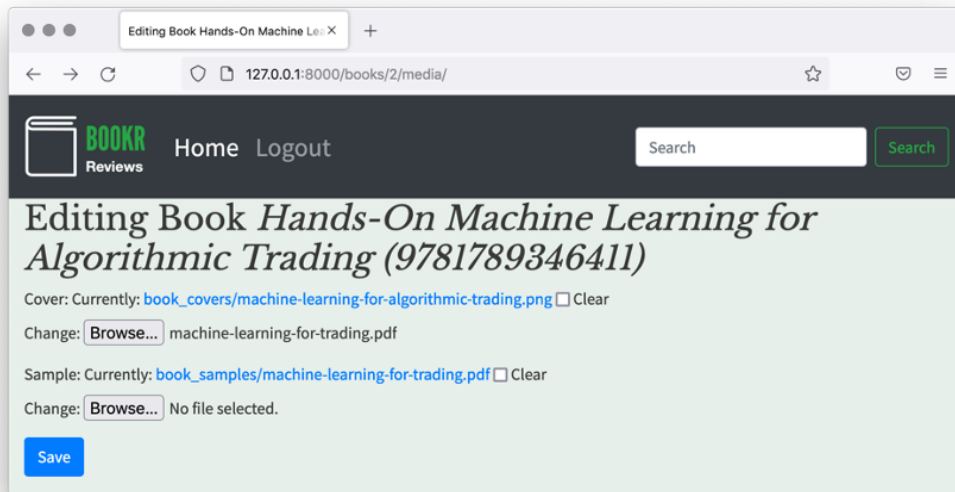


Figure 8.30: BookMediaForm with existing values

In *Activity 8.02 – displaying the cover and sample link*, you will add these uploaded files to the **Book Details** view, but for now, if you want to check that uploads have worked, you can look inside the media directory in the Bookr project:

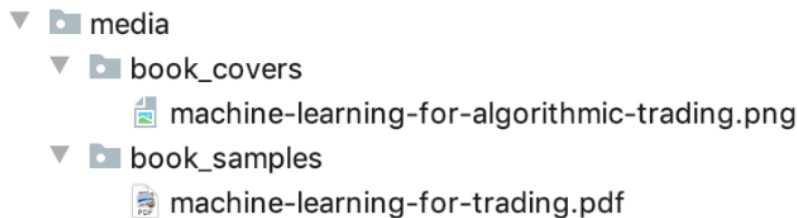


Figure 8.31: Book media

You should see the directories that were created and the uploaded files, as per *Figure 8.31*. Open an uploaded image, and you should see that its maximum dimension is 300 px.

Activity 8.02 – displaying the cover and sample link

In this activity, you will update the `book_detail.html` template to show the cover for Book (if one is set). You will also add a link to download the sample again, only if one is set. You will use the `FileField` and `ImageField` `url` attributes to generate the URLs to the media files.

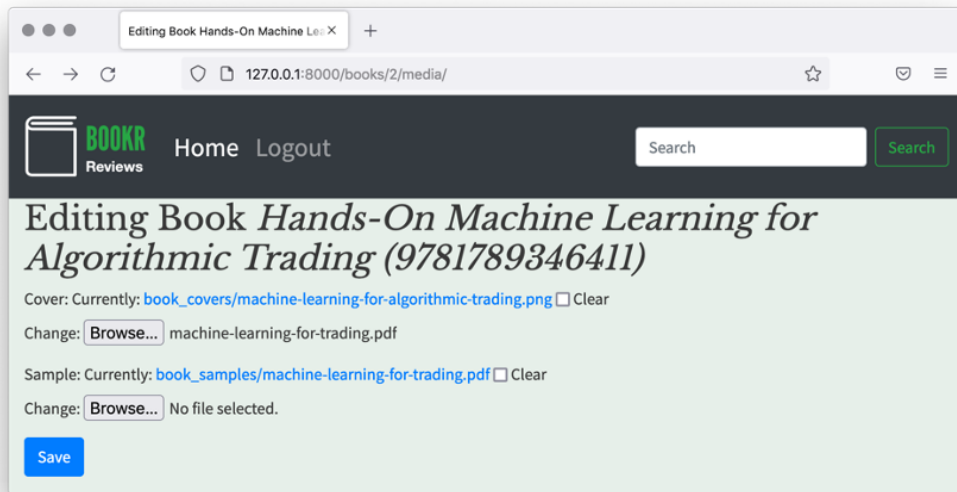


그림 8.30: 기존 값이 있는 BookMediaForm

활동 8.02 - 표지 및 샘플 링크 표시에서, 업로드한 파일을 Book Details 보기로 추가할 것입니다. 하지만 지금은 업로드가 제대로 되었는지 확인하고 싶다면, Bookr 프로젝트의 미디어 디렉토리 안에 있는 idx_5c1db5f5를 확인할 수 있습니다:

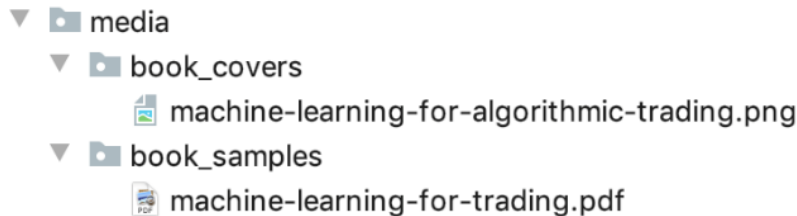


그림 8.31: 책 미디어

그림 8.31에 따라 생성된 디렉토리와 업로드된 파일을 볼 수 있어야 합니다. 업로드된 이미지를 열면 최대 크기가 300 px임을 확인할 수 있습니다.

활동 8.02 - 표지 및 샘플 링크 표시

이 활동에서는 book_detail.html 템플릿을 업데이트하여 책의 표지를 표시합니다(설정된 경우). 또한 샘플을 다시 다운로드할 수 있는 링크를 추가합니다. 단, 설정된 경우에만 가능합니다. FileField 및 ImageField URL 속성을 사용하여 미디어 idx_d5e82be6 파일에 대한 URL을 생성합니다.

These steps will help you complete this activity:

1. Inside the **Book Details** display in the `book_detail.html` view, add an `<img>` element if the book has a cover image. Then, display the cover of the book inside it. Use `<br>` after the `<img>` tag, so the image is on its own line.
2. After the **Publication Date** display, add a link to the sample file. It should only be displayed if a sample file has been uploaded. Make sure you add another `<br>` tag, so it displays correctly.
3. In the section that has a link to add a review, add another link that goes to the media page for the book. Follow the same styling as the **Add Review** link.

When you've completed these steps, you should be able to load a **Book Details** page. If the book has no cover or sample, then the page should look very similar to what it did before, except you should see the new link to the **Media** page at the bottom (*Figure 8.32*):

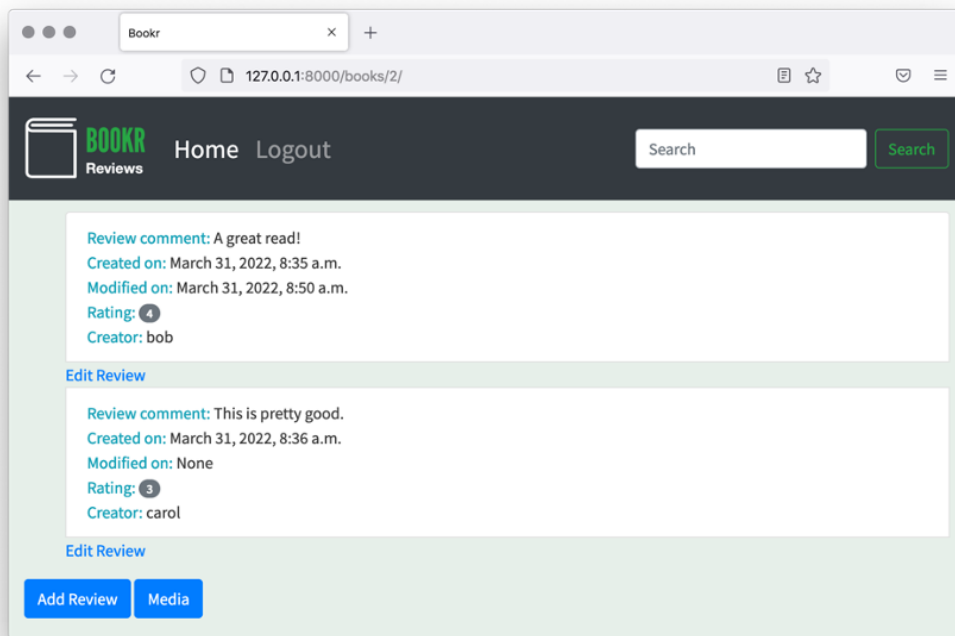


Figure 8.32: The new Media button visible on the Book Details page

Once you have uploaded a cover and/or sample for a book, the cover image and the sample link should be displayed (*Figure 8.33*):

이 단계는 이 활동을 완료하는 데 도움이 될 것입니다:

1. book_detail.html 뷰의 책 세부정보 표시 내에서 책에 표지 이미지가 있는 경우 `<img>` 요소를 추가합니다. 그런 다음 그 안에 책의 표지를 표시합니다. `<img>` 태그 뒤에 `<br>`을 사용하여 이미지가 별도의 줄에 표시되도록 합니다.
2. 출판 날짜 표시 후 샘플 파일에 대한 링크를 추가합니다. 샘플 파일이 업로드된 경우에만 표시되어야 합니다. 올바르게 표시되도록 또 다른 `<br>` 태그를 추가해야 합니다.
3. 리뷰 추가 링크가 있는 섹션에 책의 미디어 페이지로 가는 또 다른 링크를 추가합니다. 리뷰 추가 링크와 동일한 스타일을 따릅니다.

이 단계를 완료하면 책 세부정보 페이지를 로드할 수 있어야 합니다. 책에 표지나 샘플이 없으면 페이지는 이전과 매우 유사하게 보이지만, 하단에 미디어 페이지로 가는 새 링크가 표시되어야 합니다(그림 8.32):

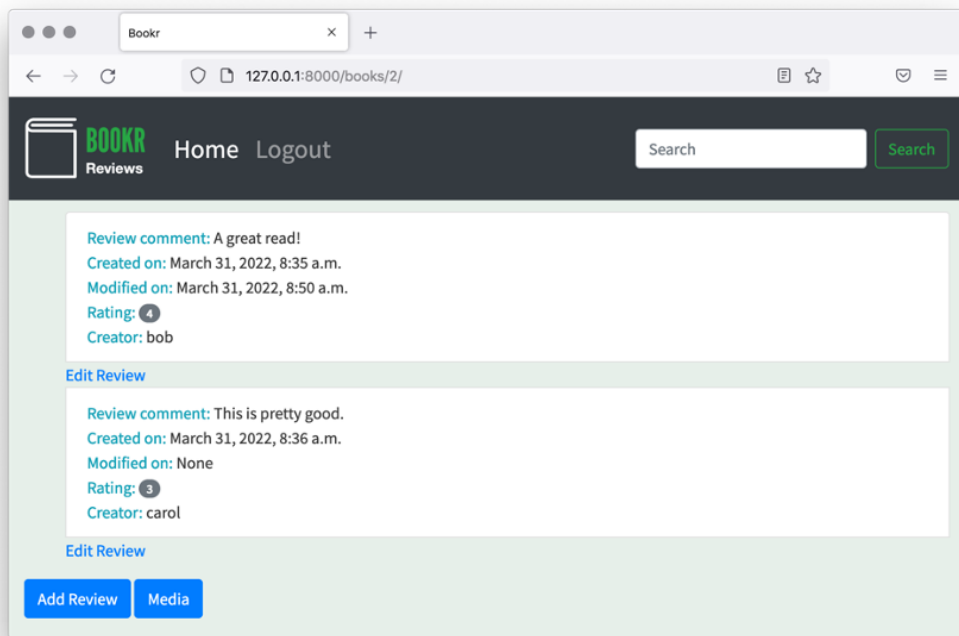


그림 8.32: 책 세부정보 페이지에 표시된 새 미디어 버튼

책에 대한 표지 및/또는 샘플을 업로드한 후에는 표지 이미지와 샘플 링크가 표시되어야 합니다(그림 8.33):

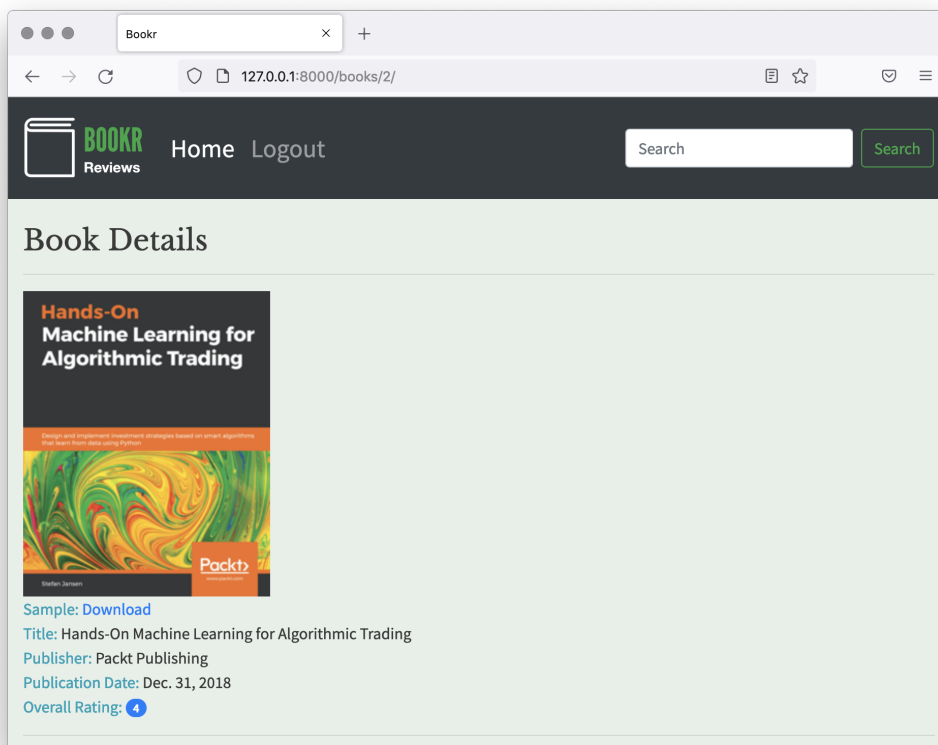


Figure 8.33: The book cover and the Sample link displayed

Summary

In this chapter, we added the `MEDIA_ROOT` and `MEDIA_URL` settings and a special URL map to serve media files. We then created a form and a view to upload files and save them to the media directory. We saw how to add the media context processor to automatically have access to the `MEDIA_URL` setting in all our templates. We then enhanced and simplified our form code by using a Django form with `FileField` or `ImageField`, instead of manually defining one in HTML.

We looked at some of the enhancements Django provides for images with `ImageField`, and how to interact with an image using Pillow. We showed an example view that would be able to serve files that required authentication using the `FileResponse` class. Then, we saw how to store files on models using `FileField` and `ImageField` and refer to them in a template using

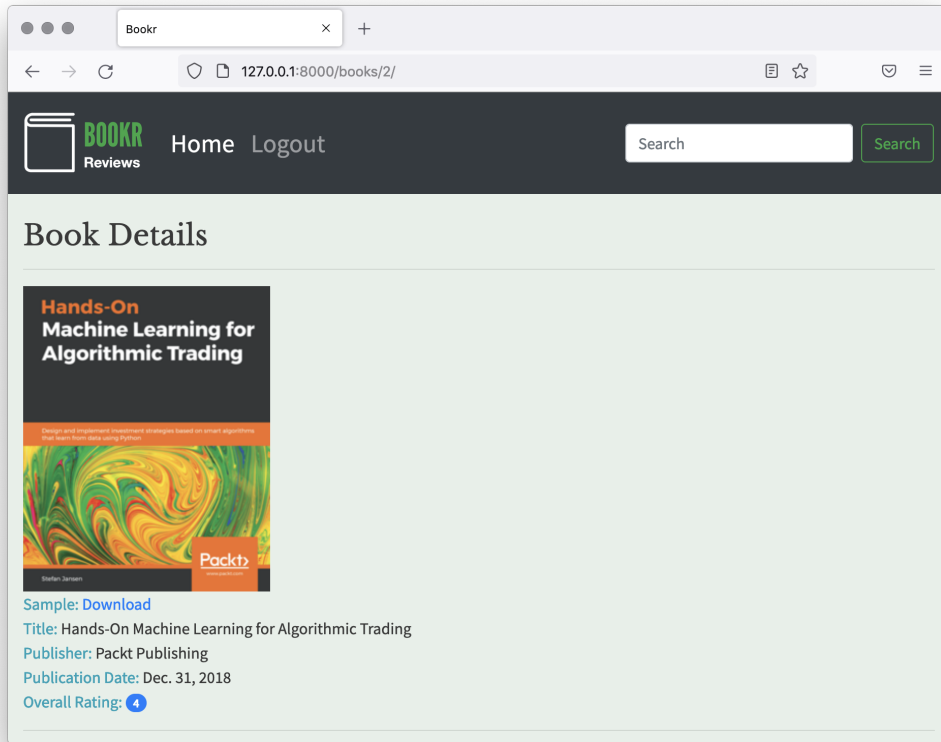


그림 8.33: 책 표지와 샘플 링크가 표시됨

요약

이 장에서는 MEDIA_ROOT 및 MEDIA_URL 설정과 미디어 파일을 제공하기 위한 특별한 URL 맵을 추가했습니다. 그런 다음 파일을 업로드하고 미디어 디렉토리에 저장하기 위한 폼과 뷰를 만들었습니다. 모든 템플릿에서 MEDIA_URL 설정에 자동으로 접근할 수 있도록 미디어 컨텍스트 프로세서를 추가하는 방법을 보았습니다. 그런 다음 HTML에서 수동으로 정의하는 대신 FileField 또는 ImageField가 있는 Django 폼을 사용하여 폼 코드를 개선하고 단순화했습니다.

우리는 ImageField로 이미지를 위한 Django가 제공하는 몇 가지 향상된 기능을 살펴보고, Pillow를 사용하여 이미지와 상호작용하는 방법을 보았습니다. 인증이 필요한 파일을 제공할 수 있는 예제 뷰를 보여주었고, FileResponse 클래스를 사용했습니다. 그런 다음 FileField 및 ImageField를 사용하여 모델에 파일을 저장하고 템플릿에서 참조하는 방법을 보았습니다.

the `FileField.url` attribute. We were able to reduce the amount of code we had to write by automatically building `ModelForm` from a model instance. Finally, we carried out two activities to enhance Bookr by adding a cover image and a sample file to the Book model.

In *Chapter 9, Sessions and Authentication*, we will learn how to add authentication to a Django application to protect it from unauthorized users.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

FileField.url 속성. 우리는 모델 인스턴스에서 ModelForm을 자동으로 생성하여 작성해야 할 코드의 양을 줄일 수 있었습니다. 마지막으로, Book 모델에 표지 이미지와 샘플 파일을 추가하여 Bookr를 향상시키기 위한 두 가지 활동을 수행했습니다.

9장, 세션 및 인증에서는 Django 애플리케이션에 인증을 추가하여 무단 사용자로부터 보호하는 방법을 배웁니다.

이 책의 PDF 사본, 코드 번들 등을 받으세요

QR 코드를 스캔하거나 (packtpub.com/unlock으로 이동) 이 책을 이름으로 검색하고, 판을 확인한 후 페이지의 단계를 따르세요.



UNLOCK NOW

참고: 송장 사본을 준비하세요. Packt 웹사이트에서 직접 구매한 경우 송장이 필요하지 않습니다.

9

Sessions and Authentication

So far, we have used Django to develop dynamic applications that allow users to interact with application models, but we have not attempted to secure these applications from unwanted use. For example, our bookr project allows unauthenticated users to add reviews and upload media. This is a critical security issue for any online web app, as it leaves the site open to the posting of spam or other inappropriate material and the vandalism of existing content. We want the creation and modification of content to be strictly limited to authenticated users who have registered with the site.

The **authentication app** supplies Django with the models for representing users, groups, and permissions. It also provides middleware, utility functions, decorators, and mixins that help us integrate user authentication into our apps. Furthermore, the authentication app allows us to group and name certain sets of users.

In *Chapter 4, Introduction to Django Admin*, we used the Admin app to create a help desk user group with the Can view log entry, the Can view permission, the Can change user, and the Can view user permissions. Those permissions could be referenced in our code using their corresponding codenames: `view_logentry`, `view_permissions`, `change_user`, and `view_user`. In this chapter, we will learn how to customize Django behavior based on specific user permissions.

Permissions control what different types of users can do in your application. Permissions can be assigned either to groups or directly to individual users. From an administrative point of view, it is cleaner to assign permissions to groups. Groups make it easier to model roles and organizational structures. If a new permission is created, it is less time-consuming to modify a few groups than to remember to assign it to a subset of users.

We are already familiar with creating users and groups and assigning permissions using several methods, such as the option of instantiating users and groups through the model using

9

세션 및 인증

지금까지 우리는 Django를 사용하여 사용자가 애플리케이션 모델과 상호작용할 수 있는 동적 애플리케이션을 개발했지만, 이러한 애플리케이션을 원치 않는 사용으로부터 보호하려고 시도하지는 않았습니다. 예를 들어, 우리의 bookr 프로젝트는 인증되지 않은 사용자가 리뷰를 추가하고 미디어를 업로드할 수 있도록 허용합니다. 이는 스팸이나 기타 부적절한 자료의 게시 및 기존 콘텐츠의 훼손에 대해 사이트를 열어두기 때문에 모든 온라인 웹 앱에 대한 중요한 보안 문제입니다. 우리는 콘텐츠의 생성 및 수정을 사이트에 등록된 인증된 사용자로 엄격히 제한하고자 합니다.

인증 앱은 사용자, 그룹 및 권한을 나타내는 모델을 Django에 제공합니다. 또한 사용자 인증을 앱에 통합하는 데 도움이 되는 미들웨어, 유틸리티 함수, 데코레이터 및 믹스인을 제공합니다. 게다가 인증 앱은 특정 사용자 집합을 그룹화하고 이름을 지정할 수 있게 해줍니다.

4장, Django Admin 소개에서는 Admin 앱을 사용하여 로그 항목 보기, 권한 보기, 사용자 변경 및 사용자 보기 권한을 가진 헬프 데스크 사용자 그룹을 생성했습니다. 이러한 권한은 해당 코드 이름(view_logentry, view_permissions, change_user, view_user)을 사용하여 코드에서 참조할 수 있습니다. 이번 장에서는 특정 사용자 권한에 따라 Django 동작을 사용자 정의하는 방법을 배울 것입니다.

권한은 다양한 유형의 사용자가 애플리케이션에서 수행할 수 있는 작업을 제어합니다. 권한은 그룹에 할당하거나 개별 사용자에게 직접 할당할 수 있습니다. 관리 관점에서 볼 때, 권한을 그룹에 할당하는 것이 더 깔끔합니다. 그룹은 역할 및 조직 구조를 모델링하는 데 더 쉽게 해줍니다. 새로운 권한이 생성되면, 사용자 하위 집합에 할당하는 것을 기억하는 것보다 몇 개의 그룹을 수정하는 것이 덜 시간 소모적입니다.

우리는 이미 여러 방법을 사용하여 사용자 및 그룹을 생성하고 권한을 할당하는 것에 익숙합니다. 예를 들어 모델을 통해 사용자 및 그룹을 인스턴스화하는 옵션을 사용하여

scripts and the convenience of creating them through the Django Admin app. The authentication app also offers us programmatic ways of creating and deleting users, groups, and permissions, and assigning relationships between them.

As we go through this chapter, we'll learn how to use authentication and permissions to implement application security and how to store user-specific data to customize the user's experience. This will help us secure the bookr project from unauthorized content changes and make it contextually relevant for different types of users. Adding this basic security to our bookr project is crucial before we consider deploying it on the internet.

This chapter begins with a brief introduction to **middleware** before delving into the concepts of **authentication models** and **session engines**. You will implement Django's authentication model to restrict permissions to only a specific set of users. Then, you will learn how to leverage Django authentication to provide a flexible approach to application security. After that, you will learn how Django supports multiple session engines to retain user data. By the end of this chapter, you will be proficient at using sessions to retain information on past user interactions and to maintain user preferences for when pages are revisited.

We will cover the following topics in this chapter:

- Middleware
- Password storage in Django
- Authentication decorators and redirection
- Enhancing templates with authentication data
- Sessions

Technical requirements

Find the solution in the Chapter09 folder in this book's GitHub repo. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

Middleware

Authentication, as well as session management (which we'll learn about in the *Sessions* section), is handled by something known as a **middleware stack**. Before we implement authentication in our bookr project, let's learn a bit about this middleware stack and its modules.

In *Chapter 3, URL Mapping, Views, and Templates*, we discussed Django's implementation of the request/response process, along with its view and rendering functionality. In addition to these, another feature that plays an extremely important role when it comes to Django's core web

스크립트와 Django Admin 앱을 통해 이를 생성하는 편리함. 인증 앱은 또한 사용자, 그룹 및 권한을 생성하고 삭제하는 프로그래밍 방식의 방법을 제공하며, 이들 간의 관계를 할당할 수 있습니다.

이 장을 진행하면서 우리는 인증 및 권한을 사용하여 애플리케이션 보안을 구현하는 방법과 사용자 경험을 맞춤화하기 위해 사용자별 데이터를 저장하는 방법을 배울 것입니다. 이는 무단 콘텐츠 변경으로부터 bookr 프로젝트를 보호하고 다양한 유형의 사용자에게 맥락적으로 관련성을 부여하는 데 도움이 될 것입니다. 인터넷에 배포하기 전에 bookr 프로젝트에 이 기본 보안을 추가하는 것이 중요합니다.

이 장은 미들웨어에 대한 간략한 소개로 시작한 후 인증 모델 및 세션 엔진의 개념으로 깊이 들어갑니다. 특정 사용자 집합에만 권한을 제한하기 위해 Django의 인증 모델을 구현할 것입니다. 그런 다음 Django 인증을 활용하여 애플리케이션 보안에 대한 유연한 접근 방식을 제공하는 방법을 배울 것입니다. 그 후, Django가 사용자 데이터를 유지하기 위해 여러 세션 엔진을 지원하는 방법을 배울 것입니다. 이 장이 끝날 때쯤이면 과거 사용자 상호작용에 대한 정보를 유지하고 페이지가 다시 방문될 때 사용자 기본 설정을 유지하기 위해 세션을 사용하는 데 능숙해질 것입니다.

이 장에서는 다음 주제를 다룰 것입니다: 미들

- 웨어
- 장고에서의 비밀번호 저장
- 인증 데코레이터 및 리디렉션 인증 데이터
- 로 템플릿 항상 세션
-

기술 요구 사항

이 책의 GitHub 리포지토리의 Chapter09 폴더에서 솔루션을 찾으세요. 리포지토리 링크에 접근하려면 서문에 있는 예제 코드 파일 다운로드 섹션의 단계를 따르세요.

미들웨어

인증과 세션 관리(세션 섹션에서 배울 예정)는 미들웨어 스택으로 알려진 것에 의해 처리됩니다. 우리의 bookr 프로젝트에서 인증을 구현하기 전에 이 미들웨어 스택과 그 모듈에 대해 조금 배워봅시다.

3장, URL 매핑, 뷰 및 템플릿에서는 요청/응답 프로세스에 대한 장고의 구현과 그 뷰 및 렌더링 기능에 대해 논의했습니다. 이 외에도 장고의 핵심 웹에 있어 매우 중요한 역할을 하는 또 다른 기능이 있습니다.

processing is **middleware**. Django's middleware refers to a variety of software components that intervene in this request/response process to integrate important functionalities such as security, session management, and authentication.

So, when we write a view in Django, we don't have to explicitly set a series of important security features in the response header. These additions to the response object are automatically made by the `SecurityMiddleware` instance after the view returns its response. As middleware components wrap the view and perform a series of preprocesses on the request and post-processes on the response, the view is not cluttered with a lot of repetitive code, and we can concentrate on coding application logic rather than worrying about low-level server behavior. Rather than building these functionalities into the Django core, Django's implementation of a middleware stack allows these components to be both optional and replaceable.

Middleware modules

When we run the `startproject` subcommand, a default list of middleware modules is added to the `MIDDLEWARE` variable in the `<project>/settings.py` file, as follows:

```
MIDDLEWARE = [  
    "django.middleware.security.SecurityMiddleware",  
    "django.contrib.sessions.middleware.SessionMiddleware",  
    "django.middleware.common.CommonMiddleware",  
    "django.middleware.csrf.CsrfViewMiddleware",  
    "django.contrib.auth.middleware.AuthenticationMiddleware",  
    "django.contrib.messages.middleware.MessageMiddleware",  
    "django.middleware.clickjacking.XframeOptionsMiddleware",  
]
```

This is a minimal middleware stack that is suitable for most Django applications. The following list elaborates on the general purpose of each module:

- `SecurityMiddleware` provides common security enhancements, such as handling SSL redirects and adding response headers to prevent common hacks
- `SessionMiddleware` enables session support and seamlessly associates a stored session with the current request
- `CommonMiddleware` implements a lot of miscellaneous features, such as rejecting requests from the `DISALLOWED_USER_AGENTS` list, implementing URL rewrite rules, and setting the Content-Length header
- `CsrfViewMiddleware` adds protection against **cross-site request forgery (CSRF)**

처리는 미들웨어입니다. Django의 미들웨어는 보안, 세션 관리 및 인증과 같은 중요한 기능을 통합하기 위해 이 요청/응답 프로세스에 개입하는 다양한 소프트웨어 구성 요소를 의미합니다.

따라서 Django에서 뷰를 작성할 때 응답 헤더에 일련의 중요한 보안 기능을 명시적으로 설정할 필요가 없습니다. 이러한 응답 객체에 대한 추가 사항은 뷰가 응답을 반환한 후 SecurityMiddleware 인스턴스에 의해 자동으로 추가됩니다. 미들웨어 구성 요소가 뷰를 감싸고 요청에 대한 일련의 전처리 및 응답에 대한 후처리를 수행하므로 뷰는 많은 반복적인 코드로 혼잡해지지 않으며, 우리는 저수준 서버 동작에 대한 걱정 없이 애플리케이션 로직 코딩에 집중할 수 있습니다. 이러한 기능을 Django 코어에 구축하는 대신, Django의 미들웨어 스택 구현은 이러한 구성 요소가 선택적이고 교체 가능하도록 합니다.

미들웨어 모듈

startproject 하위 명령을 실행할 때 기본 미들웨어 모듈 목록이 <project>/settings.py 파일의 MIDDLEWARE 변수에 추가됩니다. 다음과 같이:

```
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XframeOptionsMiddleware",
]
```

이것은 대부분의 Django 애플리케이션에 적합한 최소한의 미들웨어 스택입니다. 다음 목록은 각 모듈의 일반적인 목적을 설명합니다:

- SecurityMiddleware는 SSL 리디렉션 처리 및 일반 해킹을 방지하기 위해 idx_494da315 응답 헤더를 추가하는 등의 일반 보안 향상을 제공합니다.
- SessionMiddleware는 세션 지원을 가능하게 하고 저장된 세션을 현재 요청과 원활하게 연결합니다.
- CommonMiddleware는 DISALLOWED_USER_AGENTS 목록에서 요청을 거부하고, URL 재작성 규칙을 구현하며, Content-Length 헤더를 설정하는 등 많은 잡다한 기능을 구현합니다.
- CsrfViewMiddleware는 교차 사이트 요청 위조(CSRF)에 대한 idx_c7d3ba56 보호를 추가합니다.